

The Siemens logo is displayed in a white rectangular box in the upper left corner. The word "SIEMENS" is written in a bold, teal, sans-serif font. The background of the entire page is a low-angle photograph of a modern glass skyscraper reaching towards a blue sky with scattered white clouds. The glass panels of the building reflect the sky and clouds, creating a complex geometric pattern.

SIEMENS

Teamcenter 12.3

Rich Client
Customization

PLM00075 - 12.3

Contents

Getting started with rich client customization

Rich client customization types and requirements	1-1
Siemens Digital Industries Software customization support	1-3
Syntax definitions	1-3

Rich client customization using style sheets

Introduction to style sheets	2-1
Registering a style sheet	2-4
An example of preference hierarchy	2-4
How does the rich client style sheet viewer work?	2-8
Sample customizations using style sheets	2-10
Search for style sheets	2-10
Create a custom style sheet based on an existing style sheet	2-13
Create a custom style sheet by importing an XML file	2-16
Localizing style sheets	2-17
Use SWT instead of Swing	2-17
Style sheet definition	2-19
Rich client view rendering	2-19
XML elements	2-25
Rendering hints	2-76

Rich client customization using preferences and properties files

Add a quick search item	3-1
Configuring the worklist using .properties files	3-3
Displaying files in the viewer	3-4
Working with themes	3-4
Registry	3-6
What is the registry?	3-6
User properties files	3-7
Supported types	3-8
Registry keys	3-8
Customizing tabs	3-8
Customizing the data tabs display	3-8
Edit a custom properties file to display tabs	3-10
Sample tab customization	3-10
Add a column to view occurrence notes	3-11
Create a new type wizard	3-13
New type wizard overview	3-13
Add a menuContribution for your wizard	3-14
Utilize the extWizard extension point to create your wizard	3-15
Utilize the boTypesLoader extension point to filter the types	3-16
Utilize the extWizardRef extension point to add pages	3-16

Utilize the operation extension point to create the object	3-18
Customize the rich client properties files	3-18

Rich client customization using Eclipse plug-ins

Basic concepts about rich client customization	4-1
Understanding the Eclipse rich client platform framework	4-1
What reference material is available?	4-2
What are perspectives and views?	4-2
Process for creating rich client customizations	4-2
Introduction to SWT	4-3
Create the Eclipse rich client development environment	4-3
Ensure your customizations appear	4-5
The rich client and Eclipse 4 migration	4-6
Eclipse 4 architecture overview	4-6
Teamcenter rich client and Eclipse 4 strategy	4-7
Important changes in Eclipse 4	4-9
Creating UI components using Eclipse 4 model fragments	4-11
Sample rich client customizations	4-16
Provided rich client customization examples	4-16
Example: Create a custom perspective	4-17
Example: Create a custom view	4-18
Example: Command handlers	4-19
Example: Important files and the viewer	4-20
Example: OSGi	4-20
Example: Other contents in the sample plugin project	4-21
Hide a view	4-21
Localize your customizations	4-22
Customizing Command Suppression	4-27
Additional examples for reference only	4-31
Distributing rich client customizations	4-76
Export your custom plug-in to the rich client	4-76
Process for distributing customizations to the rich client	4-77
Package custom rich client files	4-77
Create a feature file for rich client customizations	4-77
Rich client Javadoc	4-80
User interface components documented in Javadoc	4-80
User interface components in the com.teamcenter.rac.common package	4-80
User interface components in the com.teamcenter.rac.util package	4-95
Troubleshooting rich client customization	4-127
Common problems in rich client customization	4-127
Rich client debugging	4-128
Enabling client-side logging	4-131
Listener leaks	4-134

Rich client customization reference

Adding a third-party JAR file to your plug-in	A-1
Command line options for rich client startup	A-1

Coding standards	A-5
File organization	A-5
Naming conventions	A-5
Property conventions	A-6
Source code conventions	A-6
Dialog box standards	A-7
Property beans	A-8
Common Teamcenter command IDs	A-26
Plug-in locations of perspectives	A-27
Application Integration Framework (AIF)	A-28
AIF customization and development	A-28
Integrating the Application Integration Framework (AIF) desktop with the Eclipse workbench	A-29
AIF context sensitivity	A-30

Customizing form and properties display

Why would I use forms?	B-1
Developing forms using JavaBeans	B-2
Process for developing forms using JavaBeans	B-2
Create a sample custom form using JavaBeans	B-3
Using the Teamcenter property beans	B-9
Developing custom property beans	B-10
Developing forms by extending the abstract class	B-11
Process for developing forms by extending the abstract class	B-11
Create a sample custom form by extending the abstract class	B-12
General comments	B-19
Form performance issues	B-20
Form events	B-21
Form user interface display components	B-21
Displaying a form	B-22

Index — Index-1

Tables

A-4. My Teamcenter File→New menu commands	A-27
A-5. My Teamcenter Edit menu commands	A-27
A-6. Partial list of plugin locations of perspectives	A-28

1. Getting started with rich client customization

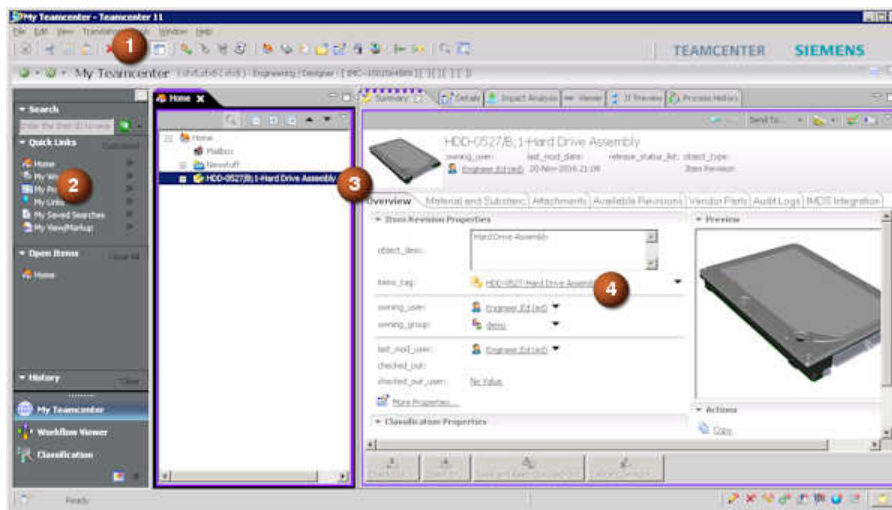
Rich client customization types and requirements	1-1
Siemens Digital Industries Software customization support	1-3
Syntax definitions	1-3

1. Getting started with rich client customization

Rich client customization types and requirements

The rich client uses several methods to control how it presents information. You need to be familiar with what you can customize, and which method controls each part of the interface.

Where can I customize?



	Location	Customization Method
1	Menu, Toolbar, and Context Menu	
	Hide or suppress	Application
	Create new	Plug-in
2	Left-hand Navigation (LHN) Panel	
	Search (Quick search)	Preference
	Quick Links	User
	Open Items	None
	History	None
	Perspectives (Applications)	User

	Location	Customization Method
3	Perspective	
	Modify existing	User
	Create new	Plug-in
4	View	XRT

What do I need?

In order to perform these customization methods, you need to know the following:

- **XML Rendering Templates (XRT)**

XML rendering templates, also called style sheets, control the layout of properties within the windows. Style sheets:

- Are easy to create and modify. (use XML)
- Are stored in the database and do not need to be deployed.
- Are the source for the organization of properties within view tabs as well as the view tabs themselves.
- Do not require any additional software.

- **Custom Plug-ins**

The core of the rich client is a set of plug-ins running on the Eclipse platform. The Eclipse platform controls the menuing, windowing, and all other general behind-the-scenes software requirements. Plug-in projects:

- Are powerful (use Java)
- Require the installation of the Eclipse IDE.
- Require knowledge of the Eclipse platform, plug-in development, workbench, and the OSGi framework.
- Are the source for modifying visibility of commands, menus, and toolbars.
- Are the source of new command locations, views, perspectives.
- Need to be deployed to each rich client installation.

• Preferences

These variables are stored within the Teamcenter database, and used to set configuration options. Teamcenter preferences:

- Are simple text variables, similar to operating system environment variables.
- Can be set using the rich client or command-line utility.
- Have multiple layers of scope from user to site.
- Do not require a deploy.

Siemens Digital Industries Software customization support

Siemens Digital Industries Software is committed to maintaining compatibility between Teamcenter product releases. If you customize functions and methods using published APIs and documented extension points, be assured that the next successive release will honor these interfaces. On occasion, it may become necessary to make behaviors more usable or to provide better integrity. Our policy is to notify customers at the time of the release *prior* to the one that contains a published interface behavior change.

Siemens Digital Industries Software does not support code extensions that use unpublished and undocumented APIs or extension points. All APIs and other extension points are unpublished unless documented in the official set of technical manuals and help files issued by Siemens Digital Industries Software Technical Communications.

The Teamcenter license agreements prohibit reverse engineering, including: decompiling Teamcenter object code or bytecode to derive any form of the original source code; the inspection of header files; and the examination of configuration files, database tables, or other artifacts of implementation. Siemens Digital Industries Software does not support code extensions made using source code created from such reverse engineering.

If you have a comment or would like to request additional extensibility, contact the Siemens Digital Industries Software customer support representatives at GTAC for further assistance.

Syntax definitions

This manual uses a set of conventions to define the syntax of Teamcenter commands, functions, and properties. Following is a sample syntax format:

```
harvester_jt.pl [bookmark-file-name bookmark-file-name ...]  
                [directory-name directory-name ...]
```

The conventions are:

Bold	Bold text represents words and symbols you must type exactly as shown. In the preceding example, you type harvester_jt.pl exactly as shown.
<i>Italic</i>	Italic text represents values that you supply. In the preceding example, you supply values for <i>bookmark-file-name</i> and <i>directory-name</i> .
<i>text-text</i>	A hyphen separates two words that describe a single value. In the preceding example, <i>bookmark-file-name</i> is a single value.
[]	Brackets represent optional elements.
...	An ellipsis indicates that you can repeat the preceding element.

Following are examples of correct syntax for the **harvester_jt.pl** command:

```
harvester_jt.pl
harvester_jt.pl assembly123.bkm
harvester_jt.pl assembly123.bkm assembly124.bkm assembly125.bkm
harvester_jt.pl AssemblyBookmarks
```

2. Rich client customization using style sheets

Introduction to style sheets	2-1
Registering a style sheet	2-4
An example of preference hierarchy	2-4
How does the rich client style sheet viewer work?	2-8
Sample customizations using style sheets	2-10
Search for style sheets	2-10
Create a custom style sheet based on an existing style sheet	2-13
Create a custom style sheet by importing an XML file	2-16
Localizing style sheets	2-17
Use SWT instead of Swing	2-17
Style sheet definition	2-19
Rich client view rendering	2-19
XML elements	2-25
Rendering hints	2-76

2. Rich client customization using style sheets

Introduction to style sheets

If you change style sheets, clear the client's cache to see the style sheet changes. Exit the rich client and restart it using the **-clean** command argument to remove the old configuration from cache.

If using the rich client to edit or register XML rendering templates (stylesheets), you must have DBA privileges.

Why use style sheets?

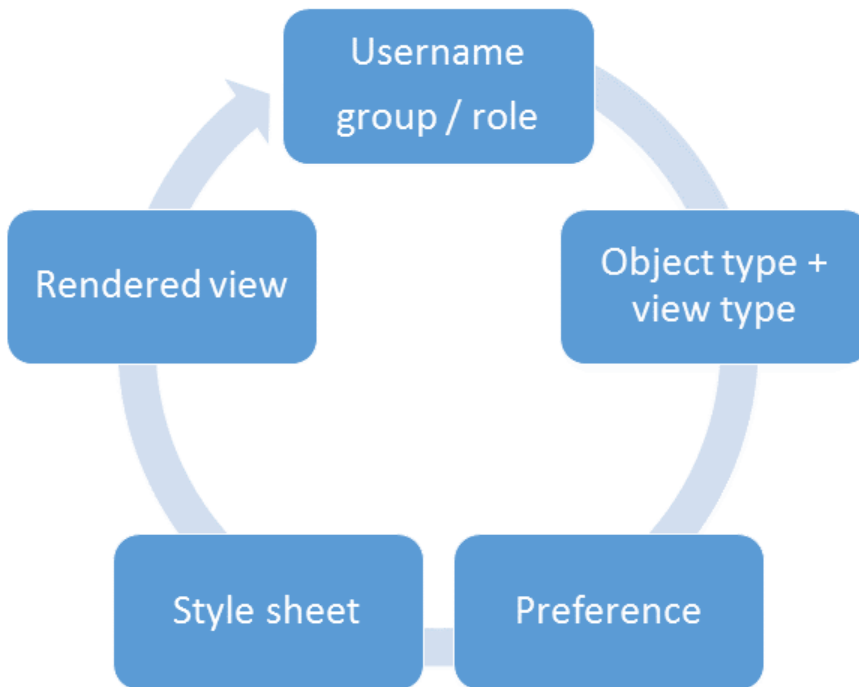
You can control the layout of the user interface based on the type of object being viewed and the credentials of the user.

- You can customize using configuration instead of coding.
- The style sheets are stored on the server. No deploy required.

Each commonly-used business object type (such as item revisions, folders, datasets, and so on) has a style sheet that defines the layout of its properties in the user interface.

How do they work?

Following are the steps Teamcenter takes to render a view using XRT.



1. The object type being viewed and the viewer being used determine the name of the preference that will be used. For example: if a **DocumentRevision** object is being viewed in the **Summary** viewer, the preference used is **DocumentRevision.SUMMARYRENDERING**
2. Since the preferences used for style sheet registration are hierarchical, a group, role, or user preference of the same name would override the site default. For example, if there is a role preference named **DocumentRevision.SUMMARYRENDERING**, it would override the site preference.
3. The appropriate preference is retrieved. It contains the name of the **XMLRenderingStylesheet** dataset that contains the rendering information.
4. The **XMLRenderingStylesheet** dataset is retrieved by name. Its contents are parsed and the view layout is generated.
5. The view is rendered and displayed to the user.

Where are they used?

Style sheets are used in the following locations.

- **Property** dialog box
Preference syntax: `.RENDERING`
Defines the layout of the **Properties** dialog box, and the **View** tab when showing properties.
- **Summary** view

Preference syntax: `.SUMMARYRENDERING`

Defines the layout of the **Summary** tab.

- **Create** dialog box

Preference syntax: `.CREATERENDERING`

Defines the layout of the dialog box used when the user chooses **File**→**New**→**Other** and some portions of the dialog box when the user chooses **File**→**New**→*object*.

To add a property to an operation dialog box, you must not only add the property to the registered style sheet but also to the operation input in the Business Modeler IDE.

- **Save As** dialog box

Preference syntax: `.SAVEASRENDERING`

Defines the layout of the **Save As** dialog box.

To add a property to an operation dialog box, you must not only add the property to the registered style sheet but also to the operation input in the Business Modeler IDE.

- **Revise** dialog box

Preference syntax: `.REVISERENDERING`

Defines the layout of the **Revise** dialog box.

To add a property to an operation dialog box, you must not only add the property to the registered style sheet but also to the operation input in the Business Modeler IDE.

- **Form** display

Preference syntax: `.FORMRENDERING`

Defines the layout of forms, such as the **Item Master** form or the **Item RevisionMaster** form.

How are they registered?

The preferences that register style sheets have the following generic syntax.

```
objectType.RENDERINGTYPE
```

objectType

Refers to the database (non-localized) name of the object, as found in the Business Modeler IDE. For example, the preference that controls the rendering for the Summary view when the user selects a Document Revision is **DocumentRevision.SUMMARYRENDERING**. If the object type selected does not have a preference, then Teamcenter will look for a preference for the object's parent in the POM. In this case, the parent of DocumentRevision is ItemRevision, so if

DocumentRevision.SUMMARYRENDERING does not exist, then **ItemRevision.SUMMARYRENDERING** will be retrieved instead.

RENDERINGTYPE

One of the supported rendering locations.

What is the syntax of a style sheet?

Style sheets are XML documents stored in **XMLRenderingStylesheet** datasets. Even though they are commonly called style sheets, they are not CSS or XSL style sheets. They are more accurately called XML

rendering templates (XRT), but both terms are used in this documentation. The XML code allows you to define a subset of properties to display, the display order, the user interface rendering components to be used, and more.

Which style sheets are provided?

To see all the available style sheets, search Teamcenter for **XMLRenderingStylesheet** datasets.

Registering a style sheet

Style sheet preferences are hierarchical preferences that take a single string value. You can set their value using any available method, they are no different than any other preference.. Following are the most common:

Registering a style sheet using the command line

You can use the **preferences_manager** command line utility to import or modify style sheet preferences like any other preference. This utility provides all of the functionality you need when working with preferences, without the restrictions you may find in some of the UI helpers.

Registering a style sheet using the rich client preferences interface

You can use the **Edit→Options** interface to register style sheet preferences, just like any other preference.

Registering a style sheet using the rich client viewer tab

The rich client has a special interface for editing style sheet content that can also modify the registration information. When you have an **XMLRenderingTemplate** dataset highlighted, the **View** tab becomes an IDE of sorts for editing the XML content. This interface saves the XML edits as well as creating or modifying the preference based on the settings.

An example of preference hierarchy

Everything in this example is based on a single preference, one which registers a style sheet to a business object for the summary view. It could be any preference as all preferences behave the same way. Since this preference definition's protection scope is **User**, you can create instances at the **Site**, **Group**, **Role**, and **User** location. This means you can control its value based on your users' current group, role, or even user name.

Example: I want the summary view's property layout for item revisions to depend on my users' login information

Following are the details of this example.

- You have three groups: Engineering, Manufacturing, and Testing.
Each group has three roles: Manager, Designer, and Viewer.
- You want a default style sheet that everyone will use unless otherwise specified.
- Your technical users need an extended set of properties.
- Your managers need a page of workflow information.
- Your designers need classification information.
- You have users that just need a simplified layout for viewing.
- You have Conner. Conner is a power-user.
Conner needs a special layout regardless of which group or role he's in.

Style sheet datasets

Five style sheet datasets are considered.

ItemRevSummary

Configured to be the default style sheet for the Item Revision summary page. This applies to everyone unless overridden.

IRSumTech

Configured to provides the extra properties for the Engineering and Manufacturing groups, but not for any other groups.

IRSumMgr

Configured to display workflow information for the Manager role, regardless of group.

IRSumDes

Configured to show the classification trace for the Designer role, regardless of group.

ConnersIRSum

Configured for Conner. Conner has his own requirements

Preference instances

Assign the style sheets to the various groups and roles, and even users if desired, by creating each preference instance with the value pointing to the respective style sheet. In this example, there are 6 preference instances created.

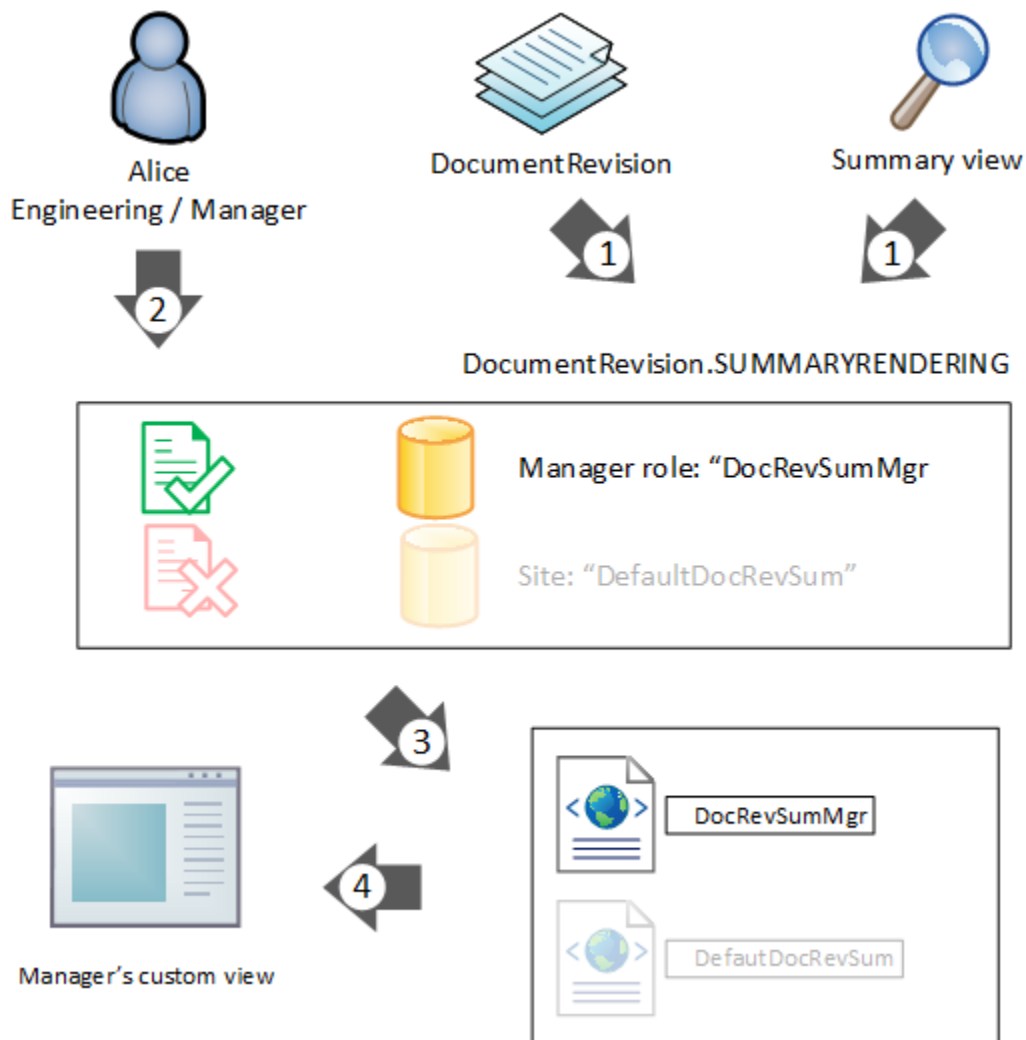
User preferences Conner: ConnersIRSum

Role preferences	Manager: IReSumMgr
	Designer: IReSumDes
Group preferences	Engineering: IReSumTech
	Manufacturing: IReSumTech
Site preference	<i>value:</i> ItemRevSum

The Viewer role and the Tester group have no preference instances created for their location.

How does Teamcenter choose which preference to use?

In this example, Alice selects a **DocumentRevision** business object and uses the **Summary** tab. When she does this, Teamcenter performs a few steps to determine which style sheet to use.



1. Based on the object type and the view location, the system knows the name of the preference instances to retrieve.

In this example, **DocumentRevision.SUMMARYRENDERING**.

There are two instances: one at the **Site** location, and one at the Manager **Role** location.

2. Based on the user's current session information, Teamcenter chooses the appropriate preference instance.

Less specific locations are overridden by more specific locations.

3. The value of the chosen preference instance is read, providing the name of the style sheet to retrieve.
4. Teamcenter uses the style sheet to render the view.

Result

Your users see a different set of information based on what group or role they are in because the client uses different style sheets.

User - Group / Role	Preference instance build-up	Resulting style sheet
Alice — Engineering / Manager	Alice: <i>none</i> Manager: IRTSumMgr Engineering: IRTSumTech Site: ItemRevSum	IRSumMgr
Ted — Manufacturing / Manager	Ted: <i>none</i> Manager: IRTSumMgr Manufacturing: IRTSumTech Site: ItemRevSum	IRSumMgr
Sue — Testing / Manager	Sue: <i>none</i> Manager: IRTSumMgr Testing: <i>none</i> Site: ItemRevSum	IRSumMgr
Bob — Engineering / Designer	Bob: <i>none</i> Designer: IRTSumDes Engineering: IRTSumTech Site: ItemRevSum	IRSumDes
Carol — Engineering / Viewer	Carol: <i>none</i> Viewer: <i>none</i> Engineering: IRTSumTech Site: ItemRevSum	IRSumTech
Pat — Testing / Viewer	Pat: <i>none</i> Viewer: <i>none</i> Testing: <i>none</i>	ItemRevSum

User - Group / Role	Preference instance build-up	Resulting style sheet
	Site: ItemRevSum	
Conner — Engineering / Manager	Conner: ConnersIRSum Manager: IRSUMMgr Engineering: IRSUMTech Site: ItemRevSum	ConnersIRSum
Conner — Testing / Viewer	Conner: ConnersIRSum Viewer: <i>none</i> Testing: <i>none</i> Site: ItemRevSum	ConnersIRSum

- Alice sees the style sheet for Managers because she does not have a user preference set to supercede it. The site preference is overridden by the Engineering group preference, which is overridden by the Manager role preference. Ted has the same result; the Manufacturing group preference is overridden by the Manager preference. Sue doesn't have a group preference, but she still gets the Manager role preference.
- Bob sees the style sheet for Designers because of his role, similar to the preceding example.
- Carol sees the tech style sheet because there is no role preference for Viewers.
- Pat's group and role do not have preferences associated with them, and neither does she have a user preference, so she gets the default style sheet defined by the site preference.
- Conner gets Conner's style sheet regardless of which group or role he's in, since a user preference supercedes all others.

How does the rich client style sheet viewer work?

The rich client's **Viewer** tab becomes an editor when the user selects an **XMLRenderingTemplate** dataset.

Rich client procedure

The rich client follows the following steps:

1. Search for a ***REGISTEREDTO** helper preference.

These preferences are based on the name of the dataset, and the value tells the system to which object type the dataset is registered.

2. Use the helper preference's value to search for the corresponding ***RENDERING** preference.
3. Display the XML content of the dataset, and populate the two fields with the registration information.

If none of the registration preferences were found, the rich client leaves them blank.

4. When the user chooses **Apply**, the rich client saves the dataset, and modifies or creates the appropriate registration and helper preference.

Style sheet registration helper preferences

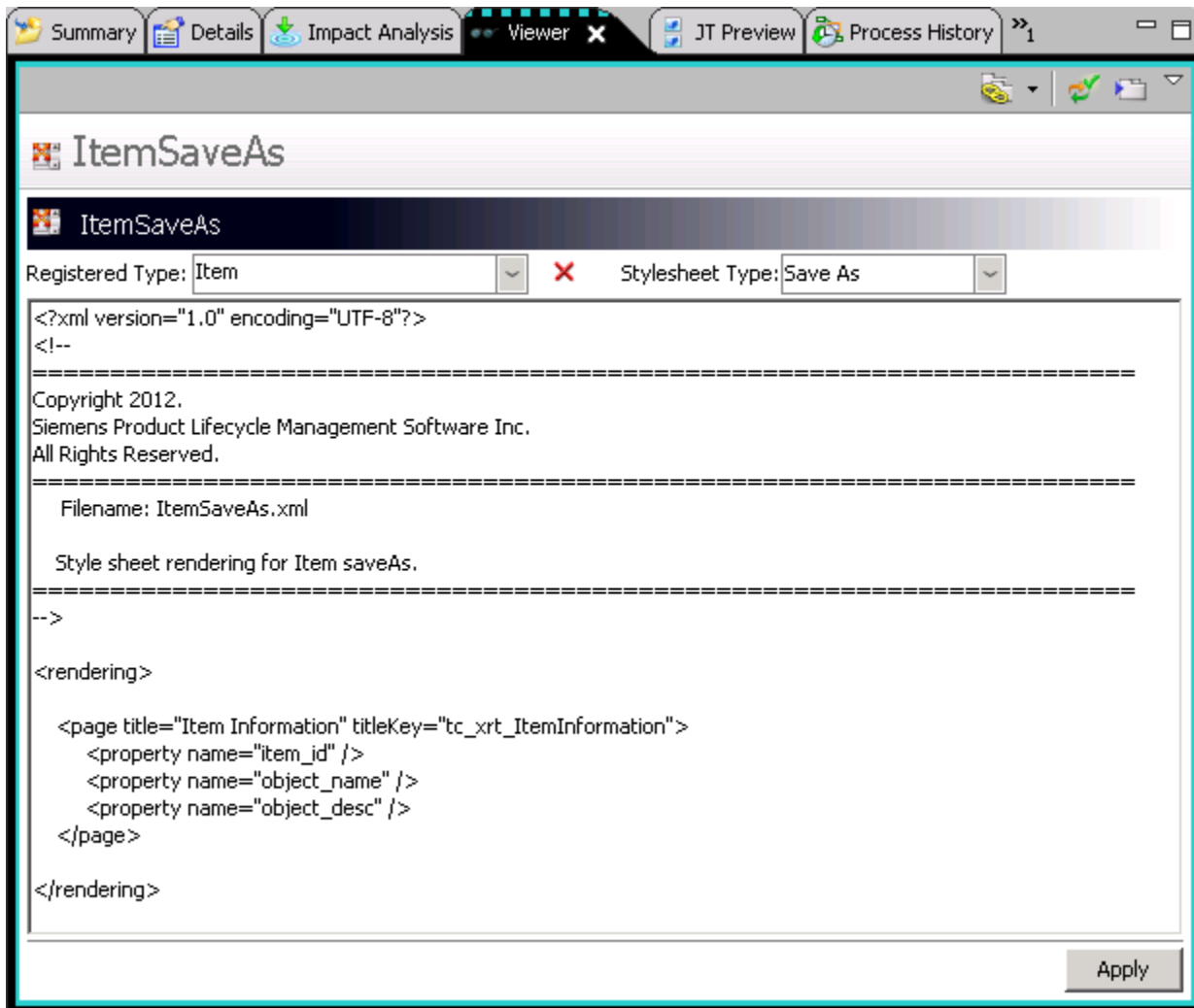
Following are the helper preferences in the order they are searched.

- **{datasetName}.FORM_REGISTEREDTO**
Corresponds to {typeName}.FORMRENDERING.
The dialog box for a **Form** object.
- **{datasetName}.SUMMARY_REGISTEREDTO**
Corresponds to {typeName}.SUMMARYRENDERING .
The **Summary** view tab.
- **{datasetName}.CREATE_REGISTEREDTO**
Corresponds to {typeName}.CREATERENDERING.
The **Create** command panel.
- **{datasetName}.SAVEAS_REGISTEREDTO**
Corresponds to {typeName}.SAVEASRENDERING.
The **Save As** command panel.
- **{datasetName}.REVISE_REGISTEREDTO**
Corresponds to {typeName}.REVISERENDERING.
The **Revise** command panel.
- **{datasetName}.REGISTEREDTO**
Corresponds to {typeName}.RENDERING.
The properties window. Either the **View Properties** dialog box, or when the **Viewer** tab shows object properties.

Example

For example, in a new installation of Teamcenter, the **ItemSaveAs** dataset might be registered to the **Save As** operation of the **Item**. If you highlight the **ItemSaveAs** dataset and switch to the **Viewer** tab, you will see its registration information in the **Registered Type** and **Stylesheet Type** fields.

When you select an **XMLRenderingStylesheet** dataset with the **Viewer** tab active, the rich client reads the {datasetName}.REGISTEREDTO preference to see



If you change the values in these fields and then click **Apply**, then the appropriate preference will be created or updated.

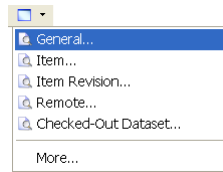
the value of the **Item.SAVEASRENDERING** preference. Its value will be **ItemSaveAs**.

Sample customizations using style sheets

Search for style sheets

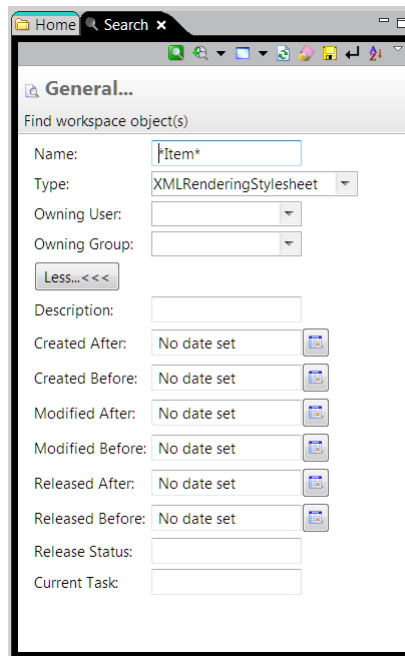
To find style sheets in the rich client, search for **XMLRenderingStylesheet** datasets.

1. Click the **Open Search View** button .
2. Click the arrow on the **Select a Search** button and choose **General**.



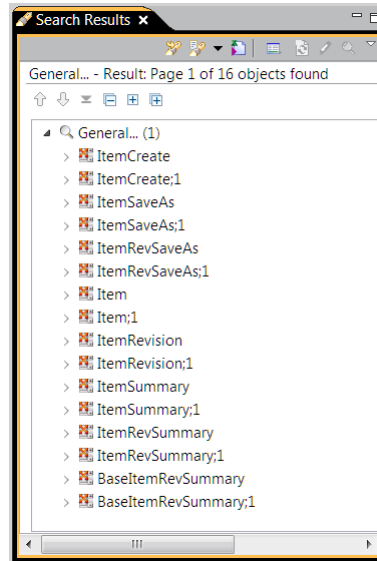
Starting the search for style sheets

3. In the **Type** box, type **XMLRenderingStylesheet**.
If you are looking for style sheets for a particular kind of object, enter a string in the **Name** box to look for those kinds of style sheets. For example, if you want to find all style sheets for items or item revisions, type ***Item*** in the **Name** box.



Searching for XMLRenderingStylesheet datasets

4. Press the Enter key or click the **Execute the Search** button .
The results are displayed in the **Search Results** view.

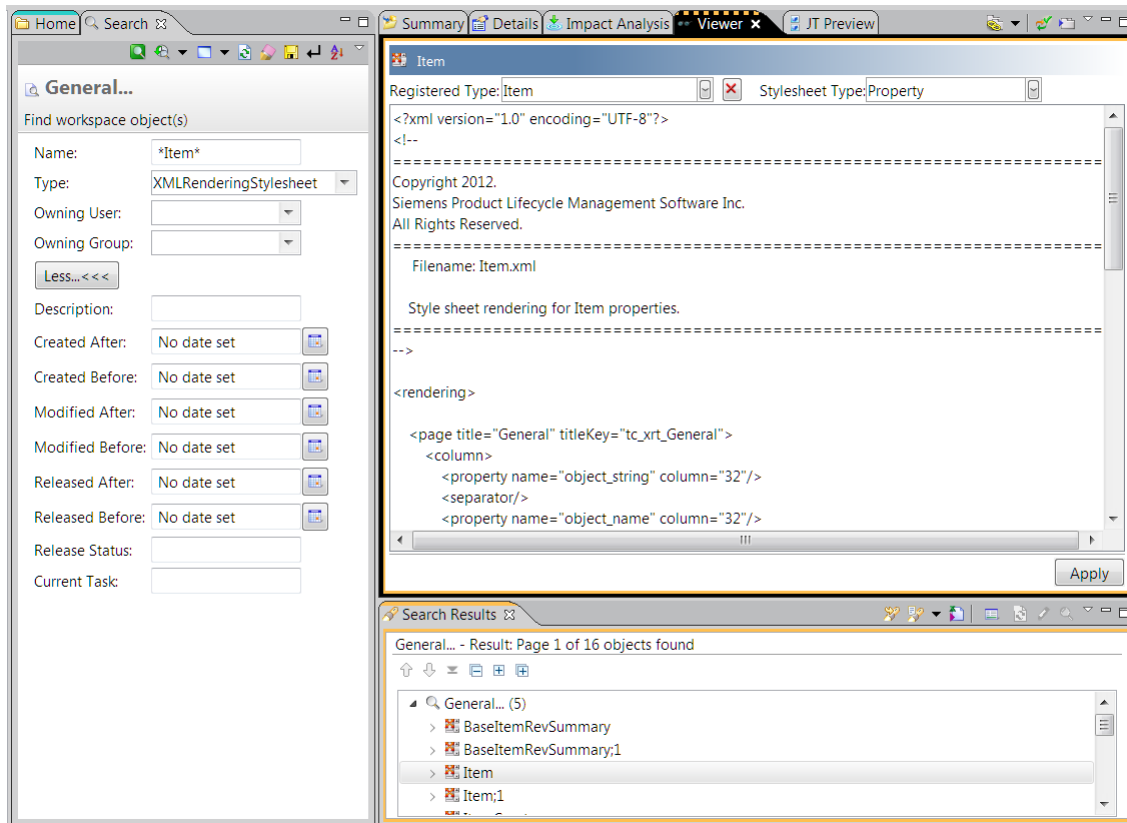


Viewing the search results for XMLRenderingStylesheet datasets

5. In the **Search Results** tab, select the style sheet you want to view. Click the **Viewer** tab to see the style sheet.

Note:

Do not double-click a style sheet (**XMLRenderingStylesheet** dataset file) in an attempt to open it. If you do, you receive an **Unable to open** error message. Instead, select the style sheet and view its contents in the **Viewer** view.



Viewing the style sheet contents in the rich client

Create a custom style sheet based on an existing style sheet

You can create your own custom style sheet.

For example, you create a custom business object in the Business Modeler IDE and install it to the rich client, and you want to create a unique style sheet to display the custom properties. To create the style sheet, in the rich client, search for **XMLRenderingStylesheet** datasets, save one as your own custom style sheet dataset, and then register the custom style sheet for use with the custom business object.

1. In the rich client, search for a style sheet you can base your new style sheet on.
2. In the **Search Results** view, select the style sheet you want to use, choose **File**→**Save As**, and rename it. For example, if you want to create a style sheet to be used with a custom **A5_MyItem** business object, you could name the style sheet **A5_MyItem**.
The new style sheet dataset is saved in your **Newstuff** folder in the **Home** view and is still displayed in the **Viewer** tab.
3. Edit the style sheet.
 - a. Change the named reference for the file by selecting the style sheet dataset and choosing **View**→**Named References**. In the **Name** column in the **Named References** dialog box, change the old name of the file to the new save as file name.

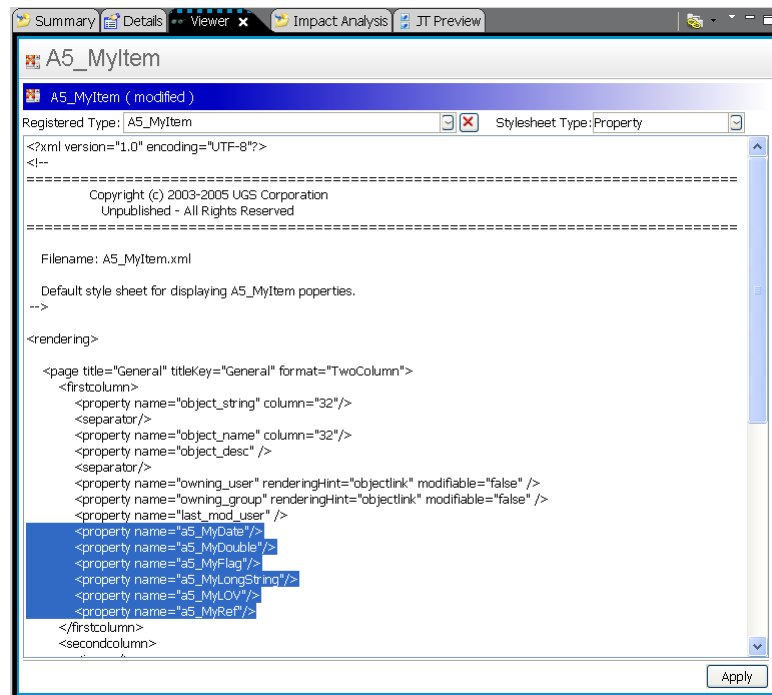
- b. In the **Viewer** tab, click the arrow in the **Registered Type** box and select the business object type you want to register it to. For example, if you have a custom **A5_MyItem** business object added to your server, select **A5_MyItem** from the list.
- c. Edit the style sheet in the **Viewer** tab to include the elements you want displayed in the layout.
For example, if you want to display custom properties, add them where you want them to appear on the page, like this:

```
<page title="General" titleKey="tc_xrt_General">
  <column>
    <property name="object_string" column="32"/>
    <separator/>
    <property name="object_name" column="32"/>
    <property name="object_desc" />
    <separator/>
    <property name="owning_user" renderingHint="objectlink"
modifiable="false" />
    <property name="owning_group" renderingHint="objectlink"
modifiable="false" />
    <property name="last_mod_user" />
    <property name="a5_MyDate"/>
    <property name="a5_MyDouble"/>
    <property name="a5_MyFlag"/>
    <property name="a5_MyLongString"/>
    <property name="a5_MyLOV"/>
    <property name="a5_MyRef"/>
  </column>
  <column>
    <image/>
  </column>
</page>
```

- d. To change the style sheet type, click the arrow in the **Stylesheet Type** box. You can choose one of the following types:

Property
Form
Summary
Create

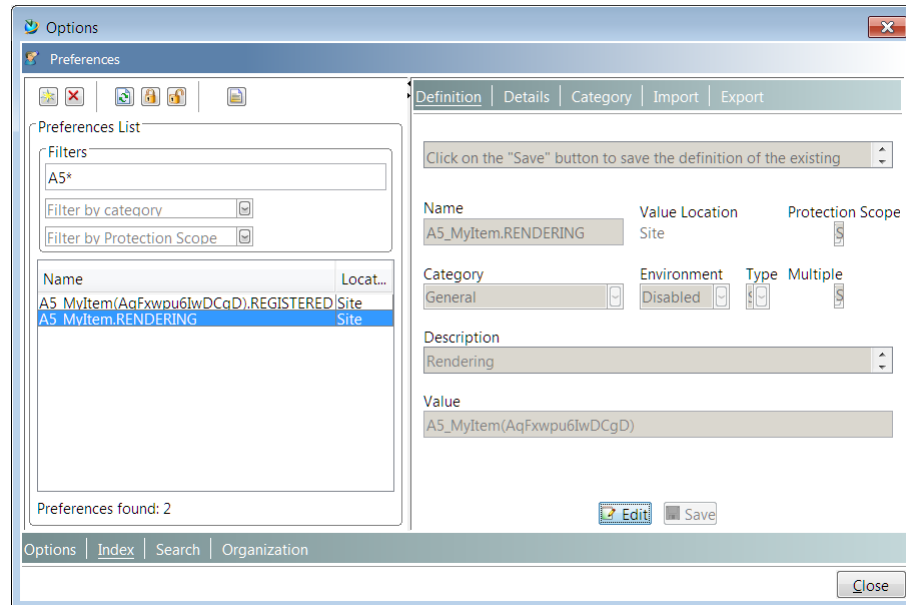
4. When you are done making changes, click the **Apply** button in the lower right corner of the view.



Create a custom style sheet

Because you used the **Registered Type** box on the **Viewer** tab to register the style sheet with a business object type, two new preferences are created (a **REGISTEREDTO** preference and a **RENDERING** preference). These preferences apply the style sheet to the business object so that the style sheet is displayed in the situation you set it for (for example, for display of the business object's property, summary, form, or create information).

5. To see the two new preferences, choose **Edit**→**Options** and at the bottom of the dialog box, click **Search**.



Viewing the <dataset_name>.REGISTEREDTO and <type_name>.RENDERING preferences

Notice how in the **Current Values** box of the <type_name>.RENDERING preference there is a number in parentheses after the name of the business object. That is a GUID number that identifies that unique business object, and that GUID number is set in the <dataset_name(dataset-UID)>.REGISTEREDTO preference. This ensures that the style sheet is applied to the correct business object.

6. To see the style sheet changes in the clients, clear the client's cache. Exit the rich client and restart it using the **-clean** command argument to remove the old configuration from cache. To make the new style sheets available for quick loading to clients, run the **generate_client_meta_cache** utility to add the new style sheets to client cache, for example:

```
generate_client_meta_cache -u=infodba -p=infodba -g=dba update
stylesheets
```

Create a custom style sheet by importing an XML file

You can create a new style sheet by creating a new dataset and associating it with an XML file.

1. In the rich client My Teamcenter application, create a new dataset of type **XMLRenderingStylesheet**.
2. Import the XML file as a named reference to the dataset as follows:
 - a. Right-click the dataset in the navigation tree and choose **Named References**. Teamcenter displays the **Named References** dialog box.
 - b. Click the **Add** button.

Teamcenter displays the **Add File** dialog box.

- c. Locate and select the XML file in your operating system directory and click **Add**. Teamcenter displays the XML file in the **Named References** dialog box.
 - d. Click **Close**.
3. Register the style sheet by setting the **Form.REGISTEREDTO** preference.
 4. To see the style sheet changes in the clients, clear the client cache. Exit the rich client and restart it using the **-clean** command argument to remove the old configuration from cache.

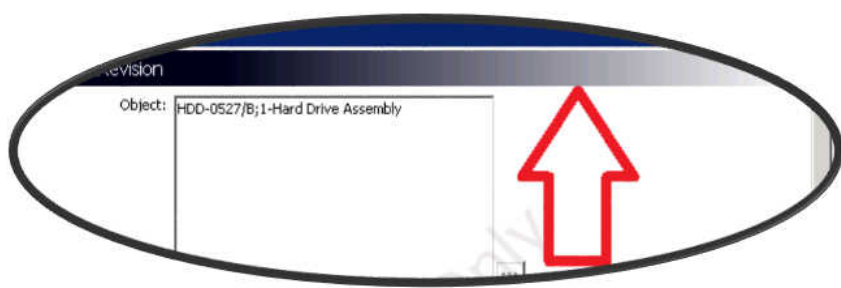
Localizing style sheets

Localization is the translation of text into the local language. XML rendering templates (XRTs), also known as style sheets, have their text localized by the TextServer. When you customize a style sheet with new text, you must enter the text to these file in the different languages to be displayed in the user interface. If you do not enter localized text strings, the unlocalized text appears in the user interface between exclamation marks (for example: **!MyText!**) except when in the **command** tag.

Use SWT instead of Swing

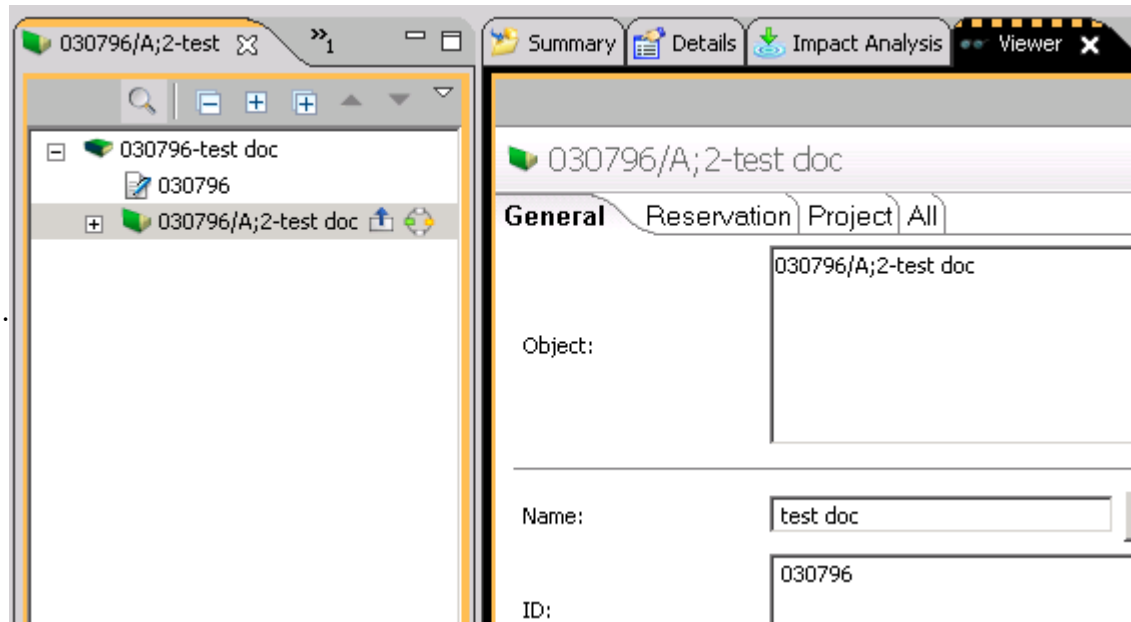
Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens Digital Industries Software encourages you to customize Teamcenter using SWT/JFace components. Siemens Digital Industries Software will discontinue Swing/AWT support in a future version. Because of the need to support legacy functionality, you may still encounter some Swing panels when using the rich client

Swing panels can be identified by the graduated bar at the top of the panel.

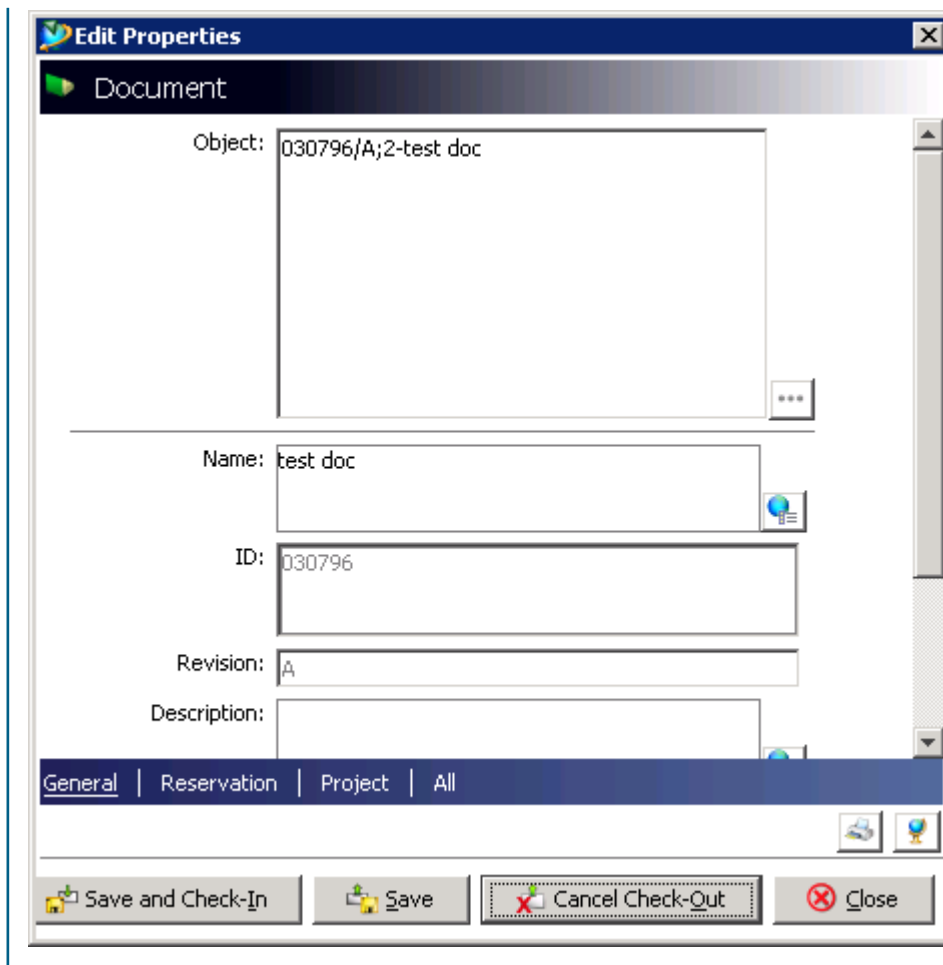


Example:

When viewing an object's properties, select it and then use the **Viewer** tab. The properties displayed in this view are rendered using SWT, but controlled by XML rendering templates (XRT), also known as style sheets. XRT is easily editable, does not need to be deployed, and can easily be tailored to any group, role, or user.



If you **View properties** using the context menu, the dialog box presented is controlled by the same XRT, but the underlying technology is Swing, and therefore limited in its functionality. Whenever possible, encourage your users to use the **Viewer** tab instead of the dialog box.



Style sheet definition

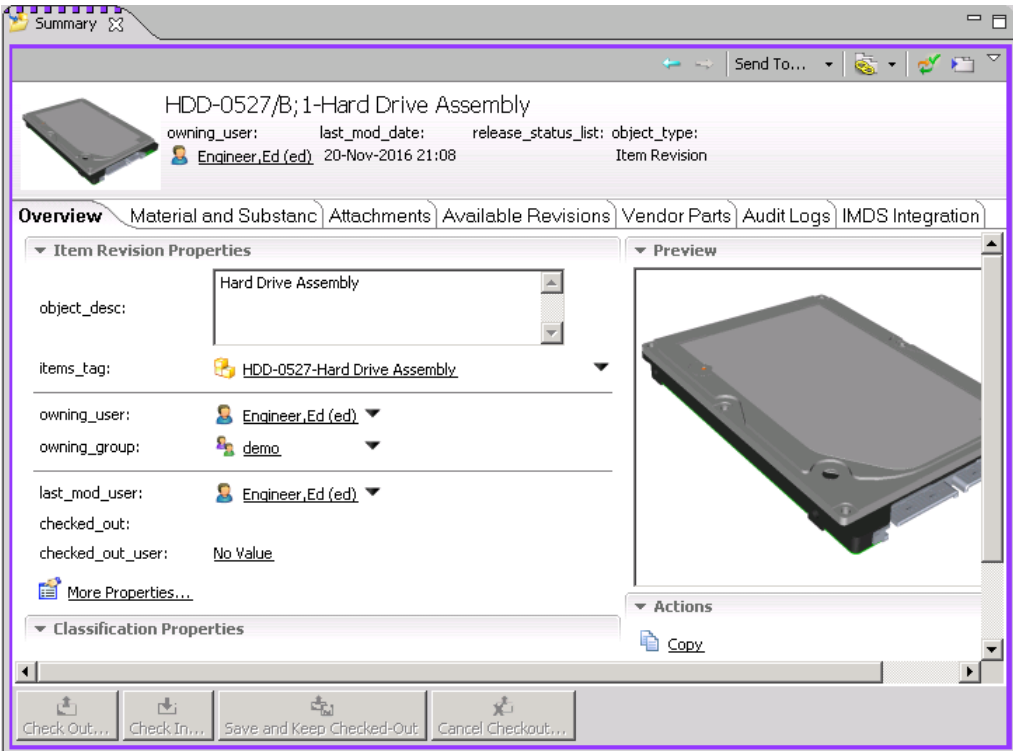
Rich client view rendering

In the Teamcenter rich client, the Eclipse view consists of several components controlled by an **XMLRenderingStylesheet** dataset. Following is a brief explanation of the various components of the rendering system.

View rendering

An individual view tab is defined by a `<rendering>` tag, and consists of two main components, the **header** and the **page**.

<rendering>



<rendering>

```

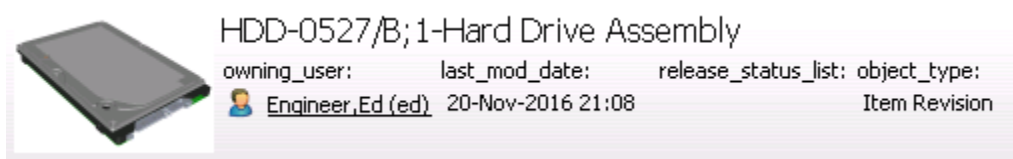
<header>
<page titleKey = ...>
<page titleKey = ...>
<page titleKey = ...>

```

View header

You control what is displayed in the header by modifying what is defined within the **<header>** tag.

<header>



<header>

```

<image source = "thumbnail"/>
<property name = "owning_user"/>
<property name = "last_mod_date"/>
<property name = "release_status_list"/>
<property name = "object_type"/>

```

View page

You control the number of pages within the view, their names, and when they appear using the **<page>** tag.

You can use **columns**, **sections**, **properties**, and **commands** within your page.

<page>

Overview
Material and Substance
Attachments
Available Revisions
Vendor Parts

```

<page titleKey="tc_xrt_Overview">
<page titleKey = "tc_xrt_MatlMaterialAndSubstanceInfo"
  title = "Material and Substance Information"
  visibleWhen = "MatlUsesMaterial != null">
<page title = "Attachments" titleKey="tc_xrt_attachments">
<page titleKey="tc_xrt_AvailableRevisions">
<page title="Related Links" titleKey="tc_xrt_RelatedLinks"
  visibleWhen="{pref:LIS_RelatedLinkTabVisible)==true">
<page titleKey = "tc_xrt_VendorParts" title = "Vendor Parts  "
  visibleWhen = "VMRepresents==null">
<page titleKey = "tc_xrt_VendorParts*" title = "Vendor Parts *"
  visibleWhen = "VMRepresents!=null">

```

Page section

You may divide your page into sections. Their names and their order on the page is controlled using the **<section>** tag.

<section>

▼ Item Revision Properties

object_desc:

items_tag: HDD-0527-Hard Drive Assembly

owning_user: Engineer, Ed (ed)

owning_group: demo

last_mod_user: Engineer, Ed (ed)

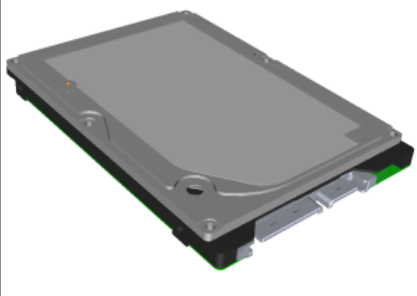
checked_out: No Value

checked_out_user: No Value

More Properties...

▼ Classification Properties

▼ Preview



▼ Actions

Copy

Revise...

New Workflow Process...

Save As

```

<section titleKey="tc_xrt_ItemRevProperties">
<section titleKey="tc_xrt_ClassificationProperties">
<section titleKey="tc_xrt_Preview">
<section titleKey="tc_xrt_actions" commandLayout="vertical">

```

Page columns

You may divide your sections on your page into columns by wrapping them in `<column>` tags. If you choose to use columns, the recommended limit is two.

<code><column></code>

<pre><page titleKey="tc_xrt_Overview"> <column> <section titleKey="tc_xrt_ItemRevProperties"> <section titleKey="tc_xrt_ClassificationProperties"> <column> <section titleKey="tc_xrt_Preview"> <section titleKey="tc_xrt_actions" commandLayout="vertical"></pre>

No columns

<pre><page titleKey="tc_xrt_Overview"> <section titleKey="tc_xrt_ItemRevProperties"> <section titleKey="tc_xrt_ClassificationProperties"> <section titleKey="tc_xrt_Preview"> <section titleKey="tc_xrt_actions" commandLayout="vertical"></pre>

Section and header properties

You control which properties are displayed within a section or header using the `<property>` tag.

<property> in a section**▼ Item Revision Properties**

object_desc:

Hard Drive Assembly

items_tag:

 HDD-0527-Hard Drive Assembly

owning_user:

 Engineer,Ed (ed) ▼

owning_group:

 demo ▼

last_mod_user:

 Engineer,Ed (ed) ▼

checked_out:

checked_out_user: No Value

 [More Properties...](#)

```

<section titleKey="tc_xrt_ItemRevProperties">
  <property name="object_desc"/>
  <property name="items_tag"/>
  <separator/>
  <property name = "owning_user" renderingHint = "objectlink"
    modifiable = "false"/>
  <property name = "owning_group" renderingHint = "objectlink"
    modifiable = "false"/>
  <separator/>
  <property name = "last_mod_user"/>
  <property name="checked_out"/>
  <property name = "checked_out_user"/>
  <command commandId="com.teamcenter.rac.properties"
    titleKey="tc_xrt_moreProperties"/>
</section>

```

<property> in a header

HDD-0527/B; 1-Hard Drive Assembly

owning_user:	last_mod_date:	release_status_list:	object_type:
 Engineer,Ed (ed)	20-Nov-2016 21:08		Item Revision

```

<header>
  <image source="thumbnail"/>
  <classificationTrace/>
  <property name = "owning_user"/>
  <property name = "last_mod_date"/>
  <property name="release_status_list"/>
  <property name = "object_type"/>

```

Property tables

You can display properties of related objects in an easy-to-read table.

<objectSet>

▼ Files

Add New...
 Paste
 Cut

Object ▲	Type	Owner
Disk_Drive_assembly	MISC	Engineer,Ed (ed)
Documentation	MS WordX	Engineer,Ed (ed)
HDD Functional Specs	MS WordX	Engineer,Ed (ed)
HDD Market Requirements	MS WordX	Engineer,Ed (ed)
HDD Market Requirements	PDF	Engineer,Ed (ed)
HDD Test Plan	MS WordX	Engineer,Ed (ed)
HDD-0527-B	UGMASTER	Engineer,Ed (ed)
HDD-CAE.docx	MS WordX	Engineer,Ed (ed)
HDD_Modal_Analysis_C	MISC	Engineer,Ed (ed)
HDD_Modal_Analysis_D	MISC	Engineer,Ed (ed)

▼ Documents

Add New...
 Paste
 Cut

Object ▲	Type	Owner
TB_6431/A;1-Security features	Technical Brief Revision	Engineer,Ed (ed)

```

<page title = "Attachments" titleKey="tc_xrt_attachments">
  <section titleKey = "tc_xrt_Files" title = "Files">
    <objectSet source = "IMAN_specification.Dataset, IMAN_reference.Dataset, ...
  <section titleKey = "tc_xrt_Documents" title = "Documents">
    <objectSet source = "IMAN_specification.DocumentRevision" ...

```

Commands

You can add commands to your sections or your tables.

<command> in table

▼ Files

Add New...
 Paste
 Cut

```

<objectSet source = "IMAN_specification.Dataset" ...
  <tableDisplay ...
  <treeDisplay ...
  <thumbnailDisplay/>
  <listDisplay/>
  <command commandId = "com.teamcenter.rac.common.AddNew"
  <command commandId = "com.teamcenter.rac.viewer.pastewithContext"
  <command commandId = "org.eclipse.ui.edit.cut"

```

<command> in a section

▼ Actions

 Copy
 Revise...
 New Workflow Process...
 Save As

```

<section titleKey="tc_xrt_actions" commandLayout="vertical">
  <command commandId = "com.teamcenter.rac.copy"/>
  <command commandId = "com.teamcenter.rac.revise"/>
  <command commandId="com.teamcenter.rac.newProcess" titleKey="tc_xrt_newProc"/>
  <command commandId="org.eclipse.ui.file.saveAs"/>

```

Other XML elements

You can use these elements to add additional information or visual input to your views.

<break>

You can use this element to add a horizontal space on the page. This is similar to the separator element, but with no visible line.

<classificationTrace>

You can add the classification category of an object. For example, **Home Electronics > Components > Hard Drives**.

<classificationProperties/>

You can add the classification properties of an object.

<label>

You can add static, formatted text to your page.

<customPanel>

You can add your own custom content.

<image>

You can add an image to the page.

XML elements

all

Lists all properties of the defined object.

ATTRIBUTES

type

Indicates whether to list all the object properties or only form properties. The valid values for this attribute are **property** and **form**.

SUPPORTED STYLE SHEETS

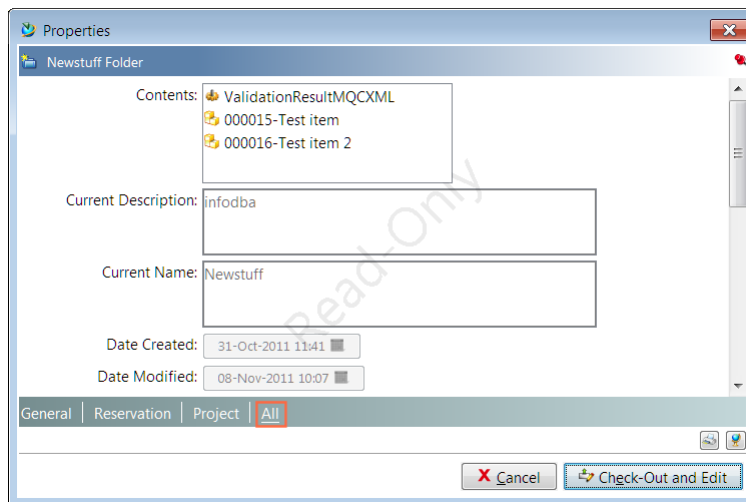
This tag can be used on the following types of style sheets (but not the New Business Object wizard):

Property
Summary
Form

EXAMPLE

Following is sample code from the **Folder.xml** XML rendering style sheet showing the **all** element:

```
<page title="All" titleKey="tc_xrt_All">
  <all type="property"/>
</page>
```



The all element in the rich client

Note:

The **<conditions>**, **<GoverningProperty>**, and **<Rules>** tags are not supported within the **<all>** tag.

break

Inserts a break in the pane.

ATTRIBUTES

None.

SUPPORTED STYLE SHEETS

This tag can be used on the following types of style sheets:

**Property
Summary
Form
Create**

EXAMPLE

```
<page titleKey="tc_xrt_General" title="General">
  <property name="project_id" icon="images/group.gif"/>
  <property name="serial_number" renderingHint="textfield" renderingStyle="headed"/>
  <property name="item_comment" renderingHint="textarea" column="30" row="5"/>
  <property name="user_data_1" renderingHint="lovcombobox" renderingStyle="titled"
border="true" />
  <separator />
  <property name="user_data_1" />
  <property name="user_data_2" />
  <break />
</page>
```

classificationProperties

Specifies that the classification properties of the current object should be displayed. Properties and their values are rendered as name/value pairs in static text.

ATTRIBUTES

None.

SUPPORTED STYLE SHEETS

This tag can be used only on **Summary** style sheets.

EXAMPLE

Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **classificationProperties** element:

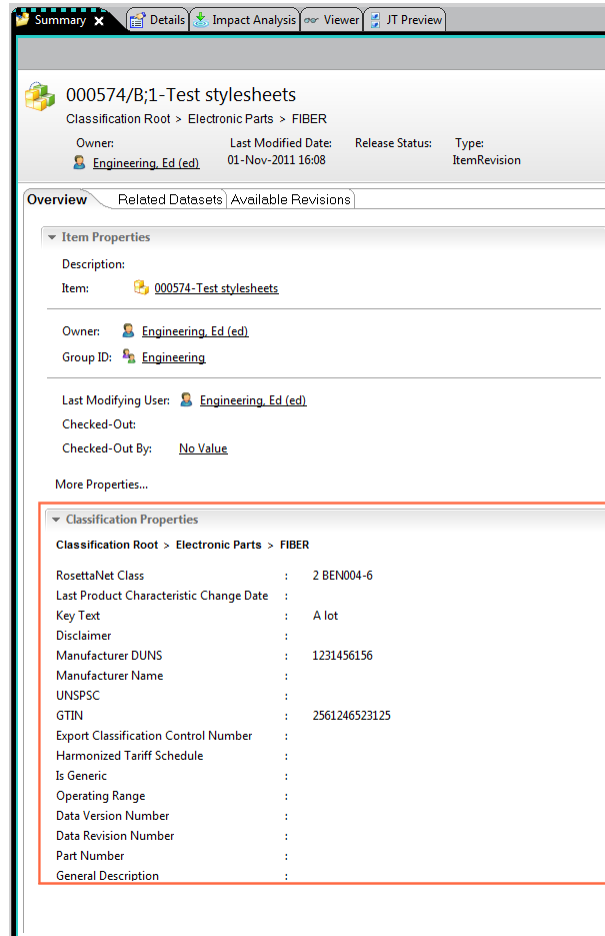
```
<page titleKey="tc_xrt_Overview">
  <column>
    <section titleKey="tc_xrt_AvailableRevisions">
      <objectSet source="revision_list.ItemRevision"
defaulttdisplay="listDisplay"
sortdirection="descending" sortby="item_revision_id">
        <tableDisplay>
          <property name="object_string"/>
          <property name="item_revision_id"/>
          <property name="release_status_list"/>
          <property name="last_mod_date"/>
          <property name="last_mod_user"/>
          <property name="checked_out_user"/>
        </tableDisplay>
        <thumbnailDisplay/>
        <treeDisplay>
          <property name="object_string"/>
          <property name="item_revision_id"/>
          <property name="release_status_list"/>
          <property name="last_mod_date"/>
          <property name="last_mod_user"/>
          <property name="checked_out_user"/>
        </treeDisplay>
        <listDisplay/>
      </objectSet>
    </section>
    <section titleKey="tc_xrt_ItemProperties">
      <property name="object_desc"/>
      <separator/>
      <property name="owning_user" renderingHint="objectlink"
modifiable="false"/>
      <property name="owning_group" renderingHint="objectlink"
modifiable="false"/>
      <property name="last_mod_user"/>
      <separator/>
      <property name="checked_out"/>
    </section>
  </column>
</page>
```

```

        <property name="checked_out_user"/>
        <separator/>
        <command commandId="com.teamcenter.rac.properties"
titleKey="tc_xrt_moreProperties"/>
    </section>
    <section titleKey="tc_xrt_ClassificationProperties">
<classificationProperties/>
</section>
</column>

```

Following is how classification properties are rendered.



classificationProperties

classificationTrace

Specifies that the classification traces of the item should be displayed, for example, **Home Care > Cleaners > Detergents**.

ATTRIBUTES

None.

SUPPORTED STYLE SHEETS

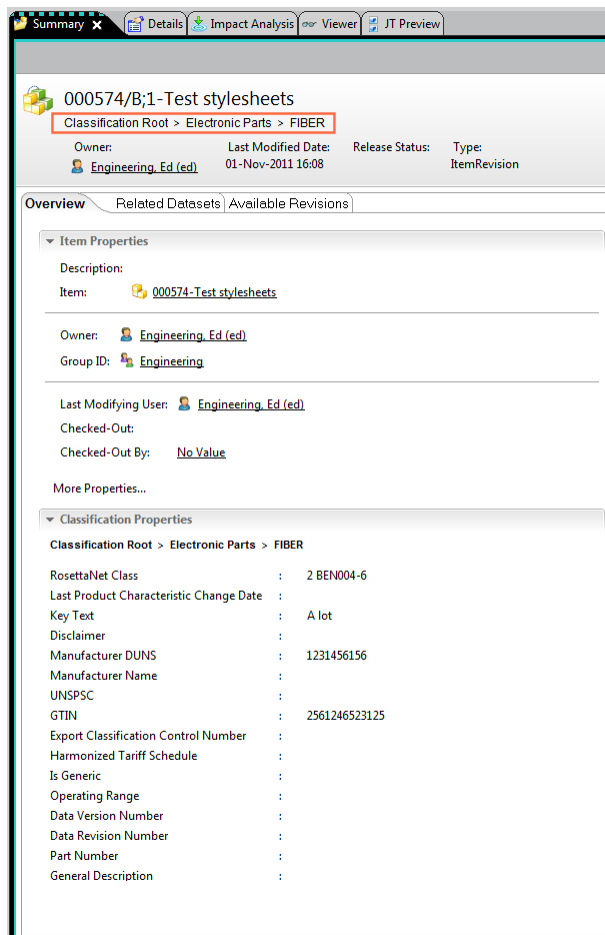
This tag can be used only on **Summary** style sheets.

EXAMPLE

Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **classificationTrace** element:

```
<header>
  <image source="thumbnail"/>
  <classificationTrace/>
  <property name="owning_user"/>
  <property name="last_mod_date"/>
  <property name="release_status_list"/>
  <property name="object_type"/>
</header>
```

Following is how the classification trace is rendered



classificationTrace

column

Defines the layout of the column defined on a page.

ATTRIBUTES

None.

SUPPORTED STYLE SHEETS

This tag can be used on the following types of style sheets (but not the New Business Object wizard):

**Property
Summary
Form**

EXAMPLE

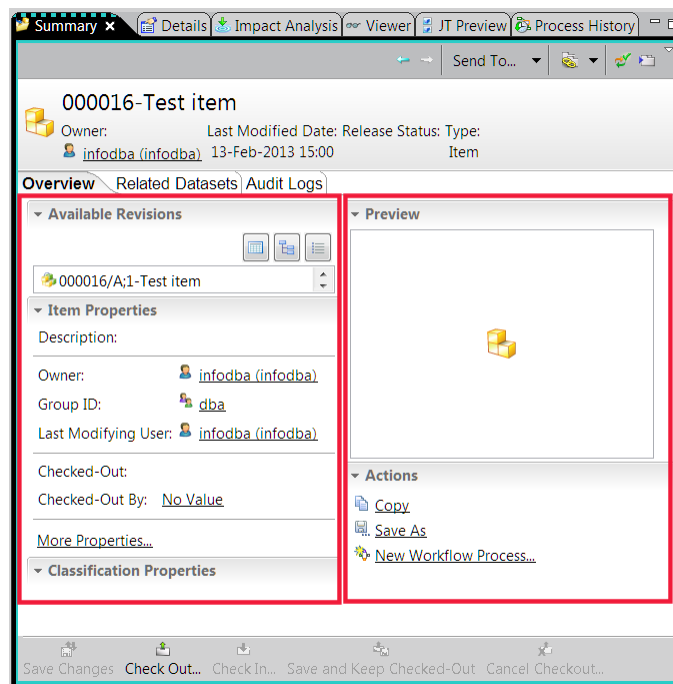
Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **column** element:

```
<page titleKey="tc_xrt_Overview">
  <column>
    <section titleKey="tc_xrt_AvailableRevisions">
      <objectSet source="revision_list.ItemRevision"
defaultdisplay="listDisplay"
sortdirection="descending" sortBy="item_revision_id">
        <tableDisplay>
          <property name="object_string"/>
          <property name="item_revision_id"/>
          <property name="release_status_list"/>
          <property name="last_mod_date"/>
          <property name="last_mod_user"/>
          <property name="checked_out_user"/>
        </tableDisplay>
        <thumbnailDisplay/>
        <treeDisplay>
          <property name="object_string"/>
          <property name="item_revision_id"/>
          <property name="release_status_list"/>
          <property name="last_mod_date"/>
          <property name="last_mod_user"/>
          <property name="checked_out_user"/>
        </treeDisplay>
        <listDisplay/>
      </objectSet>
    </section>
    <section titleKey="tc_xrt_ItemProperties">
      <property name="object_desc"/>
      <separator/>
      <property name="owning_user" renderingHint="objectlink"
modifiable="false"/>
      <property name="owning_group" renderingHint="objectlink">
```

```

modifiable="false"/>
    <property name="last_mod_user"/>
    <separator/>
    <property name="checked_out"/>
    <property name="checked_out_user"/>
    <separator/>
    <command commandId="com.teamcenter.rac.properties"
titleKey="tc_xrt_moreProperties"/>
    </section>
    <section titleKey="tc_xrt_ClassificationProperties">
        <classificationProperties/>
    </section>
</column>
<column>
    <section titleKey="tc_xrt_Preview">
        <image source="preview"/>
    </section>
    <section titleKey="tc_xrt_actions" commandLayout="vertical">
        <command actionKey="copyAction" commandId="com.teamcenter.rac.copy" />
        <command actionKey="saveAsAction"
commandId="org.eclipse.ui.file.saveAs" />
        <command actionKey="newProcessAction"
commandId="com.teamcenter.rac.newProcess"
titleKey="tc_xrt_newProc" />
    </section>
</column>
</page>

```



column element in the rich client

command

Specifies a command representation to be displayed.

ATTRIBUTES

commandId

Specifies the command to be executed. The attribute value must be a key into a property file and must be a valid command ID. This is a string attribute that is required.

icon

Specifies the icon to be displayed before the command label. The attribute value must be a key into a property file. This is an optional attribute.

renderingHint

Specifies whether the command is rendered as a hyperlink or as a button. Valid values are **hyperlink** and **commandbutton**. This attribute is optional. If the attribute is not specified, the command is rendered as a hyperlink.

title

Specifies the default string of the title for this user interface element. This attribute is used when the string in the **titleKey** attribute is not found by the TextServer. This is an optional attribute.

titleKey

Specifies the key used by the TextServer to search for the title . If it is not defined, the string defined by the **title** attribute is used. This is an optional attribute.

tooltip

Specifies the tooltip for the command. The attribute value must be a key into a property file. This attribute is optional but is required if the **icon** attribute is specified.

SUPPORTED STYLE SHEETS

This tag can be used only on the **Summary** style sheets.

Note:

The **command** tag is ignored in the **header** section. Commands cannot be added to the toolbar.

EXAMPLE

Following is sample code using the **command** element on an object set:

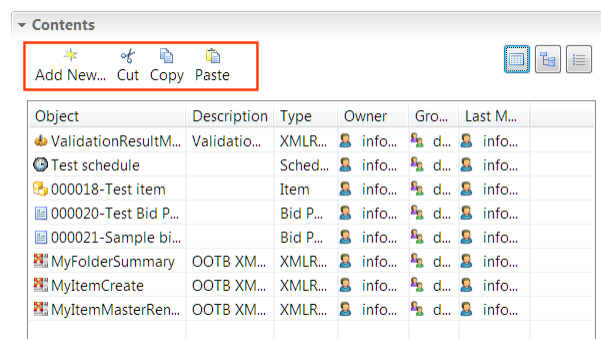
```
<objectSet source="contents.WorkspaceObject" defaultdisplay="thumbnailDisplay"
  sortBy="object_name" sortdirection="ascending">
  <tableDisplay>
    <property name="object_string"/>
  </tableDisplay>
</objectSet>
```

```

...
</tableDisplay>
<thumbnailDisplay/>
<treeDisplay>
  <property name="object_string"/>
  ...
</treeDisplay>
<listDisplay/>
<command commandId="com.teamcenter.rac.common.AddNew" renderingHint="commandbutton"/>
<command commandId="org.eclipse.ui.edit.cut" renderingHint="commandbutton">
  <parameter name="localSelection" value="true"/>
</command>
<command commandId="com.teamcenter.rac.copy" renderingHint="commandbutton"/>
<command commandId="com.teamcenter.rac.viewer.pastewithContext" renderingHint="commandbutton"/>
</objectSet>

```

In this example, the **command** element adds the **Add New**, **Cut**, **Copy**, and **Paste** buttons in the user interface.



command element

customPanel

Embeds a custom panel in a dialog box.

ATTRIBUTES

java

Specifies the fully qualified Java implementation class name responsible for building the custom user interface (for example, **com.teamcenter.rac.MyCustomPanel**). This is supported in the rich client only.

js

Specifies a JavaScript function that generates the HTML (for example, **<customPanel js="MyCustomCalendar()" />**).

SUPPORTED STYLE SHEETS

This tag can be used on the **Create** style sheets.

EXAMPLE

Following is a custom style sheet that uses the **customPanel** element. (This example works in the rich client only.)

```
<rendering>
  <page title="General" titleKey="tc_xrt_General">
    <section title="Item Information" titleKey="tc_xrt_ItemInformation">
      <property name="item_id" />
      <property name="revision:item_revision_id" />
      <property name="object_name" />
      <property name="object_desc" />
      <separator/>
      <property name="uom_tag" />
      <separator/>
      <customPanel java="com.teamcenter.rac.ui.commands.newbo.mypanel.MyPanel" />

      <section title="Additional Item Information"
titleKey="tc_xrt_AdditionalItemInformation" initialState="collapsed">
        <property name="IMAN_master_form:project_id" />
        <property name="IMAN_master_form:previous_item_id" />
        <property name="IMAN_master_form:serial_number" />
        <property name="IMAN_master_form:item_comment" />
        <property name="IMAN_master_form:user_data_1" />
        <property name="IMAN_master_form:user_data_2" />
        <property name="IMAN_master_form:user_data_3" />
      </section>

      <section title="Item Revision Information"
titleKey="tc_xrt_ItemRevisionInformation" initialState="collapsed">
        <property name="revision:IMAN_master_form_rev:project_id" />
        <property name="revision:IMAN_master_form_rev:previous_version_id" />
        <property name="revision:IMAN_master_form_rev:serial_number" />
        <property name="revision:IMAN_master_form_rev:item_comment" />
      </section>
    </section>
  </page>
</rendering>
```

```

        <property name="revision:IMAN_master_form_rev:user_data_1" />
        <property name="revision:IMAN_master_form_rev:user_data_2" />
        <property name="revision:IMAN_master_form_rev:user_data_3" />
    </section>
</page>

</rendering>

```

You must create your own custom panel to pass to the **customPanel** tag. Following is the **MyPanel.java** file that defines the custom panel:

```

package com.teamcenter.rac.ui.commands.newbo.mypanel;

import com.teamcenter.rac.ui.commands.create.bo.NewBOWizard;
import com.teamcenter.rac.util.AbstractCustomPanel;
import com.teamcenter.rac.util.IPageComplete;
import org.eclipse.swt.layout.GridData;
import org.eclipse.swt.layout.GridLayout;
import org.eclipse.swt.widgets.Composite;
import org.eclipse.swt.widgets.Label;
import org.eclipse.swt.widgets.Text;
import org.eclipse.ui.forms.widgets.FormToolkit;
public class MyPanel extends AbstractCustomPanel implements IPageComplete
{
    private Composite composite;
    private Text text;

    public MyPanel()
    {
    }

    public MyPanel( Composite parent )
    {
        super( parent );
    }

    @Override
    public void createPanel()
    {
        FormToolkit toolkit = new FormToolkit( parent.getDisplay() );
        composite = toolkit.createComposite( parent );

        GridLayout gl = new GridLayout( 2, false );
        composite.setLayout( gl );

        GridData gd = new GridData( GridData.FILL_HORIZONTAL );
        gd.grabExcessHorizontalSpace = true;

        composite.setLayoutData( gd );

        GridData labelGD = new GridData( GridData.HORIZONTAL_ALIGN_END );
        Label label = toolkit.createLabel( composite, "Object_Name: " );
        label.setLayoutData( labelGD );

        GridData typeTextGd = new GridData( GridData.FILL_HORIZONTAL );
        text = toolkit.createText( composite, "" );
        text.setText( "This is my own panel" );
        text.setLayoutData( typeTextGd );
    }
}

```

```

public boolean isPageComplete()
{
    String txt = text.getText();
    return txt.length() == 0 ? false : true;
}

@Override
public Composite getComposite()
{
    return composite;
}

@Override
public void updatePanel()
{
    if( input != null )
    {
        NewBOWizard wizard = (NewBOWizard) input;
        String msg = "";
        if( wizard.model.getTargetArray() != null )
        {
            try
            {
                msg = wizard.model.getTargetArray()[0].getProperty(
                    "object_name" ).toString();
            }
            catch( Exception e )
            {
                e.printStackTrace();
            }
        }
        else
        {
            msg = "Nothing is selected";
        }
        text.setText( msg );
    }
}

@Override
public Object getUserInput()
{
    return null;
}
}

```

Following is the resulting custom panel added to the New Business Object wizard, which is run when you choose **File**→**New**→**Other** in the rich client.

New Business Object

Object Create Information

Define business object create information
Leave ID and Revision fields blank for auto-assign.

Item Item 1

Properties

ID: *

Revision: *

Name: *

Description:

Unit of Measure:

Object_Name: Home

< Back Next > Finish Cancel

customPanel example

header

Specifies that a header area must be displayed in the page.

ATTRIBUTES

None.

SUPPORTED STYLE SHEETS

This tag can be used on the **Summary** style sheets.

EXAMPLE

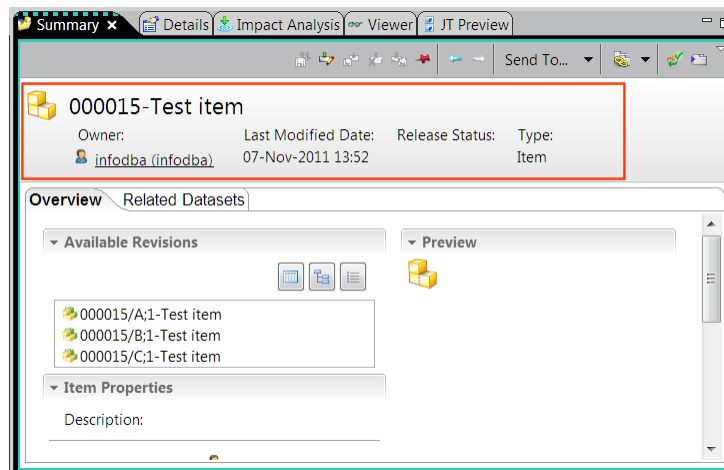
Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **header** element:

```
<header>
  <image source="thumbnail"/>
  <classificationTrace/>
  <property name="owning_user"/>
  <property name="last_mod_date"/>
  <property name="release_status_list"/>
  <property name="object_type"/>
</header>
```

Note:

The valid XML elements for this element are:

- <classificationTrace>
- <image>
- <property>



header element in the rich client

ADDITIONAL INFORMATION

The **<header>** element is optional. If it is not included, or if it does not contain any elements, the header is automatically be populated with the **object_string** as a label. This label is not selectable.

image

Specifies that an image is to be rendered. Image dimensions are always kept proportional when being scaled or resized.

ATTRIBUTES

maxheight

Specifies the maximum height in pixels to which the image should be scaled. This is a string attribute that is optional.

maxwidth

Specifies the maximum width in pixels to which the image should be scaled. This is a string attribute that is optional.

source

Specifies the source of the image to display. The attribute value can be a **thumbnail**, **preview**, or **type** keyword. This is a string attribute that is optional.

tooltip

Specifies the tooltip associated with the image. The attribute value must be a key into a property file. This is a string attribute that is optional.

Note:

For backward compatibility, if no attributes are specified and the current object type is an **Item**, **ItemRevision**, or **Dataset** business object, an attempt is made to find and render any preview image that is associated with the object.

SUPPORTED STYLE SHEETS

This tag can be used on the following types of style sheets:

**Property
Summary
Form**

EXAMPLE

Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **image** element:

```
<header>
  <image source="thumbnail"/>
  <classificationTrace/>
  <property name="owning_user"/>
  <property name="last_mod_date"/>
  <property name="release_status_list"/>
```

```
<property name="object_type"/>  
</header>
```

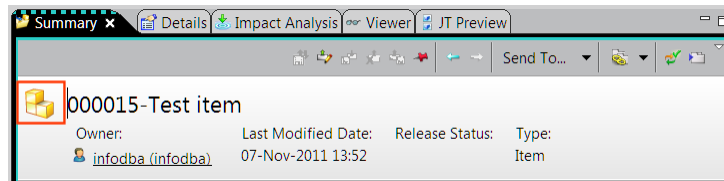


image element in the rich client

label

Specifies a label to be rendered.

ATTRIBUTES

style

Controls font style for the label text, including font size, weight, name, and style (such as italic). The format follows the CSS guideline, for example:

```
style="font-size:14pt;font-style:plain;
font-family:Tahoma;font-weight:bold"
```

textKey

Support is provided for localized values by using the TextServer. For example, the tagging `<label text="Hello World" />` displays text on the property page, and `<label textKey="k_version_name" />` displays the localized text in the provided property.

SUPPORTED STYLE SHEETS

This tag can be used on all style sheet types.

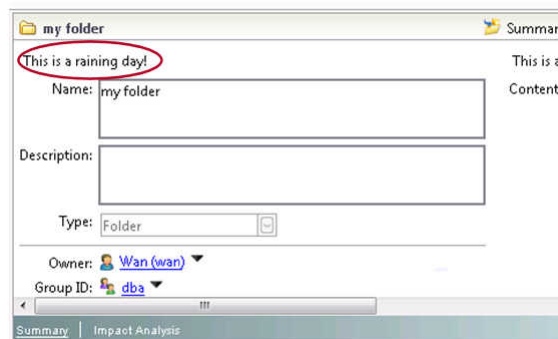
EXAMPLES

- Text

Following is sample code for the **label text** element:

```
<label text="This is a raining day!" />
```

The following figure shows the text on the page.



Label tag example

- style** attribute

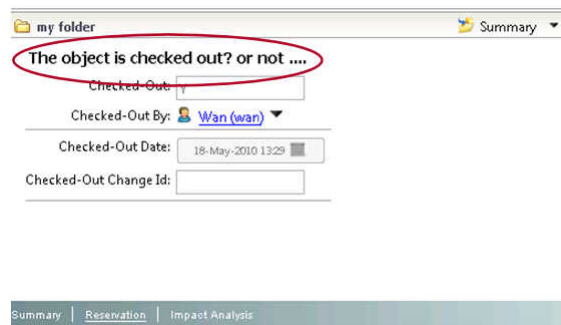
The property style sheet page tagging includes the **style** attribute within the **label text** and **property** tags to control font style. Support is provided for font size, weight, name, and style (such as italic).

The format follows the CSS guideline, for example, **style="font-size:14pt;font-style:plain;font-family:Tahoma;font-weight:bold"**.

For example:

```
<page title="Reservation" titleKey="tc_xrt_Reservation"
visibleWhen="object_desc==Testing*">
<label text="The object is checked out? or not ...."
style="font-size:14pt; font-style:plain;font-family:Tahoma; font-weight:bold" />
...
</page>
```

This results in the font style shown in the following figure.



style tagging example

- URL rendering

URL addresses included in the **label** and **property** tag are automatically rendered.

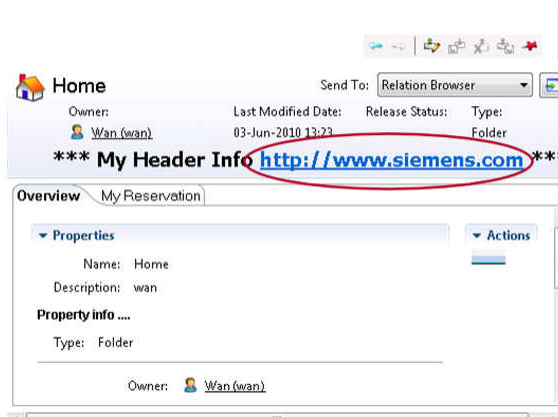
For example:

```
<label text="Press www.abcnews.com to view the latest headlines!"
style=" font-size:14pt;font-style:plain;font-family:Tahoma;font-weight:normal" />
```

Consider the following code:

```
<label text="***My Header Info http://www.siemens.com ***" />
```

Because a URL is included in the **label** tag, it is automatically rendered on the page, as shown in the following figure.



URL rendering example

listDisplay

Displays a set of objects in a list format.

ATTRIBUTES

None.

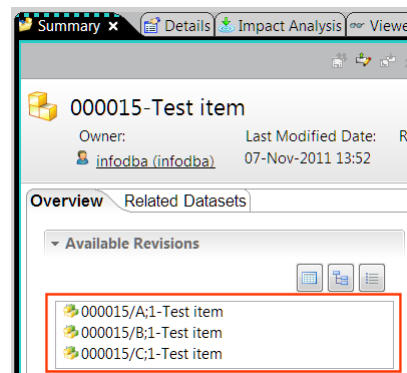
SUPPORTED STYLE SHEETS

This tag can be used on **Summary** style sheets.

EXAMPLE

Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **listDisplay** element:

```
<section titleKey="tc_xrt_AvailableRevisions">
  <objectSet source="revision_list.ItemRevision" defaultdisplay="listDisplay"
    sortdirection="descending" sortby="item_revision_id">
    <tableDisplay>
      <property name="object_string"/>
      <property name="item_revision_id"/>
      <property name="release_status_list"/>
      <property name="last_mod_date"/>
      <property name="last_mod_user"/>
      <property name="checked_out_user"/>
    </tableDisplay>
    <thumbnailDisplay/>
    <treeDisplay>
      <property name="object_string"/>
      <property name="item_revision_id"/>
      <property name="release_status_list"/>
      <property name="last_mod_date"/>
      <property name="last_mod_user"/>
      <property name="checked_out_user"/>
    </treeDisplay>
    <listDisplay/>
  </objectSet>
</section>
```

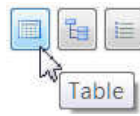


listDisplay element in the rich client

objectSet

Provides a set of display options for the selected object. This element is a replacement for the **attachments** element.

Click the appropriate button to view the object's characteristics in a table, list, or tree.



objectset buttons

The tree option is rendered as a thumbnail display.

ATTRIBUTES

defaultdisplay

Specifies the default format to use when displaying the set of objects. Valid values are **treeDisplay**, **tableDisplay**, **listDisplay**, or **thumbnailDisplay**. The default value is **listDisplay**. This is a string attribute that is optional.

sortby

Specifies the object property to sort the set of objects by prior to rendering. The default value is **object_string**. This is a string attribute that is optional.

sortdirection

Specifies the direction in which the set of objects should be sorted. Valid values are **ascending** or **descending**. The default value is **ascending**. This is a string attribute that is optional.

source

Specifies the comma-delimited set of run-time properties or relations that return the desired set of objects. The format for the attribute value is *property.type*, where *property* is the name of a relation, run-time, or reference property, and *type* represents the type of objects to be included. Multiple *property.type* values can be specified as a comma-separated list, such as **contents.Item** or **contents.ItemRevision**. When using a relation property, you must explicitly specify the relation. Subtypes are not traversed. This is a string attribute that is required.

SUPPORTED STYLE SHEETS

This tag can be used on the **Summary** style sheets.

EXAMPLE

Following is sample code using the **objectSet** tag:


```

<objectSet source="contents.WorkspaceObject" defaultdisplay="thumbnailDisplay"
sortby="object_name" sortdirection="ascending">
  <tableDisplay>
    <property name="object_string"/>
    ...
  </tableDisplay>
  <thumbnailDisplay/>
  <treeDisplay>
    <property name="object_string"/>
    ...
  </treeDisplay>
  <listDisplay/>
  <command commandId="com.teamcenter.rac.common.AddNew"
renderingHint="commandbutton"/>
  <command commandId="org.eclipse.ui.edit.cut" renderingHint="commandbutton">
    <parameter name="localSelection" value="true"/>
  </command>
  <command acommandId="com.teamcenter.rac.copy" renderingHint="commandbutton"/>
  <command commandId="com.teamcenter.rac.viewer.pastewithContext"
renderingHint="commandbutton"/>
</objectSet>

```

Note:

In an object set, you can use the **command** tag to add buttons for existing menu commands. For example, you can place cut, copy, and paste buttons on the object set.

Property tags used in an **objectSet** are able to display relation properties of the **source** objects. Use the *relationname.propertyname* format to access them. For example, to display the name of the relation type, use the following:

```
<property name = "TC_Attaches.type_string" />
```

You can use the **localSelection** parameter to specify the selected object. If the **localSelection** parameter is set to **true** for any command action, the action is performed on the object selected in the object set. For example, in case of the cut action, if the **localSelection** parameter is **true**, the cut action operates on the object selected in the object set list. If the **localSelection** parameter is **false** (or not set), the cut action operates on the object for which the summary is being shown and not on an object selected in the object set. Although the **localSelection** parameter works with all commands defined in the object set views, it only makes sense to use it with certain commands. For example, any commands such as **Revise**, **Cut**, and **Delete** that should operate on objects in the object set view should have the **localSelection** parameter set to **true**, and any commands such as **Add New** and **Paste** that do *not* operate on the object selected should not use this parameter.

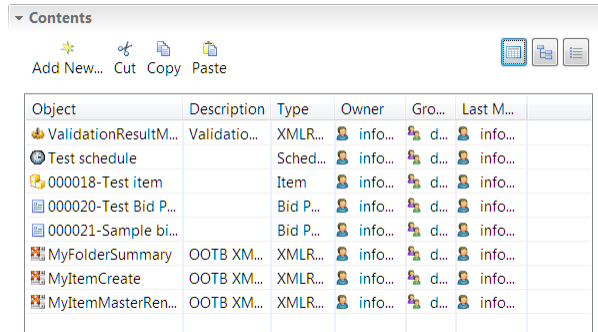
In Systems Engineering, to connect objects with any Trace Link relation (**FND_TraceLink** and its subtypes), you can use the runtime properties of the Workspace object:

- **fnd0defining_objects** (for defining objects)
- **fnd0complying_objects** (for complying objects)

These properties can be displayed using a stylesheet with the same *property.type* format. However, when you use the **Add New** command to add a new object, these properties are not set or modified.

See the Systems Engineering guide for information about configuring trace link features and configuring defining and complying object calculation.

Following is how the sample code is rendered in the user interface. This object set shows the contents of a folder.



Object	Description	Type	Owner	Gro...	Last M...
ValidationResultM...	Validatio...	XMLR...	info...	d...	info...
Test schedule		Sched...	info...	d...	info...
000018-Test item		Item	info...	d...	info...
000020-Test Bid P...		Bid P...	info...	d...	info...
000021-Sample bi...		Bid P...	info...	d...	info...
MyFolderSummary	OOTB XM...	XMLR...	info...	d...	info...
MyItemCreate	OOTB XM...	XMLR...	info...	d...	info...
MyItemMasterRen...	OOTB XM...	XMLR...	info...	d...	info...

Object set (with table button selected)

page

Presents a tab panel in a dialog box or view. If the **page** element is not defined in the XML file, a default page is created.

ATTRIBUTES

format

Specifies the format to be used for this page. This attribute can have one of these values: **OneColumn** or **TwoColumn**. The default value is **OneColumn**. This attribute is optional.

The **firstcolumn** tag defines the layout of the first column, and the **secondcolumn** tag defines the layout of the second column.

Caution:

As of Teamcenter 10.1, the **format** attribute is deprecated on the **page** tag. It is no longer needed because the **firstcolumn** and **secondcolumn** tags are deprecated.

text

Specifies the title to be displayed for the page. The attribute value must be a key into a property file. If the key is not found, the attribute value itself is displayed as static text.

Caution:

As of Teamcenter 10.1, the **text** attribute is deprecated and is replaced by the **title** and **titlekey** attributes.

title

Specifies the default string of the title for this tab. This attribute is used when the string in the **titleKey** attribute is not found in the locale file. This is an optional attribute.

titleKey

Specifies the key used to search for the title in the locale file. If it is not defined, the string defined by the **title** attribute is used. This is an optional attribute.

visibleWhen

Defines the conditional display of a tab based on one of three types of expressions comparing a property or preference to a value. The value can be **null** or a string, including a string containing the * wildcard character. Multiple values can be checked with an array property or preference. When checking an array value, use a comma as a delimiter for the values. Any properties checked must be loaded on the page using the **<property>** element. The three types of expressions check the following:

- The value of a property on the selected object

- The value of a Teamcenter preference
- The value of a property on an object related to the selected object
- To check the value of a property on the selected object, use the real (database) name of the property in the expression.
If you want to show a **"Test"** page if the **object_desc** property begins with the word **Testing**, use the following:

```
<page title="Test" titleKey="tc_xrt_testpage"
      visibleWhen="object_desc==Testing*">
```

The following examples show **"My Page"** based on the value of a property called **myProp**.
Display the page if **myProp** contains "test".

```
<page titleKey="My Page" visibleWhen="myProp==*test*">
```

Display the page if **myProp** does *not* contain "test".

```
<page titleKey="My Page" visibleWhen="myProp!=*test*">
```

Display the page if **myProp** contains no value (null).

```
<page titleKey="My Page" visibleWhen="myProp==null">
```

- To check the value of a Teamcenter preference, use **{pref:preference-name}** to differentiate it from a property-based expression. Following are some examples:
Display the page when the **TC_Enable_MyPref** preference has no value.

```
<page titleKey="My Page"
      visibleWhen="{pref:TC_Enable_MyPref}==null">
```

Display the page when the **Item_ColumnPreferences** preference contains **object_string** and **object_type**, as the first two values.

```
<page titleKey="My Page"
      visibleWhen="{pref:Item_ColumnPreferences}==object_string,object_type,*">
```

- To check a property on an object *related* to the selected object, you must include the reference or relation property name and the property name from the related object separated by a period.
Display the page when the owner of the selected object is **user1**.

```
<page title="Custom User Page"
      visibleWhen="owning_user.user_id==user1">
```

Display the page when the status of the selected object is **TCM Released**.

```
<page title="Custom Status Page"
      visibleWhen="release_status_list.name==TCM Released">
```

Display the page when two specific statuses are present on the object — both **TCM Released** and **Approved**.

```
<page title="Special Status Page"
      visibleWhen="release_status_list.name==TCM Released,Approved">
```

Display the page when there is a PDF dataset attached with a specification relation.

```
<page title="The PDF Page"
      visibleWhen="IMAN_specification.object_type==*PDF*">
```

Note:

If there is only one page, and the **visibleWhen** condition hides this page, the rich client ignores this condition and makes the page visible.

If you specify a reference property but you do not specify a property on the related object, the default value will be the secondary object's localized value - typically `object_string`.

SUPPORTED STYLE SHEETS

This tag can be used on the following types of style sheets, although the way pages appear may vary from one type to another.

Property

Summary

Form

Create (visibleWhen not supported)

SaveAs (visibleWhen not supported)

EXAMPLES

- **page** element

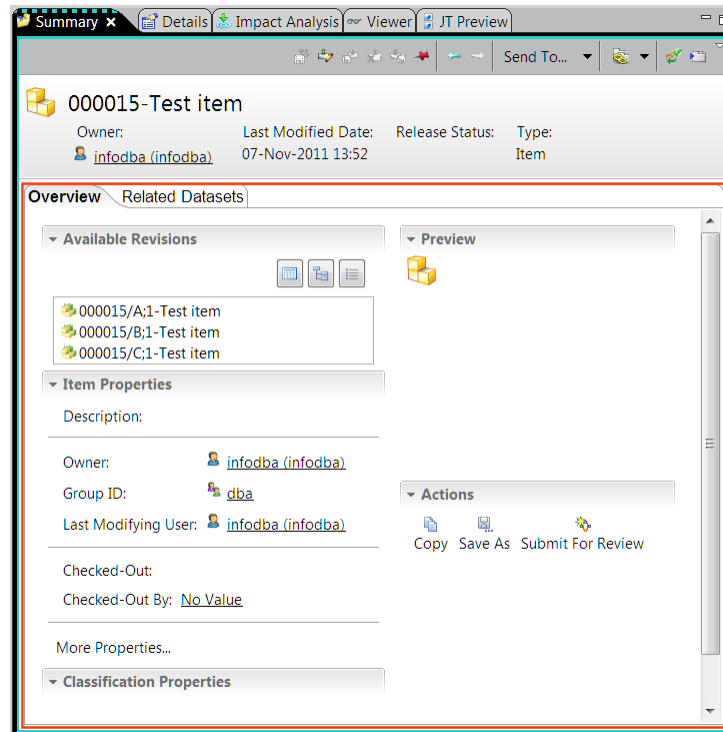
Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **page** element:

```
<page titleKey="tc_xrt_Overview">
  <column>
    <section titleKey="tc_xrt_AvailableRevisions">
      <objectSet source="revision_list.ItemRevision"
defaultdisplay="listDisplay"
sortdirection="descending" sortBy="item_revision_id">
        <tableDisplay>
          <property name="object_string"/>
          <property name="item_revision_id"/>
          <property name="release_status_list"/>
          <property name="last_mod_date"/>
          <property name="last_mod_user"/>
          <property name="checked_out_user"/>
        </tableDisplay>
      </objectSet>
    </section>
  </column>
</page>
```

```

        </tableDisplay>
        <thumbnailDisplay/>
        <treeDisplay>
            <property name="object_string"/>
            <property name="item_revision_id"/>
            <property name="release_status_list"/>
            <property name="last_mod_date"/>
            <property name="last_mod_user"/>
            <property name="checked_out_user"/>
        </treeDisplay>
        <listDisplay/>
    </objectSet>
</section>
<section titleKey="tc_xrt_ItemProperties">
    <property name="object_desc"/>
    <separator/>
    <property name="owning_user" renderingHint="objectlink"
modifiable="false"/>
    <property name="owning_group" renderingHint="objectlink"
modifiable="false"/>
    <property name="last_mod_user"/>
    <separator/>
    <property name="checked_out"/>
    <property name="checked_out_user"/>
    <separator/>
    <command commandId="com.teamcenter.rac.properties"
titleKey="tc_xrt_moreProperties"/>
</section>
<section titleKey="tc_xrt_ClassificationProperties">
    <classificationProperties/>
</section>
</column>
<column>
    <section titleKey="tc_xrt_Preview">
        <image source="preview"/>
    </section>
    <section titleKey="tc_xrt_actions" commandLayout="vertical">
        <command actionKey="copyAction" commandId="com.teamcenter.rac.copy" />
        <command actionKey="saveAsAction"
commandId="org.eclipse.ui.file.saveAs" />
        <command actionKey="newProcessAction"
commandId="com.teamcenter.rac.newProcess" titleKey="tc_xrt_newProc" />
    </section>
</column>
</page>

```



page element in the rich client

- **visibleWhen** attribute

To specify a single conditional evaluation for the component property, include the **visibleWhen** parameter on the property style sheet page, for example, **visibleWhen="object_desc!=abc"**. Consider the following code:

```
<page title="Reservation" titleKey="tc_xrt_Reservation"
visibleWhen="object_desc==Testing*">
...
</Page>
```

If the word **Testing** is used in the **object_desc** property, the **Reservation** link appears. In the following example, the **object_desc** property is blank, and therefore the **Reservation** link does not appear.

my folder Summary

This is a raining day! This is a Content:

Name: my folder

Description:

Type: Folder

Owner: Wan (wan)

Group ID: dba

Summary Impact Analysis

Condition is not met for the visibleWhen parameter

Now change the description value to **Testing my code**; the **Reservation** link appears after you save your changes, as shown in the following figure.

my folder Summary

This is a Contents: This is a Contents:

Name: my folder

Description: Testing my code.

Type: Folder

Owner: WangPeiyi (wangp)

Group ID: dba

Summary Reservation Impact Analysis

Condition is valid for the visibleWhen parameter

parameter

Passes in the name/value parameters to the parent command. This is a child element of the **command** element. An example of a parameter is **localSelection**.

ATTRIBUTES

name

Specifies the parameter name, for example, **searchName**. This is a required attribute.

value

Specifies the parameter value, for example, **CustomSearch**. This is a required attribute.

SUPPORTED STYLE SHEETS

This tag can be used on **Summary** style sheets.

EXAMPLE

Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **parameter** element:

```
<command actionKey="pasteAction" commandId="com.teamcenter.rac.viewer.pastewithContext"
renderingHint="commandbutton"/>
<command actionKey="cutAction" commandId="org.eclipse.ui.edit.cut"
renderingHint="commandbutton">
    <parameter name="localSelection" value="true"/>
</command>
```

CUSTOM COMMANDS WITH PARAMETERS

When you define a custom command for the rich client that uses parameters, you must add the parameters with the **commandParameter** tag in the **plugin.xml** file of your custom plug-in, for example:

```
<extension
    id="com.myco.command.parameter.test.commands.category"
    name="Test Category"
    point="org.eclipse.ui.commands">
    <command
        name="Test command"
        categoryId="com.teamcenter.ddp.commands.category"
        id="com.myco.command.parameter.test.commands.testCommand">
        <commandParameter id="localStage1" name="localStage1" optional="true" />
        <commandParameter id="localStage2" name="localStage2" optional="true" />
        <commandParameter id="localStage3" name="localStage3" optional="true" />
        <commandParameter id="localStage4" name="localStage4" optional="true" />
        <commandParameter id="source" name="source" optional="true" />
    </command>
</extension>
<extension
```

```
        point="org.eclipse.ui.handlers">
    <handler
        commandId="com.myco.command.parameter.test.commands.testCommand"
        class="com.myco.command.parameter.test.handlerTest">
    </handler>
</extension>
```

When you add the custom command to the style sheet, the parameters defined in the style sheet must match the **commandParameter** values defined in your plug-in, for example:

```
<command commandId="com.myco.command.parameter.test.commands.testCommand"
    renderingHint="commandbutton">
    <parameter name="localStage1" value="1"/>
    <parameter name="localStage2" value="2"/>
    <parameter name="localStage3" value="3"/>
    <parameter name="localStage4" value="4"/>
</command>
```

If you add the parameters in the style sheet, but not in the **plugin.xml** file, then when you execute the command in the rich client interface you will receive an error that no parameters are found.

property

Specifies the real (database) name, not the **display name**, of the object property you want to display. You must include at least one property in the XML definition, otherwise, the system displays an empty panel.

Note:

You cannot add the same property multiple times in the same style sheet.

ATTRIBUTES

border

Determines whether the border is displayed. Valid values are **true** and **false**. This works only with the titled style.

column

Applies only to the **textfield** and **textarea** rendering hints. It sets the number of columns.

modifiable

Specifies if the **owning_user** or **owning_group** property can be modified (**true** or **false**). For all other properties, use a property rule instead.

name

Specifies the database name of a property on the object. This is a required attribute.

Note:

When using this attribute on a **create** style sheet, which is used in the New Business Object wizard, there is additional functionality. You can specify a property from another object that is related to the original object by the **revision**, **IMAN_master_form**, or **IMAN_master_form_rev** relations. To do this, specify the relation trail followed by the name of the property on the destination related object, separated by a colon.

For example, if a **create** style sheet is registered for an item,

- to display the revision ID, you would use

```
name=revision:item_revision_id
```

- to display the **project_id** property from the item's master form.

```
name=IMAN_master_form:project_id
```

- to display the **serial_number** property from the item revision master form, you need to traverse from the item to the revision, and then to the revision's master form.

```
name=revision:IMAN_master_form_rev:serial_number
```

renderingHint

Specifies the component used to **render** this property. This is an optional attribute. If not defined, the default renderer is used based on the property type.

renderingStyle

Defines the rendering style used in the rendering component. There are three styles: headed, headless, and titled.

- **Headed**

This is the default rendering style. The property name is displayed on the left followed by the property value renderer.

- **Headless**

This style renders only the property value without displaying the property name in front of it.

- **Titled**

The property name is displayed on the top of the property value renderer.

row

Applies only to **textarea** elements. It sets the number of the rows for the element.

style

Controls font style for the label text, including font size, weight, name, and style (such as italic). The format follows the CSS guideline, for example:

```
style="font-size:14pt;font-style:plain;
font-family:Tahoma;font-weight:bold"
```

visibleWhen

Defines the conditional display of a property based on one of two types of expressions comparing a property or preference to a value. The value can be **null** or a string, including a string containing wildcard characters. Multiple values can be checked with an array property or preference. When checking an array value, use a comma as a delimiter for the values. The two types of expressions check the following:

1. The value of a property on the selected object

```
<property name="p1" visibleWhen="<Property name>==<Some value>"/>
```

```
<property name="p1" visibleWhen="<Property name>!=<Some value>"/>
```

```
<property name="p1" visibleWhen="<Property name>==null"/>
```

- `<property name="p1" visibleWhen="<Property name>!=null"/>`
2. The value of a Teamcenter preference

`<property name="p1" visibleWhen="{pref:<Preference name>}==<Some value>"/>`

`<property name="p1" visibleWhen="{pref:<preference name>}!=<Some value>"/>`

- To check the value of a property on the selected object, use the real (database) name of the property in the expression.
If you want to show a **"myprop"** property only if the **object_desc** property begins with the word **Testing**, use the following:

`<property name="myprop" visibleWhen="object_desc==Testing*"/>`

- To check the value of a Teamcenter preference, use **{pref:preference-name}** to differentiate it from a property-based expression. Following are some examples:
Display a property when the **Cust_Enable_MyProp** preference is set to **true**.

`<property name="myprop" visibleWhen="{pref:Cust_Enable_MyProp}==true">`

Note:

Using **visibleWhen** for properties is only for use in SWT-based style sheet views in the rich client. For example, **Summary** (when checked out), **Create**, and **SaveAs**.

It will only work with SWT-based property beans, and will not work with Swing-based property beans.

In the current implementation of SWT styles aheet rendering in the **Summary** view, a plain **Label** instead of a **LabelPropertyBean** is used to display property values in read-only mode (**RenderFlat**). Since the **visibleWhen** framework implementation is based on **AbstractPropertyBeans** and its children, the **visibleWhen** feature will not be available in the **Summary** view in read-only mode. It will only be available when the object is checked-out.

SUPPORTED STYLE SHEETS

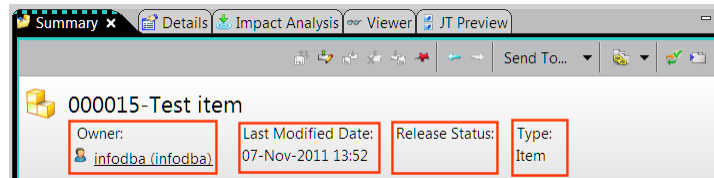
This tag can be used on the following types of style sheets:

Property
Summary
Form
Create

EXAMPLES

- property** element
Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **property** element:

```
<header>
  <image source="thumbnail"/>
  <classificationTrace/>
  <property name="owning_user"/>
</property name="last_mod_date"/>
<property name="release_status_list"/>
<property name="object_type"/>
</header>
```



property elements in the rich client

- URL rendering
URL addresses in the **label text** and **property** tags are automatically rendered. (In the rich client, URL addresses are also automatically rendered for **textfield** and **textarea** rendering hints.)
For an example, see [label](#).

rendering

Root element

ATTRIBUTES

Version

Specifies the version of the XML schema. When an older version is detected, the program automatically converts the old scheme to the new one.

SUPPORTED STYLE SHEETS

This tag is required on all types of style sheets:

Property
Summary
Form
Create

EXAMPLE

Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **rendering** element:

```
<rendering xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="XMLRenderingStylesheet_Schema.xsd">
  <header>
    <image source="thumbnail"/>
    <classificationTrace/>
    <property name="owning_user"/>
    <property name="last_mod_date"/>
    <property name="release_status_list"/>
    <property name="object_type"/>
  </header>
  .
  .
  .
</rendering>
```

section

Defines a group of elements to be displayed together within a named section.

Note:

This element is a replacement for the **view** element.

ATTRIBUTES

commandLayout

Controls the layout of commands. Valid values are **horizontal** or **vertical**.

initialstate

Specifies whether the view or section should be expanded or collapsed on initial rendering. Valid values are **expanded** or **collapsed**. The default value is **expanded**. This attribute is optional.

text

Specifies the title to be displayed on the section header. The attribute value must be a key into a property file. If the key is not found, the attribute value itself is displayed as static text. This attribute is required.

Caution:

As of Teamcenter 10.1, the **text** attribute is deprecated and is replaced by the **title** and **titlekey** attributes.

SUPPORTED STYLE SHEETS

This tag can be used on the **Summary** style sheets.

EXAMPLE

- Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **section** element:

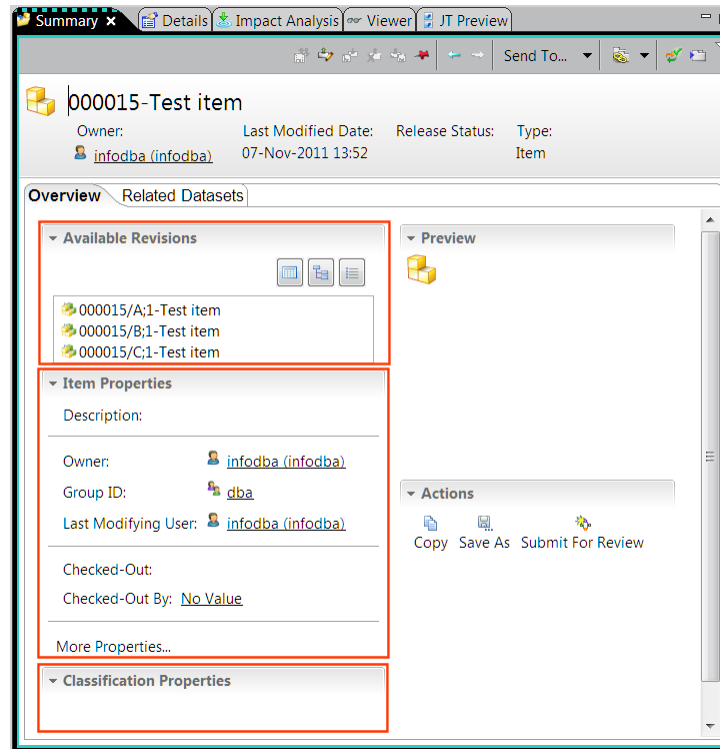
```
<page titleKey="tc_xrt_Overview">
  <column>
    <section titleKey="tc_xrt_AvailableRevisions">
      <objectSet source="revision_list.ItemRevision"
defaultdisplay="listDisplay" sortdirection="descending"
sortby="item_revision_id">
        <tableDisplay>
          <property name="object_string"/>
          <property name="item_revision_id"/>
          <property name="release_status_list"/>
          <property name="last_mod_date"/>
          <property name="last_mod_user"/>
          <property name="checked_out_user"/>
        </tableDisplay>
      </objectSet>
    </section>
  </column>
</page>
```



```

        </tableDisplay>
        <thumbnailDisplay/>
        <treeDisplay>
            <property name="object_string"/>
            <property name="item_revision_id"/>
            <property name="release_status_list"/>
            <property name="last_mod_date"/>
            <property name="last_mod_user"/>
            <property name="checked_out_user"/>
        </treeDisplay>
        <listDisplay/>
    </objectSet>
</section>
<section titleKey="tc_xrt_ItemProperties">
    <property name="object_desc"/>
    <separator/>
    <property name="owning_user" renderingHint="objectlink"
modifiabile="false"/>
    <property name="owning_group" renderingHint="objectlink"
modifiabile="false"/>
    <property name="last_mod_user"/>
    <separator/>
    <property name="checked_out"/>
    <property name="checked_out_user"/>
    <separator/>
    <command commandId="com.teamcenter.rac.properties"
titleKey="tc_xrt_moreProperties"/>
</section>
<section titleKey="tc_xrt_ClassificationProperties">
    <classificationProperties/>
</section>
</column>
<column>
    <section titleKey="tc_xrt_Preview">
        <image source="preview"/>
    </section>
    <section titleKey="tc_xrt_actions" commandLayout="vertical">
        <command commandId="com.teamcenter.rac.copy" />
        <command commandId="org.eclipse.ui.file.saveAs" />
        <command commandId="com.teamcenter.rac.newProcess"
titleKey="tc_xrt_newProc" />
    </section>
</column>
</page>

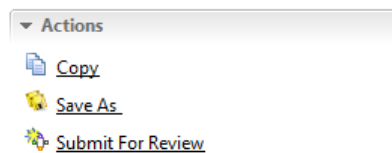
```



section elements in the rich client

- The following example demonstrates the use of the **commandLayout** attribute. The following code defines the command layout as vertical:

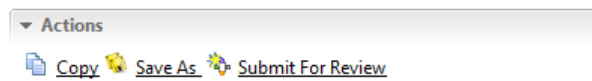
```
<section titleKey="tc_xrt_actions" commandLayout="vertical">
  <command commandId="com.teamcenter.rac.copy" />
  <command commandId="org.eclipse.ui.file.saveAs" />
  <command commandId="com.teamcenter.rac.newProcess" text="newProc" />
</section>
```



Vertical command layout

The following code defines the command layout as vertical:

```
<section titleKey="tc_xrt_actions" commandLayout="horizontal">
  <command commandId="com.teamcenter.rac.copy" />
  <command commandId="org.eclipse.ui.file.saveAs" />
  <command commandId="com.teamcenter.rac.newProcess" text="newProc" />
</section>
```



Horizontal command layout

separator

Adds a separator in the pane.

ATTRIBUTES

None.

SUPPORTED STYLE SHEETS

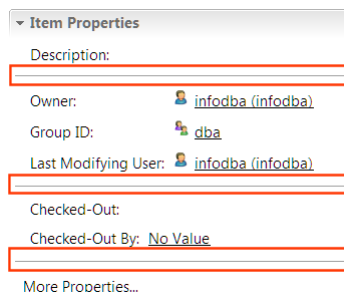
This tag can be used on the following types of style sheets and the New Business Object wizard:

Property
Summary
Form
Create

EXAMPLE

Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **separator** element:

```
<section titleKey="tc_xrt_ItemProperties">
  <property name="object_desc"/>
  <separator/>
  <property name="owning_user" renderingHint="objectlink" modifiable="false"/>
  <property name="owning_group" renderingHint="objectlink" modifiable="false"/>
  <property name="last_mod_user"/>
  <separator/>
  <property name="checked_out"/>
  <property name="checked_out_user"/>
  <separator/>
  <command commandId="com.teamcenter.rac.properties"
titleKey="tc_xrt_moreProperties"/>
</section>
```



separator elements in the rich client

tableDisplay

Displays a set of objects in a table format.

ATTRIBUTES

minRowCount

Specifies the minimum number of rows to display.

The default is **1**.

maxRowCount

Specifies the maximum number of rows to display. Set to **-1** for an unlimited number of rows.

The default is **10**.

maxColumnCharCount

Specifies the maximum number of characters for a column. Set to **-1** for unlimited column size.

The default is **80**.

SUPPORTED STYLE SHEETS

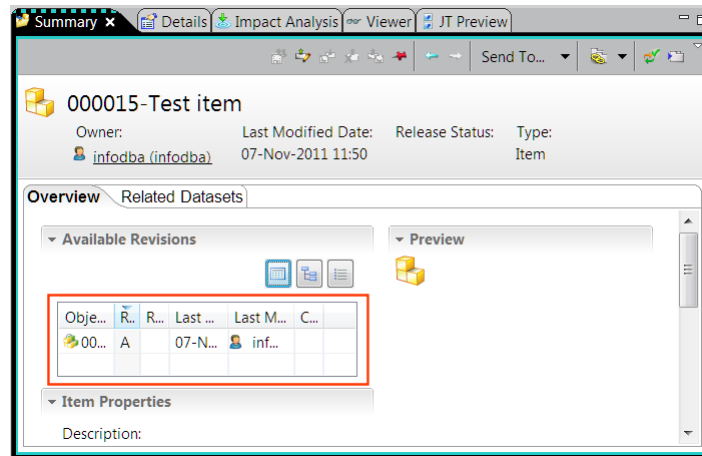
This tag can be used on the **Summary** style sheets.

EXAMPLE

Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **tableDisplay** element:

```
<section titleKey="tc_xrt_AvailableRevisions">
  <objectSet source="revision_list.ItemRevision" defaultdisplay="listDisplay"
  sortdirection="descending" sortby="item_revision_id">
    <tableDisplay>
      <property name="object_string"/>
      <property name="item_revision_id"/>
      <property name="release_status_list"/>
      <property name="last_mod_date"/>
      <property name="last_mod_user"/>
      <property name="checked_out_user"/>
    </tableDisplay>
    <thumbnailDisplay/>
    <treeDisplay>
      <property name="object_string"/>
      <property name="item_revision_id"/>
      <property name="release_status_list"/>
      <property name="last_mod_date"/>
      <property name="last_mod_user"/>
      <property name="checked_out_user"/>
    </treeDisplay>
  </listDisplay/>
</section>
```

```
</objectSet>  
</section>
```



tableDisplay element in the rich client

thumbnailDisplay

Displays a set of objects using thumbnails arranged in a grid.

ATTRIBUTES

None.

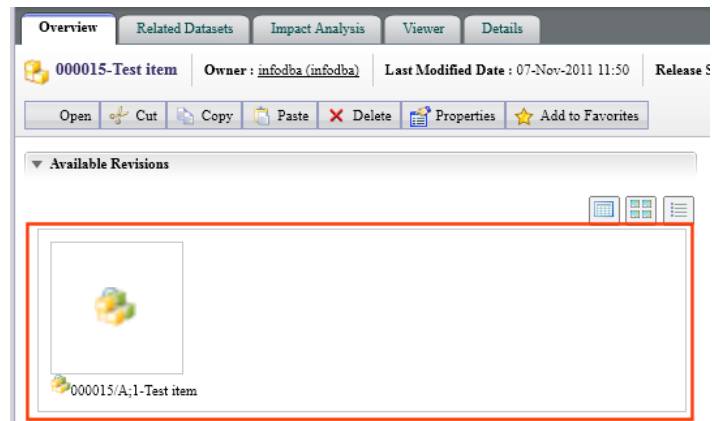
SUPPORTED STYLE SHEETS

This tag can be used on **Summary** style sheets.

EXAMPLE

Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **thumbnailDisplay** element:

```
<section titleKey="tc_xrt_AvailableRevisions">
  <objectSet source="revision_list.ItemRevision" defaultdisplay="listDisplay"
    sortdirection="descending" sortby="item_revision_id">
    <tableDisplay>
      <property name="object_string"/>
      <property name="item_revision_id"/>
      <property name="release_status_list"/>
      <property name="last_mod_date"/>
      <property name="last_mod_user"/>
      <property name="checked_out_user"/>
    </tableDisplay>
    <thumbnailDisplay/>
    <treeDisplay>
      <property name="object_string"/>
      <property name="item_revision_id"/>
      <property name="release_status_list"/>
      <property name="last_mod_date"/>
      <property name="last_mod_user"/>
      <property name="checked_out_user"/>
    </treeDisplay>
    <listDisplay/>
  </objectSet>
</section>
```



thumbnailDisplay

treeDisplay

Displays a set of objects in a tree format.

ATTRIBUTES

minRowCount

Specifies the minimum number of rows to display.

The default is **1**.

maxRowCount

Specifies the maximum number of rows to display. Set to **-1** for an unlimited number of rows.

The default is **10**.

maxColumnCharCount

Specifies the maximum number of characters for a column. Set to **-1** for unlimited column size.

The default is **80**.

SUPPORTED STYLE SHEETS

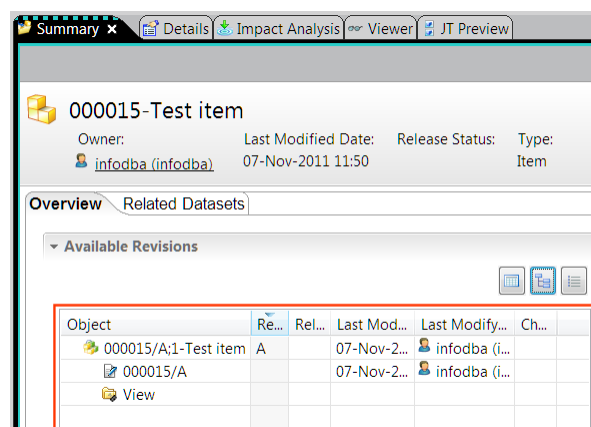
This tag can be used on **Summary** style sheets.

EXAMPLE

Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **treeDisplay** element:

```
<section titleKey="tc_xrt_AvailableRevisions">
  <objectSet source="revision_list.ItemRevision" defaultdisplay="listDisplay"
    sortdirection="descending" sortBy="item_revision_id">
    <tableDisplay>
      <property name="object_string"/>
      <property name="item_revision_id"/>
      <property name="release_status_list"/>
      <property name="last_mod_date"/>
      <property name="last_mod_user"/>
      <property name="checked_out_user"/>
    </tableDisplay>
    <thumbnailDisplay/>
    <treeDisplay>
      <property name="object_string"/>
      <property name="item_revision_id"/>
      <property name="release_status_list"/>
      <property name="last_mod_date"/>
      <property name="last_mod_user"/>
      <property name="checked_out_user"/>
    </treeDisplay>
  </objectSet>
</section>
```

```
</objectSet>
</section>
```



treeDisplay element

Rendering hints

Add a custom rendering hint

The **renderingHint** attribute on the **property** tag allows you to specify which user interface widget to use to present the corresponding property. In addition to the supported standard rendering hints, you can also add your own custom rendering hints.

For the standard rendering hints, see the **com.teamcenter.rac.viewer/plugin.xml** file for the SWT version of rendering hints, and see the **com.teamcenter.rac.common/plugin.xml** file for the Swing version of rendering hints.

The method of adding a custom rendering hint is different depending on whether you are using the SWT or Swing property bean.

SWT

The SWT property bean is used by the **Summary** view, the **New→Other** wizard, and the **Viewer** view.

Eclipse plug-in To create a custom SWT property bean:

1. Create a custom plug-in that uses the **com.teamcenter.rac.common.renderingHint** extension point.

Example:

```
<extension point="com.teamcenter.rac.common.renderingHint">
  <renderingHint id="mytextfield" priority="0">
    <propertyBean class="com.teamcenter.rac.viewer
```

```

        .stylesheet.beans.MyTextfieldPropertyBean">
    </propertyBean>
</renderingHint>
</extension>

```

2. Create a Java class by extending from the **AbstractPropertyBean** bean.
Java example follows.
3. Add your new package or class to your custom plug-in **export-package** list in the **MANIFEST.MF** file.

Example:

```
Export-Package: com.teamcenter.rac.viewer.customplugin.bean
```

Swing

The Swing property bean is used by the **Properties** dialog box, the form dialog box, and the old viewer. There are two ways you can implement the Swing property bean:

Eclipse plug-in Use the **com.teamcenter.rac.common.renderingHint** extension point.

1. Create a custom plug-in that uses the **com.teamcenter.rac.common.renderingHint** extension point.

Example:

Note the difference from the SWT version, because it is using the **legacyPropertyBean** element for the class in the extension definition:

```

<extension point="com.teamcenter.rac.common.renderingHint">
  <renderingHint id="mytextfield" priority="0">
    <legacyPropertyBean
      class="com.teamcenter.rac.stylesheet.MyPropertyTextField"/>
  </renderingHint>
</extension>

```

2. Create a Java class by extending from the **InterfacePropertyComponent** component.
Java example follows.
3. Add the new package or class to your custom plug-in **export-package** list in the **MANIFEST.MF** file.

Example:

```
Export-Package: com.teamcenter.rac.viewer.customplugin.bean
```

**Java's
properties
files**

Add the new rendering hint definition to a custom **stylesheet_user.property** file.

Each rendering hint has a key for the class path of the Java bean defined in the **com\teamcenter\rac\stylesheet.properties** file, found in the **com.teamcenter.rac.common** JAR file. You can plug in your own bean by overwriting the entry in the properties file to replace the default Java bean, or you can add new entries for custom Java beans.

The key has the following format for headed or headless beans:

```
rendering-hint.DEFINITION
```

The following format is for titled beans.

```
rendering-hint_titled.DEFINITION
```

Example:

```
textfield.DEFINITION=com.teamcenter.rac.stylesheet.
PropertyTextField

textfield_titled.DEFINITION=com.teamcenter.rac.stylesheet.
TitledPropertyTextField
```

SWT Java Class example

```
package com.teamcenter.rac.viewer.customplugin.beans;

import com.teamcenter.rac.common.controls.LOVComboBox;
import com.teamcenter.rac.kernel.TCPropertyDescriptor;
import com.teamcenter.rac.viewer.stylesheet.beans.LOVComboBoxPropertyBean;
import java.util.Map;
import org.eclipse.core.runtime.CoreException;
import org.eclipse.core.runtime.IConfigurationElement;
import org.eclipse.core.runtime.IExecutableExtension;
import org.eclipse.swt.widgets.Composite;
import org.eclipse.ui.forms.widgets.FormToolkit;
/**
 *
 */
public class MyLOVComboBoxPropertyBean
    extends LOVComboBoxPropertyBean implements IExecutableExtension
{
    /**
     *
     * Constructor
     *
     */
}
```

```

    * @param lovCombo
    */
    public MyLOVComboBoxPropertyBean( LOVComboBox lovCombo )
    {
        super( lovCombo );
    }
    /**
    * Constructor
    *
    * @param toolkit FormToolkit to use to create UI widgets
    * @param parent composite
    * @param renderFlat Flag indicating it's read only or modifiable
    * @param paramTable Map of attributes which in stylesheet file.
    * @published
    */
    public MyLOVComboBoxPropertyBean( FormToolkit toolkit, Composite parent,
        boolean renderFlat, Map paramTable )
    {
        super(toolkit, parent, renderFlat, paramTable);
    }
    @Override
    public void load( TCPropertyDescriptor desc )
        throws Exception
    {
        super.load( desc );
        System.out.println("This is mylovcombobox.");
        // force it to load the LOVs
        lovComboBox.initializeData(false);
    }
    @Override
    public void setUIFValue( final Object value )
    {
        lovComboBox.getDisplay().asyncExec( new Runnable()
        {
            public void run()
            {
                if ( value != null )
                {
                    lovComboBox.setSelectedItem( value );
                }
                else
                {
                    lovComboBox.setText(""); //$NON-NLS-1$
                }
            }
        } );
    }
    @Override
    public void setInitializationData( IConfigurationElement config,
        String propertyName, Object data )
        throws CoreException
    {
        //
    }
}

```

Swing Java Class example

```

package com.teamcenter.rac.viewer.customplugin.beans;

import com.teamcenter.rac.aif.AIFDesktop;
import com.teamcenter.rac.aif.AbstractAIFUIApplication;
import com.teamcenter.rac.kernel.TCComponentGroup;
import com.teamcenter.rac.kernel.TCProperty;
import com.teamcenter.rac.kernel.TCPropertyDescriptor;
import com.teamcenter.rac.kernel.TCSession;
import com.teamcenter.rac.stylesheet.PropertyTextField;

/**
 * My example
 */
public class MyTextFieldSwingBean
    extends PropertyTextField
{
    /**
     *
     * Constructor
     *
     * @param lovCombo
     */
    public MyTextFieldSwingBean()
    {
        super( );
    }

    @Override
    public void load( TCPropertyDescriptor desc )
        throws Exception
    {
        super.load( desc );

        System.out.println("load: This is mytextfield.");
        AbstractAIFUIApplication aifApp = AIFDesktop.getActiveDesktop()
            .getCurrentApplication();
        TCComponentGroup group = ((TCSession) aifApp.getSession())
            .getCurrentGroup();
        if (group.toString().contains("dba")) {
            // getTextField().setEditable(false);
            System.out.println("Current Group:" + group.toString());
        }
    }

    @Override
    public void save( TCProperty p )
        throws Exception
    {
        super.save(p);
        System.out.println("Save: This is mytextfield.");
    }
}

```

Rendering style

Each type of renderer supports three styles: headed, headless, and titled.

- **Headed**
Displays the property name on the left followed by the property value renderer. This is the default rendering style.
This style has two components, the **PropertyNameLabel** JavaBean for the property name and the **PropertyrenderingHint** JavaBean for the renderer (for example, **PropertyTextField** or **PropertyTextArea**).



- **Headless**
Renders only the property value without displaying the property name.
This style contains only one JavaBean, **PropertyrenderingHint**.
- **Titled**
Displays the property name above the property value renderer.
This style uses only the **TitledPropertyrenderingHint** JavaBean, for example **TitledPropertyTextField** or **TitledPropertyTextArea**.



Default renderers

The following table displays the default renderer for each type. If the rendering hint is not provided, the default renderer is used.

Type	Default renderer
string	textfield if size < 60 textarea if 60 < size < 2500 longtext if size >= 2500
char	textfield
double	textfield
float	textfield
int	textfield

Type	Default renderer
short	textfield
long	textfield
date	datebutton
logical	logical
note	textfield if size < 60 textarea if size > 60
TypedReference/ UntypedReference	objectlink
All array types	array
Any property with an LOV attached	lovuicomp In turn, the lovuicomp renderer uses the lovcombox renderer if the number of list of values does not exceed 500, and if there are more, the lovpopupbutton renderer is used.

3. Rich client customization using preferences and properties files

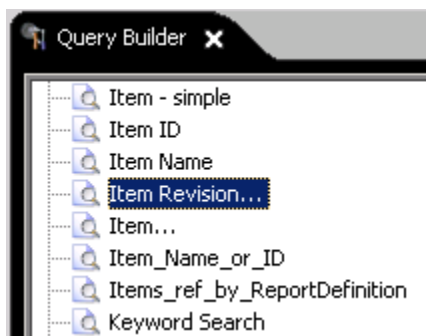
Add a quick search item	3-1
Configuring the worklist using .properties files	3-3
Displaying files in the viewer	3-4
Working with themes	3-4
Registry	3-6
What is the registry?	3-6
User properties files	3-7
Supported types	3-8
Registry keys	3-8
Customizing tabs	3-8
Customizing the data tabs display	3-8
Edit a custom properties file to display tabs	3-10
Sample tab customization	3-10
Add a column to view occurrence notes	3-11
Create a new type wizard	3-13
New type wizard overview	3-13
Add a menuContribution for your wizard	3-14
Utilize the extWizard extention point to create your wizard	3-15
Utilize the boTypesLoader extension point to filter the types	3-16
Utilize the extWizardRef extension point to add pages	3-16
Utilize the operation extension point to create the object	3-18
Customize the rich client properties files	3-18

3. Rich client customization using preferences and properties files

Add a quick search item

You can add an existing query to the quick search box at the top of the navigation pane. You can add any query from Query Builder, including queries you create yourself.

In the following example, you add the **Item Revision...** query to the quick search list, searching for the **object_name** attribute. All necessary information is found in the Query Builder.



Name:

Description:

Query Type:

Search Class:

Modifiable Query Types:

☐ Show Indented Results Revision Rule:

Attribute Selection

- ItemRevision
 - Revision
 - Item has Module?
 - Item [Item]
 - Sequence ID
 - Sequence Limit

Search Criteria | Order By

	Attribute	User Entry L10N Key	User Entry Name		Default Value
	object_name	Name	Name	=	
AND	items_tag.it...	ItemID	Item ID	=	
AND	item_revisio...	Revision	Revision	=	

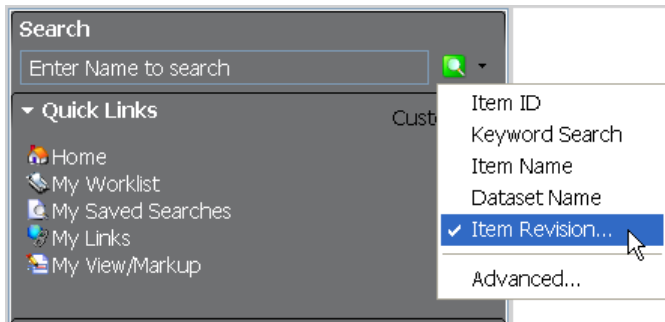
1. Edit the **Quick_Access_Queries** preference and add the name of the query to the list (for example, **Item Revision...**).

```
Item Revision...
```

2. Edit the **Quick_Access_Queries_Attribute** preference and add the **User Entry L10N Key** for the search attribute to use with the query (for example, **Name**) for **object_name**:

```
Item Revision..._SearchAttribute=Name
```

3. Test the new quick search by clicking the arrow to the right of the quick search.



Additional information

- The attribute to use for the search must exist in the query. Look at the query in Query Builder to see the attributes available.
- Any attributes used for quick search *must* have a localization (**L10N**) key registered in the TextServer. All OOTB attributes already have **L10N** keys. If you are searching with a custom attribute, you must add it to the TextServer definition.

Configuring the worklist using .properties files

You can modify properties files related to your worklist.

Worklist process and interface features that can be configured include the following:

- Application labels
- Task state labels
- System messages
- Buttons

Typically, the administrator checks first in the workflow **common_user.properties** file for the appropriate property.

- If the desired property is listed in the **common_user.properties** file, it is common to all workflow applications, and you can implement your change across all workflow applications simultaneously.
- If the desired property is not found in the **common_user.properties** file, it may be listed in the **inbox_user.properties** file. Changes made to the properties in this file are unique to the inbox.

Displaying files in the viewer

On Windows platforms, you can display Microsoft Word, PowerPoint, Excel, Portable Document Format (PDF), and text files in the **Viewer** panel. On UNIX platforms, the properties of the file are displayed instead.

Microsoft Word, Excel, PowerPoint, PDF, and text files are displayed automatically when they are named references of corresponding datasets. No additional configuration required.

To display QAF files in the viewer if Teamcenter lifecycle visualization is not already installed:

1. In the **com.teamcenter.rac.common** plug-in, create an empty **com\teamcenter\rac\common\tcviewer\tcviewer_user.properties** file.
2. In the **com.teamcenter.rac.common** plug-in, open the **tcviewer\tcviewer.properties** file, copy the entire **DatasetViewer.VIEWSEARCHORDER** line, and paste it into the new **tcviewer_user.properties** file.
3. In the **tcviewer_user.properties** file, add **UG-QuickAccess-Binary** to the end of the **DatasetViewer.VIEWSEARCHORDER** line.
4. Add the following line to the **tcviewer_user.properties** file:

```
UGMASTER.VIEWPANEL=com.teamcenter.rac.common.tcviewer.DatasetViewer
```

5. In the **com.teamcenter.rac.util** plug-in, create a **com\teamcenter\rac\util\viewer\viewer_user.properties** file, and add the following line:

```
Imager.EXTENSION=gif,jpg,qaf
```

Note:

To disable the toolbar in the viewer, go to **com/teamcenter/rac/common/tcviewer**, create the **tcviewer_user.properties** file in a text editor, and add the following entry:

```
useNevaIEViewerToolBar=false
```

Working with themes

You can change the start-up theme for the rich client and also modify existing themes, by using a text editor.

How do I know what themes are available?

Using a text editor, examine the content of **TC_ROOT/portal/plugins/configuration_nnnn/fragment.xml**. You will find each theme defined and its unique identifier (id)

Example:

In the following example, two themes are defined — **Classic** and **Light**:

```
<fragment>
  <extension
    point="org.eclipse.e4.ui.css.swt.theme">
    <theme
      basestylesheeturi="css/rac_classic.css"
      id="com.teamcenter.rac.classic"
      label="Classic">
    </theme>
    <theme
      basestylesheeturi="css/rac_lighttheme.css"
      id="com.teamcenter.rac.light"
      label="Light">
    </theme>
  </extension>
</fragment>
```

Your defined themes may differ.

How can I temporarily change which theme is used?

To launch the rich client with a specified theme once, launch the **TC_ROOT\portal\portal.bat** script specifying the **-theme=** argument followed by the **id** of the theme.

Example:

The following example will launch the rich client using the light theme this time only.

```
portal.bat -theme=com.teamcenter.rac.light
```

If you launch the rich client without the argument, it will use the default theme.

How can I permanently change which theme is used?

To launch the rich client with a specified theme every time, modify the **TC_ROOT\portal\portal.bat** script to include the **-theme=** argument followed by the **id** of the theme.

Example:

Before:

```
start Teamcenter.exe %* -vm "JRE_HOME...
```

After:

```
start Teamcenter.exe %* -theme=com.teamcenter.rac.light -vm "JRE_HOME...
```

How do I change theme colors?

The rich client applies a theme in two parts. Depending on which widget you want to modify, you need to modify the corresponding file.

- The colors for SWT widgets are defined by ***.CSS** files in the following directory:

```
TC_ROOT\portal\plugins\configuration_?version?\css
```

After making your changes, delete the rich client cache and restart the rich client.

If your changes do not appear, clear the cache by deleting the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the **%HOMEDRIVE%%HOMEPATH%\Teamcenter** directory.

On a Linux client, it is typically the **\$HOME/Teamcenter** directory.

When you delete this directory, the last state of the rich client is lost, and the user interface appears as it does at initial startup.

- The colors for Swing widgets are defined by Java registry-style ***.properties** files in the following directory:

```
TC_ROOT\portal\plugins\configuration_?version?\com\teamcenter\rac\themeName
```

Remember, after modifying any registry files, regenerate the registry and restart the rich client.

```
TC_ROOT\portal\registry\genregxml.bat
```

Registry

What is the registry?

The registry plays a vital role in the Teamcenter rich client. The registry uses the properties files to define classes, icons, internationalized text, inheritance, search order, and appearance. Our registry is fundamentally a subclass of the **ResourceBundle** object found in Java and uses the properties files to store replaceable information that can be tailored based on need. Many basic customization tasks can be accomplished using the registry.

Note:

This registry is not the Windows registry.

The key contained within the registry is contained within the product code, but the value can be modified. Every key/value pair that appears within the registry falls into one of the following categories: appearance, structural, instantiation, or localization.

- **Appearance**
Keys that define colors, fonts, and sizes.
- **Structural**
Keys that are defined (usually at the top) and are called import statements. Importing allows the nesting of property files for searching. This allows you to define a key in one place within the rich client that can be reused without being duplicated. For instance, the **OK** button text must be localized in the rich client user interface based on the current locale. Therefore, the text for **OK** is placed within the **aif_locale.properties** file.
- **Instantiation**
Keys used to construct Java objects. Teamcenter rich client makes use of reflection and dynamic instantiation within the Java language to construct objects by their string name rather than hard-coding them within the source code.
For example, each command that appears in the menu bars and toolbars is dynamically constructed and run. You can subclass a command and replace it with your own by replacing an entry within a properties file to tell the rich client to use your command rather than the base command.
- **Localization**
Keys allow localization of text and messages to be flexibly obtained based on the current locale of the OS where rich client is running. Teamcenter rich client is delivered with locales that support English, German, French, Spanish, Italian, Czech, Russian, Korean, Japanese, and Chinese. Each of these separate property files is based on the convention that JavaSoft has established. Localization keys are those for which the **ResourceBundle** class was first designed.
In any given package (directory) that uses registries, there are three property files: base, locale, and user. These properties files contain structural, instantiation, and appearance keys.
- **Base properties file**
Contains nonlocale-specific information and is the only property required in a package.
- **Locale properties file**
Contains only localization keys. The reason for having the localization separate from the base is that when keys are to be localized within the rich client, it is easy to separate the localization keys from the rest. In addition, if you are developing customization to be deployed in more than one locale, this provides a convenient separation between the two and allows for easier localization that could be outsourced to third parties for completion.

User properties files

These files contains properties that can be modified by customers. When you make modifications to the rich client, always make them in the user properties file. This protects the base system from user mistakes. It also enables a backup approach in the event that the change breaks the rich client, in which case you can reload or obtain a copy from the reference directory.

User property files are always consulted first for registry lookups. The rich client is completely loaded on demand. Therefore, when it starts up, the applications listed on the left side may or may not exist. This is not known until the user clicks on the application for the first time. In addition, commands are never loaded until requested by the user. This loading method provides a small memory footprint and enables the system to only use resources that the user has requested.

Supported types

Primitive types are supported in the registry as well. The following types are supported: color, font, integer, float, double, char, boolean, string, string array, and images. Some of the types use special formats, as follows:

- Color
red, green, blue

`255, 255, 255`

- Font
font-name, font-style, font-size

`"Times Roman, Font.ITALIC, 12"`

- String array
one,two,three

Registry keys

If a key must contain a blank or space, the space follows the backslash (\) character. If the name of the type contains a space, such as that found in the **Adobe Acrobat** type, the entry would be:

`Adobe\ Acrobat=images/acrobat.gif`

Java treats a space like an equal sign (=). Therefore, if the entry is:

`Adobe Acrobat=images/acrobat.gif`

Java interprets the key as **Adobe** and the value as **Acrobat=images/acrobat.gif**.

Customizing tabs

Customizing the data tabs display

If you have administrator privileges, you can customize how data tabs appear in Multi-Structure Manager and other applications. When a user runs one of these applications and selects the data panel,

several tabs appear in the panel. You can change the tabs that appear by editing the application properties file.

There are two types of tabs in each application:

- Tabs that always appear for a particular parent panel regardless of whether anything is selected.
- Tabs that appear only when particular components are selected in the parent panel.

You can customize how the second, selection-specific group of tabs is displayed.

To determine the tabs to display, the system checks four criteria:

- The class type of the selected display component, for example:

```
BOMLine
CfgAttachmentLine
TcItemBOPLine
```

- The subtype of the selected display component, which is generally the same as the class type. However, for BOM lines, it is the occurrence type and for attachment lines it is the relation to the parent.
- The class type of the underlying component, such as **ItemRevision**.
- The subtype of the underlying component (the component type name).

For each selection, the system checks for six properties and adds all the tabs found. You can edit these properties to change the tabs that are presented to the user:

```
Display-component-classtype.TABS
Display-component-subtype.TABS
Display-component-classtype.underlying component classtype.TABS
Display-component-classtype.underlying component subtype.TABS
Display-component-subtype.underlying component classtype.TABS
Display-component-subtype.underlying component subtype.TABS
```

For example, in the Multi-Structure Manager application, the default properties are:

```
BOMLine.TABS=Referencers, Variant, Attachments, InClassAtt, CMEViewer, Report,
IncrementalChangeInfo
TcItemBOPLine.TABS=Referencers, Variant, Attachments, InClassAtt, CMEViewer,
Report, IncrementalChangeInfo
AppGroupBOPLine.TABS=Referencers, Attachments, CMEViewer, IncrementalChangeInfo
GDELine.TABS=Referencers, InClassAtt, CMEViewer, Report, IncrementalChangeInfo
GDELinkLine.TABS=Referencers, InClassAtt, CMEViewer, Report,
IncrementalChangeInfo
MEAppearanceLine.TABS=Referencers, Attachments, CMEViewer, IncrementalChangeInfo
CfgAttachmentLine.TABS=Referencers, CMEViewer, IncrementalChangeInfo, Report
BOMLine.ItemRevision.TABS=ProductAppearance
```

```
TcItemBOPLine.ItemRevision.TABS=ProductAppearance  
CfgAttachmentLine.ItemRevision.TABS=InClassAtt
```

You can add or delete the names of tabs that are displayed for each data panel in this file.

Edit a custom properties file to display tabs

To display tabs, you can place the changes in a **_user.properties** file to override the default **.properties** file.

1. Extract the **application.properties** file from the appropriate **com.teamcenter.rac.component.jar** file using the **jar xf** command. For example, if you want to extract the **mpp.properties** file for Part Planner, the command looks like this:

```
jar xf com.teamcenter.rac.cme.legacy.jar com/teamcenter/rac/cme/mpp/mpp.properties
```

Note:

For more information about the **jar** command, see Sun's Java documentation.

2. Save the extracted file as the **application_user.properties** file.
3. In the custom properties file, edit the **.TABS** line to include the tab you want.
4. Insert the custom properties file into your own custom plug-in.

Sample tab customization

The following example is for creating new tabs for Manufacturing Process Planner.

1. In the Business Modeler IDE, create a new type as a child of the **Process** business object. When the object is deployed to the rich client and displayed in Manufacturing Process Planner, the tabs in the data pane are different than the tabs for the **Process** business object. You must customize the tabs for the new business object so that they match tabs for the parent.
2. Open the *installation-location\portal\plugins\com.teamcenter.rac.cme.legacy* JAR file and find the **mpp.properties** file in the following directory:

```
com\teamcenter\rac\cme\mpp\mpp.properties
```

3. Make a copy of the **mpp.properties** file and rename it into a customer-specific **application_user.properties** file.
4. In the custom properties file, create **.TABS** entries for your custom business object. Manufacturing Process Planner accepts the following definitions in the properties files:

```

line-type.TABS= tab-1, tab-2, tab-n
line-subtype.TABS=tab-1, tab-2, tab-n
line-type.underlying-type.TABS= tab-1, tab-2, tab-n
line-type.underlying-subtype.TABS= tab-1, tab-2, tab-n
line-subtype.underlying-type.TABS= tab-1, tab-2, tab-n
line-subtype.underlying-subtype.TABS= tab-1, tab-2, tab-n

```

line-type is the type of the BOM line, for example, **BOMLine**, **ImanItemBOPLine**, or **MfgOBvrProcess**. *line-subtype* is the subtype of a line and it can be an occurrence type or a relation type, for example, **MEConsumed** (in some cases, it is equal the line type). *underlying-type* is the type of the underlying component; a **BOPLine** can have the underlying **OperationRevision** type, **MroccessRevision** type, or other types. *underlying-subtype* is the subtype of the underlying component; like the line subtype, the underlying subtype can also be the same as the underlying type.

If a **BOMLine** type matches more than one definition, the result is the sum of tabs from all matched definitions. For example, an item name **I1** is assigned to an operation as **MEConsumed** type. The following tab lines are defined:

```

ImanItemBOPLine.TABS= Variant
ImanItemBOPLine.ItemRevision.TABS= CMEViewer
BOMLine.TABS= Referencers
BOMLine.ItemRevision.TABS= ProductAppearance

```

Selecting the **I1** item in the process structure (below the operation) matches **ImanItemBOPLine.TABS** and **ImanItemBOPLine.ItemRevision.TABS**, and as a result, the system shows the **Variant** and **CMEViewer** tabs. But selecting **I1** in the BOM structure matches **BOMLine.TABS** and **BOMLine.ItemRevision.TABS**; therefore, the system shows the **References** and **ProductAppearance** tabs.

Note:

The tabs are also defined in the **mpp.properties** file:

```

tab.CLASS
tab.ICON

```

The tab label and tooltip are defined in the **mpp_locale.properties** file:

```

tab.TABLABEL
tab.TOOLTIP

```

5. Insert the custom properties file into your own custom plug-in.

Add a column to view occurrence notes

By default, you cannot view occurrence notes on intermediate data capture (IDC) objects in the Multi-Structure Manager. However, you can view the occurrence notes if you can add a column to display them. To accomplish this, you must define the alias for the occurrence note type in the **TC_DATA**

`\structure_alias.xml` file, and then add the column for the occurrence note type in the `IDCStructureColumnsShownPref` preference.

1. Create the occurrence note on the IDC object.
 - a. Create a structure in Multi-Structure Manager.
 - b. In the data pane to the right, select an occurrence (item revision) in the structure.
 - c. Right-click a column heading and add a column for an occurrence note type (for example, **UG NAME**).
 - d. For the selected occurrence, double-click in the empty cell in that column and type in your note (for example, type **This is my occurrence note**).
 - e. In the left pane, select that occurrence in the structure (the item revision) and choose **Tools→Intermediate Data Capture**. For our example, choose the **Transfer Mode Name** of **tcm_export** because it transfers the **UG NAME** type of occurrence note. When the IDC appears on the tab, there is no content in the data panel, and there is no occurrence note column (for example, no **UG NAME** column). That's because the occurrence note type column must be added to this IDC structure view using a customization.
 - f. Exit the rich client.
2. Open the `TC_DATA\structure_alias.xml` file and add code.
 - a. Add the following under the **Alias name="BOM.Property"** node:

```
<Alias name="UG NAME" default=" " type="string" size="0"/>
```

- b. Add the following under the **AliasAlternate name="BOM.Property"** node:

```
<Alias name="UG NAME">
  <Path depth="0..0">*:*:@instancedRef</Path>
  <Xpath>/UserData/UserValue[@title='UG NAME']/@value</Xpath>
</Alias>
```

3. Regenerate the caches and restart the rich client.

Note:

```
generate_metadata_cache -force
```

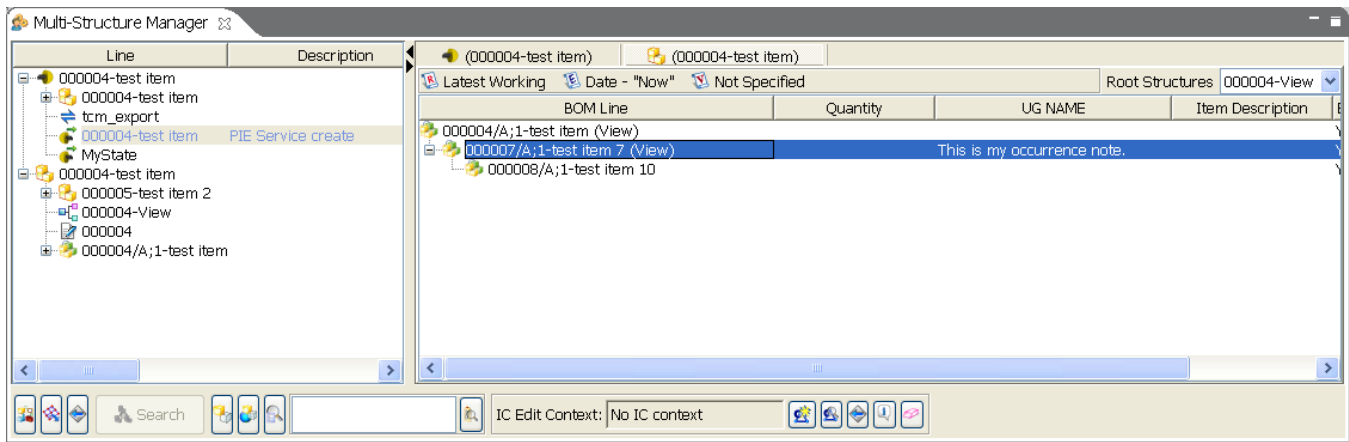
```
generate_client_meta_cache -log update all
```

4. Add the **UG NAME** value to the **IDCStructureColumnsShownPref** preference, and add a value for the new column's width to the **IDCStructureShownColumnWidthsPref** preference.

Note:

Since these are interdependent preferences, you cannot use the Options dialog box to make changes.

5. Select the item structure in My Teamcenter and send it to Multi-Structure Manager. The occurrence note displays on the IDC in the right data panel under the new column (for example, the **UG NAME** column).



New column for occurrence notes

Create a new type wizard

New type wizard overview

If you want more control over which business object types your users can create, and what the dialog boxes display, then you can create a custom New Business Object Wizard .

Using an Eclipse plug-in project, you can then create a menu contribution for your new wizard in the **File→New** menu.

You can create several new type wizards, each one allowing only a few or even just a single type of business object to be created. For complete control, limit user access to the **File→New→Object** menu. You can use Command Suppression to limit by group or role, or a plug-in project to remove it completely.

An Eclipse plug-in project is required for all the following steps.

1. Add a **menuContribution** for your wizard.
2. Utilize the **extWizard** extension point to create your wizard.

3. Utilize the **boTypesLoader** extension point to filter the types.
4. Utilize the **extWizardRef** extension point to add pages.
5. Utilize the **operation** extension point to create the object.

Add a menuContribution for your wizard

You must create some way for the user to start the wizard. Creating a **menuContribution** is the most common way to do this. The new UI selection will link to a new **command**, which will be associated with a new **handler** (Java class).

In this example, you will create a **Document** menu selection on the **File→New** menu, then create a corresponding **command** and **handler**.

Add the following menu contribution to the **menus** extension point in your project's **plugin.xml** file.

```
<extension point="org.eclipse.ui.menus">
...
  <menuContribution locationURI="menu:file.new?after=group1">
    <command
      commandId="com.mycom.newwizard.genericcustomitem"
      mnemonic="D"
      id="com.mycom.newwizard.genericcustomitem">
    </command>
  </menuContribution>
...
</extension>
```

Add the following to the **commands** extension point in your project's **plugin.xml** file.

```
<extension point="org.eclipse.ui.commands">
...
  <command
    id="com.mycom.newwizard.genericcustomitem"
    name="Document">
  </command>
...
</extension>
```

Add the following to the **handlers** extension point in your project's **plugin.xml** file.

```
<extension point="org.eclipse.ui.handlers">
...
```



```

<handler
  class="com.mycom.newwizard.handlers.CustomWizardHandler"
  commandId="com.mycom.newwizard.genericcustomitem">
</handler>
...
</extension>

```

Create the **Java** class for your **handler**.

```

package com.mycom.newwizard.handlers;

import org.eclipse.core.commands.ExecutionEvent;
import org.eclipse.core.commands.ExecutionException;

import com.teamcenter.rac.ui.commands.handlers.NewBOHandler;

public class CustomWizardHandler extends NewBOHandler {

    public Object execute( ExecutionEvent event )
        throws ExecutionException
    {
        wizardId = "com.mycom.newwizard.CustomObjectWizard";
        return super.execute( event );
    }

    public String getWizardTitle()
    {
        return "Document";
    }

}

```

Utilize the extWizard extension point to create your wizard

To create a custom type wizard you must use the **extWizard** extension point.

In this example, you create a new **extWizard** extension, with a **CustomObjectWizard** identification, which uses the existing **NewBOWizard** class.

Add the following extension to your project's **plugin.xml** file.

```

<extension
  id="extWizard"
  point="com.teamcenter.rac.util.extWizard">
  <extWizard
    class="com.teamcenter.rac.ui.commands.create.bo.NewBOWizard"

```

```

        id="com.mycom.newwizard.CustomObjectWizard">
    </extWizard>
</extension>

```

Utilize the `boTypesLoader` extension point to filter the types

You must provide a list of available business object types to the wizard. The user will be presented this list on the **BOTypePage**. The easiest way to do this is using the **boTypesLoader** extension point.

In this example, you use the **Document** business object type for your wizard.

Add the following to the **boTypesLoader** extension point in your project's **plugin.xml** file.

```

<extension point="com.teamcenter.rac.ui.commands.boTypesLoader">
...
    <BOTypesLoader base_classes="Document"
        class="com.teamcenter.rac.ui.commands.create.bo.NewBOTypesLoader"
        enableMRU="true"
        id="com.teamcenter.rac.ui.commands.create.bo.DocumentTypeloader"
        parentId="com.mycom.newwizard.CustomObjectWizard">
    </BOTypesLoader>
...
</extension>

```

The **boTypesLoader** will find not only the business object type specified as the **base_class**, but also any instantiable children of that type.

In the following example, **Document** happens to be the parent of several other instantiable business object types, including Email, Marketing Brief, and others.

To remove the children from the list, you must also use the **excludeTypes** tag to specify which child types you do not want included. Multiple type names can be entered, separated by a comma.

In this example, the Email and Marketing Brief types are removed from the list.

```

    <BOTypesLoader base_classes="Document"
        class="com.teamcenter.rac.ui.commands.create.bo.NewBOTypesLoader"
        enableMRU="true"
        id="com.teamcenter.rac.ui.commands.create.bo.DocumentTypeloader"
        parentId="com.mycom.newwizard.CustomObjectWizard">
    <excludeTypes base_class="Document"
        exclude_types="Email,CPMarketingBrief"
        typesLoaderId="com.teamcenter.rac.ui.commands.create.bo.DocumentTypeloader"
    </excludeTypes>
    </BOTypesLoader>

```

Utilize the `extWizardRef` extension point to add pages

Once you have defined the new wizard, you must add pages to it. Use the **extWizardRef** extension point to add pages. The two main pages required are **BOTypePage** and **CreatePropsPage**.

The order of the pages is controlled by the **ordinal** attribute; the pages are presented to the user in numerical order. The **BOTypePage** page must be presented before (have a lower **ordinal** number than) the **CreatePropsPage** page, since the required and available properties are determined by the business object type selected.

```
<extension
    id="itemWizardRef"
    point="com.teamcenter.rac.util.extWizardRef">
    <extWizardRef
        ordinal="100"
        pageId="com.teamcenter.rac.ui.commands.create.bo.BOTypePage"
        wizardId="com.mycom.newwizard.CustomObjectWizard">
    </extWizardRef>
    <extWizardRef
        ordinal="200"

pageId="com.teamcenter.rac.ui.commands.create.bo.CreatePropsPage"
        wizardId="com.mycom.newwizard.CustomObjectWizard">
    </extWizardRef>
</extension>
```

The **CreatePropsPage** can be controlled using an **XMLRenderingStylesheet**.

Following is an XML rendering stylesheet that could be used to display properties from both the Document and the DocumentRevision.

- **Registered Type: Document**
- **Stylesheet Type: Create**

DocumentCreate.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<rendering>
    <page title="Document Information"
        titleKey="tc_xrt_DocumentInformation">
        <section titleKey="tc_xrt_properties">
            <property name="object_name" />
            <property name="object_desc" />
            <separator/>
            <property name="revision:DocumentSubject" />
        </section>
        <section title="Additional Document Information"
            titleKey="tc_xrt_properties"
            initialState="collapsed">
            <property name="revision:DocumentTitle" />
            <property name="revision:DocumentAuthor" />
        </section>
    </page>
</rendering>
```

```
</page>
</rendering>
```

Utilize the operation extension point to create the object

Once all the properties are gathered, the business object can be created. The **NewBOOperation** command creates the new object and poasts it into the destination.

In this example, use the **operation** extension to call **NewBOOperation**.

Add the following to the **operation** extension point in your project's **plugin.xml** file.

```
<extension point="com.teamcenter.rac.ui.commands.operation">
...
  <operation
class="com.teamcenter.rac.ui.commands.create.bo.NewBOOperation"
      id="com.teamcenter.rac.ui.commands.NewItemOperation"
      wizardId="com.mycom.newwizard.CustomObjectWizard">
  </operation>
...
</extension>
```

Customize the rich client properties files

You can customize the appearance of applications in the rich client by creating user properties files that override the default properties files. The default properties files, with a **.properties** extension, are located in the various **com.teamcenter.rac.component.jar** files in the **TC_ROOT\portal\plugins** directory. You override the default file by creating a **_user.properties** file and wrapping it in an Eclipse plug-in. For example, if you want to override a property in Manufacturing Process Planner, you must override the **mpp.properties** file located in the **com.teamcenter.rac.cme.legacy.jar** file with the **mpp_user.properties** file.

1. Create the Java project.
 - a. In Eclipse, choose **File→New→Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Then click **Next**.
 - c. In the **Project name** box, type the project name, which should be in the form of **com.mycom.project-name**. For example, type **com.mycom.propertiesfile**. Click **Next**.
 - d. Under **Options**, ensure the **Generate an activator** and **This plug-in will make contributions to the UI** check boxes are selected. Click **Next**.
 - e. Clear the **Create a plug-in using one of these templates** check box. Click **Finish**.

2. Modify the properties file.

- a. Navigate to the `TC_ROOT\portal\plugins` directory, and open the JAR file that contains the properties file you want to override. The file name has the form *property.properties*. For example, to override the **mpp.properties** file, open the **com.teamcenter.rac.cme.legacy.jar** file.
- b. Extract the properties file you want to override from the JAR file. For example, extract the **mpp.properties** file.

Note:

A JAR file uses ZIP compression and can be opened using standard unjar or unzip tools.

- c. In a text editor, open the properties file to determine the syntax of the property you want to modify.
- d. Create a new *property_user.properties* in a text editor. For example, if you open the **mpp.properties** file, create the **mpp_user.properties** file.
- e. Type the entries you want to use to override the defaults.
- f. Save and exit the file.

3. Update the project tabs.

- a. In Eclipse, click your project tab and click its **Dependencies** tab.
- b. Under **Required Plug-ins**, click the **Add** button.
- c. Select the following plug-ins from the list by holding down the Ctrl key while you click them:
 - **com.teamcenter.rac.aifrcp**
 - **com.teamcenter.rac.common**
 - **com.teamcenter.rac.external**
 - **com.teamcenter.rac.kernel**
 - **com.teamcenter.rac.neva**
 - **com.teamcenter.rac.tcapps**
 - **com.teamcenter.rac.util**

- d. Click **OK**.
- e. Click your project's **Runtime** tab.
- f. Under **Exported Packages**, click the **Add** button.
- g. Select your project and click **OK**.
- h. Click your project's **Extensions** tab.
- i. Click the **Add** button.
- j. Select the **com.teamcenter.rac.util.tc_properties** extension point.
- k. Click **Finish**.
- l. Click the **plugin.xml** tab. Type text in the file so it looks like the following:

```
<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.6"?>
<plugin>
  <extension
    point="com.teamcenter.rac.util.tc_properties">
  </extension>
</plugin>
```

If the text is different, edit it in the tab to resemble the example.

- m. Save the project by choosing **File→Save All**.
4. Create a package.
 - a. In the **Package Explorer** view, right-click your project and choose **New→Package**.
 - b. In the **Name** box, type the path name where the properties file was originally. For example, if you originally extracted the **mpp.properties** file, the package name should be **com.teamcenter.rac.cme.mpp**.
 - c. Click **Finish**.
 - d. From Windows Explorer, drag your modified *property_user.properties* file and drop it on the package you created in **Package Explorer**.
 - e. In **Package Explorer**, select **plugin.xml**.
 - f. Click the **MANIFEST.MF** tab and type your new package at the end of the **Export-Package** line. For example, if your project name is **com.mycom.propertiesfile** and the new package is called **com.teamcenter.rac.cme.mpp**, the line should read:

```
Export-Package: com.mycom.propertiesfile, com.teamcenter.rac.cme.mpp
```

The **MANIFEST.MF** file should look like the following:

```
Manifest-Version: 1.0
Bundle-ManifestVersion: 2
Bundle-Name: Propertiesfile
Bundle-SymbolicName: com.mycom.propertiesfile;singleton:=true
Bundle-Version: 1.0.0.qualifier
Bundle-Activator: com.mycom.propertiesfile.Activator
Bundle-Vendor: MYCOM
Require-Bundle: org.eclipse.ui,
    org.eclipse.core.runtime,
    com.teamcenter.rac.aifrcp;bundle-version="8000.2.0",
    com.teamcenter.rac.common;bundle-version="8000.2.0",
    com.teamcenter.rac.external;bundle-version="8000.2.0",
    com.teamcenter.rac.kernel;bundle-version="8000.2.0",
    com.teamcenter.rac.neva;bundle-version="1.0.0",
    com.teamcenter.rac.tcapps;bundle-version="8000.2.0",
    com.teamcenter.rac.util;bundle-version="8000.2.0"
Bundle-RequiredExecutionEnvironment: JavaSE-1.6
Bundle-ActivationPolicy: lazy
Export-Package: com.mycom.propertiesfile, com.teamcenter.rac.cme.mpp
```

- g. Click your project's **Runtime** tab. If one or more of the items listed in the **Export-Package** line in the previous step are missing, add the missing ones by clicking the **Add** button, selecting the missing packages, and clicking **OK**.
5. Save and export your changes.
6. Debug in Eclipse.
 - a. In Eclipse, choose **Run**→**Debug Configurations**.
 - b. In the **Debug** dialog box, under **Java Application** on the left-hand side, select the configuration you want to debug.
 - c. At the bottom of the dialog box, click the **Debug** button.
This launches the application in debug mode. To change the perspective to the debug perspective, choose **Window**→**Open Perspective**→**Debug**.

You can then debug your application.

7. Run the **genregxml** script.
If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded when the rich client starts. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory. The **RegistryLoader** file is added during rich client startup.

- a. Delete the old registry loader file:

```
TC_ROOT\portal\registry\RegistryLoader.xml.gz
```

- b. Run the following script:

```
TC_ROOT\portal\registry\genregxml
```

When the script is finished, a new **RegistryLoader.xml.gz** file is created.

8. Verify the customization.

4. Rich client customization using Eclipse plug-ins

Basic concepts about rich client customization	4-1
Understanding the Eclipse rich client platform framework	4-1
What reference material is available?	4-2
What are perspectives and views?	4-2
Process for creating rich client customizations	4-2
Introduction to SWT	4-3
Create the Eclipse rich client development environment	4-3
Ensure your customizations appear	4-5
The rich client and Eclipse 4 migration	4-6
Eclipse 4 architecture overview	4-6
Teamcenter rich client and Eclipse 4 strategy	4-7
Important changes in Eclipse 4	4-9
Creating UI components using Eclipse 4 model fragments	4-11
Sample rich client customizations	4-16
Provided rich client customization examples	4-16
Example: Create a custom perspective	4-17
Example: Create a custom view	4-18
Example: Command handlers	4-19
Example: Important files and the viewer	4-20
Example: OSGi	4-20
Example: Other contents in the sample plugin project	4-21
Hide a view	4-21
Localize your customizations	4-22
Customizing Command Suppression	4-27
Additional examples for reference only	4-31
Distributing rich client customizations	4-76
Export your custom plug-in to the rich client	4-76
Process for distributing customizations to the rich client	4-77
Package custom rich client files	4-77
Create a feature file for rich client customizations	4-77
Rich client Javadoc	4-80
User interface components documented in Javadoc	4-80
User interface components in the com.teamcenter.rac.common package	4-80
User interface components in the com.teamcenter.rac.util package	4-95
Troubleshooting rich client customization	4-127
Common problems in rich client customization	4-127
Rich client debugging	4-128
Enabling client-side logging	4-131
Listener leaks	4-134

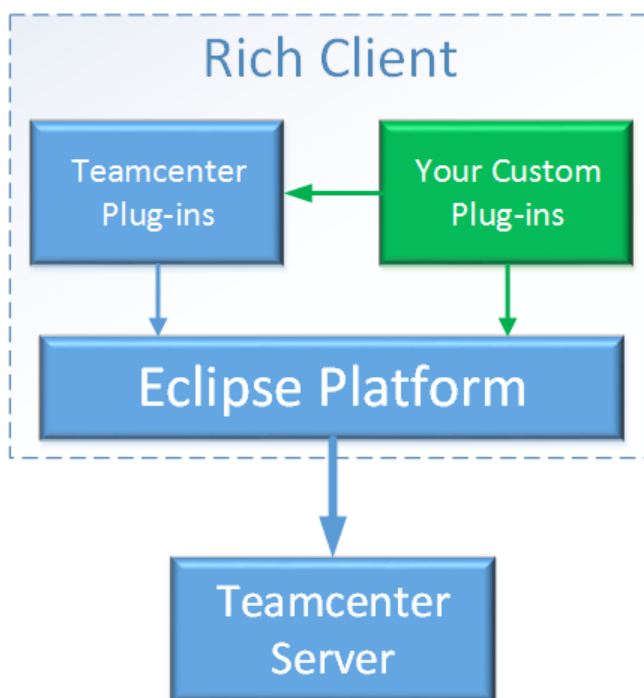
4. Rich client customization using Eclipse plug-ins

Basic concepts about rich client customization

Understanding the Eclipse rich client platform framework

When the rich client is installed, Java archive (JAR) files are installed in the `TC_ROOT\portal\plugins` directory. These files comprise the rich client and the resources required to run it. When you add a new custom plug-in, you deploy it into this directory.

The Teamcenter rich client is hosted within the Eclipse rich client platform (RCP) framework. The RCP is a general purpose application framework which provides strong support for modular and extensible component-based development through the use of plug-ins.



You can find more information about the RCP's features, advantages, and use at the Eclipse Web site:

<http://www.eclipse.org/>

For more information about using Eclipse, see the *Platform Plug-in Developer Guide*, which can be found at the following location:

<http://help.eclipse.org/indigo/index.jsp>

To customize the rich client UI beyond the capabilities of style sheets, you need to use the Eclipse IDE.

What reference material is available?

There are two major sources of reference material available for rich client customization.

- The **JavaDoc** documents the rich client Java API.
- The **sample files** provide working examples of the most common types of customization.

What are perspectives and views?

Within the rich client user interface, application functionality is provided in *perspectives* and *views*.

View The basic display component that displays related information in a UI window.

Perspective A collection of one or more views and their layout.

Some applications use a perspective with multiple views to arrange how functionality is presented. Other applications use a perspective with a single view.

You can use the **HiddenPerspectives** preference to prevent the display of some Teamcenter perspectives in the rich client.

If your site has online help installed, you can access application and view help from the rich client **Help** menu or by pressing F1.

Process for creating rich client customizations

When you create a rich client customization, follow this general process to create, test, and distribute the customization:

1. Set up your Eclipse environment for rich client customization.
2. Create the desired customization.
3. If you create a custom rich client plug-in, export the plug-in to the `TC_ROOT\portal\plugins` directory or a shared directory.
4. Clear the cache and register any new plug-in to ensure the customization appears in the rich client.
5. Run the rich client to verify the customization.
6. After testing is successful, distribute the customization to your organization.

Introduction to SWT

The Standard Widget Toolkit (SWT), maintained by the Eclipse Foundation, is a graphical toolkit for use with the Java platform. It is an alternative to the Abstract Window Toolkit (AWT) and Swing Java toolkits.

For more information about SWT, see the following URL:

<http://www.eclipse.org/swt/>

Because the rich client already uses SWT, the necessary files are already loaded into your Eclipse environment from `TC_ROOT\portal\plugins` when you defined the target platform. Therefore, when you create a Java file, you can use the **`import org.eclipse.swt.widgets`** command to include SWT in your coding.

Note:

Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens Digital Industries Software encourages you to customize Teamcenter using SWT/JFace components. Siemens Digital Industries Software will discontinue Swing/AWT support in a future version.

To become familiar with SWT, you can download SWT samples and run the SWT tutorial in Eclipse.

Create the Eclipse rich client development environment

You must use the Eclipse plug-in development environment in order to do advanced rich client customization. The prerequisites are:

- The Teamcenter rich client must be installed.
- Java must be installed.

The following steps are required to customize the rich client with Eclipse:

1. **Set up Eclipse.**
2. **Set the Eclipse preferences.**
3. **Launch the rich client from Eclipse.**

For the supported versions of Eclipse and Java, see the **hardware and software certifications** page on GTAC.

Set up Eclipse

1. Install the Eclipse IDE with the Plug-in Development functionality.

2. Create a batch file that sets the Teamcenter environment, starts the server, and launches Eclipse using the JDK command line parameters.

Use the following template to create your batch file:

```
set FMS_HOME=TC_ROOT\tccs
set JAVA_HOME=jre-install-directory
set JRE_HOME=jre-install-directory
set CLASSPATH=TC_ROOT\portal
set PATH=%FMS_HOME%\bin;%FMS_HOME%\lib;TC_ROOT\portal;%PATH%
start "TAO ImR" /min cmd /c "TC_ROOT\iiopservers\start_imr.bat"
Eclipse-install-directory\eclipse.exe -vm jdk-install-directory\bin\javaw
```

You can use the `TC_ROOT\portal\portal.bat` file as an example for this batch file.

3. Run the batch file you just created to launch Eclipse in a Teamcenter environment.

Set the Eclipse preferences

1. Verify that the supported Java Runtime Environment (JRE) version is listed and checked.

Find this option in **Window**→**Preferences** under the **Java** node, in the **Installed JREs** section.

2. Add the Teamcenter rich client as a target platform.

Find this option in **Window**→**Preferences** under the **Plug-in Development** node, in the **Target Platform** section.

Choose the `TC_ROOT\portal` as the target directory.

In the **Target Platform** dialog box, ensure the new target is checked..

Launch the rich client from Eclipse

1. Configure the **Debug Configurations** to launch the rich client.
2. Add the **VM arguments** for memory in the **Arguments** tab.

```
-Xmx1g -XX:MaxPermSize=512m
```

3. After applying the changes, launch the rich client.

In the **Debug Configurations** window, click **Debug**.

The rich client should launch to the **Login** page, and you should be able to log in normally.

Ensure your customizations appear

You can run the rich client from Eclipse, and thereby test if your customizations appear in the rich client.

But running your customization from Eclipse is not a true test of whether your customizations will work in an end user's rich client installation. After you create a customization, you must perform the following steps to ensure it appears in the rich client when it is run outside the Eclipse IDE.

You must perform these steps whenever you create a new plug-in to add to the rich client, which should be in most cases of rich client customization (with the exception of style sheets). You must always wrap your rich client customizations in a plug-in to ensure that the out-of-the-box rich client is unchanged, thereby maintaining its integrity the next time you upgrade to a newer version of Teamcenter.

Note:

If you customize the out-of-the-box rich client files, you can lose those customizations the next time you upgrade.

1. **Export your custom plug-in** from Eclipse to the rich client.
2. Run the `TC_ROOT\portal\registry\genregxml.bat` file to register new plug-ins and other changes with the rich client.

When the script is finished, a new `TC_ROOT\portal\registry\RegistryLoader.xml.gz` file is created.

Note:

If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory.

3. Ensure that the **-clean** and **-initialize** arguments are added to the **portal.bat** file (between the **start Teamcenter.exe** command and **%***).

Run the **portal.bat** file to launch the rich client to verify that your customizations appear. To confirm that the plug-in is registered in the rich client, choose **Help→About→Installation Details** and search the list to see the registered plug-ins.

4. If your customizations still do not appear, to clear the cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

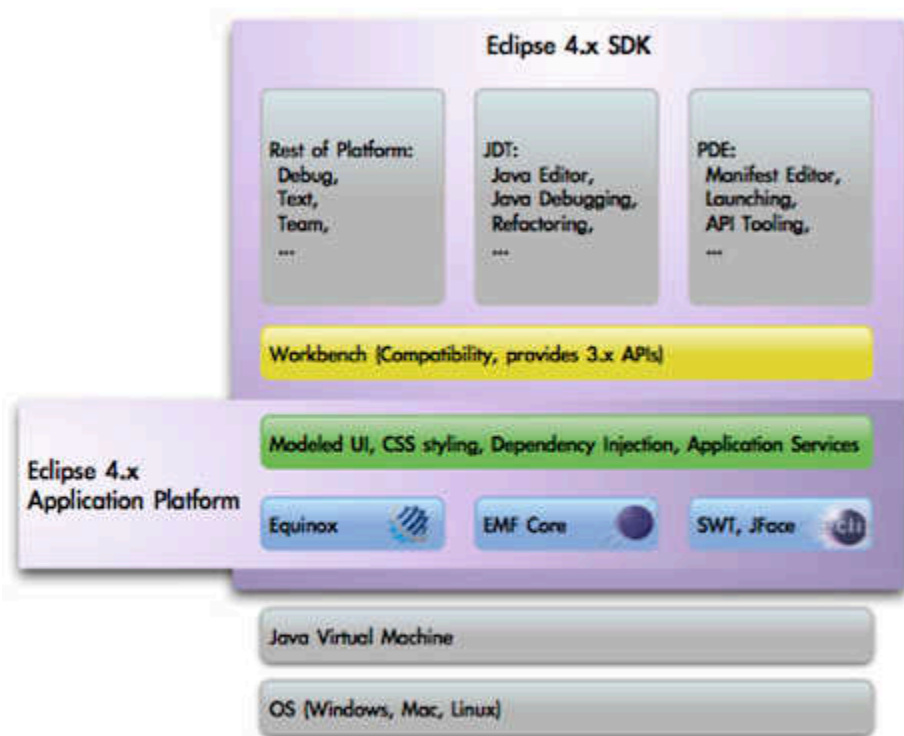
On a Windows client, it is typically the `%HOMEDRIVE%%HOMEPATH%\Teamcenter` directory. On a Linux client, it is typically the `$HOME/Teamcenter` directory.

If you delete this directory, the last state of the rich client is lost, and the user interface appears as it does at initial startup.

The rich client and Eclipse 4 migration

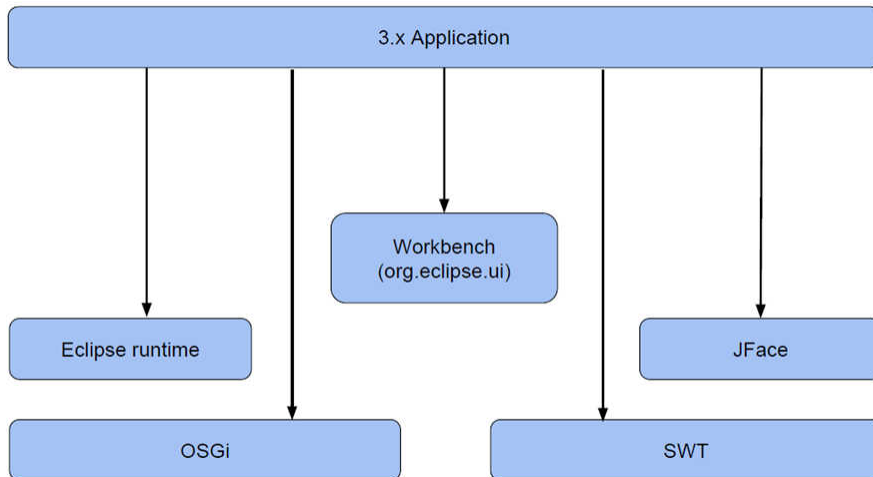
Eclipse 4 architecture overview

Following is a high-level overview of the Eclipse 4.x SDK architecture. Most of the functionality is very similar to the Eclipse 3.x platform.

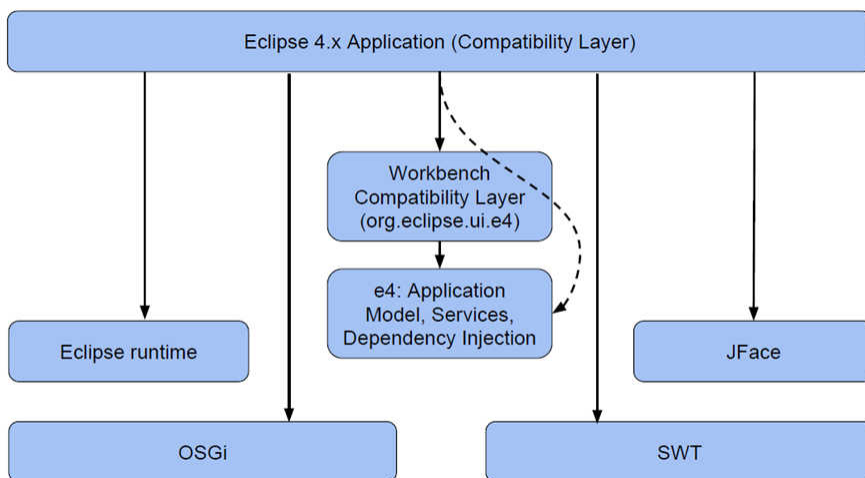


Comparing Eclipse 3 to Eclipse 4

The Eclipse 4 Application Platform (E4AP) differs from the Eclipse 3.x platform in the implementation of the Workbench (i.e. **org.eclipse.ui.workbench plugin**), and the technologies this new implementation is based on.



On top of E4AP, the 4.0 Workbench offers an implementation of the 3.x Workbench APIs, called the compatibility layer, to provide backwards compatibility for existing well-behaved Eclipse 3.x applications.



Teamcenter rich client and Eclipse 4 strategy

Compatibility layer

Due to the length of time the Teamcenter rich client has been available and the volume of customer customizations based on it, the transition from E3 to E4 will take advantage of E4's Compatibility Layer. One important criteria for existing applications to work well on the compatibility layer is that they should not use any internal workbench API. Aside from this, there should be no source code changes required. This is especially true for existing applications, as it requires almost no migration effort. Additionally, the Eclipse IDE itself is still based on the E3.x API, so if you extend it, you automatically need to use the compatibility layer. If your application uses 3.x internal API, you may have to adjust your code as this code might have been changed.

Use the E4 application model

You still might want to use some of the benefits of the Eclipse 4 programming model, especially for newly developed components and migrate existing Eclipse 3.x RCP application to E4 in stages. This approach allows you to develop new parts of the application using the benefits of the Eclipse 4 programming model as well as reuse existing components. The Teamcenter rich client integrates Eclipse 4 components to create a rich client specific application model used by the compatibility layer, register it as the application model of the rich client, and add new E4 components to it. This allows contributing E4 API-based views, menus, and toolbars to the rich client application. The relevant model in the Teamcenter rich client, `application.e4xmi`, can be found in the `configuration_12000.0.0` libraries. There are some limitation to do so from rich client customizations due to mixed hybrid application, but you still can leverage E4 application model to learn and evaluate E4, planning your migration for a future Teamcenter release. Furthermore, your newly developed E4 components based upon the Teamcenter 12 application model can be integrated into a pure E4 application without any adaptations.

No impacts on extension point

Teamcenter 12 uses the 3.x compatibility layer along with the new E4 application model. Technically, there is no impact on existing customization code to use Eclipse 3.X extension point to define perspective, view, command, command expression, handler, menucontribution. The extension points introduced by previous versions of Teamcenter still work in Teamcenter 12.0. In addition, you can contribute new views, commands, handlers through model fragment using the new E4 way or by migrating your old extension points to model fragments.

Eclipse internal code change

- **The presentation layer has been removed**

The package `org.eclipse.ui.presentations` in `org.eclipse.ui.workbench` plugin and the `org.eclipse.ui.presentationFactories` extension point have been removed in Eclipse 4.6.

Per Eclipse, using the presentation API and extension point to customize the workbench appearance will no longer have any effect. Clients are encouraged to try out the provisional new API in Eclipse 4.X for performing equivalent workbench customization. Complete rendering control can be achieved by supplying an `org.eclipse.e4.ui.workbench.IPresentationEngine`. Customization of fonts, spacing, and color can be achieved by supplying custom CSS style sheets via the `org.eclipse.e4.ui.css.swt.theme` extension point. Customers using internal `org.eclipse.ui.*` APIs need to refactor code to remove API dependency and find replacement. Other internal APIs may not be guaranteed to work in Eclipse 4.X platform. Test your code before refactoring.

- **Certain Eclipse plug-ins have been removed**

The following Eclipse plugins have been removed in Eclipse 4.6:

```
org.eclipse.core.runtime.compatibility
org.eclipse.update.core
org.eclipse.update.ui
```

See complete porting guides from Eclipse 3.7 to Eclipse 4.6 for API deprecations removal information.

Important changes in Eclipse 4

For convenience, following is a list of commonly used patterns that have changed.

visibleWhen checkEnabled

In Teamcenter 11.X based upon Eclipse 3.8 RCP, we could use `checkEnabled="true"` in the `<visibleWhen>` clause for `<menuContribution>` to determine the visibility based upon enabled state of the command, for example `handler.isEnabled()`. This is specific to Eclipse 3.X-based API and is no longer supported in Eclipse 4.X.

This will always return false in an Eclipse 4 application with the compatibility layer. In E4, handlers can be installed on parts, windows, as well as globally on the `MApplication`. Enablement expressions are now handled by the handler class itself through `@CanExecute` methods. To workaround this issue in the compatibility layer, please add `enabledWhen` or `activeWhen` conditions in your handler extension point in the `<visibleWhen>` block.

See example Customer Command 2 in the examples plugin for reference. Use Case: We contribute "Custom Command 2" to File menu. The command should be visible and enabled only when Customer List View is selected and in Customer Example perspective. If Customer Overview is selected, this command should be removed from File menu.

Fast view has been removed

Eclipse 4.x no longer supports the Fast View functionality, so the **Show View as Fast View** button has been removed from Teamcenter 12. Views can still be opened using **Window→Show View**.

Save Perspective As event

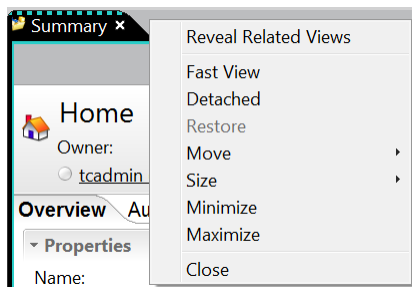
In Teamcenter rich client based upon Eclipse 3.8 RCP, Eclipse fires the `org.eclipse/ui/Perspective/SavedAs` event for **Save Perspective As**. Now in the Teamcenter rich client based upon Eclipse 4.6, it now fires the `org.eclipse/e4/ui/LifeCycle/perpSaved` event.

The `org.eclipse/ui/Perspective/SavedAs` event will not work in Teamcenter 12. If you are doing some customization for save perspective, use the `org.eclipse/e4/ui/LifeCycle/perpSaved` listen event instead.

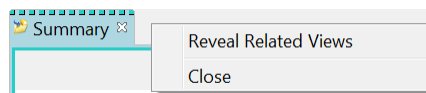
Context Menus on the View Tab

Eclipse 4 removes many of the context menus from the view tabs.

Rich client on Eclipse 3



Rich client on Eclipse 4



Following are the menus removed and the Eclipse 4 alternative, if available:

Fast View

No Alternative.

Detached

Drag and drop the view outside the application.

Move

Drag and drop the view inside the application.

Size

No Alternative.

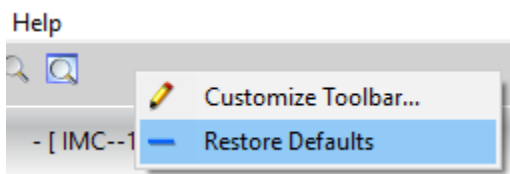
Minimize, Maximize, and Restore

Use the view control buttons in the top right of the view.



Coolbar is not supported

The CoolbarManager2 from Eclipse 3 has been replaced by the CoolbarToTrimManager in Eclipse 4. Creation of the context menu and addint at a particular location is now done through the EModel Service.



CommandContributionItem

In Eclipse 3, the Menu Manager returned a CommandContributionItem. In Eclipse 4, it returns a HandledContributionItem instead. An example of this, named **HandledContribution Item Example**, is in the [provided sample customization](#).

Creating UI components using Eclipse 4 model fragments

Beginning with Teamcenter 12, the rich client runs on the Eclipse 4.6 platform utilizing the Eclipse 3.x compatibility layer. The rich client registers an application model in order to leverage some E4 features. Following are the new programming models used in the rich client.

The E4 application model

The E4 application model is used to describe the structure of an application. This is similar to the DOM of a web page that describes the layout and structure of the user interface. This application model contains the visual elements as well as some non-visual elements of the application.

Components

The visual components are windows, parts, menus, toolbars, and so on. Non-visual components are handlers, commands, key bindings and expression.

Model elements

Each model element has attributes which describe its current state, like the size and the position of a window. The application model also expresses the relationship of the model elements via a hierarchy.

Widgets

The individual user interface widgets, which are displayed in a part, are not defined via the application model. The content of the part is still defined by your source code.

Model

The model itself is created and maintained using EMF, and uses EMF-style patterns for creation and adding elements to containers. The base of the application model is typically defined as a static file. By default, it is called **Application.e4xmi** and is located in the main directory of the plug-in which defines the product extension. This file is read at application startup and is used to construct the initial application model. Changes made by the user are persisted and re-applied at startup.

E4 tools

The E4 tools provide UI tooling to develop and Eclipse 4 RCP application. Parts, commands, and handlers can be added into the application model through the *Application Model Editor*. After opening eclipse, use the update manager to install **Eclipse e4 Tools Developer Resources**, once you double click the application model, the Application Model Editor will be opened in your IDE.

Model fragments

Siemens Digital Industries Software does not recommend modifying the OOTB Teamcenter Application Model to add a new customized view implemented in custom plugin.

In order to statically contribute a new part using the E4 pattern, you contribute to the Application Model in a custom plugin using a *model fragment*. A model fragment statically specifies model elements and the location in the application model to which they should be contributed. In these fragments you contribute to an existing model element using model fragment extension points. A model fragment file

typically uses the **.e4xmi** extension. In the model fragment editor, specify the ID of the element in the application model you are contributing to.

Dependency injection

Dependency injection separates the configuration of an object from its behavior.

The Eclipse 4 Application Platform has adopted the use of Dependency Injection (DI) to circumvent several issues:

- Code frequently used global singleton accessors, like **Platform** and **PlatformUI**, or required navigating a deep chain of dependencies.
- Client code needed to know the internals of the Eclipse code base.
- The use of singletons tightly coupled the consumers of *things* to their producer or provider, which inhibited reuse.

Rather than require client code to know how to access a service, the client instead describes the service required, and the platform is responsible for configuring the object with an appropriate service. DI shields the client code from knowing the origin of the injected objects. With DI, one object supplies the dependencies of another object.

A dependency is an object that can be used (a service).

An injection is the passing of a dependency to a dependent object (a client) that would use it.

Example:

In Eclipse 3, a view accessed the Eclipse Selection Service using the PlatformUI singleton.

```
public class DetailsView extends ViewPart {

    @Override
    public void createPartControl(Composite parent) {
        //create tableViewer...

        final ISelectionService ss = getSite().getWorkbenchWindow().getSelectionService();
        ss.addPostSelectionListener(myListener);
        myListener.selectionChanged(null, ss.getSelection());
    }

    protected void setSelection(Contact contact) {
        // ... do something with the selection
    }

    private final ISelectionListener myListener = new ISelectionListener() {
        @Override
        public void selectionChanged(IWorkbenchPart part, ISelection selection) {
            if (!(selection instanceof IStructuredSelection)) {
                setSelection(null);
                return;
            }
        }
    }
}
```

```

    final IStructuredSelection ss = (IStructuredSelection) selection;
    if (ss.isEmpty() || !(ss.getFirstElement() instanceof Contact)) {
        setSelection(null);
        return;
    }
    setSelection((Contact) ss.getFirstElement());
}
};
}

```

Example:

In Eclipse 4, you can get the activeSelection without accessing the selectionService by defining a **DetailsView**.

```

public class DetailsView {

    @Inject
    public DetailsView(Composite parent) {
        //create tableViewer
        ...
    }

    @Inject
    public void setSelection( @Optional @Named(IServiceConstants.ACTIVE_SELECTION)
Contact
    contact) {
        // ... do something with the selection
    }
}

```

In Eclipse 3.x you have a Java method chain.

```
getSite().getWorkbenchWindow().getSelectionService()
```

E4 uses a magic class or string.

```
@Optional @Named(IServiceConstants.ACTIVE_SELECTION).
```

Annotations

Some of E4's injectors are based upon standard JSR 330 annotations. Additional annotations, such as **@Optional** or **@Persist**, are specific for Eclipse 4.

To get an overview of commonly used annotations, the following list shows all described annotations with the bundle defining them. If you use any of these annotations, you will need a dependency or a package import to these bundles.

Annotation	Bundle
@Active	org.eclipse.e4.core.contexts
@Creatable	org.eclipse.e4.core.di.annotations

Annotation	Bundle
@CanExecute	org.eclipse.e4.core.di.annotations
@Execute	org.eclipse.e4.core.di.annotations
@Inject	javax.inject
@Named	javax.inject
@Optional	org.eclipse.e4.core.di.annotations
@Persist	org.eclipse.e4.ui.di
@PersistState	org.eclipse.e4.ui.di
@PostConstruct	javax.annotation
@ProcessAdditions	org.eclipse.e4.ui.workbench.lifecycle
@ProcessRemovals	org.eclipse.e4.ui.workbench.lifecycle
@PostContextCreate	org.eclipse.e4.ui.workbench.lifecycle
@PreDestroy	javax.annotation
@PreSave	org.eclipse.e4.ui.workbench.lifecycle
@Singleton	javax.inject
@Focus	org.eclipse.e4.ui.di
@AboutToShow, @AboutToHide	org.eclipse.e4.ui.di
@EventTopic	org.eclipse.e4.core.di.extensions
@UIEventTopic	org.eclipse.e4.ui.di

CSS

Eclipse 4 provides an easy way to style the UI widgets using CSS. The new css declarative styling support provides developers with the flexibility of styling their user interface based on a set of properties defined within a CSS stylesheet. CSS selectors used in Eclipse are identifiers, which relate to widgets or other elements, for example predefined pseudo classes. E4 CSS isn't HTML CSS, E4AP has its own CSS rendering engine which can parse eclipse specific CSS Selector.

CSS style sheets can be used to modify SWT widget properties. The SWT widget class is used as an element type, such as **Shell**, **Button**, **Table**. Eclipse publishes a **table** to show the mapping between CSS properties defined for SWT widgets and SWT widget calls.

Model elements of your Eclipse application, like **MPartStack**, **MPart**, or **MTrimbar**, can also be used as selectors for styling. The CSS selectors are based on the Java classes for the model elements.

Example:

You can hide the minimize and maximize button of a **MPartStack** via the following CSS rule.

```
.MPartStack {
    swt-maximize-visible: false;
```



```
swt-minimize-visible: false;
}
```

Example:

You can assign a CSS ID tag on SWT widget class directly. In CSS, define the **RACPerspectiveHeader** selector.

```
#RACPerspectiveHeader {
    color: #004664;
    background-color: #FFFFFF;
}
```

Example:

In the **RACPerspectiveHeader.java**, set this selector codefully.

```
public class RACPerspectiveHeader
    extends Composite
{
    public RACPerspectiveHeader( final Composite parent, final int style )
    {
        super( parent, style );

        m_parentComposite = parent;

        GridLayout gridLayout = new GridLayout( 4, false );
        gridLayout.marginBottom = 1;
        gridLayout.marginHeight = 0;
        setLayout( gridLayout );

        GridData gd = new GridData();

        gd.minimumWidth = 2048;

        setLayoutData( gd );

        setData( ICSSConstants.ECLIPSE_CSS_ID,
            ICSSConstants.ID_RACPerspectiveHeader );

        init();

        ...
    }
}
```

Example:

Some SWT widget states are captured in pseudo selectors.

```
Button:checked {
    background-color: #FF0000;
}
```

Example:

The style bits, normally passed through the constructor, are available through the style attribute.

```
Button[style~='SWT.CHECK']{
    color: #E2E2E2;
}
```

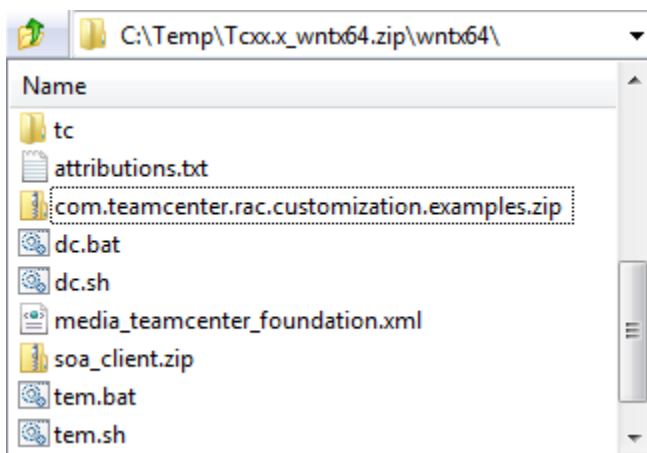
Sample rich client customizations

Provided rich client customization examples

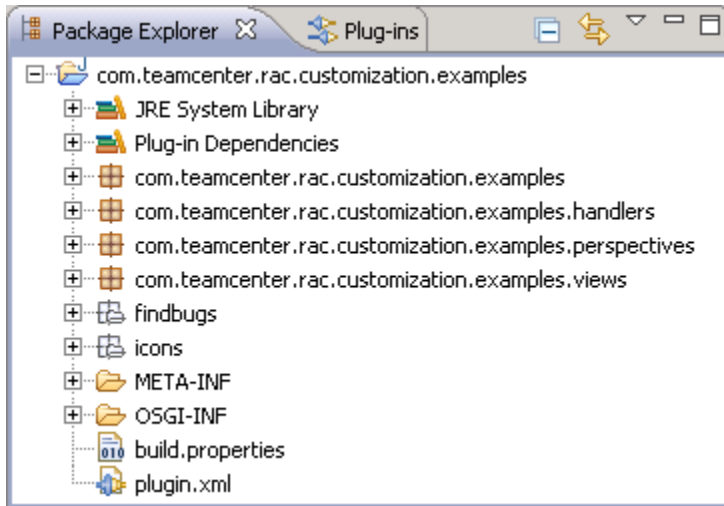
You can use the provided rich client customization example project as a guide to how typical customizations are accomplished.

com.teamcenter.rac.customization.examples.zip

This project is found in the Teamcenter installation zip file specific to your operating system, *Tcxx.x_wntx64.zip* or *Tcxx.x_lnx64.zip*.



Import this project into your Eclipse rich client platform (RCP) plugin development environment to see the examples in action.



The core of any plugin is the **plugin.xml** file. This contains the information needed to connect the custom code into Teamcenter.

Example listing

Following are some of the examples provided:

- Create a custom perspective (application).
- Create custom views and hide views.
- Add custom menu contributions including **File→New**.
- Add an entry to the **Sent To** menu.
- Permanently remove an existing menu contribution.
- See Teamcenter's use of OSGI services.

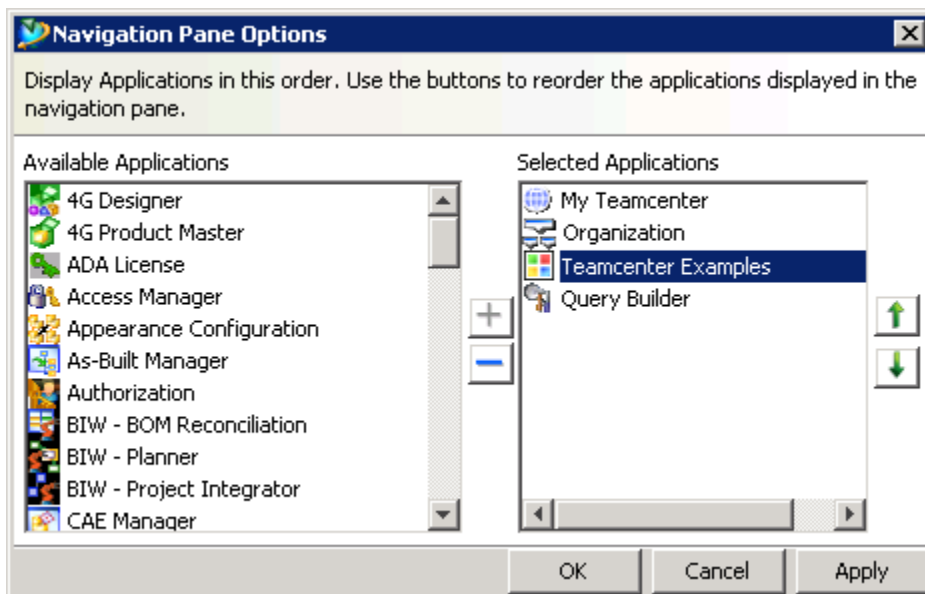
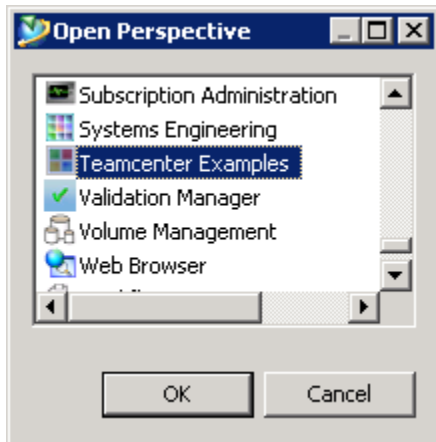
Example: Create a custom perspective

You might want to create your own perspective to the rich client. A perspective is also called an application, and is a collection of related views. When referring to an existing view, you must not use `viewDef`, since the view is already defined, use `viewRef` instead.

Included with the provided sample project is the **perspectives** package. This package contains a custom perspective which is the focus of many of the other customizations provided.

com.teamcenter.rac.customization.examples.perspectives

The definition for a custom perspective, **Teamcenter Examples**, is contained in the *TeamcenterExamplePerspective.java* class. You can examine it in the rich client by choosing **Window**→**Open Perspective** or by opening it with the navigation pane.



This custom perspective implements two layout interfaces and extends the **AbstractRACPerspective** class. It also loads custom views.

Examine the source code to see more details.

Example: Create a custom view

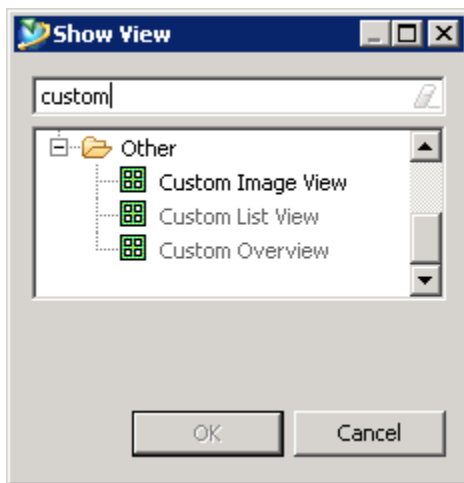
You might want to create your own views in the rich client. Included with the provided sample project is the **views** package. This package contains custom views which demonstrate the various ways to display information and also how to assign icons to the views.

com.teamcenter.rac.customization.examples.views

The definitions for several custom views are contained in the following classes.

- CustomImageView.java
- CustomListView.java
- CustomOverview.java

You can examine them in the rich client by choosing **Window**→**Show View** or by opening the provided perspective.



Custom Image View	Demonstrates a custom image viewer, which extends the <i>Thumbnail2DFileView</i> class.
Custom List View	Demonstrates a custom list viewer, which extends the <i>AbstractContentViewPart</i> , and implements the <i>IContentView</i> interface.
Custom Overview	Demonstrates a custom list viewer, which extends the <i>AbstractContentViewPart</i> , and implements the <i>IContentView</i> interface.

Examine the source code to see more details.

Example: Command handlers

com.teamcenter.rac.customization.examples.handlers

When creating custom commands, you also need custom handlers. Several handlers have been provided as examples. Match them with the commands that have been provided.

Some of the examples extend the **AbstractHandler** class, and some extend the **NewBOHandler** class.

The abstract handler

The basic Eclipse handler class that you extend for nearly all commands.

The business object handler

A Teamcenter provided handler class specifically for working with the New Business Object wizard.

Example: Important files and the viewer

The dynamic nature of Eclipse plugins is controlled by several important files. The sample plugin contains the following:

- MANIFEST.MF
- plugin.xml
- build.properties
- component_overview.xml
- OpenServiceComponent.xml

To examine any of these files, double-click them to open the viewer. Opening the **plugin.xml** file in the viewer displays several tabs, showing the majority of the configuration for the plugin on a single location. Changes made on the **Extensions** tab, for example are reflected in the **plugin.xml** file, and vice versa. Using the viewer instead of manually editing is a good way to help validate your configuration.

Take the time to examine the **Extensions** and **Dependencies** tabs, as they contain the most information.

Example: OSGi

An OSGi (Open Service Gateway Initiative) service is a Java object. It is registered in the Service Registry under the name of a Java interface. Unlike extensions, which are scanned and registered during start-up, services are not automatically registered. To register a service, you must create a Component Definition in the project's OSGi folder, and then make a call to the OSGi API to register the object as a service.

You will find several OSGi service examples in the sample plugin project.

component_overview.xml

Implemented in the **DisplayObjectInOverview** class. See that source file for details.

OpenServiceComponent.xml

Implemented in the **CustomizationExampleOpenService** class. See that source file for details.

These files define the services used in the sample plugin project.

Example: Other contents in the sample plugin project

You must examine the plugin project to get the full details of all the samples provided. Many of them will only appear when you have the **Teamcenter Examples** perspective open.

Command contribution Menu, toolbar and popup examples are given.

Custom new business object wizard Two custom new business object wizards are available at **File→New** when the **Custom Overview** is active.

Permanently hidden existing command The **Help→Help Library** is permanently removed from the menu.

Hide a view

You can hide views based upon user context using the Command Suppression perspective in the rich client. However, in order to hide a view from all users at all times, you must modify the **plugin.xml** file of your Eclipse rich client plug-in project. The visibility is controlled using the **org.eclipse.ui.activities** extension point. By binding an empty activity to the view, the view is suppressed regardless of user context.

In the following example, an empty activity, **Hide Custom List View**, is created, and then attached to a **Custom List View** view.

```
<extension point="org.eclipse.ui.activities">
  <activity
    id="com.teamcenter.rac.customization.examples.hidecustomlistview"
    name="Hide Custom List View">
  </activity>
  <activityPatternBinding
    activityId="com.teamcenter.rac.customization.examples.hidecustomlistview"
    isEqualityPattern="true"
    pattern="com.teamcenter.rac.customization.examples/
      com.teamcenter.rac.customization.examples.view.CustomListView">
  </activityPatternBinding>
</extension>
```

The actual **name** and **id** used for the empty activity are not relevant, however it is recommended to choose something descriptive.

Localize your customizations

You can localize (present in different languages) the rich client by either modifying the **plugin.xml** file or the **application_locale.properties** file. Although modifying the properties is still supported, Siemens Digital Industries Software recommends that you modify the Eclipse **plug-in.xml** file instead.

1. Create the project.
 - a. In Eclipse, choose **File**→**New**→**Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Then click **Next**.
 - c. In the **New Plug-in Project** dialog box **Plug-in Project** pane, type *com.mycom.l10n* in the **Project name** box. Click **Next**.
 - d. In the **New Plug-in Project** dialog box **Content** pane, do the following:
 - A. Under **Options**, ensure the **This plug-in will make contributions to the UI** check box is selected.
 - B. Click the **No** button next to **Would you like to create a rich client application?**
 - C. Click **Next**.
 - e. Ensure the **Create a plug-in using one of these templates** check box is selected and select **Hello, World Command** in the list. Click **Finish**.
2. Create the project files.
 - a. Add the following line to the end of the **MANIFEST.MF** file:

```
Bundle-Localization: plugin
```

The **MANIFEST.MF** file should appear as follows:

```
Manifest-Version: 1.0
Bundle-ManifestVersion: 2
Bundle-Name: L10n
Bundle-SymbolicName: com.mycom.l10n; singleton:=true
Bundle-Version: 1.0.0.qualifier
Bundle-Activator: com.mycom.l10n.Activator
Bundle-Vendor: MYCOM
Require-Bundle: org.eclipse.ui,
    org.eclipse.core.runtime
Bundle-RequiredExecutionEnvironment: JavaSE-1.6
Bundle-ActivationPolicy: lazy
Bundle-Localization: plugin
```


- b. Create a **plugin.properties** file in the *com.mycom.l10n* project by selecting the project and choosing **File→New→File**.

The empty **plugin.properties** file appears directly under the *com.mycom.l10n* project at the same level as the **plugin.xml** file.

- c. Add the following name/value pairs to the **plugin.properties** file:

```
myMenuItem.label=My Hello world
myTooltip.tip=My Say hello world
myCommand.name=My Command
```

- d. Edit the **plugin.xml** file replacing hard-coded values with **%keyname** for the name/value pairs you typed in the **plugin.properties** file. For example:

```
label="%myMenuItem.label"
```

The **plugin.xml** file should appear as follows:

```
<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.6"?>
<plugin>
  <extension point="org.eclipse.ui.commands">
    <category
      name="Sample Category"
      id="com.mycom.l10n.commands.category">
    </category>
    <command
      name="%myCommand.name"
      categoryId="com.mycom.l10n.commands.category"
      id="com.mycom.l10n.commands.sampleCommand">
    </command>
  </extension>
  <extension point="org.eclipse.ui.handlers">
    <handler
      commandId="com.mycom.l10n.commands.sampleCommand"
      class="com.mycom.l10n.handlers.SampleHandler">
    </handler>
  </extension>
  <extension point="org.eclipse.ui.bindings">
    <key
      commandId="com.mycom.l10n.commands.sampleCommand"
      contextId="org.eclipse.ui.contexts.window"
      sequence="M1+6"
      schemeId="org.eclipse.ui.defaultAcceleratorConfiguration">
    </key>
  </extension>
  <extension point="org.eclipse.ui.menus">
    <menuContribution
      locationURI="menu:org.eclipse.ui.main.menu?after=additions">
      <menu
        label="%myMenuItem.label"
        mnemonic="M"
        id="com.mycom.l10n.menus.sampleMenu">
        <command
```

```

        commandId="com.mycom.l10n.commands.sampleCommand"
        mnemonic="S"
        icon="icons/sample.gif"
        id="com.mycom.l10n.menus.sampleCommand">
    </command>
</menu>
</menuContribution>
<menuContribution
    locationURI="toolbar:org.eclipse.ui.main.toolbar?after=additions">
    <toolbar
        id="com.mycom.l10n.toolbars.sampleToolbar">
        <command
            commandId="com.mycom.l10n.commands.sampleCommand"
            icon="icons/sample.gif"
            tooltip="%myTooltip.tip"
            id="com.mycom.l10n.toolbars.sampleCommand">
        </command>
    </toolbar>
</menuContribution>
</extension>
</plugin>

```

- e. Add a Spanish localization by creating a **plugin_es.properties** file at the same level as the **plugin.properties** file. The keys are identical in the **plugin_es.properties** file but have Spanish values.

```

myMenuItem.label=Mí Hola mundo
myTooltip.tip=Mí para Decir hola mundo
myCommand.name=Mí Orden

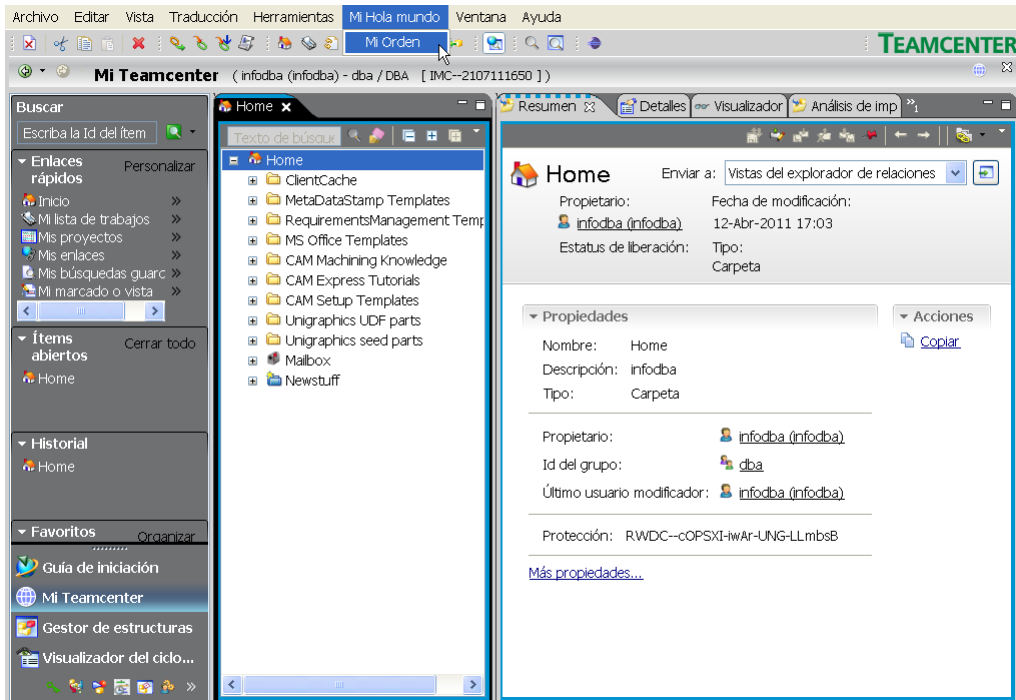
```

3. Verify the customization so far.

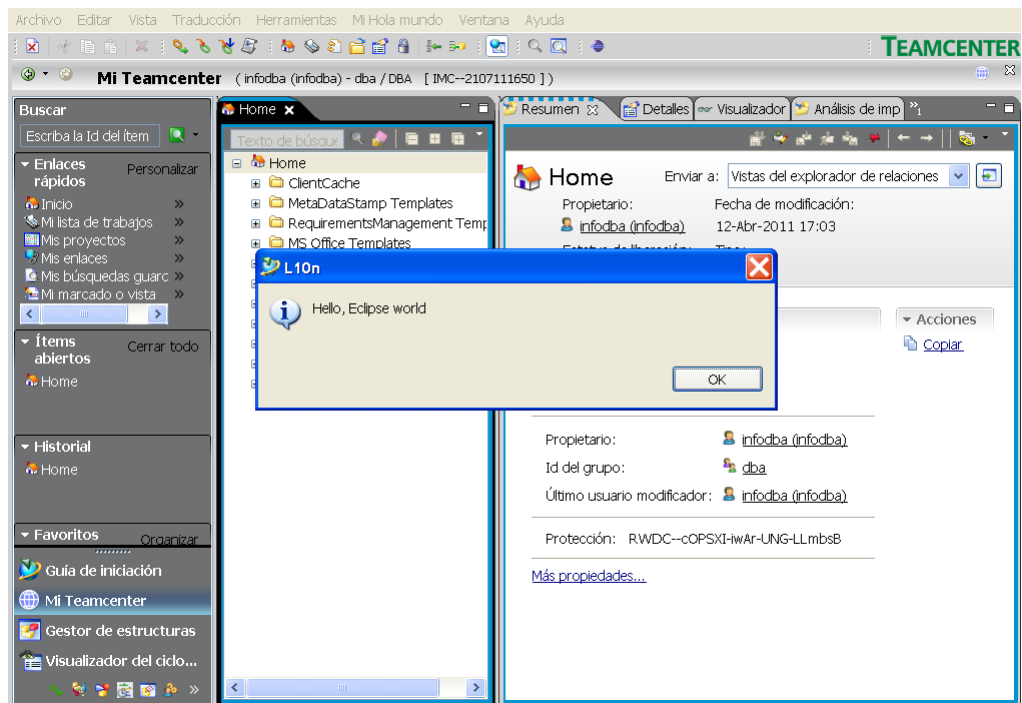
- a. Create a new configuration for the Spanish locale by choosing **Run→Run Configurations** (or **Run→Debug Configurations**) and specifying **-nl es** instead of **-nl \${target.nl}** on the **Arguments** tab. For example:

```
-os ${target.os} -ws ${target.ws} -arch ${target.arch} -nl es
```

- b. To verify the configuration so far, choose **Run→Run Configurations** to start the rich client from your Eclipse environment. Clear the workspace by checking **Clear** in the launch configuration dialog box.
- c. Choose **Mí Hola Mundo→Mí Orden** to run the custom command.



The UI displays the proper localization.



However, the message displayed by the custom command is not localized. You localize the message in the following step.

4. Localize the message.

- a. In Eclipse, choose **Source**→**Externalize Strings**.

This creates the **Messages.java** and **messages.properties** files in the *com.mycom.l10n.handlers* package.

- b. Create a **messages_es.properties** file at the same level as the **messages.properties** file, and add the following content to the new file:

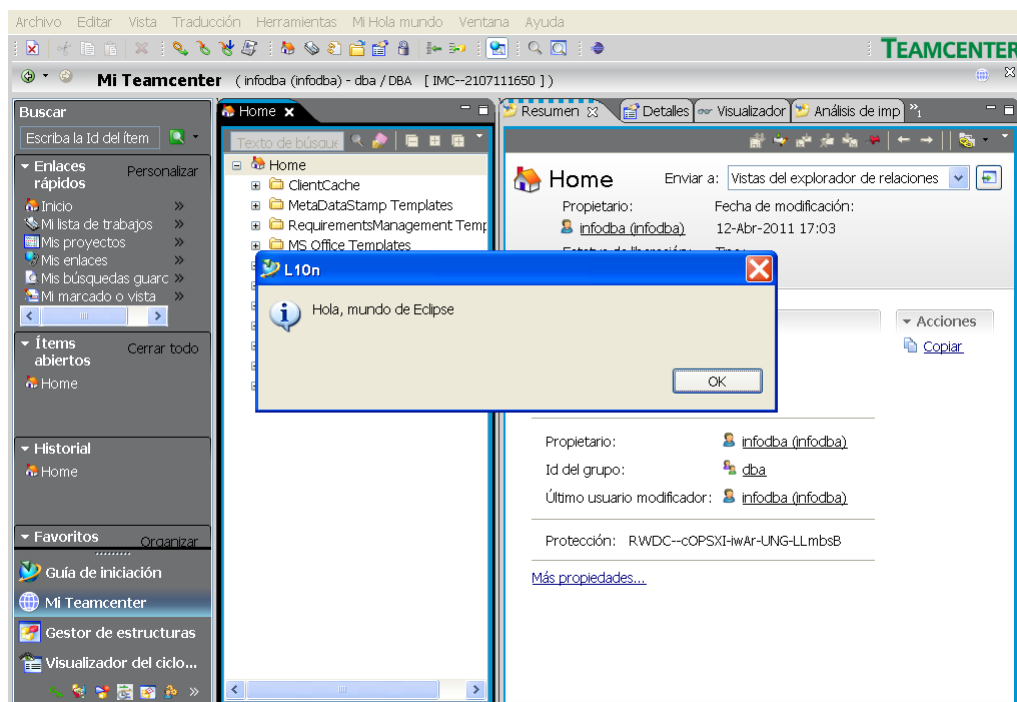
```
SampleHandler_0=L10n
SampleHandler_1=Hola, mundo de Eclipse
```

5. Verify the customization.

- a. Choose **Run**→**Run Configurations** and choose the Spanish locale to start the rich client from your Eclipse environment. Clear the workspace by checking **Clear** in the launch configuration dialog box.

- b. Choose **Mi Hola Mundo**→**Mi Orden** to run the custom command.

The localized message is displayed.



For more information about localizing Eclipse plug-ins, see the Eclipse documentation. Also, you can unzip any of the Teamcenter plug-ins to see how the rich client uses localization. The **plugin.properties** and **plugin.xml** files in the **aifrcp** plug-in are good examples.

6. Package the project.

If you want to package the project for distribution, export it to a JAR file and place it in the `TC_ROOT\portal` directory.

- a. Right-click the project, choose **Export**→**Plug-in Development**→**Deployable plug-ins and fragments**, select the `TC_ROOT\portal` directory as the destination, and click **Finish**.

The `com.mycom.project-name` JAR file is automatically generated into the `TC_ROOT\portal\plugins` directory.

- b. Run the `TC_ROOT\portal\registry\genregxml` file to register the plug-in with the rich client.

When the script is finished, a new `TC_ROOT\portal\registry\RegistryLoader.xml.gz` file is created.

If you make changes to any of the `.properties` files, or you add new plug-ins or change plug-in content, you must run the `genregxml` script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the `portal\Registry` directory.

- c. To clear the cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the `%HOMEDRIVE%%HOMEPATH%\Teamcenter` directory. On a Linux client, it is typically the `$HOME/Teamcenter` directory.

Customizing Command Suppression

Introduction to customizing Command Suppression

You can suppress menu commands using the Command Suppression application in Teamcenter. If you want to make your custom plug-ins conform to the Command Suppression application, you must add the proper coding to the plug-ins.

The rich client uses Eclipse declarative commands, menu contributions, and handlers to define the vast majority of menu bar and toolbar items. Eclipse uses a Boolean expression based syntax to allow control over visibility of any specific command in a menu using the **visibleWhen** expression. Just like Eclipse provides some sources like **activeContexts**, **activePartId**, and so on, a rich client **rac_command_suppression** source is defined.

Every command contribution in a custom `plugin.xml` file must have a **visibleWhen** expression consuming the **rac_command_suppression** source using the **with** expression. (That is, those command contributions using the `org.eclipse.ui.menus` extension point to contribute a command using the **menuContribution** source.) Using the **rac_command_suppression** source ensures that the command is visible only when it is not suppressed.

The **rac_command_suppression** source gets called whenever the workbench state changes, for example, when new commands are suppressed, when switching between perspectives, and so on.

The **with** expression (also referred to as the *Command Suppression expression*) consuming the **rac_command_suppression** source must be specified in the **visibleWhen** expression of every command contribution. A template of this Command Suppression expression follows:

```
<with variable="rac_command_suppression">
  <not>
    <iterate operator="or">
      <equals value="command_ID_of_the_command_contribution"/>
    </iterate>
  </not>
</with>
```

Replace *command_ID_of_the_command_contribution* with the command ID of the respective command contribution.

Using the Command Suppression expression in the plugin.xml file

Using the Command Suppression expression in a rich client **plugin.xml** file varies based on the following scenarios:

- The command contribution does not contain a **visibleWhen** expression.
Example before changes:

```
<menuContribution locationURI="menu:edit?after=cut.ext">
  <command commandId="org.eclipse.ui.edit.cut" id="org.eclipse.ui.edit.cut"
    label="%cutAction.NAME">
  </command>
</menuContribution>
```

Example after changes:

```
<menuContribution locationURI="menu:edit?after=cut.ext">
  <command commandId="org.eclipse.ui.edit.cut" id="org.eclipse.ui.edit.cut"
    label="%cutAction.NAME">
    <visibleWhen>
      <with variable="rac_command_suppression">
        <not>
          <iterate operator="or">
            <equals value="org.eclipse.ui.edit.cut"/>
          </iterate>
        </not>
      </with>
    </visibleWhen>
  </command>
</menuContribution>
```

- The command contribution contains a **visibleWhen** expression with a nested **and** expression.
Example before changes:

```

<menuContribution locationURI="popup:org.eclipse.ui.popup.any
?after=org.eclipse.ui.edit.paste">
    <command commandId="com.teamcenter.rac.pasteDuplicate">
        <visibleWhen>
            <and>
                <reference definitionId="com.teamcenter.rac.cme.mpp.inMainView"/>
                <reference definitionId="com.teamcenter.rac.tcapps.
isPasteDuplicateAllowed"/>
            </and>
        </visibleWhen>
    </command>
</menuContribution>

```

Example after changes:

```

<menuContribution locationURI="popup:org.eclipse.ui.popup.any
?after=org.eclipse.ui.edit.paste">
    <command commandId="com.teamcenter.rac.pasteDuplicate">
        <visibleWhen>
            <and>
                <reference definitionId="com.teamcenter.rac.cme.mpp.inMainView"/>
                <reference definitionId="com.teamcenter.rac.tcapps.
isPasteDuplicateAllowed"/>
                <with variable="rac_command_suppression">
                    <not>
                        <iterate operator="or">
                            <equals value="com.teamcenter.rac.pasteDuplicate"/>
                        </iterate>
                    </not>
                </with>
            </and>
        </visibleWhen>
    </command>
</menuContribution>

```

- The command contribution contains a **visibleWhen** expression with nested expressions (not the **and** expression).

Example before changes:

```

<menuContribution locationURI="menu:file?after=save.ext">
    <command commandId="com.teamcenter.rac.importAMRuleTree"
icon="icons/importamruletree_16.png" mnemonic="%importAMRuleTreeAction.MNEMONIC">
        <visibleWhen>
            <reference definitionId="com.teamcenter.rac.accessmanager.inMainView"/>
        </visibleWhen>
    </command>
</menuContribution>

```

Example after changes:

```

<menuContribution locationURI="menu:file?after=save.ext">
    <command commandId="com.teamcenter.rac.importAMRuleTree"
icon="icons/importamruletree_16.png" mnemonic="%importAMRuleTreeAction.MNEMONIC">
        <visibleWhen>
            <and>
                <reference definitionId="com.teamcenter.rac.accessmanager.inMainView"/>
                <with variable="rac_command_suppression">

```

```

        <not>
            <iterate operator="or">
                <equals value="com.teamcenter.rac.importAMRuleTree"/>
            </iterate>
        </not>
    </with>
</and>
</visibleWhen>
</command>
</menuContribution>

```

Command Suppression constraints

Following are constraints on Command Suppression customization:

- The Command Suppression application cannot suppress commands that are dynamic contributions using the **<dynamic>** **</dynamic>** tag in the **menuContribution** section of the **org.eclipse.ui.menus** extension point.
- Any contributions that are done statically in the code (for example, the **Window** menu) cannot be suppressed.
- Any contributions that are done using Eclipse actions cannot be suppressed.

Naming convention for extensions and Command Suppression

Every command contribution in a plug-in has the **visibleWhen** expression containing the **rac_command_suppression** source provider. The naming convention of the extensions in a plug-in should always start with the bundle symbolic name to clearly indicate which plug-in is providing the contribution.

Assume that the *com.mycom.myapp* plug-in has the *com.mycom.myapp* bundle symbolic name in the **META-INF/MANIFEST.MF**. Assume that this plug-in creates the **Sample Perspective** perspective and makes contributions to the global menu bar and toolbar that are visible in the perspective. The perspective ID starts with the bundle symbolic name, for example:

```
com.mycom.myapp.perspectives.samplePerspectiveId
```

Because the command contributions from the *com.mycom.myapp* plug-in should be visible only in the **Sample Perspective** perspective, a reference definition can be defined and associated on all command contributions, for example:

```

<definition id="com.mycom.myapp.inMainPerspective">
    <with variable="activeContexts">
        <iterate operator="or">
            <or>
                <equals value="com.mycom.myapp.perspectives.samplePerspectiveId"/>
            </or>
        </iterate>
    </with>
</definition>

```


Assume that the plug-in contributes a sample command. This means that the plug-in must define a command ID using the **org.eclipse.ui.commands** extension point adhering to the naming convention, for example:

```
com.mycom.myapp.sampleCommand
```

To make this sample command visible only in the **Sample Perspective** perspective, and if it is not suppressed from the **Command Suppression** perspective, use a combination of this reference definition and the command suppression source provider, for example:

```
<command commandId="com.mycom.myapp.sampleCommand" tooltip="%sampleCommand.TIP">
  <visibleWhen>
    <and>
      <reference definitionId="com.mycom.myapp.inMainPerspective"/>
      <with variable="rac_command_suppression">
        <not>
          <iterate operator="or">
            <equals value="com.mycom.myapp.sampleCommand"/>
          </iterate>
        </not>
      </with>
    </and>
  </visibleWhen>
</command>
```

Additional examples for reference only

Add a table viewer

You can create a table in its own dialog box and add a menu command to launch it.

1. Create the project.
 - a. In Eclipse, choose **File**→**New**→**Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Then click **Next**.
 - c. In the **New Plug-in Project** dialog box **Plug-in Project** pane, type **com.mycom.tableviewer** in the **Project name** box. Click **Next**.
 - d. In the **New Plug-in Project** dialog box **Content** pane, do the following:
 - A. Under **Options**, ensure the **This plug-in will make contributions to the UI** check box is checked.
 - B. Click the **No** button next to **Would you like to create a rich client application?**
 - C. Click **Next**.

- e. Ensure the **Create a plug-in using one of these templates** check box is selected and select **Hello, World Command** in the list. Click **Finish**.

2. Update the project tabs.

- a. Click the **Dependencies** tab, click the **Add** button, and select the following:

```
com.teamcenter.rac.aifrcp
com.teamcenter.rac.common
com.teamcenter.rac.kernel
com.teamcenter.rac.util
```

- b. Click the **Runtime** tab, click the **Add** button, and select the following:

```
com.mycom.tableviewer
com.mycom.tableviewer.handlers
```

3. Create the project files.

- a. Edit the *com.mycom.tableviewer* plug-in's **plugin.xml** file. Replace the entire contents of the **plugin.xml** file with the following code:

```
<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.4"?>
<plugin>
  <extension
    point="org.eclipse.ui.commands">
    <category
      name="Sample Category"
      id="com.mycom.tableviewer.commands.category">
    </category>
    <command
      name="Sample Command"
      categoryId="com.mycom.tableviewer.commands.category"
      id="com.mycom.tableviewer.commands.sampleCommand">
    </command>
  </extension>
  <extension
    point="org.eclipse.ui.handlers">
    <handler
      commandId="com.mycom.tableviewer.commands.sampleCommand"
      class="com.mycom.tableviewer.handlers.TableViewerHandler">
    </handler>
  </extension>
  <extension
    point="org.eclipse.ui.bindings">
    <key
      commandId="com.mycom.tableviewer.commands.sampleCommand"
      contextId="org.eclipse.ui.contexts.window"
      sequence="M1+6"
      schemeId="org.eclipse.ui.defaultAcceleratorConfiguration">
    </key>
  </extension>
</extension>
```

```

        point="org.eclipse.ui.menus">
<menuContribution
    locationURI="menu:org.eclipse.ui.main.menu?after=additions">
    <menu
        label="Viewer"
        mnemonic="V"
        id="com.mycom.tableviewer.menus.sampleMenu">
        <command
            commandId="com.mycom.tableviewer.commands.sampleCommand"
            id="com.mycom.tableviewer.menus.sampleCommand"
            label="TableViewer"
            mnemonic="T">
            <visibleWhen>
                <reference
                    definitionId="com.teamcenter.rac.ui.inMainPerspective">
                </reference>
            </visibleWhen>
        </command>
    </menu>
</menuContribution>
<menuContribution
    locationURI="toolbar:org.eclipse.ui.main.toolbar?after=additions">
    <toolbar
        id="com.mycom.tableviewer.toolbars.sampleToolbar">
        <command
            commandId="com.mycom.tableviewer.commands.sampleCommand"
            icon="icons/sample.gif"
            id="com.mycom.tableviewer.toolbars.sampleCommand"
            label="TableViewer"
            tooltip="Explore Tableviewer">
        </command>
    </toolbar>
</menuContribution>
</extension>
</plugin>

```

- b. Open the *com.mycom.tableviewer.handlers* package, select the **SampleHandler.java** file, choose **File→Rename**, and rename the file to **TableViewerHandler.java**.

Replace the contents of the **TableViewerHandler.java** file with the following:

```

package com.mycom.tableviewer.handlers;
import org.eclipse.core.commands.AbstractHandler;
import org.eclipse.core.commands.ExecutionEvent;
import org.eclipse.core.commands.ExecutionException;
/**
 * Our TableViewerHandler extends AbstractHandler
 * @see org.eclipse.core.commands.AbstractHandler
 */
public class TableViewerHandler extends AbstractHandler {
    /**
     * The constructor.
     */
    public TableViewerHandler() {
    }
    /**
     * the command has been executed, so extract extract the needed information
     * from the application context.

```

```

        */
        public Object execute(ExecutionEvent event) throws ExecutionException {
            return new TableViewerExample();
        }
    }
}

```

- c. Right-click the *com.mycom.tableviewer.handlers* package, choose **New**→**Class**, and add the following Java files:

Row.java
TableContentProvider.java
TableLabelProvider.java
TableViewerExample.java

- d. Replace the contents of the **Row.java** file with the following:

```

package com.mycom.tableviewer.handlers;
public class Row {
    private String key;
    private String value;
    public Row(String key, String value) {
        setKey(key);
        setValue(value);
    }
    public void setKey(String key) {
        this.key = key;
    }
    public String getKey() {
        return key;
    }
    public void setValue(String value) {
        this.value = value;
    }
    public String getValue() {
        return value;
    }
}

```

- e. Replace the contents of the **TableContentProvider.java** file with the following:

```

package com.mycom.tableviewer.handlers;
import java.util.ArrayList;
import java.util.List;
import org.eclipse.jface.viewers.IStructuredContentProvider;
import org.eclipse.jface.viewers.Viewer;
public class TableContentProvider implements IStructuredContentProvider {
    public Object[] getElements(Object parent) {
        List results = new ArrayList();
        if (parent instanceof ArrayList) {
            results = (ArrayList) parent;
        }
        return results.toArray();
    }
    public void dispose() {
    }
    public void inputChanged(Viewer viewer, Object oldInput, Object newInput) {
    }
}

```

```
    }
}
```

- f. Replace the contents of the **TableLabelProvider.java** file with the following:

```
package com.mycom.tableviewer.handlers;
import org.eclipse.jface.viewers.ITableLabelProvider;
import org.eclipse.jface.viewers.LabelProvider;
import org.eclipse.swt.graphics.Image;
public class TableLabelProvider extends LabelProvider implements
    ITableLabelProvider {
    public Image getColumnImage(Object element, int columnIndex) {
        return null;
    }
    public String getColumnText(Object element, int columnIndex) {
        Row row = (Row) element;
        switch (columnIndex) {
            case 0:
                return row.getKey();
            case 1:
                return row.getValue();
        }
        return null;
    }
}
```

- g. Replace the contents of the **TableViewerExample.java** file with the following:

```
package com.mycom.tableviewer.handlers;
import java.util.Arrays;
import java.util.Iterator;
import java.util.List;
import org.eclipse.jface.viewers.CellEditor;
import org.eclipse.jface.viewers.ColumnWeightData;
import org.eclipse.jface.viewers.ICellModifier;
import org.eclipse.jface.viewers.ISelection;
import org.eclipse.jface.viewers.IStructuredSelection;
import org.eclipse.jface.viewers.TableLayout;
import org.eclipse.jface.viewers.TableViewer;
import org.eclipse.jface.viewers.TextCellEditor;
import org.eclipse.swt.SWT;
import org.eclipse.swt.events.SelectionAdapter;
import org.eclipse.swt.events.SelectionEvent;
import org.eclipse.swt.layout.FillLayout;
import org.eclipse.swt.layout.GridData;
import org.eclipse.swt.layout.GridLayout;
import org.eclipse.swt.widgets.Button;
import org.eclipse.swt.widgets.Composite;
import org.eclipse.swt.widgets.Control;
import org.eclipse.swt.widgets.Display;
import org.eclipse.swt.widgets.Shell;
import org.eclipse.swt.widgets.Table;
import org.eclipse.swt.widgets.TableColumn;
import org.eclipse.swt.widgets.TableItem;
import com.teamcenter.rac.aif.AbstractAIFDialog;
import com.teamcenter.rac.util.Registry;
/**
 * This is an example program showing how to use a TableViewer
```

```

*/
public class TableViewerExample {
    private Table table;
    private TableViewer tableViewer;

    /** class registry */
    private final static Registry reg =
Registry.getRegistry( TableViewerExample.class );
    private final static String[] COLUMN_HEADINGS =
{ reg.getString( "Object"),reg.getString( "Type" )};
    public TableViewerExample() {
        createTableViewerExample();
    }
    private void createTableViewerExample() {
        Shell shell = new Shell(SWT.SHELL_TRIM);
        shell.setImage( Registry.getRegistry(
AbstractAIFDialog.class ).getImage(
"aifDesktop.ICON" ) );
        shell.setText(reg.getString("details"));
        shell.setLayout(new FillLayout());
        createContents(shell);
        shell.setSize(400, 400);
        shell.open();
        while (!shell.isDisposed()) {
            if (!Display.getDefault().readAndDispatch())
                Display.getDefault().sleep();
        }
    }
    protected Control createContents(Composite parent) {
        final Composite composite = new Composite(parent, SWT.NONE);
        GridLayout layout = new GridLayout(1, false);
        layout.verticalSpacing = 10;
        composite.setLayout(layout);
        GridData data = new GridData(GridData.FILL_BOTH);
        data.grabExcessHorizontalSpace = true;
        composite.setLayoutData(data);
        table = new Table(composite, SWT.BORDER | SWT.V_SCROLL | SWT.MULTI
| SWT.FULL_SELECTION);
        table.setLinesVisible(true);
        table.setHeaderVisible(true);
        data = new GridData(SWT.FILL, SWT.FILL, true, false);
        data.heightHint = 300;
        table.setLayoutData(data);
        TableLayout tableLayout = new TableLayout();
        table.setLayout(tableLayout);
        tableLayout.addColumnData(new ColumnWeightData(10, 50, true));
        TableColumn column = new TableColumn(table, SWT.NONE);
        column.setText(COLUMN_HEADINGS[0]);
        column.setAlignment(SWT.LEFT);
        tableLayout.addColumnData(new ColumnWeightData(15, 200, true));
        column = new TableColumn(table, SWT.NONE);
        column.setText(COLUMN_HEADINGS[1]);
        column.setAlignment(SWT.LEFT);
        tableViewer = new TableViewer(table);
        tableViewer.setColumnProperties(COLUMN_HEADINGS);
        tableViewer.setContentProvider(new TableContentProvider());
        tableViewer.setLabelProvider(new TableLabelProvider());
        CellEditor[] editors = new CellEditor[2];
        editors[0] = new TextCellEditor(table);
        editors[1] = new TextCellEditor(table);
    }
}

```

```

tableViewer.setCellEditors(editors);
tableViewer.setCellModifier(new TableCellModifier());
Composite buttonComposite = new Composite(composite, SWT.NONE);
FillLayout fillLayout = new FillLayout(SWT.HORIZONTAL);
fillLayout.spacing = 10;
buttonComposite.setLayout(fillLayout);
Button addButton = new Button(buttonComposite, SWT.PUSH);
addButton.setText(reg.getString("Add"));
addButton.addSelectionListener(new SelectionAdapter() {
    public void widgetSelected(SelectionEvent e) {
        Row row = new Row("", "");
        tableViewer.add(row);
        table.setTopIndex(table.getItemCount());
        table.select(table.getItemCount() - 1);
        tableViewer.editElement(row, 0);
    }
});
Button deleteButton = new Button(buttonComposite, SWT.PUSH);
deleteButton.setText(reg.getString("Delete"));
deleteButton.addSelectionListener(new SelectionAdapter() {
    public void widgetSelected(SelectionEvent e) {
        ISelection selection = tableViewer.getSelection();
        if (selection instanceof IStructuredSelection) {
            Iterator iterator = ((IStructuredSelection) selection)
                .iterator();
            while (iterator.hasNext()) {
                Object obj = iterator.next();
                tableViewer.remove(obj);
            }
        }
    }
});

Button closeButton = new Button(buttonComposite, SWT.PUSH);
closeButton.setText(reg.getString("Close"));
closeButton.addSelectionListener(new SelectionAdapter() {
    public void widgetSelected(SelectionEvent e) {
        composite.getParent().dispose();
    }
});
return composite;
}

class TableCellModifier implements ICellModifier {
    public boolean canModify(Object element, String property) {
        return true;
    }

    public Object getValue(Object element, String property) {
        Object result = null;
        Row row = (Row) element;
        List<String> list = Arrays.asList(COLUMN_HEADINGS);
        int columnIndex = list.indexOf(property);
        switch (columnIndex) {
            case 0:
                result = row.getKey();
                break;
            case 1:
                result = row.getValue();
                break;
        }
    }
}

```

```

        return result;
    }
    public void modify(Object element, String property, Object value) {
        List<String> list = Arrays.asList(COLUMN_HEADINGS);
        int columnIndex = list.indexOf(property);
        TableItem tableItem = (TableItem) element;
        Row row = (Row) tableItem.getData();
        switch (columnIndex) {
            case 0:
                String key = (String) value;
                if (key.length() > 0) {
                    row.setKey(key);
                }
                break;
            case 1:
                String v = (String) value;
                if (v.length() > 0) {
                    row.setValue(v);
                }
                break;
        }
        tableViewer.update(row, null);
    }
}

```

- h. Right-click the **com.mycom.tableviewer.handlers** package, choose **New→File**, and add a **handlers.properties** file.
- i. Replace the contents of the **handlers.properties** file with the following:

```

Object=Object
Type=Type
details=Details
Add=Add
Delete=Delete
Close=Close

```

- j. Choose **File→Save All**.

4. Package the project.

To package the project for distribution, export it to a JAR file and place it in the *TC_ROOT\portal* directory.

- a. Right-click the project, choose **Export→Plug-in Development→Deployable plug-ins and fragments**, select the *TC_ROOT\portal* directory as the destination, and click **Finish**.

The **com.mycom.project-name** JAR file is automatically generated into the *TC_ROOT\portal\plugins* directory.

- b. Run the *TC_ROOT\portal\registry\genregxml* file to register the plug-in with the rich client.

If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory.

- On a Windows client, it is typically the `%HOMEDRIVE%%HOMEPATH%\Teamcenter` directory. On a Linux client, it is typically the `$HOME/Teamcenter` directory.

- a. Launch My Teamcenter and choose **Viewer**→**TableViewer** on the menu bar, or click the **Explore TableViewer** button on the toolbar.



-
- The screenshot shows a 'Details' window with a blue title bar. The window contains a table with two columns: 'Object' and 'Type'. The table is empty. At the bottom of the window, there are three buttons: 'Add', 'Delete', and 'Close'.

Add a tree viewer

You can create a tree in its own dialog window and add a menu command to launch it. This example displays a file explorer tree that reads the contents of the hard disk.

By default, this example reads the **C:** drive on Windows systems. To read the **/user** drive on UNIX systems, change code in the **Explorerer.java** file.

1. Create the project.
 - a. In Eclipse, choose **File→New→Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Then click **Next**.
 - c. In the **New Plug-in Project** dialog box **Plug-in Project** pane, type **com.mycom.treeviewer** in the **Project name** box. Click **Next**.
 - d. In the **New Plug-in Project** dialog box **Content** pane, do the following:
 - A. Under **Options**, ensure the **This plug-in will make contributions to the UI** check box is selected.
 - B. Click the **No** button next to **Would you like to create a rich client application?**
 - C. Click **Next**.
 - e. Ensure the **Create a plug-in using one of these templates** check box is selected and select **Hello, World Command** in the list. Click **Finish**.
2. Update the project tabs.
 - a. Click the **Dependencies** tab, click the **Add** button, and select the following:

com.teamcenter.rac.aifrcp
com.teamcenter.rac.common
com.teamcenter.rac.kernel
com.teamcenter.rac.util
 - b. Click the **Runtime** tab, click the **Add** button, and select the following:

com.mycom.treeviewer
com.mycom.treeviewer.handlers
3. Create the project files.

- a. Edit the **com.mycom.treeviewer** plug-in's **plugin.xml** file. Replace the entire contents of the **plugin.xml** file with the following code:

```
<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.4"?>
<plugin>
  <extension
    point="org.eclipse.ui.commands">
    <category
      name="Sample Category"
      id="com.mycom.treeviewer.commands.category">
    </category>
    <command
      name="Sample Command"
      categoryId="com.mycom.treeviewer.commands.category"
      id="com.mycom.treeviewer.commands.sampleCommand">
    </command>
  </extension>
  <extension
    point="org.eclipse.ui.handlers">
    <handler
      commandId="com.mycom.treeviewer.commands.sampleCommand"
      class="com.mycom.treeviewer.handlers.TreeViewerHandler">
    </handler>
  </extension>
  <extension
    point="org.eclipse.ui.bindings">
    <key
      commandId="com.mycom.treeviewer.commands.sampleCommand"
      contextId="org.eclipse.ui.contexts.window"
      sequence="M1+6"
      schemeId="org.eclipse.ui.defaultAcceleratorConfiguration">
    </key>
  </extension>
  <extension
    point="org.eclipse.ui.menus">
    <menuContribution
      locationURI="menu:org.eclipse.ui.main.menu?after=additions">
    <menu
      label="Viewer"
      mnemonic="V"
      id="com.mycom.treeviewer.menus.sampleMenu">
    <command
      commandId="com.mycom.treeviewer.commands.sampleCommand"
      id="com.mycom.treeviewer.menus.sampleCommand"
      label="TreeViewer"
      mnemonic="T"
      tooltip="Explore Treeviewer">
    <visibleWhen>
      <reference
        definitionId="com.teamcenter.rac.ui.inMainPerspective">
      </reference>
    </visibleWhen>
    </command>
    </menu>
  </menuContribution>
  <menuContribution
    locationURI="toolbar:org.eclipse.ui.main.toolbar?after=additions">
    <toolbar
```

```

        id="com.mycom.treeviewer.toolbars.sampleToolBar">
        <command
            commandId="com.mycom.treeviewer.commands.sampleCommand"
            icon="icons/sample.gif"
            id="com.mycom.treeviewer.toolbars.sampleCommand"
            label="TreeViewer"
            tooltip="Explore Treeviewer">
        </command>
    </toolbar>
</menuContribution>
</extension>
</plugin>

```

- b. Open the **com.mycom.treeviewer.handlers** package, select the **SampleHandler.java** file, choose **File→Rename**, and rename the file to **TreeViewerHandler.java**.

Replace the contents of the **TreeViewerHandler.java** file with the following:

```

package com.mycom.treeviewer.handlers;
import org.eclipse.core.commands.AbstractHandler;
import org.eclipse.core.commands.ExecutionEvent;
import org.eclipse.core.commands.ExecutionException;
/**
 * TreeViewerHandler extends AbstractHandler, an IHandler base class.
 * @see org.eclipse.core.commands.IHandler
 * @see org.eclipse.core.commands.AbstractHandler
 */
public class TreeViewerHandler extends AbstractHandler {
    /**
     * The constructor.
     */
    public TreeViewerHandler() {
    }
    /**
     * the command has been executed, so extract extract the needed information
     * from the application context.
     */
    public Object execute(ExecutionEvent event) throws ExecutionException {
        Explorer explorer = new Explorer();
        explorer.setBlockOnOpen(true);
        explorer.open();
        return null;
    }
}

```

- c. Right-click the **com.mycom.treeviewer.handlers** package, choose **New→Class**, and add the following Java files:

Explorer.java
FileTreeExplorerContentProvider.java
FileTreeExplorerLabelProvider.java

- d. Replace the contents of the **Explorer.java** file with the following:

```

package com.mycom.treeviewer.handlers;
import java.io.File;
import org.eclipse.jface.viewers.TreeViewer;
import org.eclipse.swt.widgets.*;
import org.eclipse.jface.window.ApplicationWindow;
public class Explorer extends ApplicationWindow {
    public Explorer() {
        super(null);
    }
    protected Control createContents(Composite parent) {
        TreeViewer tv = new TreeViewer(parent);
        tv.setContentProvider(new FileTreeExplorerContentProvider());
        tv.setLabelProvider(new FileTreeExplorerLabelProvider());
        // if for Linux/Unix platforms
        // tv.setInput(new File("/usr/"));
        tv.setInput(new File("C:\\"));
        return tv.getTree();
    }
}

```

Note:

By default, this example reads the **C:** drive on Windows systems. To read the **/user** drive on UNIX systems, change code in the **Explorer.java** file.

- e. Replace the contents of the **FileTreeExplorerContentProvider.java** file with the following:

```

package com.mycom.treeviewer.handlers;
import java.io.File;
import org.eclipse.jface.viewers.ITreeContentProvider;
import org.eclipse.jface.viewers.Viewer;
public class FileTreeExplorerContentProvider implements ITreeContentProvider
{
    public Object[] getChildren(Object element)
    {
        Object[] children = ((File) element).listFiles();
        return children == null ? new Object[0] : children;
    }
    public Object[] getElements(Object element)
    {
        return getChildren(element);
    }
    public boolean hasChildren(Object element)
    {
        return getChildren(element).length > 0;
    }
    public Object getParent(Object element)
    {
        return ((File)element).getParent();
    }

    public void dispose()
    {
        //No implementation
    }
    public void inputChanged(Viewer viewer, Object old_input, Object new_input)
    {
        //No implementation
    }
}

```

```
    }
}
```

- f. Replace the contents of the **FileTreeExplorerLabelProvider.java** file with the following:

```
package com.mycom.treeviewer.handlers;
import java.io.File;
import org.eclipse.jface.viewers.LabelProvider;
public class FileTreeExplorerLabelProvider extends LabelProvider
{
    public String getText(Object element)
    {
        return ((File) element).getName();
    }
}
```

- g. Choose **File**→**Save All**.

4. Package the project.

To package the project for distribution, export it to a JAR file and place it in the *TC_ROOT\portal* directory.

- a. Right-click the project, choose **Export**→**Plug-in Development**→**Deployable plug-ins and fragments**, select the *TC_ROOT\portal* directory as the destination, and click **Finish**.

The **com.mycom.project-name** JAR file is automatically generated into the *TC_ROOT\portal\plugins* directory.

- b. Run the *TC_ROOT\portal\registry\genregxml* file to register the plug-in with the rich client.

When the script is finished, a new *TC_ROOT\portal\registry\RegistryLoader.xml.gz* file is created.

Note:

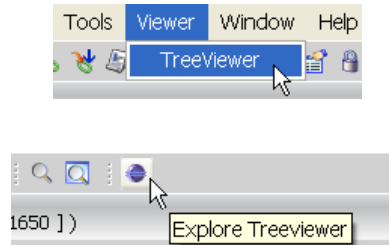
If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory.

- c. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

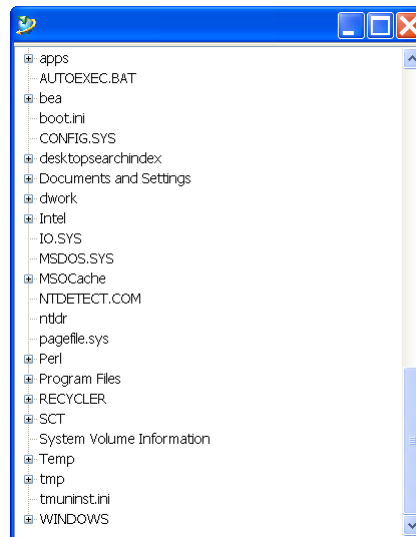
On a Windows client, it is typically the **%HOMEDRIVE%%HOMEPATH%\Teamcenter** directory. On a Linux client, it is typically the **\$HOME/Teamcenter/** directory.

5. Verify the customization.

- a. Launch My Teamcenter and choose **Viewer**→**TreeViewer** on the menu bar, or click the **Explore TreeViewer** button on the toolbar.



- b. Browse the tree structure.



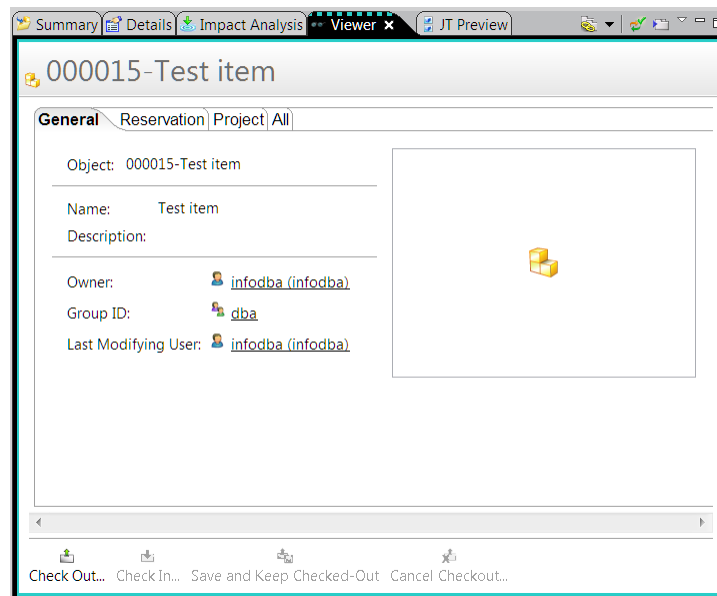
Create a custom Viewer view

The **Viewer** view in My Teamcenter displays information about the selected object. In this customization example using SWT, you create two custom **Viewer** view definitions: a **HelloWorldViewer** definition that displays only the text **Hello selected-object-name!** for the selected object, and a **MyPropertyViewer** definition that displays properties and **Check In / Check Out** buttons at the bottom of the view.

This SWT customization example extends the `com.teamcenter.rac.viewer.ViewerViewRegistry` extension point to override the standard **TextViewer** viewer and replace it with a new viewers. In the example Java files, you implement **IViewerFactory** and **ISubViewer** components, and extend the **ViewerViewRegistry** extension point to register the new **SubView** definition.

After creating the view, you must update the `defaultViewerConfig.VIEWERCONFIG` preference to include this new view for the data type you want. For example, if you set the custom view for the text dataset type, the custom view appears when you select that dataset type.

Following is an example of a default **Viewer** view.



1. Create the project.
 - a. In Eclipse, choose **File**→**New**→**Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Click **Next**.
 - c. In the **New Plug-in Project** wizard **Plug-in Project** dialog box, type **com.mycom.customviewerview** in the **Project name** box. Click **Next**.
 - d. In the **New Plug-in Project** wizard **Content** dialog box, do the following:
 - A. Under **Options**, ensure the **This plug-in will make contributions to the UI** check box is selected.
 - B. Click the **No** button next to **Would you like to create a rich client application?**
 - C. Click **Next**.
 - e. Clear the **Create a plug-in using one of these templates** check box. Click **Finish**.
2. Update the project tabs.
 - a. Click the **Overview** tab and select the **This plug-in is a singleton** check box.
 - b. Click the **Dependencies** tab, click the **Add** button, and select the following plug-ins:

org.eclipse.ui


```
org.eclipse.core.runtime
com.teamcenter.rac.aifrcp
com.teamcenter.rac.common
com.teamcenter.rac.kernel
com.teamcenter.rac.util
com.teamcenter.rac.viewer
```

3. Right-click the **com.mycom.customviewerview** project, choose **New→Package**, and create the following packages:

```
com.mycom.customviewerview.factory
com.mycom.customviewerview.viewer
```

4. Create the **HelloWorldViewer** definition.

- a. Create the project files.

- A. Right-click the **com.mycom.customviewerview.factory** package, choose **New→Class**, and create the **HelloWorldViewerFactory** class.
- B. Right-click the **com.mycom.customviewerview.viewer** package, choose **New→Class**, and create the following classes:

```
HelloWorldViewer
HelloWorldViewerContentProvider
```

- C. Replace the code in the **HelloWorldViewerFactory.java** file with the following:

```
package com.mycom.customviewerview.factory;

import org.eclipse.swt.widgets.Display;

import com.mycom.customviewerview.viewer>HelloWorldViewer;
import com.mycom.customviewerview.viewer>HelloWorldViewerContentProvider;
import com.teamcenter.rac.common.tcviewer.factory.AbstractSWTViewerFactory;
import com.teamcenter.rac.util.viewer.IViewerEvent;
import com.teamcenter.rac.util.viewer.SubViewerListener;
import com.teamcenter.rac.util.viewer.ViewerEvent;

/**
 * This factory builds HelloWorldViewer.
 */
public class HelloWorldViewerFactory extends AbstractSWTViewerFactory
{
    @Override
    public void loadViewer( final SubViewerListener listener )
    {
        Runnable runnable = new Runnable()
        {
            public void run()
            {
```

```

        try
        {
            //HelloWorldViewer viewer = new HelloWorldViewer( m_parent );
            HelloWorldViewer viewer = new
HelloWorldViewer( getParent() );
            viewer.setContentProvider( new
HelloWorldViewerContentProvider());
            listener.done( viewer );
        }
        catch( Exception e)
        {
            ViewerEvent viewerError = new ViewerEvent( this,
IViewerEvent.NOVIEWDATAFOUND );
            listener.error( viewerError );
        }
        finally
        {
        }
    }
};
Display.getDefault().asyncExec( runnable );
}
}

```

D. Replace the code in the **HelloWorldViewer.java** file with the following:

```

package com.mycom.customviewerview.viewer;

import org.eclipse.swt.SWT;
import org.eclipse.swt.layout.FillLayout;
import org.eclipse.swt.widgets.Composite;
import org.eclipse.swt.widgets.Control;
import org.eclipse.swt.widgets.Label;
import com.teamcenter.rac.common.tcviewer.TCComponentViewerInput;
import com.teamcenter.rac.kernel.TCComponent;
import com.teamcenter.rac.util.AdapterUtil;
import com.teamcenter.rac.util.viewer.IViewerEvent;
import com.teamcenter.rac.util.viewer.ViewerEvent;
import com.teamcenter.rac.viewer.view.AbstractSwtSubViewer;

/**
 * A simple "Hello World" example viewer.
 */
public class HelloWorldViewer
    extends AbstractSwtSubViewer //implements IAdaptable
{
    private Composite m_composite = null;
    private Label m_label = null;
    private TCComponent m_comp = null;

    public HelloWorldViewer( Composite parent )
    {
        m_composite = new Composite( parent , SWT.NONE );
        m_composite.setLayout(new FillLayout());

        m_label = new Label( m_composite, SWT.None );
        m_label.setText( "" );
        m_label.setVisible(true);
        m_label.setEnabled(true);
    }
}

```

```

    }

    /*
     * Currently this is just getting the TCComponent object
     * from the viewer input.
     */
    @Override
    public void setInput( Object viewerInput )
    {
        TCComponentViewerInput input = (TCComponentViewerInput)
AdapterUtil.getAdapter( viewerInput,
        TCComponentViewerInput.class );
        m_comp = (TCComponent)input.getViewableObj();
        super.setInput( m_comp );
    }

    @Override
    public void inputChanged( Object input, Object oldInput )
    {
        HelloWorldViewerContentProvider cp =
(HelloWorldViewerContentProvider)
getContentProvider();
        String text = cp.getText( input );
        m_label.setText( text );

        //Notify the host viewer to reload.
        ViewerEvent viewerEvent = new ViewerEvent( this,
IViewerEvent.RELOADVIEW );
        viewerEvent.queueEvent();

        //Viewer SHOW_HEADER event will make this customized view generate
viewer header.
        ViewerEvent viewerEvent2 = new ViewerEvent( this,
IViewerEvent.SHOW_HEADER );
        viewerEvent2.queueEvent();
    }

    @Override
    public Control getControl() {
        return m_composite;
    }

    @Override
    public void refresh() {
        m_composite.layout();
    }

```

- E. Replace the code in the **HelloWorldViewerContentProvider.java** file with the following:

```

package com.mycom.customviewerview.viewer;

import org.eclipse.jface.viewers.IContentProvider;
import org.eclipse.jface.viewers.Viewer;

/**
 * ContentProvider for the HelloWorldViewer
 */
public class HelloWorldViewerContentProvider implements IContentProvider

```

```

{
    @Override
    public void dispose()
    {
        //Do nothing.
    }

    @Override
    public void inputChanged( Viewer viewer, Object oldInput, Object
newInput )
    {
        //Do nothing.
    }

    public String getText( Object input )
    {
        return "Hello " + input.toString() + "!";
    }
}

```

- F. To create an empty **plugin.xml** file, click the **Extensions** tab and click the **Add** button in the **Extensions** view, and click the **Cancel** button in the **New Extension** dialog box.

A **plugin.xml** file is added to the project.

- G. Click the **plugin.xml** tab and replace the code in the **plugin.xml** file with the following:

```

<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.4"?>
<plugin>
    <extension
        point="com.teamcenter.rac.viewer.ViewerViewRegistry">
        <viewer
            autoCheckout="false"
            factoryClassName="com.mycom.customviewerview.factory.
HelloWorldViewerFactory"
            id="HelloWorldViewer"
            isSwing="false"
            priority="100">
            <enableWhen
                checkEnabled="false">
            </enableWhen>
        </viewer>
        <viewer
            autoCheckout="false"
            factoryClassName="com.teamcenter.rac.common.tcviewer.factory.
SwingViewerFactory"
            id="SwingPropertyViewer"
            isSwing="true"
            priority="100"
            viewPanel="com.teamcenter.rac.common.tcviewer.GenericViewer">
            <enableWhen
                checkEnabled="false">
            </enableWhen>
        </viewer>
    </extension>
</plugin>

```

Note:

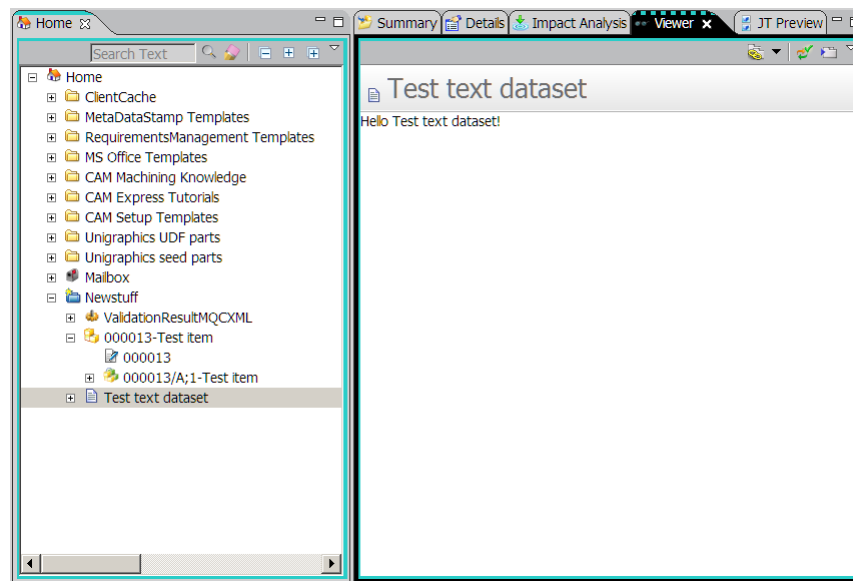
Copying and pasting the **plugin.xml** file content is a quick way to provide you with extensions for this example. However, when you create your own extensions, instead of placing content directly in the **plugin.xml** file, you work in the project's **Extensions** tab. Therefore, review the **Extensions** tab to see how you can create extensions yourself.

- H. Choose **File**→**Save All**.
- b. Verify the customization.
 - A. Choose **Run**→**Run Configurations** to start the rich client from your Eclipse environment. Clear the workspace by checking **Clear** in the launch configuration dialog box.
 - B. Edit the **defaultViewerConfig.VIEWERCONFIG** preference to point to the new **HelloWorldViewer** definition when you select a text dataset.

In My Teamcenter, choose **Edit**→**Options**→**Search**, open the preference, and change the **Text.TextViewer=Text** value to **Text.HelloWorldViewer**.

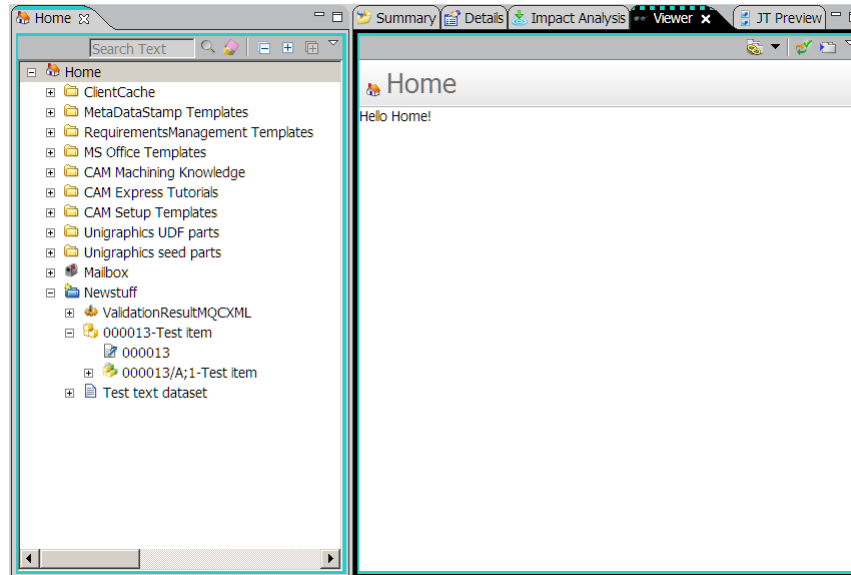
- C. In My Teamcenter, create a text dataset by choosing **File**→**New**→**Dataset**, selecting **More** on the left side of the **New Dataset** dialog box, and selecting **Text** from the list of available dataset types.
- D. Select the text dataset and click the **Viewer** view.

The message **Hellodataset-name!** is displayed in the **Viewer** view.



- E. If you edit the **Viewer** view preferences to use the new **HelloWorldViewer** definition for another object type, the viewer is presented.

For example, if you edit the **defaultViewerConfig.VIEWERCONFIG** preference to include the **Folder.HelloWorldViewer** value, when you select a **Folder** object such as the **Home** folder, the **Viewer** view displays the following message: Hello, Home!



HelloWorldViewer view specified for another object type

5. Create the **MyPropertyViewer** definition.

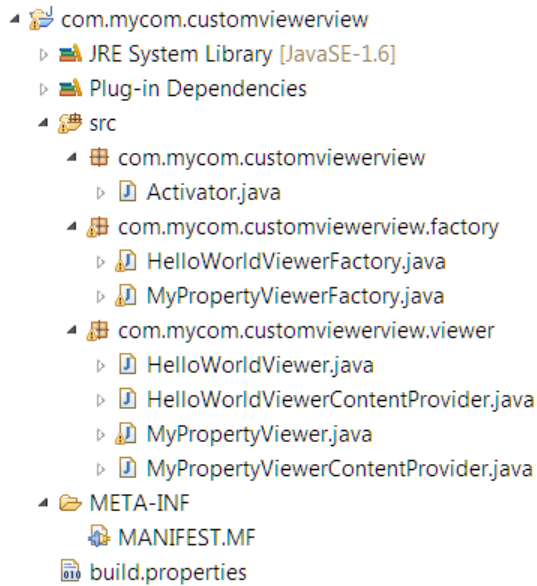
This example displays custom properties named **w2str**, **w2integer**, and **w2date** in a custom **Viewer** view. To see the properties, you must create them on a **Text** dataset, or create your own custom properties and modify the sample code.

- a. Create the project files.

- A. Right-click the **com.mycom.customviewerview.factory** package, choose **New→Class**, and create the **MyPropertyViewerFactory** class.
- B. Right-click the **com.mycom.customviewerview.viewer** package, choose **New→Class**, and create the following classes:

MyPropertyViewer
MyPropertyViewerContentProvider

The plug-in structure looks like this:



C. Replace the code in the **MyPropertyViewerFactory.java** file with the following:

```
package com.mycom.customviewerview.factory;

import org.eclipse.swt.widgets.Display;

import com.mycom.customviewerview.viewer.MyPropertyViewer;
import com.mycom.customviewerview.viewer.MyPropertyViewerContentProvider;
import com.teamcenter.rac.common.tcviewer.factory.AbstractSWTViewerFactory;
import com.teamcenter.rac.util.viewer.IViewerEvent;
import com.teamcenter.rac.util.viewer.SubViewerListener;
import com.teamcenter.rac.util.viewer.ViewerEvent;

/**
 */
public class MyPropertyViewerFactory extends AbstractSWTViewerFactory
{
    @Override
    public void loadViewer( final SubViewerListener listener )
    {
        Runnable runnable = new Runnable()
        {
            public void run()
            {
                try
                {
                    MyPropertyViewer viewer = new
MyPropertyViewer( getParent() );
                    viewer.setContentProvider( new
MyPropertyViewerContentProvider() );
                    listener.done( viewer );
                }
                catch( Exception e)
                {
                    e.printStackTrace();
                    ViewerEvent viewerError = new ViewerEvent( this,
```

```

        IViewerEvent.NOVIEWDATAFOUND );
        listener.error( viewerError );
    } finally {
    }
}
};
Display.getDefault().asyncExec( runnable );
}
}

```

D. Replace the code in the **MyPropertyViewer.java** file with the following:

```

// @<COPYRIGHT>@
// =====
// Copyright 2011.
// Siemens Product Lifecycle Management Software Inc.
// All Rights Reserved.
// =====
// @<COPYRIGHT>@

package com.mycom.customviewerview.viewer;

import java.util.Arrays;
import java.util.Date;

import org.eclipse.core.runtime.IProgressMonitor;
import org.eclipse.swt.SWT;
import org.eclipse.swt.layout.GridData;
import org.eclipse.swt.layout.GridLayout;
import org.eclipse.swt.widgets.Composite;
import org.eclipse.swt.widgets.Control;
import org.eclipse.swt.widgets.Label;
import org.eclipse.ui.ISaveablePart;

import com.teamcenter.rac.aif.kernel.AIFComponentChangeEvent;
import com.teamcenter.rac.aif.kernel.AIFComponentEvent;
import com.teamcenter.rac.aif.kernel.AIFComponentPropertyChangeEvent;
import com.teamcenter.rac.aif.kernel.InterfaceAIFComponent;
import com.teamcenter.rac.aif.kernel.InterfaceAIFComponentEventListener;
import com.teamcenter.rac.aifrcp.AIFUtility;
import com.teamcenter.rac.common.SoaPropertyHelper;
//import com.teamcenter.rac.common.controls.LOVComboBox;
import com.teamcenter.rac.common.tcviewer.TCComponentViewerInput;
import com.teamcenter.rac.kernel.TCComponent;
import com.teamcenter.rac.kernel.TCProperty;
import com.teamcenter.rac.util.AdapterUtil;
import com.teamcenter.rac.util.MessageBox;
import com.teamcenter.rac.util.SWTUIUtilities;
import com.teamcenter.rac.util.controls.DateControl;
import com.teamcenter.rac.util.controls.TextControl;
import com.teamcenter.rac.util.event.ClientEventDispatcher;
import com.teamcenter.rac.util.event.IClientEvent;
import com.teamcenter.rac.util.viewer.IViewerEvent;
import com.teamcenter.rac.util.viewer.ViewerEvent;
import com.teamcenter.rac.viewer.stylesheet.StylesheetRenderingConstants;
import com.teamcenter.rac.viewer.utils.CheckInOutComposite;
import com.teamcenter.rac.viewer.view.AbstractSwtSubViewer;

```



```

/**
 * Example Viewer which shows how to display a list of properties. This example
 * contains textfield, textarea, date, integer. If you have a LOV attached, you
 * can un-comment the lines for lov related code. (The example uses a string LOV.)
 * To use this example, you need to add extension
"com.teamcenter.rac.viewer.ViewerViewRegistry" and
 * modify preference "defaultViewerConfig.VIEWERCONFIG". For more detail, please refer to
 * RAC Customization Guide.
 */
public class MyPropertyViewer
    extends AbstractSwtSubViewer implements ISaveablePart, InterfaceAIFComponentEventListener
{
    private Composite m_composite = null;
    private TCComponent m_comp = null;

    private Label m_w2StrLabel = null;
    private TextControl m_w2StrText;
    private TCProperty m_w2StrProp;

    private Label m_w2IntegerLabel = null;
    private TextControl m_w2IntegerText;
    private TCProperty m_w2IntProp;

    private Label m_w2DateLabel = null;
    private DateControl m_w2DateButton;
    private TCProperty m_w2DateProp;

    private Label m_w2ObjDescLabel = null;
    private TextControl m_w2ObjDescText;
    private TCProperty m_w2ObjDescProp;

    // private Label m_lovLabel;
    // private LOVComboBox m_lovComboBox;
    // private TCProperty m_lovProp;

    public static String STR_PROP = "aos0str";
    public static String INT_PROP = "aos0integer";
    public static String DATE_PROP = "aos0date";

    public MyPropertyViewer( Composite parent )
    {
        GridLayout parentLayout = SWTUIUtilities.tightGridLayout( 1 );

        Composite mainComposite =new Composite( parent , SWT.NONE );
        mainComposite.setLayout(parentLayout);

        m_composite = new Composite( mainComposite , SWT.NONE );
        m_composite.setLayout(new GridLayout(2, false) );
        m_composite.setLayoutData( new GridData( SWT.FILL, SWT.FILL, true, true ) );

        m_w2StrLabel = new Label( m_composite, SWT.None );
        m_w2StrText = new TextControl( m_composite, SWT.SINGLE | SWT.FLAT | SWT.BORDER );

        GridData m_w2StrTextGd = new GridData();
        m_w2StrTextGd.horizontalAlignment = SWT.FILL;
        m_w2StrTextGd.verticalAlignment = SWT.CENTER;
        m_w2StrTextGd.widthHint = 30 *
        StylesheetRenderingConstants.DEFAULT_TEXTFIELD_COL_WIDTH;
        m_w2StrText.getTextWidget().setLayoutData( m_w2StrTextGd );

```

```

m_w2IntegerLabel = new Label( m_composite, SWT.None );
m_w2IntegerText = new TextControl( m_composite, SWT.SINGLE | SWT.FLAT | SWT.BORDER );
m_w2IntegerText.getTextWidget().setLayoutData( m_w2StrTextGd );

m_w2DateLabel = new Label( m_composite, SWT.None );
m_w2DateButton = new DateControl( m_composite );

m_w2ObjDescLabel = new Label( m_composite, SWT.None );
m_w2ObjDescText = new TextControl( m_composite, SWT.WRAP | SWT.MULTI | SWT.V_SCROLL
| SWT.BORDER );

GridData taGd = new GridData();
taGd.horizontalAlignment = SWT.FILL;
taGd.verticalAlignment = SWT.CENTER;
// gd.grabExcessHorizontalSpace = true;
taGd.widthHint = 30 * StylesheetRenderingConstants.DEFAULT_TEXTAREA_COL_WIDTH;
taGd.heightHint = 5 * StylesheetRenderingConstants.DEFAULT_TEXTAREA_ROW_HEIGHT;
m_w2ObjDescText.getTextWidget().setLayoutData( taGd );

//      m_lovLabel = new Label( m_composite, SWT.None );
//      m_lovComboBox = new LOVComboBox(m_composite, SWT.None);

CheckInOutComposite m_cicoComposite = new CheckInOutComposite( mainComposite );
m_cicoComposite.panelLoaded();

AIFUtility.getDefaultSession().addAIFComponentEventListener( this );
}

/*
 * Currently this is just getting the TCComponent object
 * from the viewer input.
 */
@Override
public void setInput( Object viewerInput )
{
    TCComponentViewerInput input = (TCComponentViewerInput)
AdapterUtil.getAdapter( viewerInput,
    TCComponentViewerInput.class );
    m_comp = (TCComponent)input.getViewableObj();
    super.setInput( m_comp );
}

@Override
public void inputChanged( Object input, Object oldInput )
{
    MyPropertyViewerContentProvider cp = (MyPropertyViewerContentProvider)
getContentProvider();

    m_w2StrLabel.setText( cp.getPropertyDisplayName(STR_PROP) );
    m_w2StrProp = cp.getTCProperty(STR_PROP);
    m_w2StrText.getTextWidget().setText( m_w2StrProp.getStringValue() );

    m_w2IntegerLabel.setText( cp.getPropertyDisplayName( INT_PROP ) );
    m_w2IntProp = cp.getTCProperty( INT_PROP );
    int ii = m_w2IntProp.getIntValue();
    if( !m_w2IntProp.getNullVerdict() )
    {
        m_w2IntegerText.getTextWidget().setText( Integer.toString(ii) );
    }
}

```

```

    }

    m_w2DateLabel.setText(cp.getPropertyDisplayName( DATE_PROP ));
    m_w2DateProp = cp.getTCProperty( DATE_PROP );
    m_w2DateButton.setDate( m_w2DateProp.getDateValue() );

    m_w2ObjDescLabel.setText(cp.getPropertyDisplayName("object_desc"));
    m_w2ObjDescProp = cp.getTCProperty("object_desc");
    m_w2ObjDescText.getTextWidget().setText(m_w2ObjDescProp.getStringValue());

//    m_lovLabel.setText(cp.getPropertyDisplayName("w2Str_e"));
//    m_lovProp = cp.getTCProperty("w2Str_e");
//    m_lovComboBox.setLOVComponent(m_lovProp.getLOV());
//    m_lovComboBox.setSelectedItem( m_lovProp.getPropertyData() );

    //Notify the host viewer to reload.
    ViewerEvent viewerEvent = new ViewerEvent( this, IViewerEvent.RELOADVIEW );
    viewerEvent.queueEvent();
}

@Override
public Control getControl()
{
    return m_composite;
}

@Override
public void refresh()
{
    if( m_composite == null || m_composite.isDisposed() )
    {
        return;
    }
    m_composite.layout();

    if( m_comp.isCheckedOut() )
    {
        // Make panel writable
        m_w2StrText.getTextWidget().setEditable(true);
        m_w2IntegerText.getTextWidget().setEditable(true);
        m_w2DateButton.setEnabled(true);
        m_w2ObjDescText.getTextWidget().setEditable(true);
    }
    else
    {
        // make panel read-only or just show values in label like OOTB behavior.
        m_w2StrText.getTextWidget().setEditable(false);
        m_w2IntegerText.getTextWidget().setEditable(false);
        m_w2DateButton.setEnabled(false);
        m_w2ObjDescText.getTextWidget().setEditable(false);
    }
}

// @Override
// public Object getAdapter( Class adapter )
// {
//     if( adapter.equals( IContributionItem[].class ) )
//     {
//         return getCommandContributions();
//     }

```

```

//
//      return null;
//  }

//  private IContributionItem[] getCommandContributions()
//  {
//      List<IContributionItem> list = new ArrayList<IContributionItem>();
//
//      CommandContributionItemParameter contributionParameters = new
CommandContributionItemParameter(
//          PlatformUI.getWorkbench(), "", "com.teamcenter.rac.checkOut",
CommandContributionItem.STYLE_PUSH );
//      contributionParameters.mode = CommandContributionItem.MODE_FORCE_TEXT;
//      list.add( new CommandContributionItem( contributionParameters ) );
//
//      contributionParameters = new
CommandContributionItemParameter( PlatformUI.getWorkbench(), "",
//          "com.teamcenter.rac.checkIn", CommandContributionItem.STYLE_PUSH );
//      contributionParameters.mode = CommandContributionItem.MODE_FORCE_TEXT;
//      list.add( new CommandContributionItem( contributionParameters ) );
//
//      contributionParameters = new
CommandContributionItemParameter( PlatformUI.getWorkbench(), "",
//          "com.teamcenter.rac.saveCheckOut", CommandContributionItem.STYLE_PUSH );
//      contributionParameters.mode = CommandContributionItem.MODE_FORCE_TEXT;
//      list.add( new CommandContributionItem( contributionParameters ) );
//
//      contributionParameters = new
CommandContributionItemParameter( PlatformUI.getWorkbench(), "",
//          "com.teamcenter.rac.cancelCheckOut", CommandContributionItem.STYLE_PUSH );
//      contributionParameters.mode = CommandContributionItem.MODE_FORCE_TEXT;
//      list.add( new CommandContributionItem( contributionParameters ) );
//
//      return list.toArray( new IContributionItem[list.size()] );
//  }

// Implementation of ISaveablePart
@Override
public boolean isDirty()
{
    // Check if the value has been changed
    String txt = m_w2StrText.getTextWidget().getText();
    if( !txt.equals(m_w2StrProp.getStringValue()) )
    {
        return true;
    }

    txt = m_w2IntegerText.getTextWidget().getText();
    if( !txt.isEmpty() && m_w2IntProp.getNullVerdict() ||
        txt.isEmpty() && !m_w2IntProp.getNullVerdict() )
    {
        return true;
    }
    if(!txt.isEmpty() && !txt.equals(Integer.toString(m_w2IntProp.getIntValue())))
    {
        return true;
    }

    Date date = m_w2DateButton.getDate();
    if( (date == null && !m_w2DateProp.getNullVerdict()) ||

```

```

        (date != null && m_w2DateProp.getNullVerdict()))
    {
        return true;
    }
    if( date != null && !date.equals(m_w2DateProp.getDateValue()) )
    {
        return true;
    }

    txt = m_w2ObjDescText.getTextWidget().getText();
    if( !txt.equals(m_w2ObjDescProp.getStringValue()) )
    {
        return true;
    }

//      Object obj = m_lovComboBox.getSelectedObject();
//      if( obj == null && !m_lovProp.getNullVerdict() ||
//      obj != null && m_lovProp.getNullVerdict() )
//      {
//          return true;
//      }
//      if( obj != null && !obj.equals(m_lovProp.getPropertyData()) )
//      {
//          return true;
//      }
    return false;
}

// Implementation of ISaveablePart
@Override
public boolean isSaveAsAllowed()
{
    return false;
}

// Implementation of ISaveablePart
@Override
public boolean isSaveOnCloseNeeded()
{
    return true;
}

// Implementation of ISaveablePart
@Override
public void doSaveAs()
{
    //not supported
}

// Implementation of ISaveablePart
@Override
public void doSave( final IProgressMonitor monitor )
{
    Exception ex2 = null;
    try
    {
        // If you want to setBusy(true), make sure it's reset to false after operation
        // is done.
        // Otherwise, after you modify the object, then select another object, the
        confirmation dialog
    }

```

```

        // about saving changes will not be displayed as the code considers the Viewer
is still busy.
        //setBusy( true );

        // Save any changes in the viewer
        TCProperty[] props = new TCProperty[] {m_w2StrProp, m_w2IntProp, m_w2DateProp,
m_w2ObjDescProp};
        //TCProperty[] props = new TCProperty[] {m_w2ObjDescProp, m_lovProp};

        String txt = m_w2StrText.getTextWidget().getText();
        m_w2StrProp.setStringValueData(txt);

        txt = m_w2IntegerText.getTextWidget().getText();
        if( txt.isEmpty() )
        {
            m_w2IntProp.setNullVerdict(true);
        }
        else
        {
            int ii = Integer.parseInt(txt);
            m_w2IntProp.setIntValueData(ii);
        }

        Date date = m_w2DateButton.getDate();
        m_w2DateProp.setDateValueData(date);

        txt = m_w2ObjDescText.getTextWidget().getText();
        m_w2ObjDescProp.setStringValueData(txt);

//        Object obj = m_lovComboBox.getSelectedObject();
//        if( obj == null )
//        {
//            m_lovProp.setNullVerdict(true);
//        }
//        else
//        {
//            m_lovProp.setStringValueData(obj.toString());
//        }
        SoaPropertyHelper.setPropertiesService( m_comp, props );
    }
    catch( Exception e )
    {
        ex2 = e;
        MessageBox.post(e);
    }
    finally
    {
        ClientEventDispatcher.fireEventLater( MyPropertyViewer.this,
            IClientEvent.SS_VIEWER_SAVE_COMPLETE, TCComponent.class, m_comp,
Exception.class,
            ex2 );
    }
}

// Implement InterfaceAIFComponentEventListener
@Override
public void processComponentEvents( AIFComponentEvent[] events )
{
    if( getInput() == null )
    {

```

```

        // nothing to be done.
        return;
    }

    boolean requireUpdate = false;
    Arrays.sort( events );
    for( AIFComponentEvent event : events )
    {
        InterfaceAIFComponent targetComponent = event.getComponent();

        if( targetComponent == getInput() ) // changed.
        {
            if( event instanceof AIFComponentPropertyChangeEvent )
            {
                // For property change event, don't need to refresh the whole panel
            }
            else if( event instanceof AIFComponentChangeEvent )
            {
                requireUpdate = true;
            }
        }
    }

    if( requireUpdate )
    {
        SWTUIUtilities.asyncExec( new Runnable()
        {
            // update the check in/out UI widgets only
            @Override
            public void run()
            {
                refresh();
            }
        } );
    }
}
}
}

```

For the **Check-in**, **Check-out**, and **Cancel Check-out** buttons, you do not need to implement the **IAdaptable** or **ISaveablePart** methods. Instead, create the **CheckInOutComposite** component and add it to your viewer, for example:

```

CheckInOutComposite m_cicoComposite = new
CheckInOutComposite( mainComposite );
m_cicoComposite.panelLoaded();

```

In the **doSave()** method, run the save complete event so that checkin can be launched after saving for the **Check-in** button:

```

ClientEventDispatcher.fireEventLater( MyPropertyViewer.this,
    IClientEvent.SS_VIEWER_SAVE_COMPLETE, TCCComponent.class, m_comp,
    Exception.class,
    ex2 );

```

For **Form** objects, the form display in the **Viewer** view is now in SWT tagging. To use the legacy Swing **FormViewer** object in the **Viewer** view for all forms, modify the

defaultViewerConfig.VIEWERCONFIG preference and add the following entry: **Form.FORMViewer**. To use the legacy Swing **FormViewer** object for a certain form type, add an entry to the **defaultViewerConfig.VIEWERCONFIG** preference for the form type that must be displayed in the legacy view, for example, *form-type-name.FORMViewer*.

- E. Replace the code in the **MyPropertyViewerContentProvider.java** file with the following:

```
package com.mycom.customviewerview.viewer;

import org.eclipse.jface.viewers.IContentProvider;
import org.eclipse.jface.viewers.Viewer;

import com.teamcenter.rac.kernel.TCComponent;
import com.teamcenter.rac.kernel.TCException;
import com.teamcenter.rac.kernel.TCProperty;
import com.teamcenter.rac.kernel.TCPropertyDescriptor;
import com.teamcenter.rac.util.AdapterUtil;

/**
 * ContentProvider for the MyPropertyViewer
 */
public class MyPropertyViewerContentProvider implements IContentProvider
{
    private TCComponent m_comp;

    @Override
    public void dispose()
    {
    }

    @Override
    public void inputChanged( Viewer viewer, Object oldInput, Object
newInput )
    {
        String[] propNames = new String[] { "w2str", "w2integer", "w2date",
"object_desc" };
        m_comp = (TCComponent)AdapterUtil.getAdapter(newInput,
TCComponent.class);
        if( m_comp == null )
        {
            return;
        }

        try
        {
            // Pre-load properties in one call to reduce the network calls.
            m_comp.getTCProperties(propNames);
        }
        catch (TCException e)
        {
            e.printStackTrace();
        }
    }

    /**
     * Return the TCProperty
     * @param propName The property name

```



```

    * @return TCProperty
    */
    public TCProperty getTCProperty(String propName)
    {
        try
        {
            return m_comp.getTCProperty(propName);
        }
        catch (TCEException e)
        {
            e.printStackTrace();
        }
        return null;
    }

    /**
     * Return the property's display name
     * @param propName The property name
     * @return String
     */
    public String getPropertyDisplayName( String propName )
    {
        try
        {
            TCPropertyDescriptor propDesc = m_comp.getTypeComponent()
                .getPropertyDescriptor(propName);
            return propDesc.getDisplayName();
        }
        catch (TCEException e)
        {
            e.printStackTrace();
        }
        return propName;
    }

    /**
     * Return the property value
     * @param propName The property name
     * @return Object
     */
    public Object getPropertyValue(String propName)
    {
        try
        {
            TCProperty prop = m_comp.getTCProperty(propName);
            if( prop != null )
            {
                return prop.getPropertyData();
            }
        }
        catch (Exception e)
        {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
        return null;
    }
}

```

F. Click the **plugin.xml** tab and add the code in **bold** to the **plugin.xml** file:

```

<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.4"?>
<plugin>
  <extension
    point="com.teamcenter.rac.viewer.ViewerViewRegistry">
    <viewer
      autoCheckout="false"
      factoryClassName="com.mycom.customviewerview.factory.
HelloWorldViewerFactory"
      id="HelloWorldViewer"
      isSwing="false"
      priority="100">
      <enableWhen
        checkEnabled="false">
      </enableWhen>
    </viewer>
    <viewer
      autoCheckout="false"
      factoryClassName="com.mycom.customviewerview.factory.
MyPropertyViewerFactory"
      id="MyPropertyViewer"
      isSwing="false"
      priority="100">
      <enableWhen
        checkEnabled="false">
      </enableWhen>
    </viewer>
    <viewer
      autoCheckout="false"
      factoryClassName="com.teamcenter.rac.common.tcvviewer.factory.
SwingViewerFactory"
      id="SwingPropertyViewer"
      isSwing="true"
      priority="100"
      viewPanel="com.teamcenter.rac.common.tcvviewer.GenericViewer">
      <enableWhen
        checkEnabled="false">
      </enableWhen>
    </viewer>
  </extension>
</plugin>

```

Note:

Copying and pasting content into the **plugin.xml** file is a quick way to provide you with extensions for this example. However, when you create your own extensions, instead of placing content directly in the **plugin.xml** file, you work in the project's **Extensions** tab. Therefore, review the **Extensions** tab to see how you can create extensions yourself.

- G. Choose **File**→**Save All**.
- b. Verify the customization.

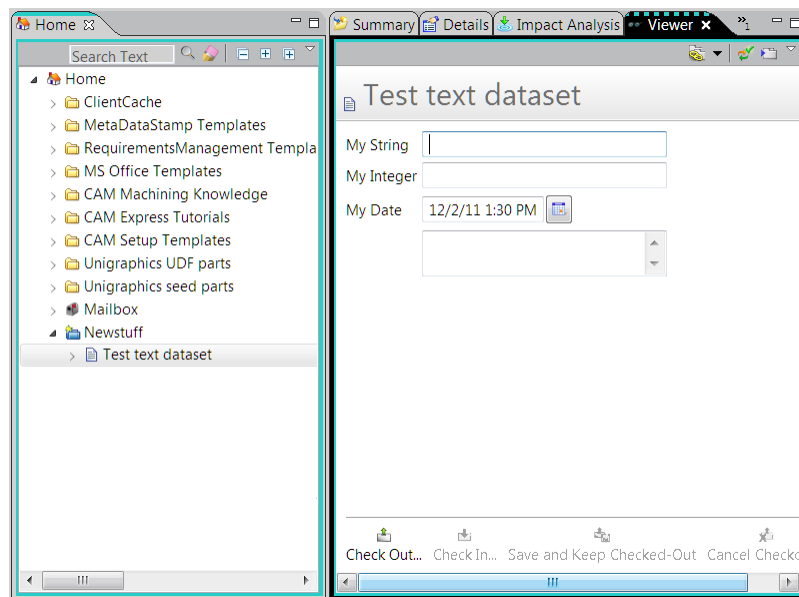
- A. Choose **Run**→**Run Configurations** to start the rich client from your Eclipse environment. Clear the workspace by selecting the **Clear** check box in the launch configuration dialog box.
- B. Edit the **defaultViewerConfig.VIEWERCONFIG** preference to point to the new **MyPropertyViewer** viewer when you select a text dataset.

In My Teamcenter, choose **Edit**→**Options**→**Search**, open the preference, and change the **Text.HelloWorldViewer** value to **Text.MyPropertyViewer** to load the other viewer when you select a text dataset.

- C. Select the text dataset and click the **Viewer** view.

The **Viewer** view displays the dataset information using the **MyPropertyViewer** definition.

Notice the checkin and checkout button at the bottom of the view.



Viewer view using the MyPropertyViewer definition

6. Package the project.

If you want to package the project for distribution, export it to a JAR file and place it in the **TC_ROOT\portal** directory.

- a. Right-click the project, choose **Export**→**Plug-in Development**→**Deployable plug-ins and fragments**, select the **TC_ROOT\portal** directory as the destination, and click **Finish**.

The **com.mycom.project-name** JAR file is automatically generated into the **TC_ROOT\portal\plugins** directory.

- b. Run the `TC_ROOT\portal\registry\genregxml` file to register the plug-in with the rich client.

When the script is finished, a new `TC_ROOT\portal\registry\RegistryLoader.xml.gz` file is created.

Note:

If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory.

- c. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the `%HOMEDRIVE%%HOMEPATH%\Teamcenter` directory. On a Linux client, it is typically the `$HOME/Teamcenter/` directory.

Add a toggle menu item

You can create toggle menu items and tie them to state inside the rich client. To illustrate the process, this example adds an **Error Level** menu with toggled menu items. Select an object in My Teamcenter and select the toggle on the custom **Error Level** menu.

1. Create the project.
 - a. In Eclipse, choose **File**→**New**→**Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Then click **Next**.
 - c. In the **New Plug-in Project** dialog box **Plug-in Project** pane, type **com.mycom.toggle** in the **Project name** box. Click **Next**.
 - d. In the **New Plug-in Project** dialog box **Content** pane, do the following:
 - A. Under **Options**, ensure the **This plug-in will make contributions to the UI** check box is selected.
 - B. Click the **No** button next to **Would you like to create a rich client application?**
 - C. Click **Next**.
 - e. Ensure the **Create a plug-in using one of these templates** check box is selected and select **Hello, World Command** in the list. Click **Finish**.

2. Update the project tabs.

- a. Click the **Dependencies** tab, click the **Add** button, and select the following:

```
com.teamcenter.rac.aifrcp
com.teamcenter.rac.common
com.teamcenter.rac.kernel
com.teamcenter.rac.util
```

- b. Click the **Runtime** tab, click the **Add** button, and select the following:

```
com.mycom.toggle
com.mycom.toggle.handlers
```

3. Create the project files.

- a. Edit the **com.mycom.toggle** plug-in's **plugin.xml** file. Replace the entire contents of the **plugin.xml** file with the following code:

```
<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.4"?>
<plugin>
  <extension
    point="org.eclipse.ui.commands">
    <category
      name="Sample Category"
      id="com.teamcenter.rac.toggleexample.commands.category">
    </category>
    <command
      categoryId="com.mycom.toggle.commands.category"
      defaultHandler="com.mycom.toggle.handlers.ToggleInfoHandler"
      id="com.mycom.toggle.commands.infoCommand"
      name="Info">
    <state
      class="org.eclipse.ui.handlers.RegistryToggleState:true"
      id="org.eclipse.ui.commands.toggleState">
    </state>
    </command>
    <command
      categoryId="com.mycom.toggle.commands.category"
      defaultHandler="com.mycom.toggle.handlers.ToggleWarnHandler"
      id="com.mycom.toggle.commands.warnCommand"
      name="Warn">
    <state
      class="org.eclipse.ui.handlers.RegistryToggleState:false"
      id="org.eclipse.ui.commands.toggleState">
    </state>
    </command>
    <command
      categoryId="com.mycom.toggle.commands.category"
      defaultHandler="com.mycom.toggle.handlers.ToggleErrorHandler"
      id="com.mycom.toggle.commands.errorCommand"
      name="Error">
    <state
      class="org.eclipse.ui.handlers.RegistryToggleState:false"
```

```

        id="org.eclipse.ui.commands.toggleState">
    </state>
</command>
</extension>
<extension
    point="org.eclipse.ui.menus">
    <menuContribution
        locationURI="menu:org.eclipse.ui.main.menu?after=additions">
        <menu
            id="com.mycom.toggle.menus.sampleMenu"
            label="Error Level"
            mnemonic="L">
            <command
                commandId="com.mycom.toggle.commands.infoCommand"
                label="Info"
                mnemonic="I"
                style="toggle">
            <visibleWhen>
                <reference
                    definitionId="com.teamcenter.rac.ui.inMainPerspective">
                </reference>
            </visibleWhen>
            </command>
            <command
                commandId="com.mycom.toggle.commands.warnCommand"
                label="Warn"
                mnemonic="W"
                style="toggle">
            <visibleWhen>
                <reference
                    definitionId="com.teamcenter.rac.ui.inMainPerspective">
                </reference>
            </visibleWhen>
            </command>
            <command
                commandId="com.mycom.toggle.commands.errorCommand"
                label="Error"
                mnemonic="E"
                style="toggle">
            <visibleWhen>
                <reference
                    definitionId="com.teamcenter.rac.ui.inMainPerspective">
                </reference>
            </visibleWhen>
            </command>
        </menu>
    </menuContribution>
</extension>
</plugin>

```

- b. Open the **com.mycom.toggle.handlers** package, select the **SampleHandler.java** file, choose **File→Rename**, and rename the file to **ToggleInfoHandler.java**.

Replace the contents of the **ToggleInfoHandler.java** file with the following:

```

package com.mycom.toggle.handlers;
import org.eclipse.core.commands.AbstractHandler;
import org.eclipse.core.commands.Command;

```

```

import org.eclipse.core.commands.ExecutionEvent;
import org.eclipse.core.commands.ExecutionException;
import org.eclipse.ui.IWorkbenchWindow;
import org.eclipse.ui.handlers.HandlerUtil;
import org.eclipse.jface.dialogs.MessageDialog;
import com.teamcenter.rac.util.Registry;
/**
 * Our sample handler extends AbstractHandler, an IHandler base class.
 * @see org.eclipse.core.commands.IHandler
 * @see org.eclipse.core.commands.AbstractHandler
 */
public class ToggleInfoHandler extends AbstractHandler {

    private Registry reg ;
    /**
     * The constructor.
     */
    public ToggleInfoHandler() {
    }
    /**
     * the command has been executed, so extract the needed information
     * from the application context.
     */
    public Object execute(ExecutionEvent event) throws ExecutionException {
        Command command = event.getCommand();
        boolean checked = HandlerUtil.toggleCommandState(command);
        IWorkbenchWindow window = HandlerUtil
            .getActiveWorkbenchWindowChecked(event);
        reg = Registry.getRegistry(this);
        if (checked) {
            MessageDialog.openInformation(window.getShell(),
reg.getString("Info.TITLE"),
                reg.getString("Info.MESSAGE"));
        }
        return null;
    }
}

```

- c. Right-click the **com.mycom.toggle.handlers** package, choose **New→Class**, and add the following Java files:

ToggleErrorHandler.java
ToggleWarnHandler.java

- d. Replace the contents of the **ToggleErrorHandler.java** file with the following:

```

package com.mycom.toggle.handlers;
import org.eclipse.core.commands.AbstractHandler;
import org.eclipse.core.commands.Command;
import org.eclipse.core.commands.ExecutionEvent;
import org.eclipse.core.commands.ExecutionException;
import org.eclipse.jface.dialogs.MessageDialog;
import org.eclipse.ui.IWorkbenchWindow;
import org.eclipse.ui.handlers.HandlerUtil;
import com.teamcenter.rac.util.Registry;
public class ToggleErrorHandler extends AbstractHandler {

```

```

private Registry reg ;
public ToggleErrorHandler() {
}
@Override
public Object execute(ExecutionEvent event) throws ExecutionException {
    Command command = event.getCommand();
    boolean checked = HandlerUtil.toggleCommandState(command);
    IWorkbenchWindow window = HandlerUtil
        .getActiveWorkbenchWindowChecked(event);
    reg = Registry.getRegistry(this);
    if (checked) {
        MessageDialog.openError(window.getShell(),
reg.getString("Error.TITLE"),
        reg.getString("Error.MESSAGE"));
    }
    return null;
}
}

```

- e. Replace the contents of the **ToggleWarnHandler.java** file with the following:

```

package com.mycom.toggle.handlers;
import org.eclipse.core.commands.AbstractHandler;
import org.eclipse.core.commands.Command;
import org.eclipse.core.commands.ExecutionEvent;
import org.eclipse.core.commands.ExecutionException;
import org.eclipse.jface.dialogs.MessageDialog;
import org.eclipse.ui.IWorkbenchWindow;
import org.eclipse.ui.handlers.HandlerUtil;
import com.teamcenter.rac.util.Registry;
public class ToggleWarnHandler extends AbstractHandler {
    private Registry reg ;

    public ToggleWarnHandler() {
    }
    @Override
    public Object execute(ExecutionEvent event) throws ExecutionException {

        Command command = event.getCommand();
        boolean checked = HandlerUtil.toggleCommandState(command);
        IWorkbenchWindow window = HandlerUtil
            .getActiveWorkbenchWindowChecked(event);
        reg = Registry.getRegistry(this);
        if (checked) {
            MessageDialog.openWarning(window.getShell(),
reg.getString("Warn.TITLE"),
            reg.getString("Warn.MESSAGE"));
        }
        return null;
    }
}

```

- f. Right-click the **com.mycom.toggle.handlers** package, choose **New→File**, and add a **handlers.properties** file.
- g. Replace the contents of the **handlers.properties** file with the following:


```
Info.TITLE=ToggleInfo
Info.MESSAGE=Info menu is toggled off
Error.TITLE=ToggleError
Error.MESSAGE=Error menu is toggled off
Warn.TITLE=ToggleWarn
Warn.MESSAGE=Warn menu is toggled off
```

h. Choose **File**→**Save All**.

4. Package the project.

To package the project for distribution, export it to a JAR file and place it in the `TC_ROOT\portal` directory.

a. Right-click the project, choose **Export**→**Plug-in Development**→**Deployable plug-ins and fragments**, select the `TC_ROOT\portal` directory as the destination, and click **Finish**.

The `com.mycom.project-name` JAR file is automatically generated into the `TC_ROOT\portal\plugins` directory.

b. Run the `TC_ROOT\portal\registry\genregxml` file to register the plug-in with the rich client.

When the script is finished, a new `TC_ROOT\portal\registry\RegistryLoader.xml.gz` file is created.

Note:

If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the `portal\Registry` directory.

c. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the `%HOMEDRIVE%%HOMEPATH%\Teamcenter` directory. On a Linux client, it is typically the `$HOME/Teamcenter/` directory.

5. Verify the customization.

a. Launch My Teamcenter and choose **Error Level** on the menu bar.

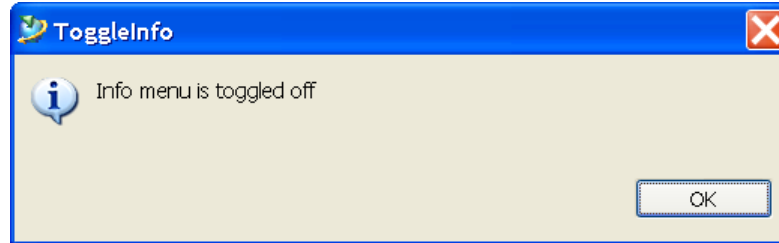
The error levels are displayed.



Error Level toggle menus

- b. Select toggles on the menu.

When you clear a toggle, a dialog box is displayed.



Toggle state dialog box

Context menu suppression

Overview of context menu suppression

The Teamcenter rich client uses Eclipse's platform menu contribution mechanism for displaying menus and toolbars. By default, all available commands are shown on a menu or toolbar unless the Teamcenter Command Suppression application has been configured by the administrator. Due to the nature of the platform, Command Suppression is unable to control the contents of a context menu.

Note:

A context menu is also referred to as a shortcut menu, right-click menu, or popup menu.

To reduce clutter, context menu suppression allows control over the contents of a context menu using an XML file. The elements in this file control which commands are suppressed, and in which view, application context, or with which types of objects.

Implementation

This XML file is stored in the Teamcenter database as a **Context Menu Suppression (Fnd0ContextMenuSuppRuleXML)** dataset and is registered using the **TC_ContextMenuSuppression** preference. The value of the preference is the name of the dataset.

Note:

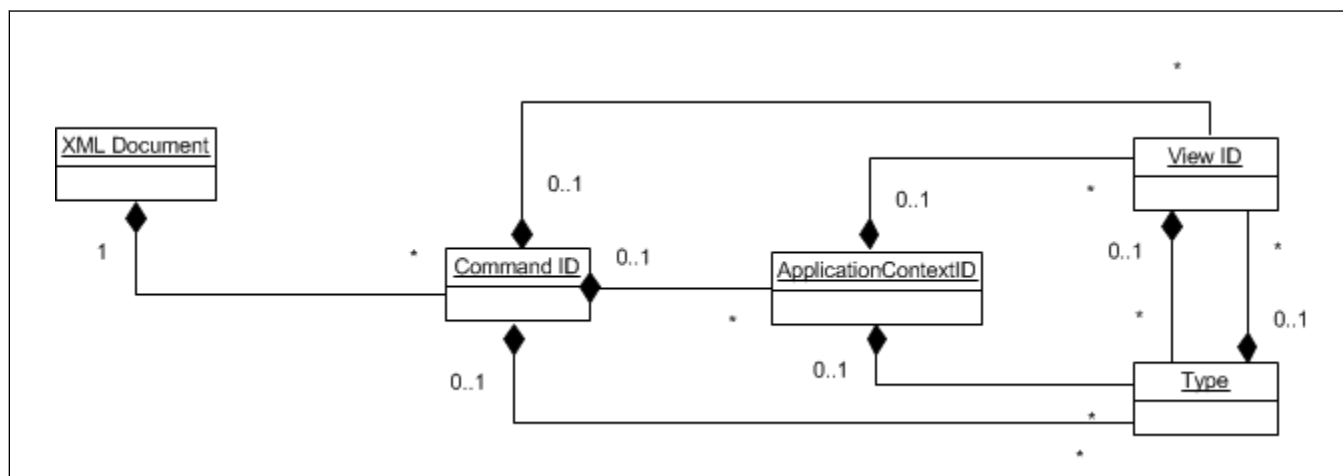
Ensure that **Context Menu Suppression** datasets maintain unique names; however, this is not required by Teamcenter.

The preference can have a site, group, role, or user protection scope, so suppression rules can be implemented based on the user's current credentials. If there are two datasets and two preferences, one pair for each of two groups, a user may see one set of commands on the context menu while in one group, and then, after switching groups, see a different set of commands.

XML file elements

The context menu suppression XML rules consist of the following elements.

- **Command ID**
- **View ID**
- **Type Name**
- **ApplicationContext ID**



XML schema definition

Command ID **Command ID** is a root node for each context menu suppression rule. The **Command ID** node can have one or more **View ID**, **Type Name**, or **ApplicationContext ID** nodes.

View ID Teamcenter applications consist of one or more views, each of which has an associated **View ID**. A rule with a **View ID** node restricts the suppression definition to a specific view. **View ID** elements can hold one or many **Type Name** elements. In such cases, commands can be suppressed for all types in a given view.

Type Name Specifies the database name of the object type for which a command must be suppressed when the selection is made in the rich client interface. This object type can be abstract, persistent, or dynamic.

- A rule defined for an abstract type is inherited by all of its child types.
- A rule defined for a persistent type is applied to the instance selected.
- A rule defined for a dynamic type is applied to the type it represents.

Caution:

Do not write suppression rules for dynamic types like **BOMLine**.

When a **BOMLine** is selected from Structure Manager, if the underlying component is an item revision for example, the suppression rules for item revision are applied instead.

ApplicationContext ID The **ApplicationContext ID** filter can be defined under the root element. This filter allows the administrator to write a suppression rule with respect to a specific application like Structure Manager, My Teamcenter, or Systems Engineering, and so forth. The **ApplicationContext ID** element can hold view or type elements.

When elements are nested, it is considered to be an **AND** grouping, while elements at the same level are considered to be an **OR** grouping.

Example 1

```
<command id="com.teamcenter.rac.expand">
  <view id="com.teamcenter.rac.ui.views.DetailsView"/>
  <view id="com.teamcenter.rac.ui.views.SummaryView"/>
  <type name="Folder"/>
  <type name="Dataset"/>
</applicationContext>
</command>
```

In this example, the **expand** command is suppressed when the current view is the **Details** view or the **Summary** view or if the object type is **Folder** or any **dataset**.

Example 2

```
<command id="org.eclipse.ui.edit.cut">
  <applicationContext id
="com.teamcenter.rac.ui.perspectives.navigatorContext">
    <view id="com.teamcenter.rac.ui.views.TCComponentView">
      <type name="Item"/>
    </view>
```

```

    <view id ="com.teamcenter.rac.ui.views.SummaryView">
      <type name="Folder"/>
    </view>
    <type name="BOMLine">
      <view id ="com.teamcenter.rac.pse.PSEView"/>
    </type>
  </applicationContext>
</command>

```

In this example, the **cut** command is suppressed for the My Teamcenter application when:

```

(type=Item AND view=Teamcenter Component)
OR
(type=Folder AND view=Summary)
OR
(type=BOMLine AND view=Structure Manager)

```

Getting the internal names

Because the XML file requires Teamcenter's *internal* names for objects, commands, views, and application contexts, it is important to know how to find them. Using the display name does not work.

Here are a few common examples.

Display name	Internal name
My Teamcenter (application context)	com.teamcenter.rac.ui.perspectives.navigatorContext
Summary (view)	com.teamcenter.rac.ui.views.SummaryView
Document Revision (object type)	DocumentRevision

To find complete listings of Teamcenter's *internal* names, refer to the following sources:

Objects	The Business Modeler IDE is the best place to find the internal names for all objects.
Commands, views, and application contexts	The DumpCMSConfigInfo utility generates a listing of these names.

XML prolog

A full context menu suppression file must have a proper XML prolog, and all suppression elements must be enclosed within **contextMenuSuppression** tags.

```
<?xml version='1.0' encoding='UTF-8' ?>
<contextMenuSuppression>
  <command id ="org.eclipse.ui.edit.cut">
    <applicationContext id
="com.teamcenter.rac.ui.perspectives.navigatorContext">
      <type name="Item"/>
    </applicationContext>
  </command>
  <command ...>
    ...
  </command>
</contextMenuSuppression>
```

Caveats

- This is not a hierarchical or inheritance system. Only a single dataset is evaluated at any given time. If multiple preferences are defined, for example, a group and a role preference, standard Teamcenter preference evaluation precedence determines which preference is valid, and therefore which dataset is used. If the user changes group or role, the preferences are reevaluated, which may result in a different dataset being used.
- The **Summary** view's **Send To...** menu is built using the same rules as the context menu's **Send To...** command. Suppressing one of the **Send To...** commands from the context menu also suppresses it from the **Summary** view's **Send To...** menu.
- There is no upgrade mechanism provided for the context menu suppression rule XML. Any of the contents related to **Command ID**, **View ID**, and **ApplicationContext ID** are subject to change.

Distributing rich client customizations

Export your custom plug-in to the rich client

To test your custom plug-in, export it as a JAR file to the rich client `TC_ROOT\portal\plugins` directory.

1. In Eclipse, right-click the customization project and choose **Export**.
2. In the **Export** dialog box, choose **Plug-in Development**→**Deployable plug-ins and fragments**.
3. Click **Next**.
4. Click the **Browse** button to the right of the **Directory** dialog box and browse to the `TC_ROOT\portal` directory.
5. Click **Finish**.

The JAR file is automatically generated into the `TC_ROOT\portal\plugins` directory.

Before running the rich client to verify the customization, run the `TC_ROOT\portal\registry\genregxml` file to register the plug-in with the rich client, and clear the rich client cache.

Process for distributing customizations to the rich client

The basic customization technique is to create a plug-in that contains the customizations and deploy the custom plug-in JAR file to the rich client installation's `TC_ROOT\portal\plugins` directory. This way, when you upgrade to a newer version of Teamcenter, you can simply install the custom plug-in without having to alter files in the Teamcenter installation.

Perform the following steps to distribute your rich client customizations created in Eclipse:

1. Package your custom files.
2. Create a feature file to install the custom files.

Package custom rich client files

When you create customizations to the rich client, typically they are comprised of JAR files exported from Eclipse that must be installed to the `TC_ROOT\portal\plugins` directory. Package these JAR files into a ZIP file so you can install them using Teamcenter Environment Manager (TEM).

1. To create a plug-in JAR file of your customization, in Eclipse right-click the project and choose **Export→Plug-in Development→Deployable plug-ins and fragments**.
2. Zip your custom JAR file into a `rac_feature-name.zip` file (for example, `rac_samplecust.zip`) with the root path for the JAR file set to `\plugins`.
The use of `rac_` in the name signifies that the ZIP file contains a rich client feature. For examples of these files, see the `portal\compressed_files` directory on the Teamcenter installation source.
3. Create a directory structure to hold the ZIP files, for example, `portal\compressed_files\`.
You can use any directory structure you want. For convenience, this structure is the same used on the Teamcenter installation source.
4. Copy your ZIP files to the `portal\compressed_files\` directory.
5. Create a feature file so you can install the custom files using TEM.

Create a feature file for rich client customizations

After you have packaged your rich client customization files into a ZIP file, you can create a feature file so that you can install them using Teamcenter Environment Manager (TEM).

1. Package your rich client customization files JAR files into a ZIP file.

2. In a text editor, create an XML feature file that points to the ZIP file, for example, **feature_rac_mycust.xml**.

Tip:

For examples of **feature_rac_feature-name.xml** files, see the **install\modules** directory on the installation source.

Following is an example of a feature file:

```
<?xml version="1.0" encoding="UTF-8"?>
<feature>
  <name value="Rich Client Customizations"/>
  <size value="21"/>
  <os value="windows" version="5.1"/>
  <os value="sunos" version="5.10"/>
  <os value="hp-ux" version="11.11"/>
  <root value="true"/>
  <group value="package"/>
  <singular value="true"/>
  <guid value="7560C67ED5B01B65C39B8DF7364066C8"/>
  <relation>
    <depends>
      <or value="FF18D25DA73019F880BCFFBC0029CA28"/>
      <or value="E2564104E1B964BB23D78078DBA34EEA"/>
      <or value="A2564824E1B434AC23D70178DBA34BCA"/>
    </depends>
  </relation>
  <files>
    <code>
      <unzip file="portal/compressed_files/rac_samplecust.zip"
        todir="portal"/>
    </code>
  </files>
</feature>
```

In the feature file, change the following lines:

- In the **size** tag, change the **value** text to the approximate size (in megabytes) of your customization files.
- Add or remove **os** tags to match the platforms where you use rich client.
- In the **guid** tag, change the **value** text to a unique value exactly 32 characters in length and composed of any combination of the numbers 0–9 and the letters A–F.
- In the **unzip** tag, change the **file** text to the name of the ZIP file that contains your customizations.
To ensure that the **todir="portal"** line correctly unzips to the **portal\plugins** directory, ensure that **plugins** is set as the location in the ZIP file.

Tip:

If your customizations include a packaged Business Modeler IDE template project, you can edit the packaged project's feature file to include the new rich client feature by adding a node similar to the following. This is not required; you can still have a separate feature file for the rich client customizations.

```
<feature>
  <name value="feature-display-name"/>
  <property name="feature_id" value="rac"/>
  <property name="bmide_optional" value="true"/>
  <relation>
    <depends>
      <or value="FF18D25DA73019F880BCFFBC0029CA28"/>
      <or value="E2564104E1B964BB23D78078DBA34EEA"/>
      <or value="A2564824E1B434AC23D70178DBA34BCA"/>
    </depends>
  </relation>
  <size value="0"/>
  <singular value="true"/>
  <guid value="85B3827CD5A0FA61220398F6C60C21AB"/>
  <files>
    <code>
      <unzip file="portal/compressed_files/rac_feature-name.zip"
        todir="portal"/>
    </code>
  </files>
</feature>
```

3. Copy the XML feature file to the `TC_ROOT\install\install\modules` directory on the installation source.
4. Open the `TC_ROOT\install` directory and run Teamcenter Environment Manager (TEM).
5. Click **Configuration Manager** and click **Next**.
6. Click **Perform maintenance on an existing configuration** and click **Next**.
7. Ensure the correct configuration is selected and click **Next**.
8. Ensure **Add/Remove Features** is selected and click **Next**.
9. Under the **Extensions** node, select the check box for your new files. The name of the box is the one you gave it in the feature file. Click **Next**.
10. In the **Confirm Selections** panel, click **Next** to start installing your customization files.

Rich client Javadoc

User interface components documented in Javadoc

Teamcenter provides user interface components that you can use in your rich client customizations. The Javadoc documentation for these rich client components are in the **JavaDoc.zip** file within the *Teamcenter-version_pub.zip* file provided with the Teamcenter installation source. After unzipping the **JavaDoc.zip** file, look at the Javadoc found in the following packages:

- **javadoc\com.teamcenter.rac.common\index.html**
Contains user interface components that you can use in your rich client customizations. For a listing of APIs, see *User interface components in the com.teamcenter.rac.common package*. (The list contains a sample of the components available. For a complete list, see the Javadoc.)
- **javadoc\com.teamcenter.rac.util\index.html**
Contains generic components, layout managers, and JavaBeans. For a listing of APIs, see *User interface components in the com.teamcenter.rac.util package*. (The list contains a sample of the components available. For a complete list, see the Javadoc.)

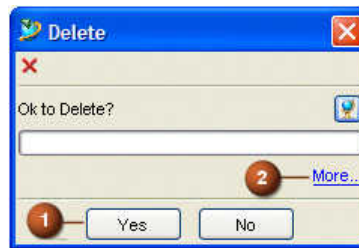
User interface components in the com.teamcenter.rac.common package

AbstractProgressDialog

This class contains the definition for the **AbstractProgressDialog** class, which displays the progress of multiple components individually, as they are processed. The **AbstractProgressDialog** class has the following features:

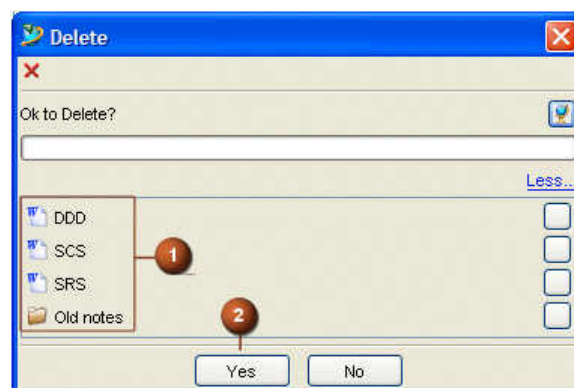
- The status of the operation, such as in-progress, successful completion, or failure of completion, is indicated for each component on the dialog box in the following figure.
- If an operation fails, the tooltip on the error symbol describes the error.
- If the error symbol is clicked, the system displays a detailed error message.
- If confirmation is required before the operation can proceed, the **AbstractProgressDialog** component can display the confirmation flag.
- If confirmation is not required, the operation processes the first component as soon as the dialog box is displayed.
- When an operation is in progress, a **Stop** button displays on the dialog box and can be used to stop the operation. When the delete operation is aborted, the operation in progress on a component cannot be stopped; however, the operation is stopped before the next component is deleted.

The following figures show the behavior of the delete operation.



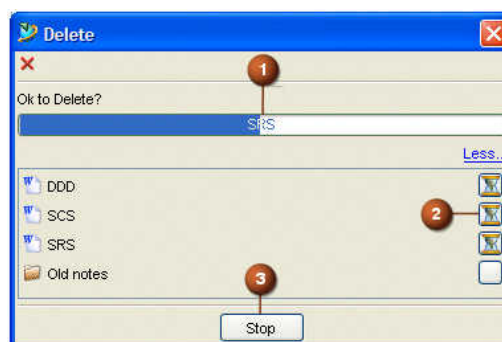
Delete dialog

- 1 Click **Yes** to initiate **Delete** operation.
 - 2 Click **More** to see components to be deleted.
-



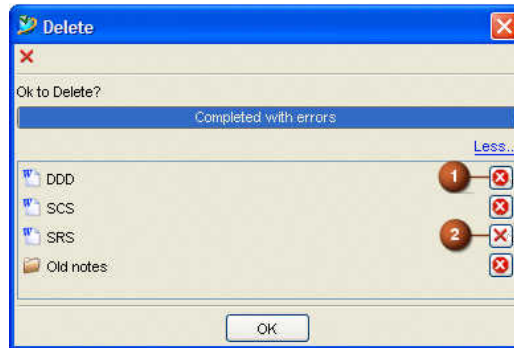
Expanded Delete dialog

- 1 Components to be deleted.
 - 2 Click **Yes** to initiate **Delete** operation.
-



Progress indicators

- 1 Overall progress indicator.
 - 2 Hourglasses indicate components in the midst of being deleted.
 - 3 If **Stop** is clicked, the components being processed are deleted and the operation is stopped.
-



Completion indicators

- 1 Indicates unsuccessful completion of the operation.
 - 2 Indicates successful completion of the operation.
-

The following Teamcenter commands subclass from the **AbstractProgressDialog** class:

- Cut
- Copy
- Paste
- Delete
- Check-In
- Check-Out
- Transfer Check-Out
- Publish
- Unpublish

The following code shows how the **AbstractProgressDialog** class is used to display the components and execute the delete operation. The dialog box calls the **initializeDialog** method in its constructor. The **initializeDialog** method sets the display parameters for the components.

```

public class DeleteDialog extends AbstractProgressDialog
{
    //The Constructor calls the initializeDialog method
    initializeDialog(targets);

    //The initialize Dialog sets the methods for the display in the
    AbstractProgressDialog
    public void initializeDialog (AIFComponentContext[] targets)
    {
        try
        {
            // set the title for the dialog
            setDialogTitle(r.getString("command.TITLE"));
            //display the components that are successfully processed
            setDisplaySuccessComponents(true);

            //Set the icon to be displayed on the dialog
            setCommandIcon(r.getImageIcon("delete.ICON"));

            //Set the icon that needs to be set if the components is successfully processed
            setSuccessIcon(r.getImageIcon("delete.ICON"));
            //set the confirmation to true. The user has to press the Yes button
            //to initiate the operation.
            setConfirmationText(r.getString("confirmationText"));

            // for delete, don't need to show parent
            setShowParentFlag(false);
            // display objects that needs to be cut and their parents on dialog
            setTCComponents(targets);
        }
        catch ( Exception ex )
        {
            //Show the messageBox and return
        }
    }
    // ask user's confirmation before starting operation
    setConfirmationFlag(true);
}
//The abstract method to be implemented to execute the selected operation
protected void getOperations ( AIFComponentContext compContext )
{
    // In here the developer has to use addOperation method to add the
    // operation.
    addOperation ( new DeleteOperation( compContext ) );
}
// There's also an overloaded getOperations() method which can be used by the developers.
protected void getOperations ( TCComponent parentComp, AIFComponentContext[] ctxts )
{
    // Create whatever operation you want to based on the parent component
    // and the contexts.
    // Then use the addOperation() method to add the operation.
    addOperation ( createdOperation );
}
}

```

The **startOperation()** method is called when you click the **Yes** button. This method builds the **DeleteOperation** class, which in turn calls the **startProcess()** method from the **AbstractProgressDialog** class. The **startProcess()** method sets the focus on the first displayed component, sets the in-progress icon against the component, and calls the **getOperations()** method, which is an abstract method that

the subclass must implement. The **getOperations()** method in the **DeleteDialog** class calls the **DeleteOperation** class on the first component.

The methods implemented by the subclass are shown in the following code:

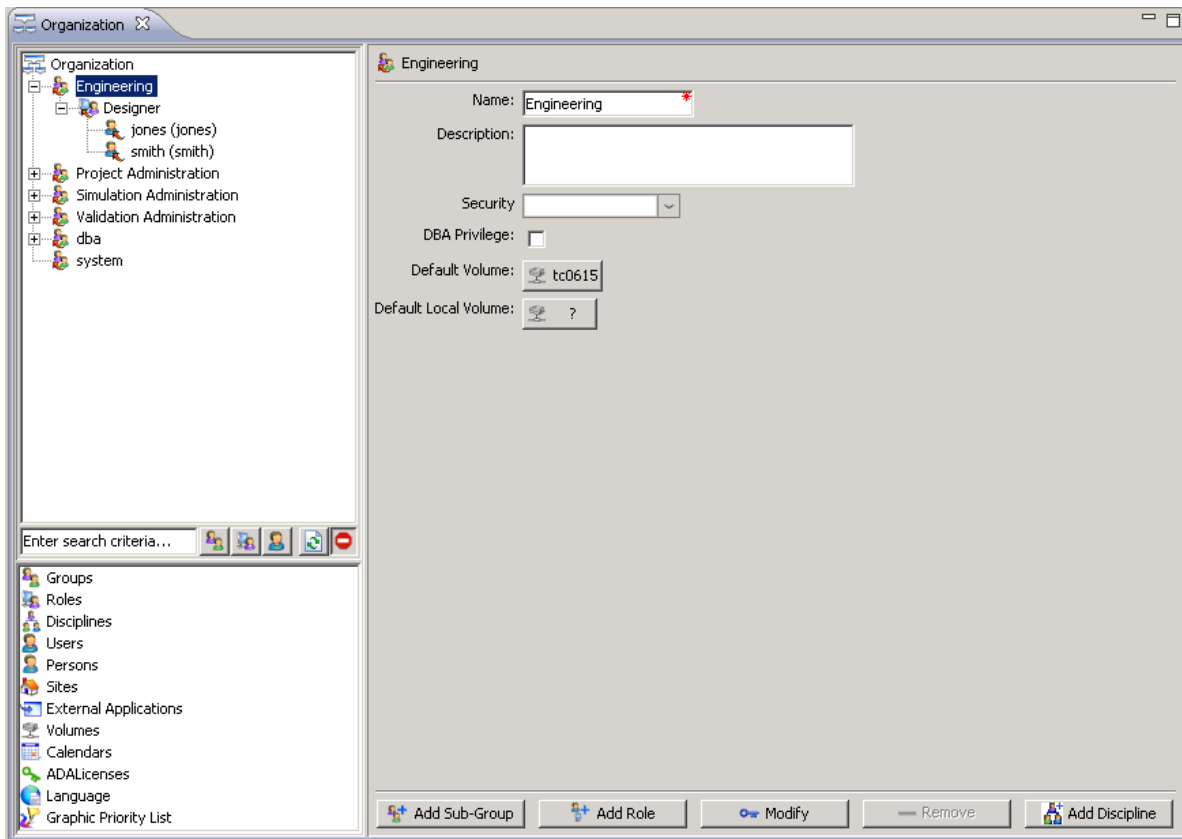
```
setDialogTitle(r.getString("command.TITLE"));
setDisplaySuccessComponents(true);
setCommandIcon(r.getImageIcon("delete.ICON"));
setSuccessIcon(r.getImageIcon("delete.ICON"));
setConfirmationText(r.getString("confirmationText"));
setShowParentFlag(false);
setTCComponents(targets);
setConfirmationFlag(true);
public void run()
public void execute( AIFComponentContext compContext ) throws Exception
(This method builds the appropriate operation on the component)
public void startOperation()
(This method calls the startProcess() method from the AbstractProgressDialog
```

ExpansionRule

The **ExpansionRule** component gets the specific components of related types that are attached to a component. Users can set the relations for specific types of components and use the **getChildren()** method to get the required children. The **ExpansionRule** component also provides the ability to filter out unwanted children.

GroupPanel

The **GroupPanel** reusable component extends the **JPanel** component and can be used by any application to display group information (as shown in the following figure).



Group panel in the Organization Selection dialog box

The **GroupPanel** component can be constructed and added to the dialog box for display (as shown in the following code example):

```
GroupPanel groupPanel = new GroupPanel (grpComponent);;
mainSplitPane.setRightComponent (groupPanel);
```

Lists of values (LOVs)

Lists of values (LOVs) are lists in property boxes in the user interface. In the end-user client applications, the format of the LOV box appears as a table rather than a simple list (except for hierarchical and range LOVs). This allows for the display of columns that show the value's description and other properties. The **LOVDisplayer** Java class is the primary class used to define the list of values display.

Safety Code: 1

Value	Description
1	Meets safety standards
2	Does not meet safety standards
35	Under review for safety
40	Safety approval pending

List of values display

The list of values table format allows users to:

- Sort the values in the columns.
- Scroll the list of values using a scroll bar.
- Resize the box containing the list of values.
- Type in the LOV box to narrow the choices when there are many values to choose from. You can even use an asterisk (*) for wildcard searching.

Note:

If end users type text when looking for a value in the format of an integer, double, or date, they must type the exact value.

Following are the list of values Java classes:

```
com.teamcenter.rac.common.lov.common.ILOVConstants
com.teamcenter.rac.common.lov.common.ILOVDataService
com.teamcenter.rac.common.lov.common.LOVColumnInfo
com.teamcenter.rac.common.lov.common.LOVPropertyKey
com.teamcenter.rac.common.lov.common.LOVPropertyValue
com.teamcenter.rac.common.lov.common.LOVRow
com.teamcenter.rac.common.lov.view.components.LOVDataDisplayView
com.teamcenter.rac.common.lov.view.components.LOVDisplayer
com.teamcenter.rac.common.lov.view.components.LOVSelectionDisplayView
com.teamcenter.rac.common.lov.view.components.LOVTreeTable
com.teamcenter.rac.common.lov.view.components.LOVWindowResizer
com.teamcenter.rac.common.lov.view.controls.LOVDataDisplayView
com.teamcenter.rac.common.lov.view.controls.LOVDisplayer
com.teamcenter.rac.common.lov.view.controls.LOVSelectionDisplayView
com.teamcenter.rac.common.lov.viewmodel.LOVDataDisplayProperties
com.teamcenter.rac.common.lov.viewmodel.LOVDataServiceImpl
com.teamcenter.rac.common.lov.viewmodel.LOVSelectionDisplayProperties
com.teamcenter.rac.common.lov.viewmodel.LOVViewModel
```

Note:

As of Teamcenter 10.1, the following list of values Java classes are deprecated:

```
com.teamcenter.rac.common.lov.InterDependentLOVPanel
com.teamcenter.rac.common.lov.LOVUIComponent
com.teamcenter.rac.common.lov.LOVPanel
com.teamcenter.rac.common.lov.LOVPopupButton
com.teamcenter.rac.common.lov.LOVDialog
com.teamcenter.rac.common.lov.LOVComboBox
```



```
com.teamcenter.rac.common.lov.HierarchicalLOVComponent
com.teamcenter.rac.common.lov.LOVHierarchyPanel
com.teamcenter.rac.common.controls.SWTLovPopupButton
com.teamcenter.rac.common.controls.SWTLovPopupDialog
com.teamcenter.rac.common.controls.LOVUIComponent
com.teamcenter.rac.common.controls.LOVComboBox
```

Note:

In rich client forms, the Tab key moves focus from field to field, except for lists of values. To move focus between lists of values, users can press Ctrl+Tab.

MRUButton

The **MRUButton** component is used by applications to maintain a list of previously referenced **InterfaceAIFComponent** objects. It is generic and works with any AIF application within the AIF framework. When one of the objects is clicked, the **open** method for the application is invoked with the component.

The **MRUButton** component maintains a list of components supplied by the application. The application must populate the components to the **MRUButton** component and determine the number of objects to place inside the component. Different behaviors can be associated with the use of the **MRUButton** component in different applications. A purge limit is commonly associated with the **MRU** button; however, the **MRUButton** component shown in the following figure does not employ a purge limit.

The code in the following example constructs the **MRUButton** component:

```
mruButton = new MRUButton ( explorerApp );
mruButton.setSuggestedVerticalAlignment(MRUButton.TOP);
mruButton.loadMRUEntries();
```

OpenByNameButton

The **OpenByNameButton** component allows an application to open new objects that can be manipulated. It also allows users to issue queries and view properties of the objects located by the query.

The objects that are found by the query are loaded into the table. Instead of loading all objects, only the first set is loaded. To load the next set, click the **Load Next** button. To load all objects, click the **Load All** button. The **Load Previous** button loads the previous set and appends it to the current selection. Once the object is located, double-click the object to open it in the application.

Objects can be selected in the table and copied to the clipboard. The **Copy** button is active only when an item has been selected.

The **OpenByNameButton** component enforces the limitation that a query can only be performed on a single object type, for example, item revisions or folders. This is usually acceptable, as most applications

can only support one root object type. Examples of this are My Teamcenter, which only supports folders as root objects, and Structure Manager, which only supports occurrences as root objects.

The **OpenByNameButton** component is subclassed from the **AbstractPopupButton** class, which allows it to be inserted into the user interface where it is treated like any other Java button.

This composition allows you to add things to uniquely identify the button and its purpose, such as tooltips and icons.

The following code shows how to construct the **OpenByNameButton** component for the My Teamcenter application, which only searches on folders:

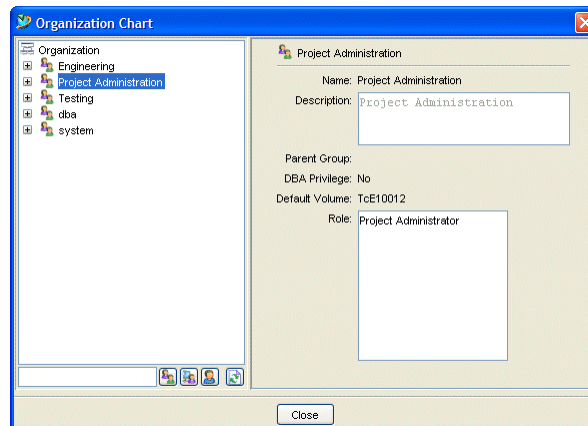
```
OpenByNameButton openByNameButton = new OpenByNameButton(explorerApp, "Folder");
```

OrgSelectionDialog

The **OrgSelectionDialog** reusable component displays the organization chart in tree form. The root of the chart is a root group, and nodes in the tree represent groups, roles, and users. The tree displays hierarchies within the organization.

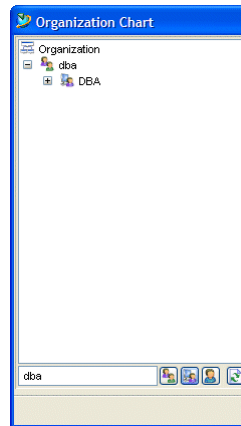
This dialog box consists of a vertical split pane. The left pane displays the tree, and the right pane displays information about the selected role, group, or user node.

The dialog box shown in the following figure displays information about a fictional organization. The first-level nodes in the tree represent groups. Groups can be expanded to display the hierarchical groups or roles contained within the group. Roles can be expanded to display the users assigned to the role.



OrgSelectionDialog component

In addition, the dialog box provides the ability to search for a specific group, role, or user within the organization (as shown in the following figure). If you click the **Reload** button, the tree displays all top-level groups in the organization. Both figures show the features of the **OrgSelectionDialog** component.



Organization dialog box search feature

The **Access** dialog box presents the **OrgSelectionDialog** component as a separate dialog box. The **OrgSelectionDialog** component handles its own events and display; therefore, dialog boxes that use this dialog component must invoke the **OrgSelectionDialog** component. The code in the following example shows the **OrgSelectionDialog** component constructed from the **AccessDialog** component:

```

JButton selectUserButton = new JButton(appReg.getImageIcon("selectUser.ICON"));
selectUserButton.addActionListener ( new ActionListener()
{
    public void actionPerformed (ActionEvent e)
    {
        //Create the OrgSelectionDialog
        orgSelectionDialog = new OrgSelectionDialog(parent, target);
        orgSelectionDialog.addPropertyChangeListener(AccessDialog.this);
        orgSelectionDialog.setVisible(true);
    }
});

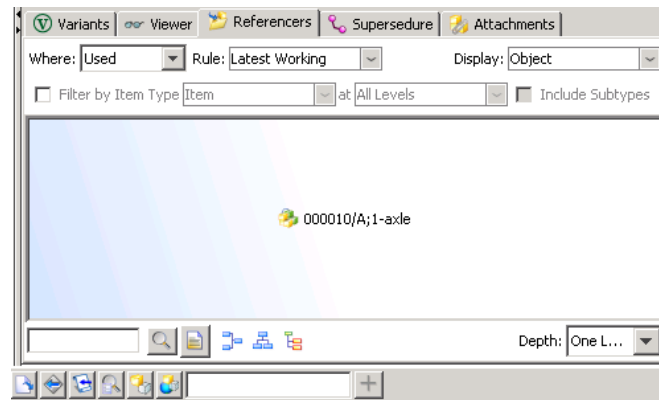
```

The **OrgSelectionDialog** component is constructed and added as a **propertyChangeListener** to the **AccessDialog** component, and the visibility of the dialog component is set to **true**. The Organization Selection displays when you click the **Select User** button in the **Access** dialog box.

ReferencersPanel

The **ReferencersPanel** component displays where-used and where-referenced diagrams. An object can be sent to the **Referencers** panel, where the user can double-click it to search for where the object is used or referenced.

When a component is sent to the **Referencers** panel, whether it defaults to the where-referenced or where-used display depends on the component type. Users can define which components to display in the panel.

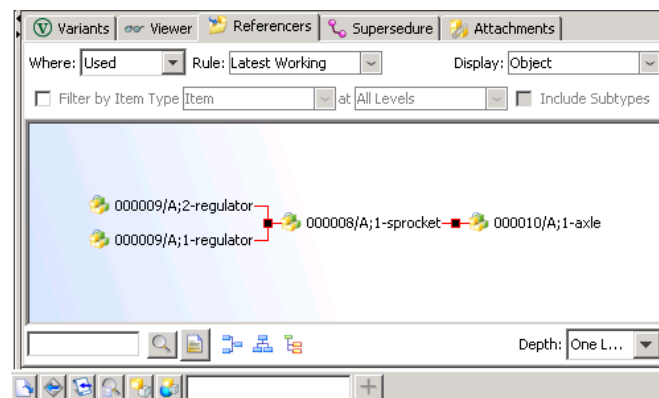


Referencers panel

The **Referencers** panel can display the nodes in reverse horizontal style, vertical style, or tree-look style. Three layout managers are used to accomplish this:

- **ReferencersReverseHorizontalNodeLayout**

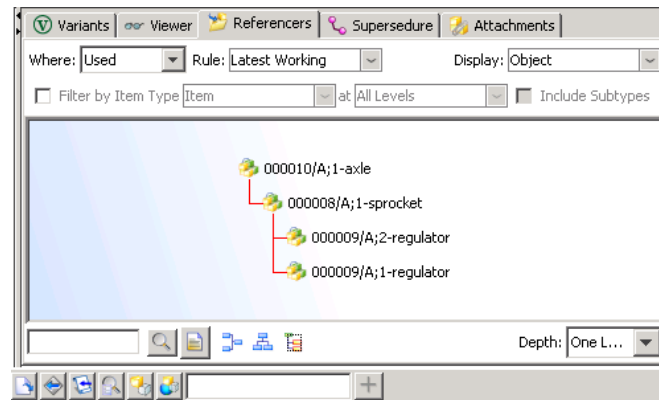
Displays the nodes in reverse horizontal order where the structure is expanded from right to left horizontally and can display where-referenced and where-used information in different colors. The **ReferencersReverseHorizontalNodeLayout** layout manager extends from the **ReverseHorizontalNodeLayout** component.



Referencers reverse horizontal node layout

- **ReferencersTreeLookNodeLayout**

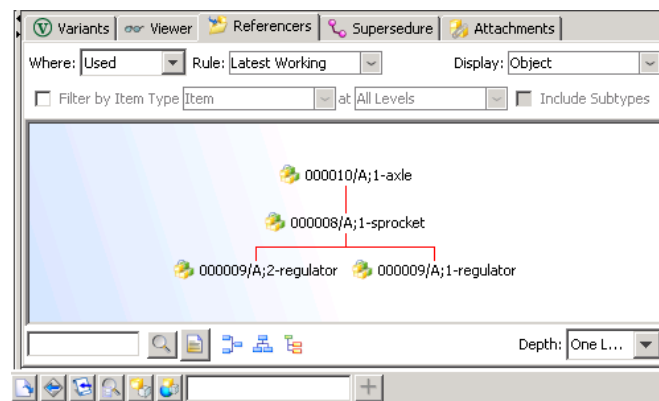
Displays the nodes in a **JTree** manner and can display where-referenced and where-used information in different colors. The **ReferencersTreeLookNodeLayout** layout manager extends from the **TreeLookNodeLayout** component.



Referencers tree look node layout

- **ReferencersVerticalNodeLayout**

Displays the nodes in vertical order expanded from top to bottom and can display different where-referenced and where-used information in different colors. The **ReferencersVerticalNodeLayout** layout manager extends from the **VerticalNodeLayout** component.



Referencers vertical node layout

ReferencerUINode

The **ReferencerUINode** component is an extension of the **TCComponentUINode** component. It adds an attribute that tells if a where-used or where-referenced expansion is associated with the node.

Note:

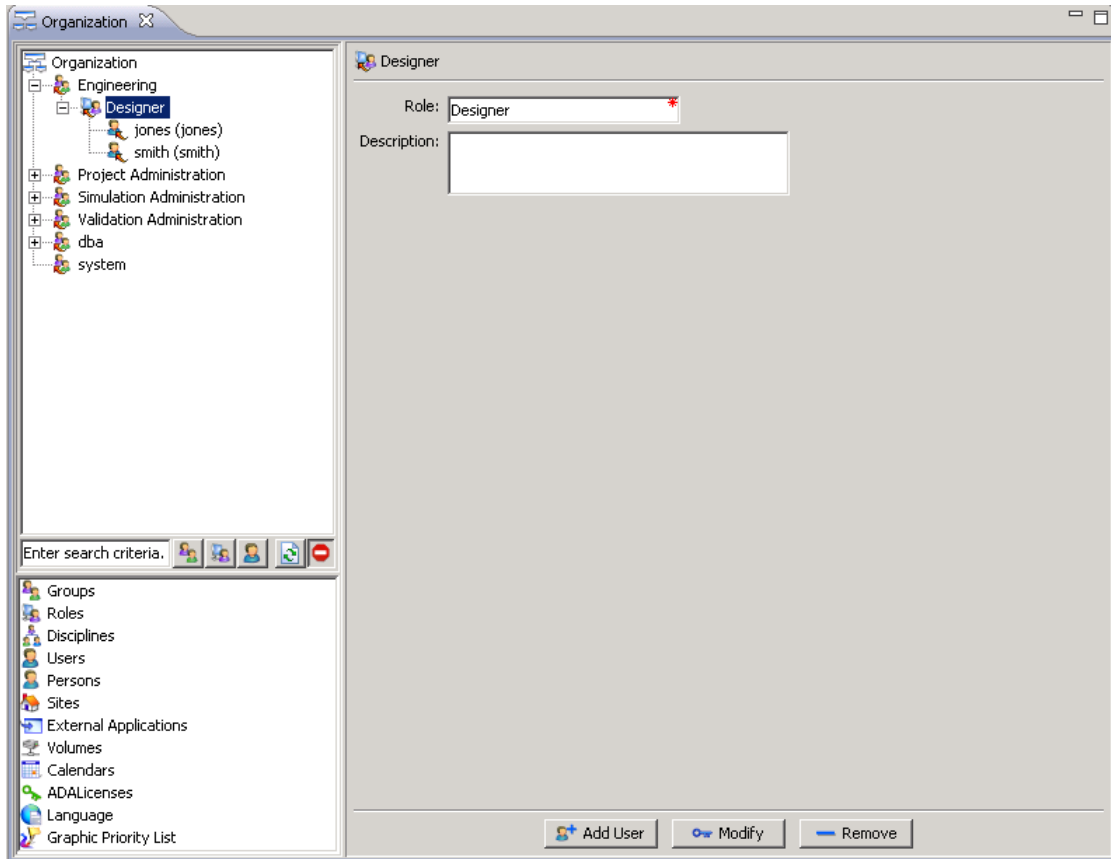
This component is a Swing class. Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens Digital Industries Software encourages you to customize Teamcenter using SWT/JFace components. Siemens Digital Industries Software will discontinue Swing/AWT support in a future version.

This example shows how to construct a **ReferencersPanel** component and set a component in it:

```
referencersPanel = new ReferencersPanel(explorerApp, false, false, false, false);
referencersPanel.setComponent ( c );
```

RolePanel

The **RolePanel** reusable component extends the **JPanel** component and can be used by any application to display role information (as shown in the following figure).



Role panel in the Organization Selection dialog box

The following code shows how the **RolePanel** component is constructed and added to the dialog box.

```
RolePanel rolePanel = new RolePanel (roleComponent);
mainSplitPane.setRightComponent (rolePanel);
```

TCComponentUINode

The **TCComponentUINode** component is an extension of the **AbstractUINode** class, and creates a UI node based on an **TCComponent** object.

Note:

This component is a Swing class. Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens Digital Industries Software encourages

you to customize Teamcenter using SWT/JFace components. Siemens Digital Industries Software will discontinue Swing/AWT support in a future version.

The UI node is presented with the **TCComponent** object name and rendered icon (as shown in the following figure).



Item revision UI component

The following code shows the code used to create an **TCComponentUINode** component as the root in a **GraphPane** panel.

```
// myFolder is an TCComponent
TCComponentUINode node = new TCComponentUINode ( myFolder );
GraphPane panel = new GraphPane ( new VerticalNodeLayout() );
// add the UI node to the panel and set it as the root
panel.setRoot ( node );
```

TCConstants

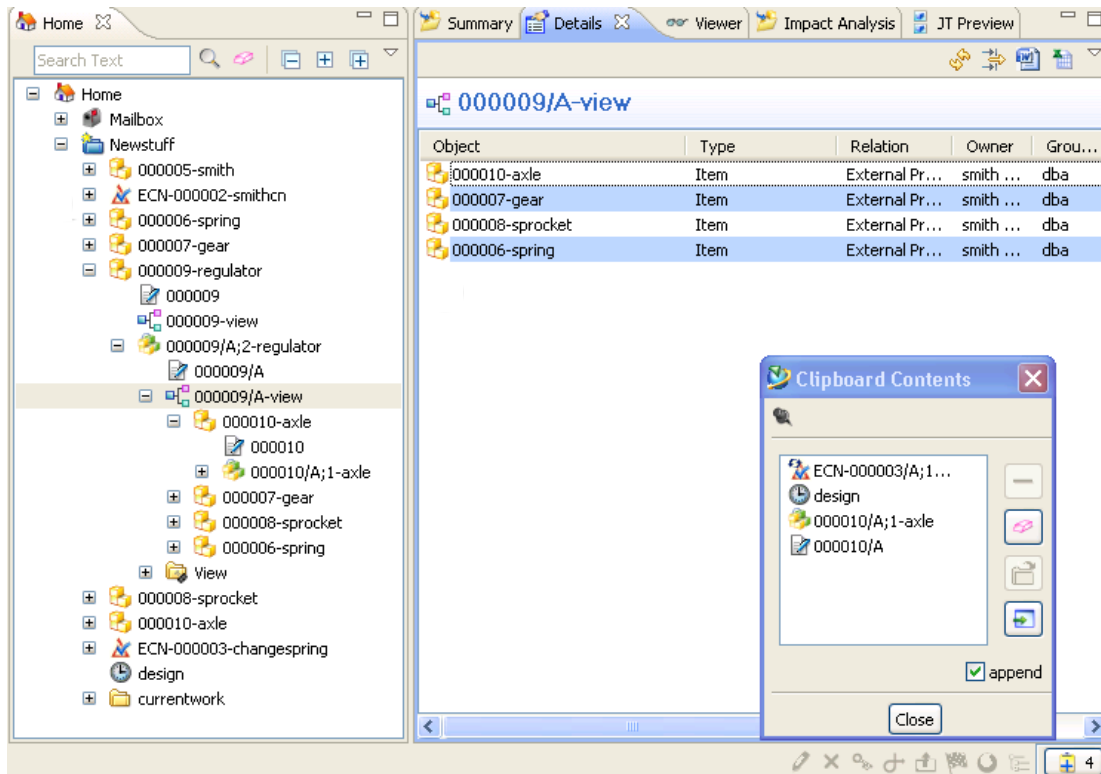
The **TCConstants** class contains constants used across Teamcenter and its related packages. Use the static variables defined in this class rather than hard-coding strings.

TCTypeRenderer

The **TCTypeRenderer** class is an implementation of a renderer that returns an icon based on either the Teamcenter object type or an object property. Many Teamcenter components use this render class.

Note:

This component is a Swing class. Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens Digital Industries Software encourages you to customize Teamcenter using SWT/JFace components. Siemens Digital Industries Software will discontinue Swing/AWT support in a future version.



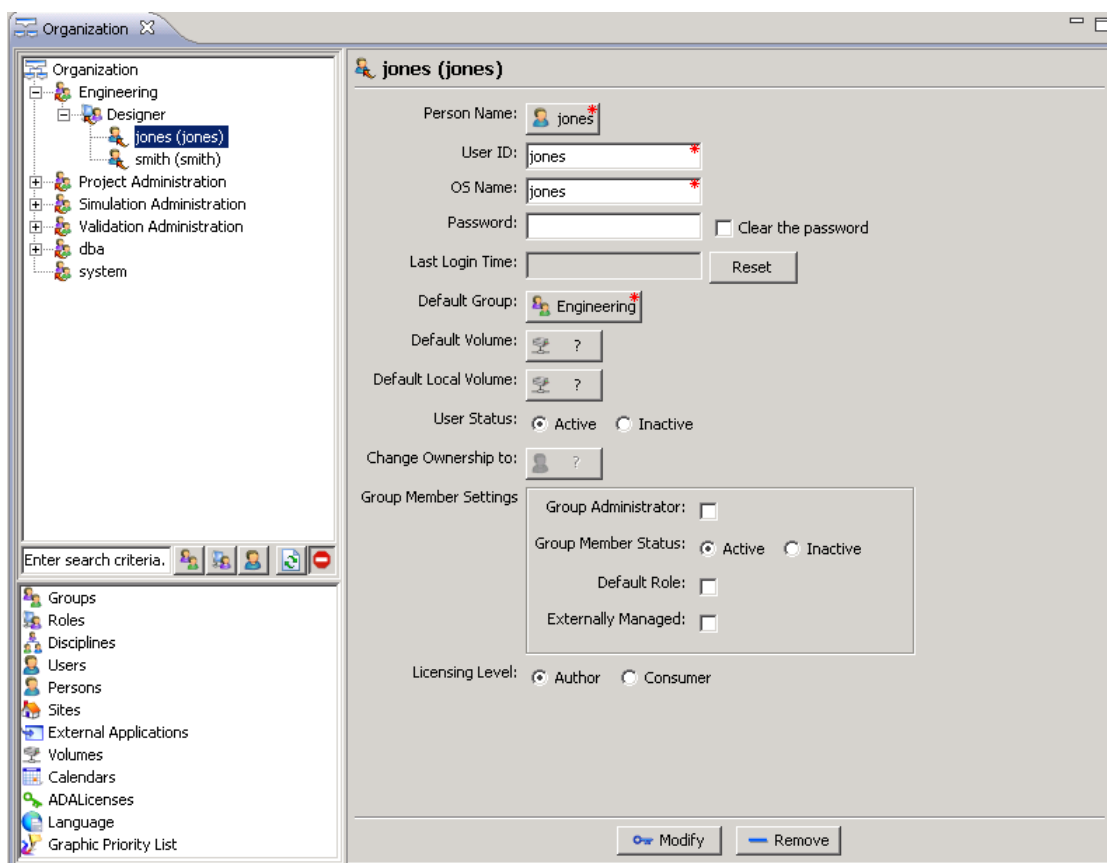
Usage of the TCTypeRenderer class

The following code example shows how to obtain the icon for an **TCComponent** object using the **TCTypeRenderer** class:

```
String text;
if ( comp instanceof TCComponent )
{
    TCComponent ic = (TCComponent)comp;
    text = ic.toString();
    JButton bt = new JButton();
    bt.setText ( text );
    bt.setIcon ( TCTypeRenderer.getIcon ( ic, false ) );
}
```

UserPanel

The **UserPanel** reusable component extends the **JPanel** component and can be used by any application to display group information (as shown in the following figure).



User panel in the Organization Selection dialog box

The **UserPanel** component can be constructed and added to the dialog box for display (as shown in the following code example).

```
UserPanel userPanel = new UserPanel (roleComponent);
mainSplitPane.setRightComponent (userPanel);
```

User interface components in the com.teamcenter.rac.util package

AbstractDialog

The **AbstractDialog** component enhances the functionality of the **JDialog** component, allowing you to instantiate the dialog box from a nonvisible application and keep it modal. The **AbstractDialog** component also provides **centerToScreen** methods.

The **AbstractDialog** class must be inherited by another class in order to work. For example, the **StringViewerDialog** component extends the **AbstractDialog** class, as follows:

```
public class StringViewerDialog extends AbstractDialog
{
    // Class implementation here.
};
```

AbstractPopupButton

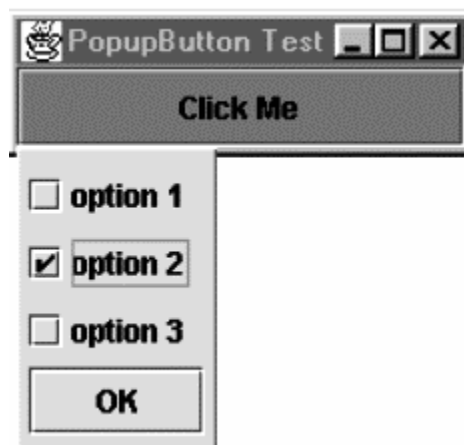
The **AbstractPopupButton** class creates a custom popup window from a button. This allows a small UI component to display a larger window that functions like a dialog box.

The **AbstractPopupButton** component works like a **JComboBox** component. It initially displays as a button (as shown in the following figure).



Initial state of an AbstractPopupButton component

It displays a popup window when the button is clicked (as shown in the following figure). Classes that extend from the **AbstractPopupButton** class must implement the UI for the popup window.



AbstractPopupButton component popup window

The following example adds three check boxes and a button to the popup window. When you click the **OK** button, the popup window closes.

```
import java.awt.*;
import java.awt.event.*;
import java.util.*;
import javax.swing.*;
import com.teamcenter.rac.util.*;
public class PopupButtonTest extends AbstractPopupButton
{
    PopupButtonTest (String txt)
    {
        super(txt);
    }
    public void initPopupWindow()
```

```

{
    // get the panel for popup window
    JPanel popupWin = getPanel();
    JPanel main = new JPanel ( true );
    main.setLayout ( new VerticalLayout(2,4,4,4,4) );

    JCheckBox op1 = new JCheckBox ( "option 1" );
    JCheckBox op2 = new JCheckBox ( "option 2" );
    JCheckBox op3 = new JCheckBox ( "option 3" );
    JButton okButton = new JButton ( "OK" );
    okButton.addActionListener( new ActionListener()
    {
        public void actionPerformed ( ActionEvent e )
        {
            // post down the popup window
            postDown();
        }
    });
    main.add ( "top", op1 );
    main.add ( "top", op2 );
    main.add ( "top", op3 );
    main.add ( "top", okButton );
    // add the created panel to the popup window
    popupWin.add ( "Center", main );
}
public static void main(String s[])
{
    WindowListener l = new WindowAdapter()
    {
        public void windowClosing(WindowEvent e) {System.exit(0);}
    };
    Frame f = new Frame("PopupButton Test");
    f.addWindowListener(l);
    f.add("Center", new PopupButtonTest ("Click Me"));
    f.pack();
    f.setSize(new Dimension(350,350));
    f.show();
}
}

```

GenericTableModel

The **GenericTableModel** class is an implementation of a custom **TableModel** component that stores and caches the strings it manages.

Note:

This component is a Swing class. Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens Digital Industries Software encourages you to customize Teamcenter using SWT/JFace components. Siemens Digital Industries Software will discontinue Swing/AWT support in a future version.

The following example shows code used to create a **GenericTableModel** component:

```

//Create the Table to show the history
columnNames.addElement(appReg.getString("date"));

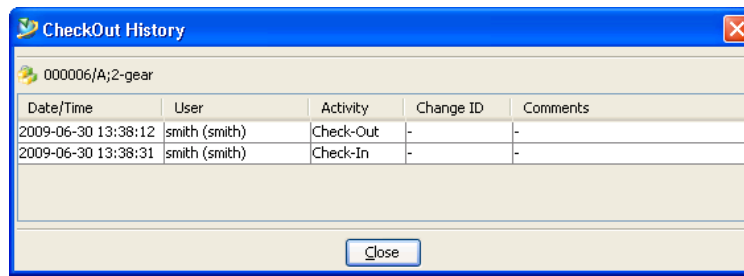
```

```

columnNames.addElement(appReg.getString("user"));
columnNames.addElement(appReg.getString("activity"));
columnNames.addElement(appReg.getString("changeId"));
columnNames.addElement(appReg.getString("comments"));
        dataModel = new GenericTableModel(columnNames, 0);
dataModel.markColumnEditable(0, true);
JTable historyTable = new JTable (dataModel);
.....

```

The following figure shows a table created with the **GenericTableModel** component.



Date/Time	User	Activity	Change ID	Comments
2009-06-30 13:38:12	smith (smith)	Check-Out	-	-
2009-06-30 13:38:31	smith (smith)	Check-In	-	-

Table created using GenericTableModel component

iTextArea

The **iTextArea** component is a subclass of the **JTextArea** class and displays all the same behaviors. Use this component to achieve a consistent look and feel across the system. The goal is to use this component across the entire commercial Teamcenter interface, so that when standards change or are enhanced the changes can be placed in this class and be inherited by all implementing components.

Using the **iTextArea** subclass provides the following benefits over using the **JTextArea** class:

- **Field selection when focus is gained**

Data is automatically selected when the focus is gained within the text area.

- **Improved focus traversal**

The up arrow and down arrow keys provide next and previous focus traversal abilities.

- **Require signaling**

You no longer must override the paint method to implement the paint, as required. Instead, invoke the **setRequired(Boolean state)** method.

iTextField

The **iTextField** component is a subclass of the **JTextField** class and displays all the same behaviors. Use this component to achieve a consistent look and feel across the system. The goal is to use this component across the entire commercial Teamcenter interface, so that when standards change or are enhanced the changes can be placed in this class and be inherited by all implementing components.

Using the **iTextField** subclass provides the following benefits over using the **JTextField** class:

- **Field selection when focus is gained**

Data is automatically selected when the focus is gained within the text field.

- **Improved focus traversal**

The up arrow and down arrow keys provide next and previous focus traversal abilities.

- **Require signaling**

You no longer must override the paint method to implement the paint, as required. Instead, invoke the **setRequired(Boolean state)** method.

Layout manager

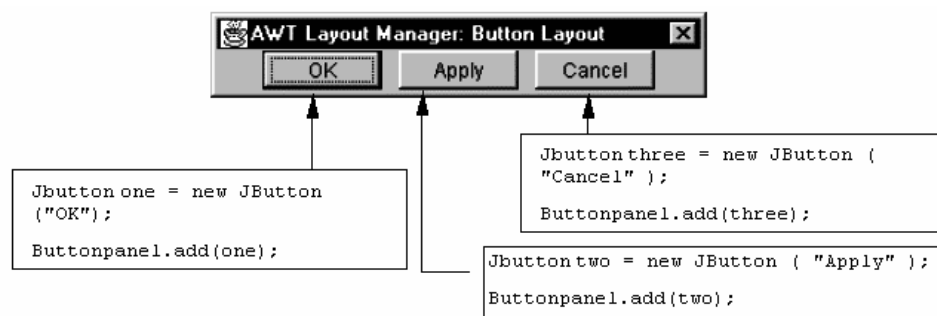
ButtonLayout

Horizontal ButtonLayout layout manager examples

The following figure shows the **ButtonLayout** layout manager with horizontal orientation and center alignment.

Note:

This component is a Swing class. Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens Digital Industries Software encourages you to customize Teamcenter using SWT/JFace components. Siemens Digital Industries Software will discontinue Swing/AWT support in a future version.



Horizontal button layout with center alignment

The constructor is called, as follows:

```

buttonPanel.setLayout (new ButtonLayout (ButtonLayout.HORIZONTAL,
ButtonLayout.CENTER)
  
```

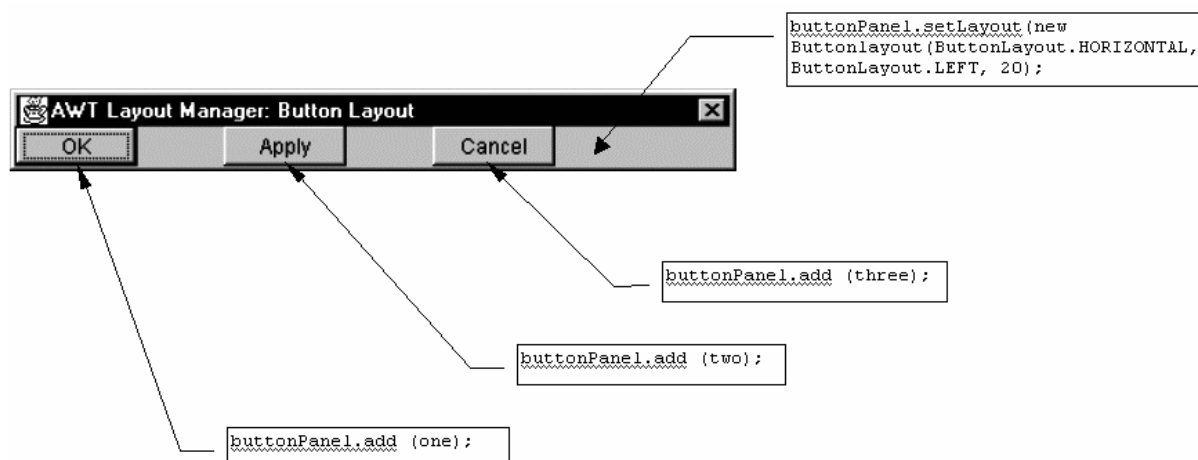
Buttons are added in the order in which they are displayed. In the previous figure, the sequence is **OK**, **Apply** and **Cancel**.

The following figure shows the dialog box when it is resized. The components maintain their orientation and alignment.



Results of resizing the dialog box

The following figure shows the **ButtonLayout** layout manager with horizontal orientation, left alignment, and a 20-unit gap between the buttons.

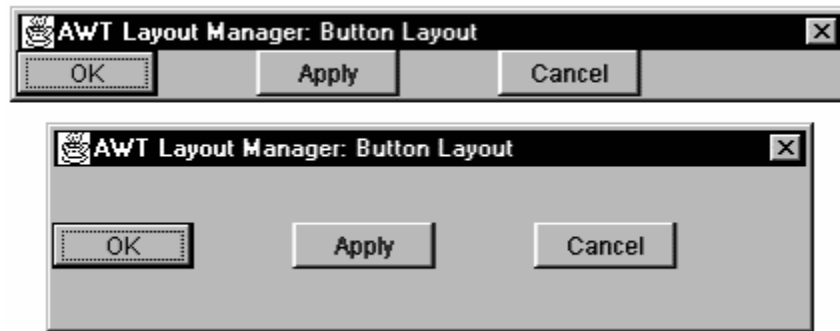


Horizontal button layout with left alignment and a 20-unit gap

The constructor is called, as follows:

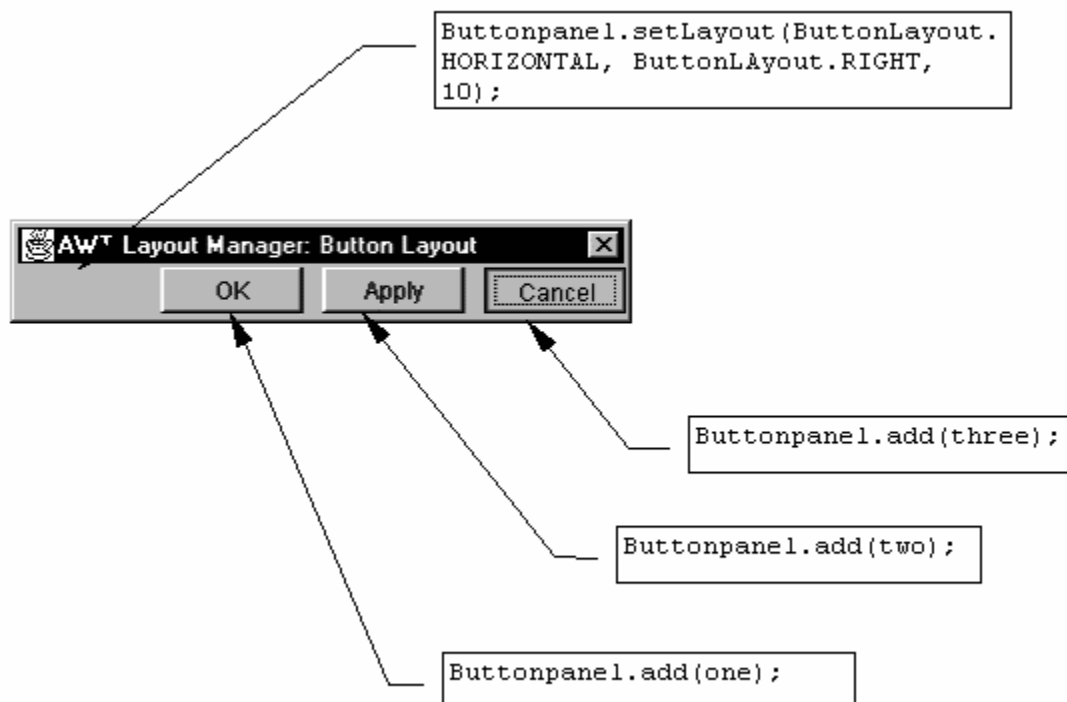
```
buttonPanel.setLayout (new ButtonLayout (ButtonLayout.HORIZONTAL,
ButtonLayout.LEFT, 20);
```

Since the alignment is set to **LEFT**, the first button added is aligned to the left corner of the dialog box and the remaining buttons are placed to the right of the first, with a 20-unit gap between buttons. The following figure shows the dialog box when it is resized.



Results of resizing the dialog box

The following figure shows the **ButtonLayout** layout manager with horizontal orientation, right alignment, and a 20-unit gap between the buttons.



Horizontal button layout with right alignment and a 20-unit gap

The constructor is called, as follows:

```
buttonPanel.setLayout (new ButtonLayout (ButtonLayout.HORIZONTAL,
ButtonLayout.RIGHT, 20);
```

Since the alignment is set to **RIGHT**, the first button added is aligned to the right corner of the dialog box and the remaining buttons are placed to the left of the first, with a 20-unit gap between buttons. The following figure shows the dialog box when it is resized.



Results of resizing the dialog box

The following example shows the code used to create the previous examples:

```
public class testlayout extends JDialog
{
    protected JButton one, two, three;
    protected JPanel buttonPanel;
    public testlayout ( Frame parent, String title )
    {
        super ( parent, title, false );
        // Create a new panel with a ButtonLayout Manager
        buttonPanel = new JPanel();
        buttonPanel.setLayout ( new ButtonLayout (ButtonLayout.HORIZONTAL));
        // Create three buttons
        one = new JButton ( "OK" );
        two = new JButton ( "Apply" );
        three = new JButton ( "Cancel" );
        // Add the buttons to the Panel created
        buttonPanel.add (one );
        buttonPanel.add (two);
        buttonPanel.add (three);
        this.add (buttonPanel);
        this.pack ();
    }
}
```

The **buttonPanel** panel is created. The panel uses the **ButtonLayout** layout manager and assumes the default values for its parameters. Three buttons are created and added to the panel. The buttons are positioned horizontally, in the center of the panel, with a 10-unit gap between buttons. The buttons are

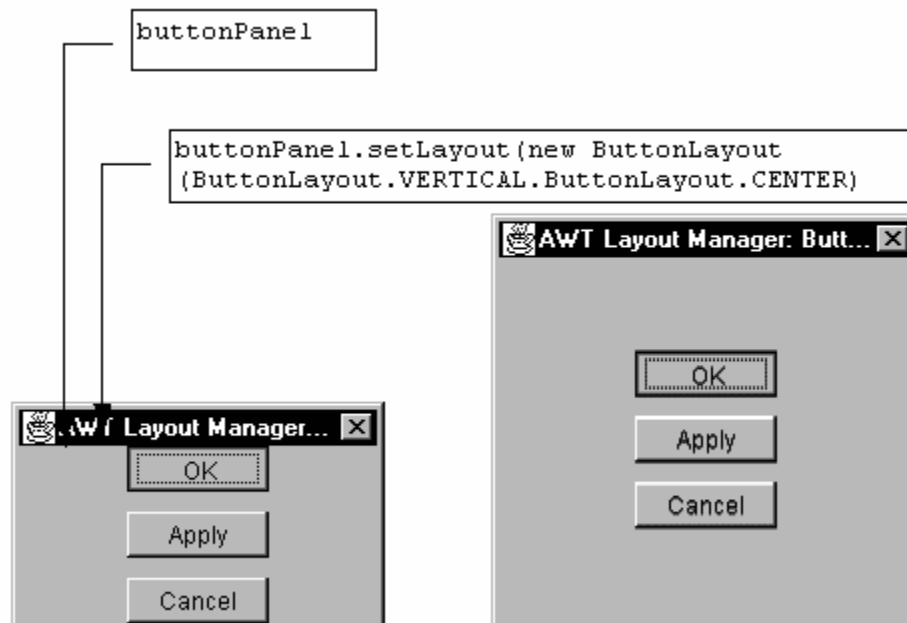
not resized when the panel is resized. The position of the buttons in relation to the edges of the panel also remains unchanged.

Vertical BorderLayout layout manager examples

The following figure shows the **ButtonLayout** layout manager with vertical orientation and center alignment.

Note:

This component is a Swing class. Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens Digital Industries Software encourages you to customize Teamcenter using SWT/JFace components. Siemens Digital Industries Software will discontinue Swing/AWT support in a future version.



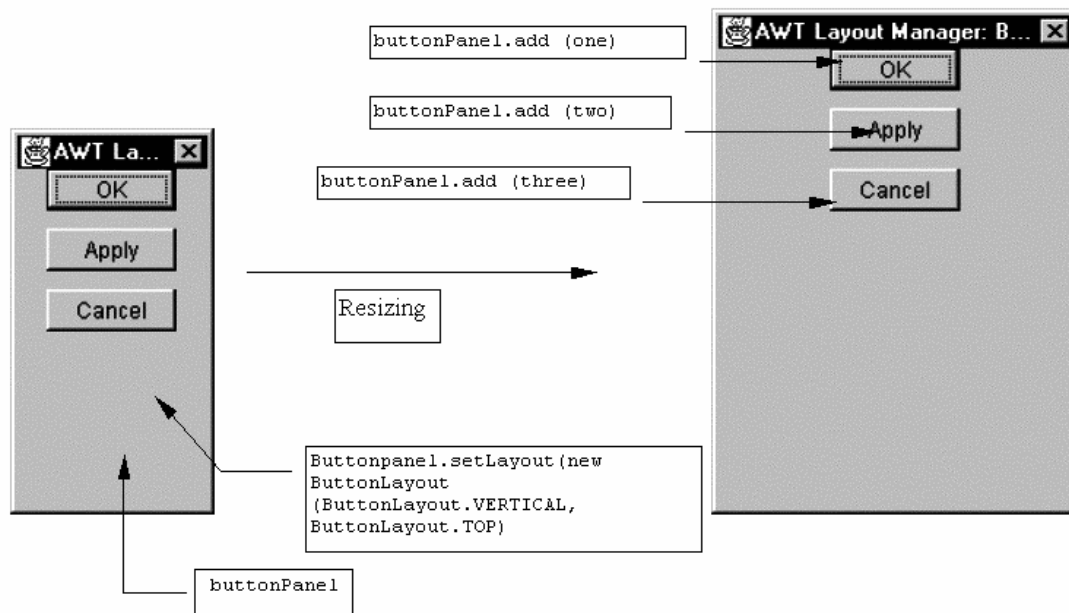
Vertical button layout with center alignment

The constructor is called, as follows:

```
buttonPanel.setLayout (new Buttonlayout(ButtonLayout.VERTICAL,
ButtonLayout.CENTER);
```

When the panel is resized, the buttons maintain their size and alignment in relation to the dialog box. The buttons are placed in the sequence in which they are added to the panel.

The following figure shows the **ButtonLayout** layout manager with vertical orientation and top alignment.

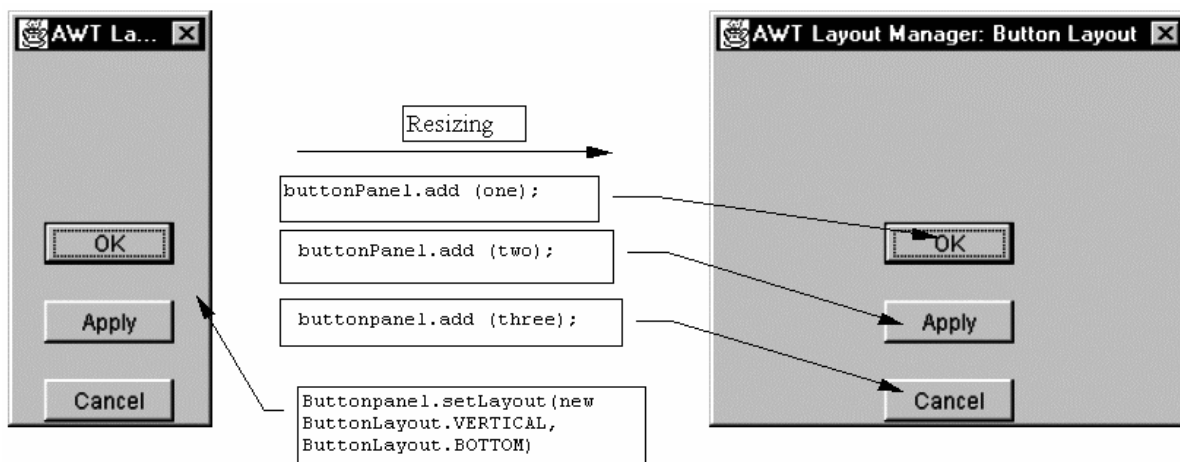


Vertical button layout with top alignment

The constructor is called, as follows:

```
buttonPanel.setLayout (new ButtonLayout (ButtonLayout.VERTICAL,
ButtonLayout.TOP) ;
```

The following figure shows the **ButtonLayout** layout manager with vertical orientation and bottom alignment.



Vertical button layout with bottom alignment

The constructor is called, as follows:

```
buttonPanel.setLayout (new ButtonLayout (ButtonLayout.VERTICAL,
ButtonLayout.BOTTOM);
```

The following figure shows the code used to create the previous examples:

```
public class testlayout extends JDialog
{
    protected JButton one, two, three;
    protected JPanel buttonPanel;
    public testlayout ( Frame parent, String title )
    {
        super ( parent, title, false );
        // Create a new panel with a ButtonLayout Manager
        buttonPanel = new JPanel();
        buttonPanel.setLayout ( new ButtonLayout(ButtonLayout.HORIZONTAL));
        // Create three buttons
        one = new JButton ( "OK" );
        two = new JButton ( "Apply" );
        three = new JButton ( "Cancel" );
        // Add the buttons to the Panel created
        buttonPanel.add (one );
        buttonPanel.add (two);
        buttonPanel.add (three);
        this.add (buttonPanel);
        this.pack ();
    }
}
```

The **buttonPanel** panel is created. The panel uses the **ButtonLayout** layout manager and assumes default values for its parameters. Three buttons are created and added to the panel. The buttons are positioned horizontally in the center of the panel with a 10-unit gap between buttons. Resizing a **ButtonLayout** panel does not resize the buttons. The position of the buttons relative to the four edges of the panel also remains unchanged.

HorizontalLayout

The **HorizontalLayout** layout manager positions children and attachments horizontally in the **container** object.

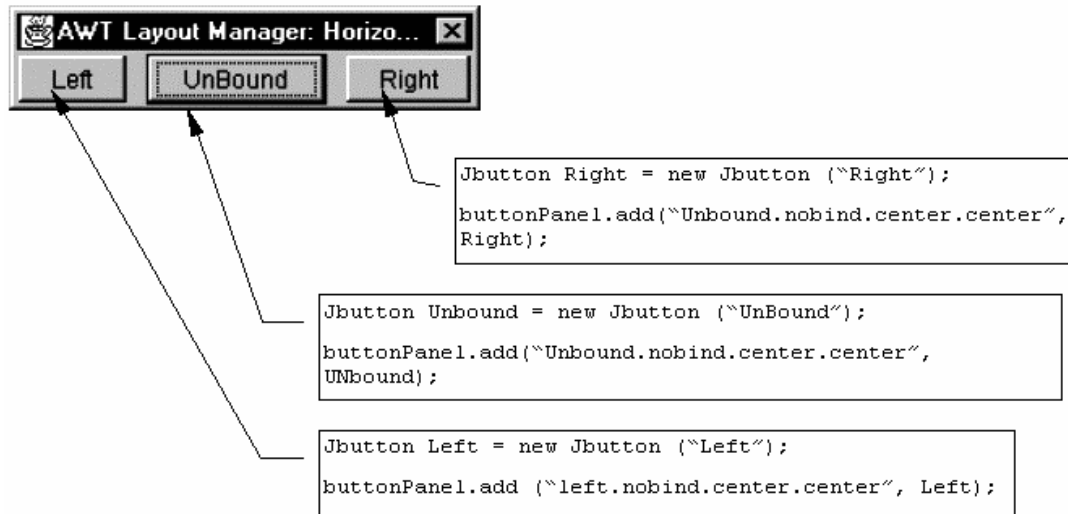
Note:

This component is a Swing class. Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens Digital Industries Software encourages you to customize Teamcenter using SWT/JFace components. Siemens Digital Industries Software will discontinue Swing/AWT support in a future version.

When a component is added to the **container** object, the position of the component is determined by the parameters passed in the **name** field. These parameters are **{Attachment, bind, HorizontalAlignment, VerticalAlignment}**. The default positioning for a component added without correct formatting in the **name** field is **(left.bind.center.center)**. **bind** and **nobind** indicate that the component will be resized or maintained based on the space available. Top components are positioned

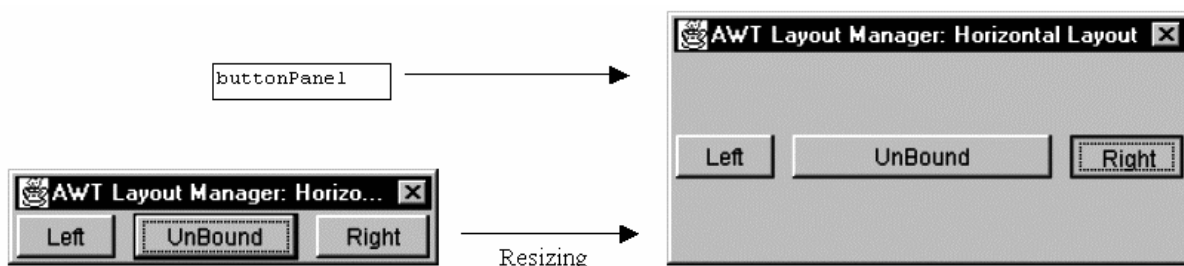
first, followed by bottom components and unbound components. If you do not call the **add** function with a name string in the argument, an exception occurs.

The following figure shows the use of the **HorizontalLayout** layout manager.



Horizontal layout with center alignment

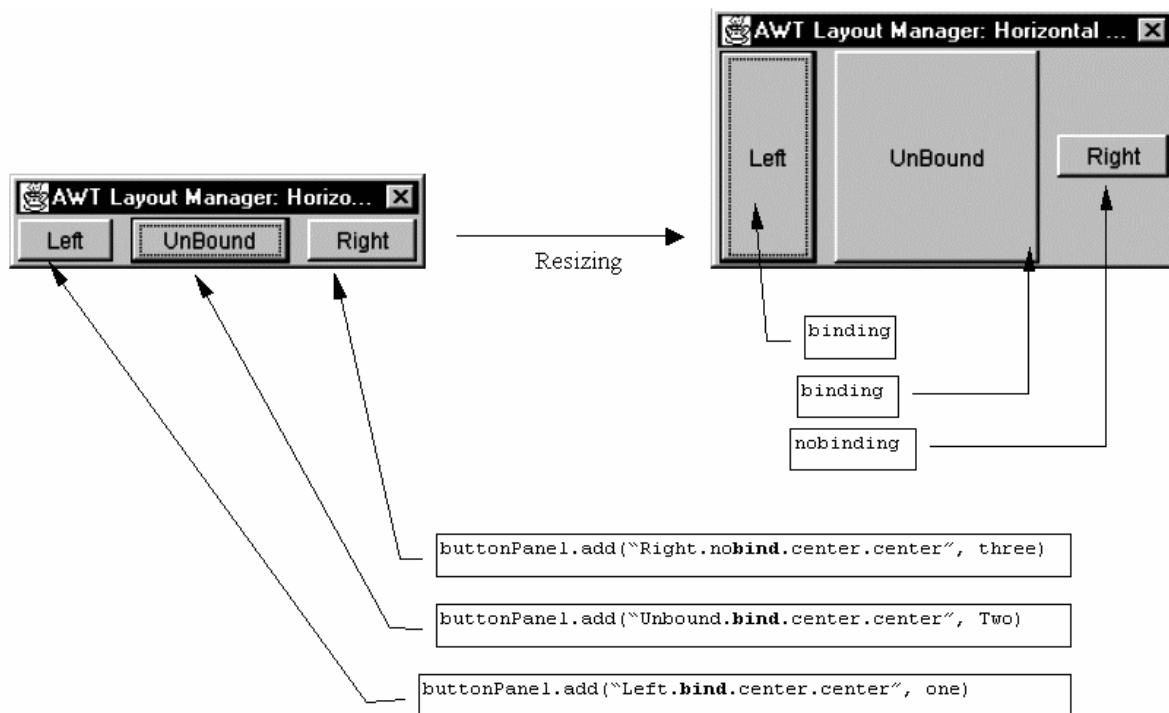
The following figure shows the results when the dialog box is resized.



Results of resizing the dialog box

When the dialog box is resized, the **Left** and **Right** buttons maintain their shape, size, and alignment in relation to the edges of the dialog box because they are added with a **nobind** parameter. The **Unbound** button is resized when the dialog box is resized because it is added to the dialog box using a **bind** parameter.

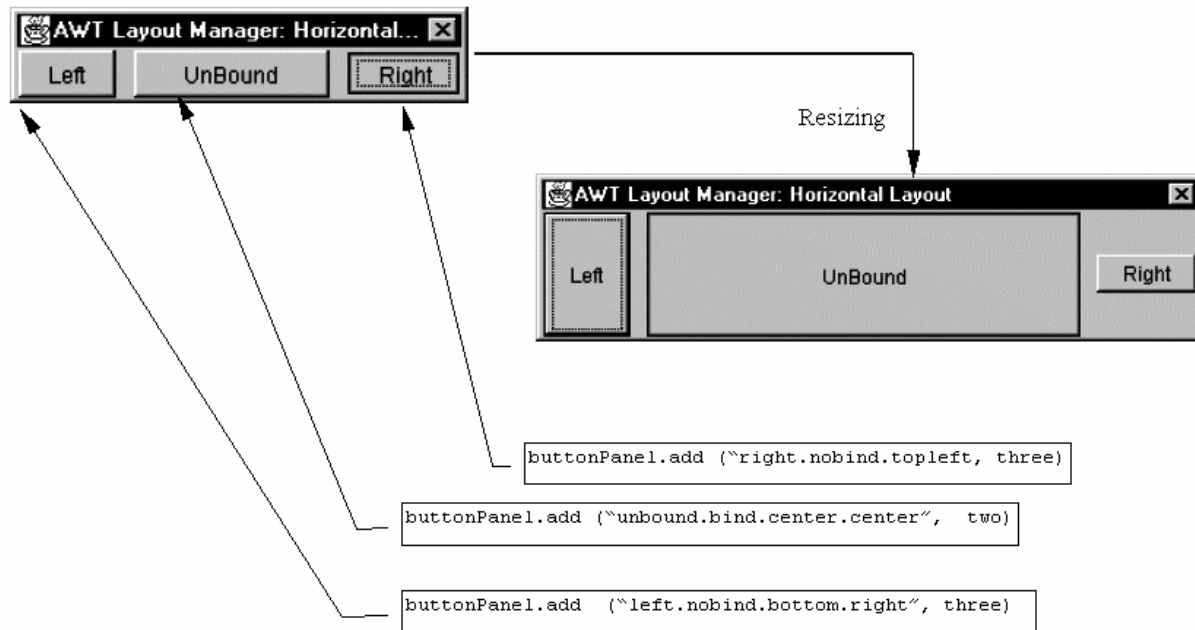
The following figure shows the dialog box when the placement of the components is changed according to the parameters indicated in the figure. The behavior of the dialog boxes when resized is also shown.



Horizontal layout with components added

When you resize dialog boxes, components added with the **bind** parameter are resized.

In the following figure, the **Left** and **Unbound** buttons are added with a **bind** parameter and the **Right** button is added with a **nobind** parameter. When the dialog box is resized, the **Left** and **Unbound** buttons are resized, but the **Right** button maintains its size and alignment.



Horizontal layout with components added

In addition to controlling the placement of the components within the dialog box, the **LayoutManager** component controls the spacing of the components relative to the edges of the dialog box by passing the parameters into the constructor for the **LayoutManager** component. The order in which the parameters are passed into the constructor is important. The constructor is as follows:

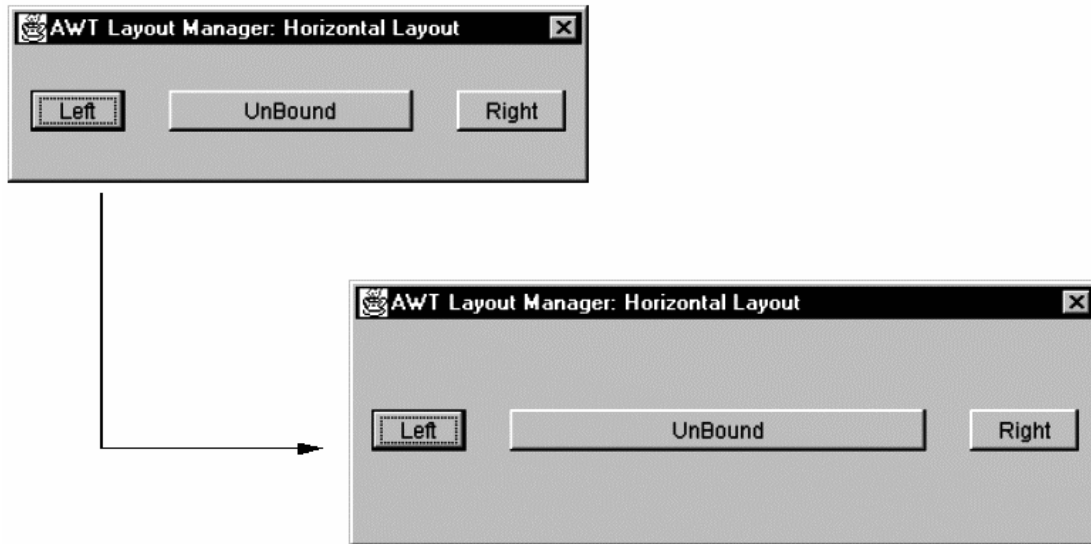
```
new HorizontalLayout ( vgap, lm, rm, tm, bm);
```

vgap indicates the distance between the components in the dialog box. **lm** indicates the left margin, **rm** indicates the right margin, **tm** indicates the top margin, and **bm** indicates the bottom margin. All parameters are expressed in integers and measured in pixels.

The following figure shows the dialog box that results when the following parameters are passed to the constructor:

```
new HorizontalLayout (25, 10, 10, 25, 25);
```

The following figure also shows the behavior when the dialog box is resized. The margins and spacing between components are maintained.



Horizontal layout with parameters

The following example shows the code used to create the previous examples:

```
public class testlayout extends JDialog
{
    protected JButton one, two, three;
    protected JPanel buttonPanel;

    public testlayout( Frame parent, String title )
    {
        super ( parent, title, false );
        buttonPanel = new JPanel();
        buttonPanel.setLayout ( new HorizontalLayout(10,2,2,2,0));
        one = new JButton ( "Left" );
        two = new JButton ( "Right" );
        three = new JButton ( "UnBound" );
        buttonPanel.add ( "left.nobind.center.center", one );
        buttonPanel.add ( "unbound.nobind.center.center", three );
        buttonPanel.add ( "right.nobind.center.center", two );
        this.add (buttonPanel);
        this.pack ();
    }

    public static main void ( String [] args){
    {
        Frame f = new Frame ( "Horizontal Layout Test" );
        f.resize ( 100,100 );
        f.show();
        testlayout d = new testlayout ( f, "AWT Layout Manager: Horizontal Layout" );
        d.show();
    }
    }
}
```

The **buttonPanel** panel is created. The **HorizontalLayout** layout manager is used and three buttons are created and added to the panel. The placement of the buttons in each of the examples is determined by the parameters passed in the **name** field.

PropertyLayout

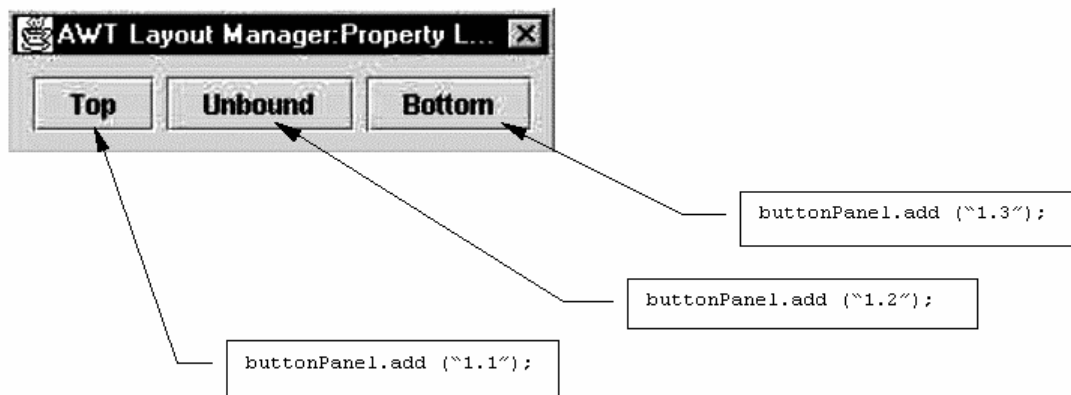
The **PropertyLayout** layout manager positions children and attachments vertically within the **container** object.

Note:

This component is a Swing class. Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens Digital Industries Software encourages you to customize Teamcenter using SWT/JFace components. Siemens Digital Industries Software will discontinue Swing/AWT support in a future version.

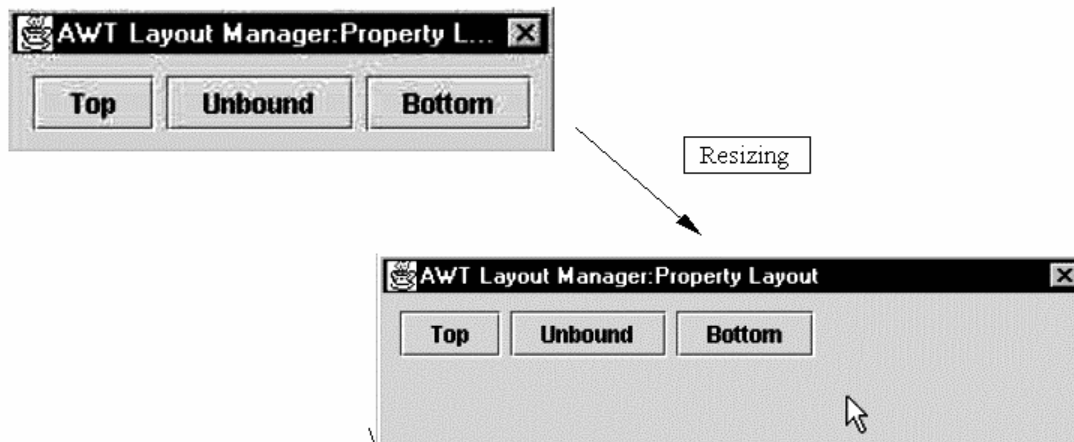
When a component is added to the **container** object, its position is determined by a mask passed through the **name** field that describes how to position the component and whether resizing of the component is desired. The parameters are **Row**, **Column**, **HorizontalAlignment**, **VerticalAlignment**, **HorizontalAttachment**, and **VerticalAttachment**. The default positioning for a component added without correct formatting in the **name** field is **1.1.center.center.preferred.preferred**.

The following figure shows the use of the **PropertyLayout** layout manager with default parameters for each of the components.



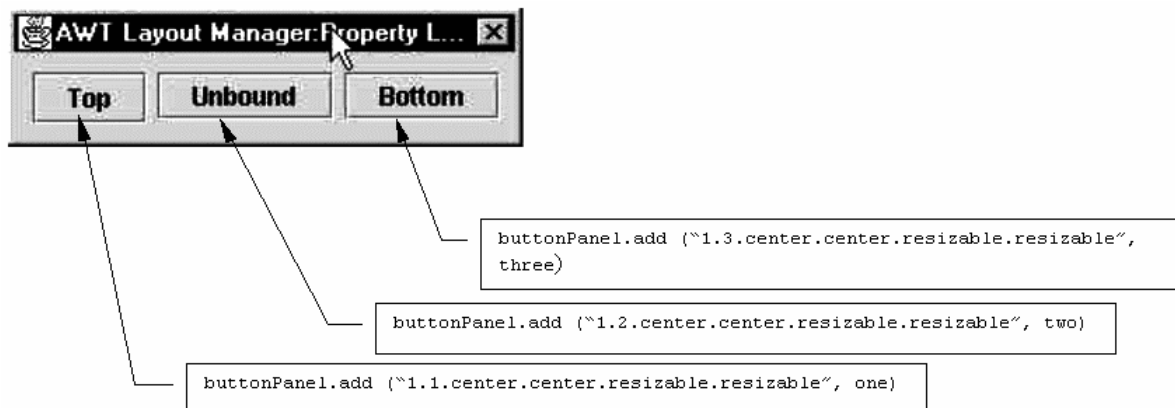
Property layout with components added

The code creates a **buttonPanel** panel using the **PropertyLayout** layout manager. Three buttons are created and added to the panel in three separate columns. When resized, the placement and size of the components remain unchanged (as shown in the following figure). The default positioning of the components in the dialog box is **1.1.center.center.preferred.preferred**. Because the horizontal alignment and vertical alignment parameters are set to **preferred**, the components are not resized along with the dialog box.



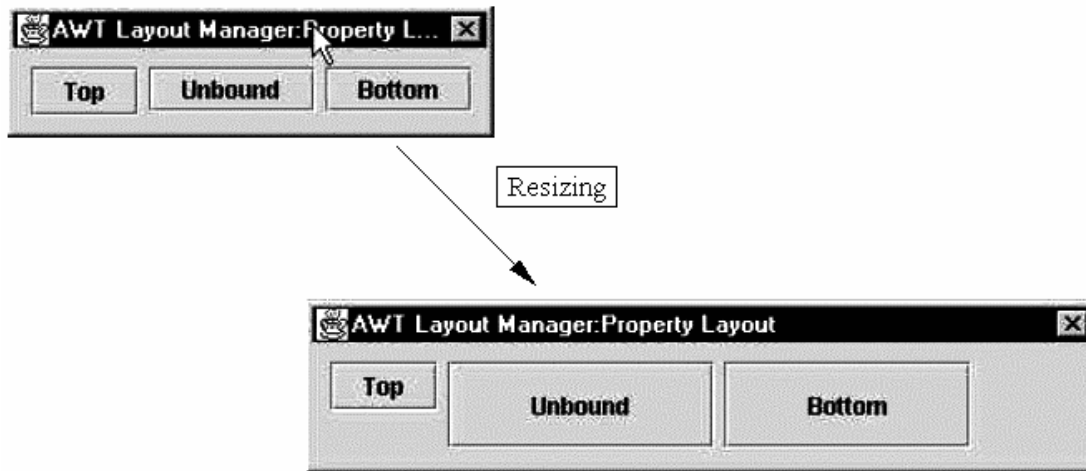
Results of resizing the dialog box

The following figure shows the layout of a dialog box using the **PropertyLayout** layout manager. The components are placed according to the parameters indicated in the figure. Note that the **HorizontalAttachment** and **VerticalAttachment** parameters are set to **resizable**.



Property layout with components added

The following figure shows the behavior when the dialog box is resized. All components are resized except the component placed in the column 1, row 1 position. The other components in the dialog box are resized both horizontally and vertically.



Results of resizing the dialog box

In addition to controlling the placement of components in the dialog box, the **LayoutManager** component controls the spacing of the components relative to the edges of the dialog box. This is achieved by the parameters passed into the constructor. The order in which the parameters are passed into the constructor is important. The constructor is as follows:

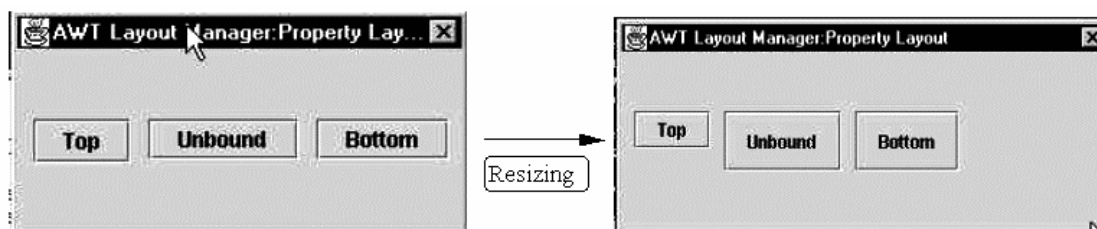
```
new PropertyLayout ( hgap, vgap, lm, rm, tm, bm);
```

hgap indicates the horizontal distance between components in the dialog box. **vgap** indicates the vertical distance between components in the dialog box. **lm** indicates the left margin, **rm** indicates the right margin, **tm** indicates the top margin, and **bm** indicates the bottom margin. All parameters are expressed in integers and measured in pixels. The components are added to the dialog box horizontally, that is, they are placed on the dialog box in the same row but in different columns.

The following figure shows the dialog box that results from passing the following parameters to the constructor:

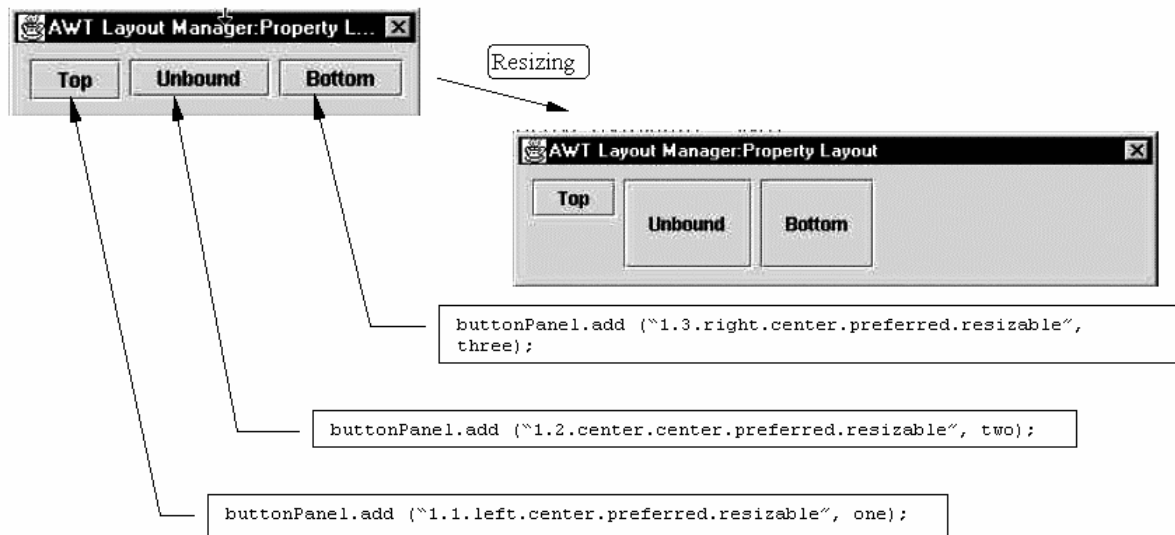
```
new PropertyLayout (10, 20, 10, 10, 40, 40);
```

The following figure also shows the behavior when the dialog box is resized. Only the left and top margins and spacing between components are maintained when the dialog box is resized.



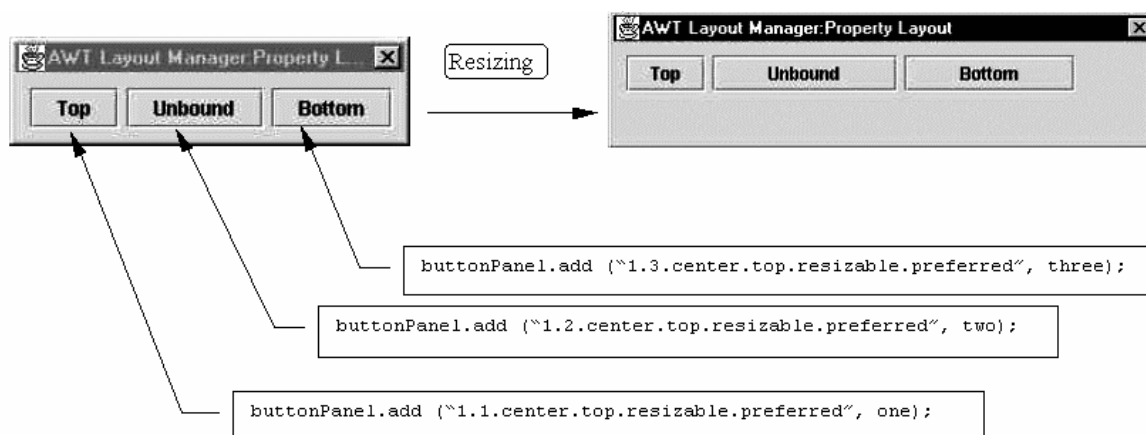
PropertyLayout Manager with margin setup

The following figure shows a dialog box in which the components are only resizable vertically. The **HorizontalAttachment** parameter is set to **preferred**. The buttons in the figure do not seem aligned due to the positions selected for their placement.



Results of resizing the dialog box

The following figure shows a dialog box in which the components are only resizable vertically. The **HorizontalAttachment** parameter is set to **preferred**.



Results of resizing the dialog box

The following example shows the code used to create the previous examples:

```
public class testlayout extends JDialog
{
    protected JButton one, two, three;
    protected JPanel buttonPanel;
    public testlayout( Frame parent, String title )
```

```

{
    super ( parent, title, false );
    buttonPanel = new JPanel();
    buttonPanel.setLayout ( new PropertyLayout());
    one = new JButton ( "Top" );
    two = new JButton ( "Bottom" );
    three = new JButton ( "UnBound" );
    buttonPanel.add ( "1.1 ", one );
    buttonPanel.add ( "1.2", two );
    buttonPanel.add ( "1.3", three );
    this.getContentPane().add (buttonPanel);
    this.pack ();
}
public static void main ( String[] args ){
{
    Frame f = new Frame ( "Vertical Layout Test" );
    f.show();
    testlayout d = new testlayout ( f, "AWT Layout Manager: Vertical Layout" );
    d.show();
}
}
}

```

VerticalLayout

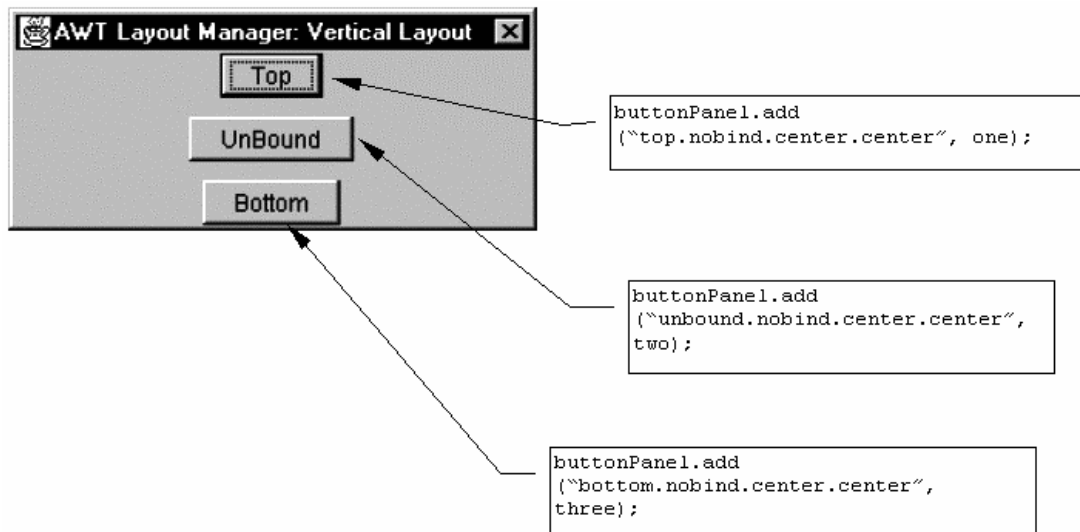
The **VerticalLayout** layout manager positions children and attachments vertically in the **container** object.

Note:

This component is a Swing class. Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens Digital Industries Software encourages you to customize Teamcenter using SWT/JFace components. Siemens Digital Industries Software will discontinue Swing/AWT support in a future version.

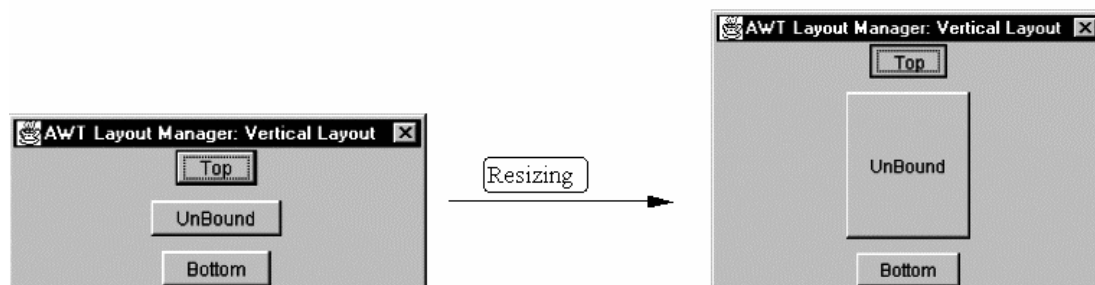
When a component is added to the **container** object, the position of the component is determined by the parameters passed in the **name** field. These parameters are **{Attachment.Binding.HorizontalAlignment.VerticalAlignment}**. The default positioning for a component added without correct formatting in the **name** field is **(top.center.center.bind)**. Top components are positioned first, followed by bottom components and unbound components. If you do not call the **add** function with a name string in the argument, an exception occurs.

The following figure shows the use of a **VerticalLayout** layout manager. The placement of the components within the dialog box is determined by the parameters passed into the **name** field.



Vertical layout with components added

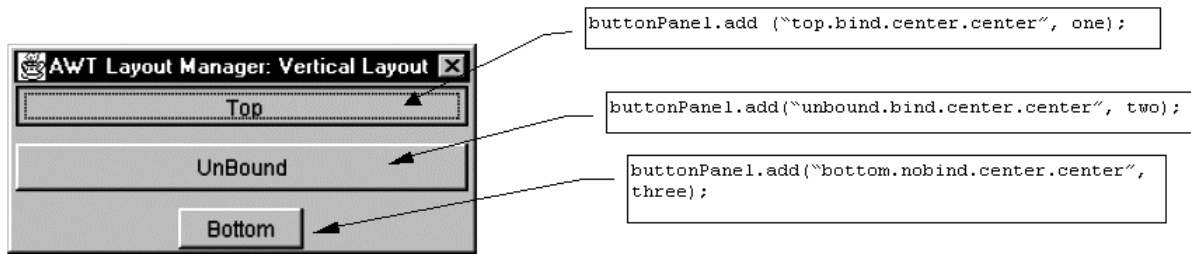
The following figure shows the results of resizing the dialog box.



Results of resizing the dialog box

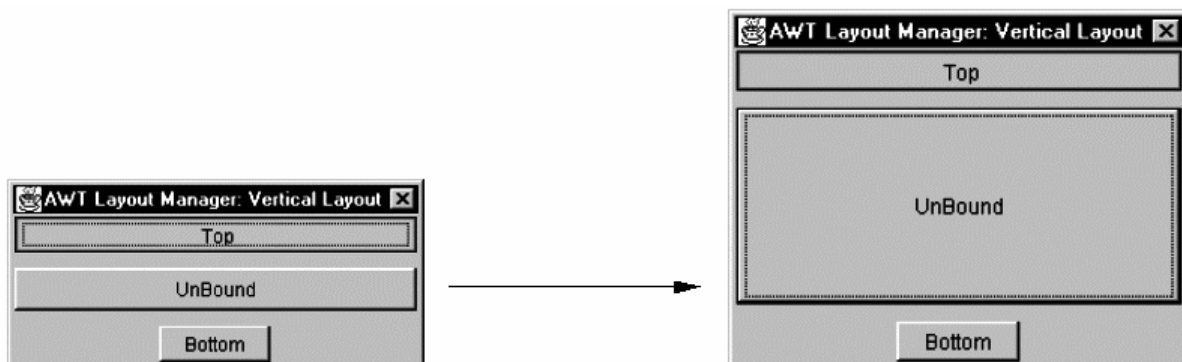
The code creates a new **buttonPanel** panel. The layout in the panel is set to **VerticalLayout** with default parameters. Three buttons are created and added to the panel. The top components are positioned first, followed by the bottom components and unbound components. All the buttons are added to the panel with a **nobind** parameter and placed in the center of the dialog box. When the dialog box is resized, the **Unbound** button is resized, but the **Top** and **Bottom** components are not.

The following figure shows the layout of the components added with a **bind** parameter. The **Top** and **Unbound** components are added with a **bind** parameter. The **Bottom** component is added with a **nobind** parameter.



Horizontal layout with components added

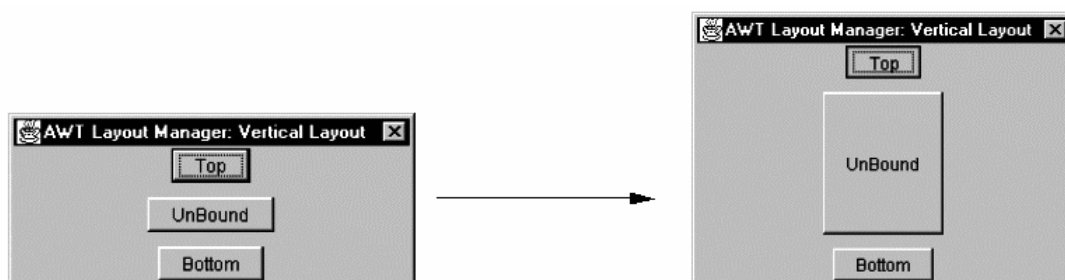
The following figure shows the resizing behavior of the components added with a **bind** parameter.



Results of resizing the dialog box

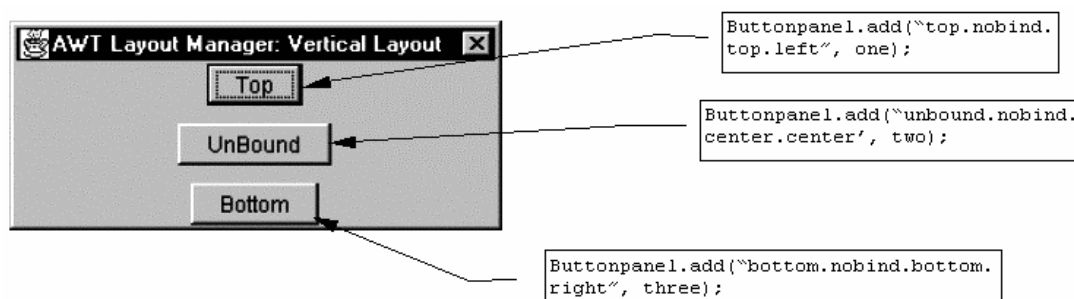
Upon resizing the dialog box, the **unbound** component is resized. The **top** component, which is added with a **bind** parameter, remains attached to the edges of the dialog box. The **bottom** component, added with a **nobind** parameter, does not change.

The following figure shows the behavior of the dialog box when resized.



Results of resizing the dialog box

The following figure shows the layout of the dialog boxes when the components are added with the parameters indicated in the figure.



Vertical layout with components added

In addition to controlling the placement of components in the dialog box, the **LayoutManager** component also controls the spacing of the components relative to the edges of the dialog box. This is achieved by the parameters passed into the **LayoutManager** constructor. The order in which the parameters are passed into the constructor is important. The constructor is:

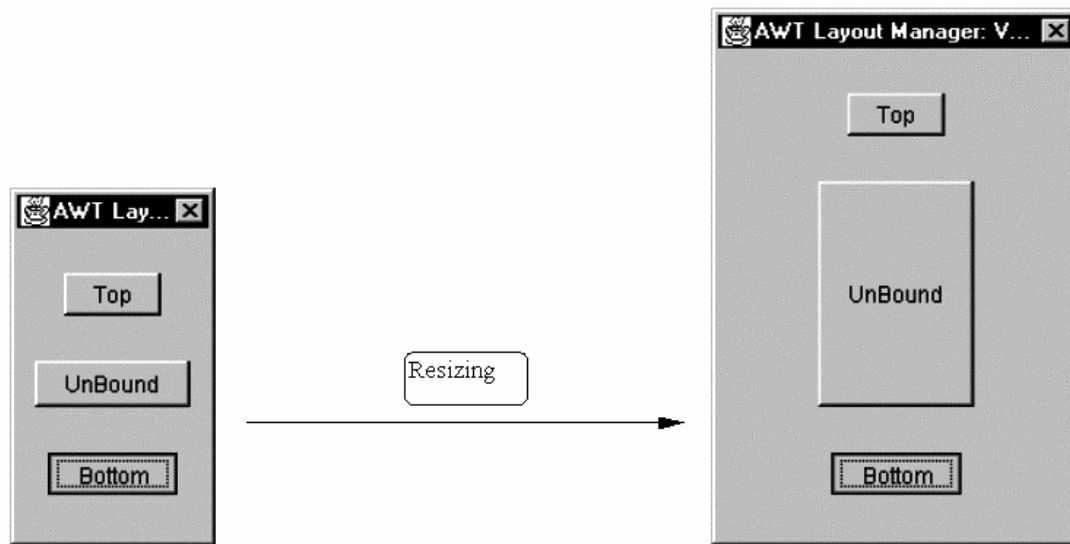
```
new VerticalLayout ( vgap, lm, rm, tm, bm);
```

vgap indicates the distance between the components in the dialog box. **lm** indicates the left margin, **rm** indicates the right margin, **tm** indicates the top margin, and **bm** indicates the bottom margin. All parameters are expressed in integers and measured in pixels.

The following figure shows the dialog box layout when the following parameters are passed to the constructor:

```
new VerticalLayout (25, 10, 10, 25, 25);
```

The behavior of the dialog box when resized is also shown in the following figure. The dialog box maintains the margins and the spacing between the components when resized.



Vertical layout with margin setup

The following figure shows the code used to create the previous examples:

```
public class testlayout extends JDialog
{
    protected JButton one, two, three;
    protected JPanel buttonPanel;
    public testlayout( Frame parent, String title )
    {
        super ( parent, title, false );
        buttonPanel = new JPanel();
        buttonPanel.setLayout ( new VerticalLayout(10,2,2,2,0));
        one = new JButton ( "Top" );
        two = new JButton ( "UnBound" );
        three = new JButton ( "Bottom" );
        buttonPanel.add ( "top.nobind.center.center", one );
        buttonPanel.add ( "unbound.nobind.center.center", two );
        buttonPanel.add ( "bottom.nobind.center.center", three);
        this.add (buttonPanel);
        this.pack ();
    }
    public static void main ( String[] args ){
        {
            Frame f = new Frame ( "Vertical Layout Test" );
            f.resize ( 100,100 );
            f.show();
            testlayout d = new testlayout ( f, "AWT Layout Manager: Vertical Layout" );
            d.show();
        }
    }
}
```

MessageBox

The **MessageBox** class communicates informational, warning, working, and error messages to the user (as shown in the following figure).



MessageBox

The **MessageBox** class is a specialized **JDialog** class that provides the ability to create and display a wide variety of message boxes to the user. Examples of the information that appears in message boxes include help, detailed messages, and general messages with icons that are set based on the type of **MessageBox** component.

The following example shows code used to create a **MessageBox** component:

```
JFrame frm = new JFrame();
JPanel displayPanel = new JPanel(new BorderLayout());
JButton invokeButton = new JButton("Invoke MessageBox");
final MessageBox msgBox = new MessageBox(frm, "Some Message", "Title", MessageBox.ERROR);

displayPanel.add("Center", invokeButton);

frm.getContentPane().add(displayPanel);
invokeButton.addActionListener(new ActionListener()
{
    public void actionPerformed(ActionEvent e)
    {
        msgBox.setVisible(true);
    }
});

frm.addWindowListener(new WindowAdapter()
{
    public void windowClosing(WindowEvent e)
    {
        System.exit(0);
    }
});
frm.pack();
frm.validate();

frm.setVisible(true);
```

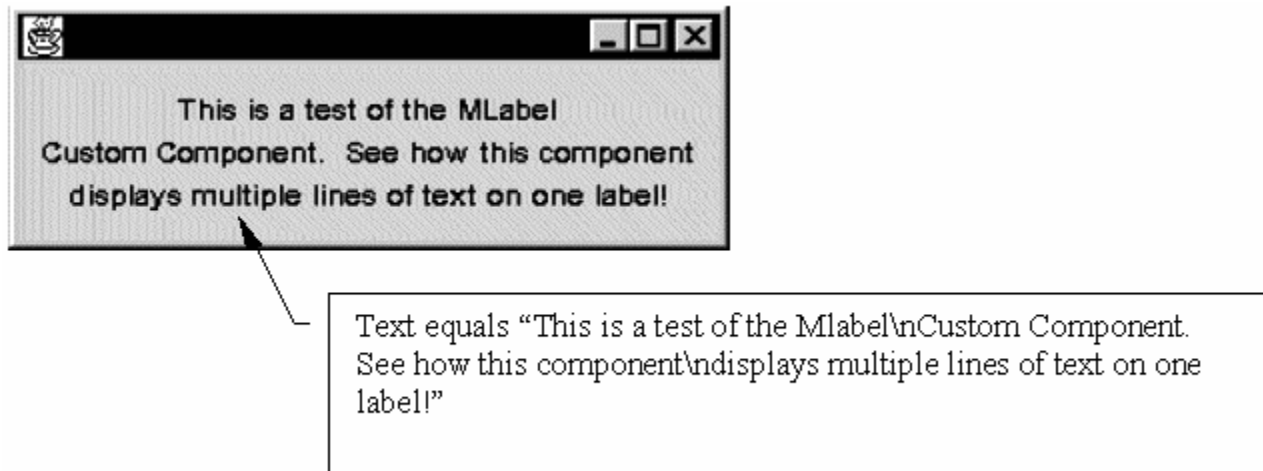
The following figure shows the message box produced by the code.



MessageBox produced from sample code

MLabel

The **MLabel** component displays text on multiple lines (as shown in the following figure). The current AWT label and **JLabel** components are only capable of displaying one line of text. The backslash and *n* character (*\n*) delimit the lines of text.

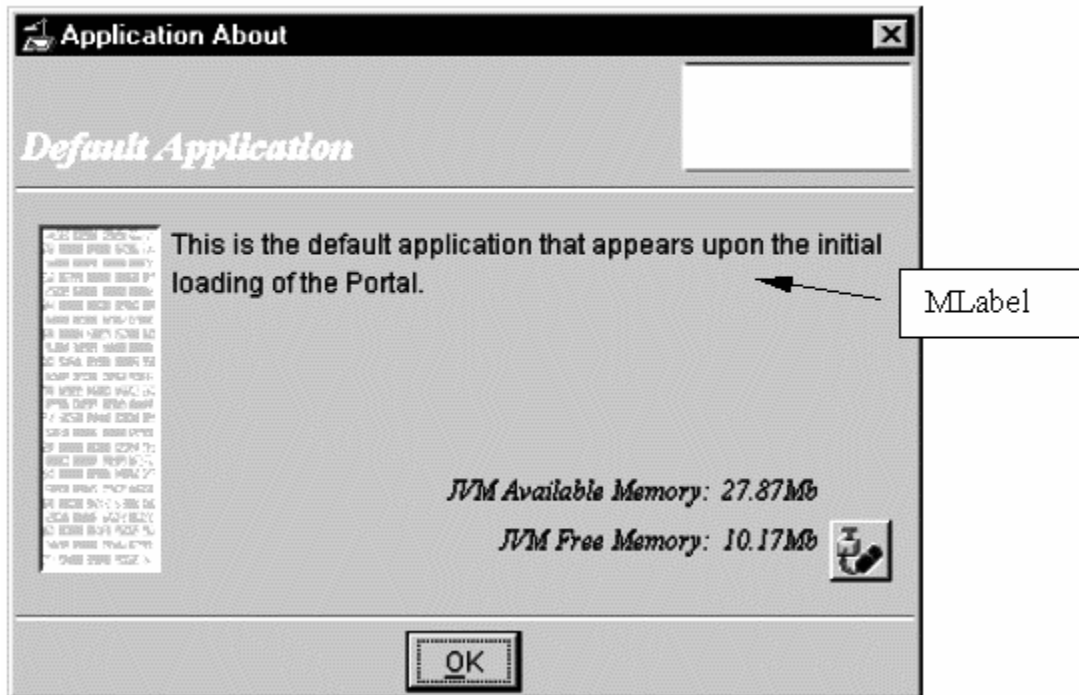


MLabel component

The code in the following example constructs an **MLabel** component:

```
int fontSize = getFont().getSize();
Font fontText = new Font("TimesRoman", Font.PLAIN, fontSize+4);
Font f2 = new Font("TimesRoman", Font.BOLD, fontSize+32);
MLabel labelBanner = new MLabel("TC Portal\nDesktop");
labelBanner.setFont(f2);
labelBanner.setTextAlignment(MLabel.CENTER);
MLabel labelVersion = new MLabel("Version 6.0\nUpdate version: 0");
labelVersion.setFont(fontText);
labelVersion.setTextAlignment(MLabel.CENTER);
```

The following figure shows the results of the code.



MLabel component produced from sample code

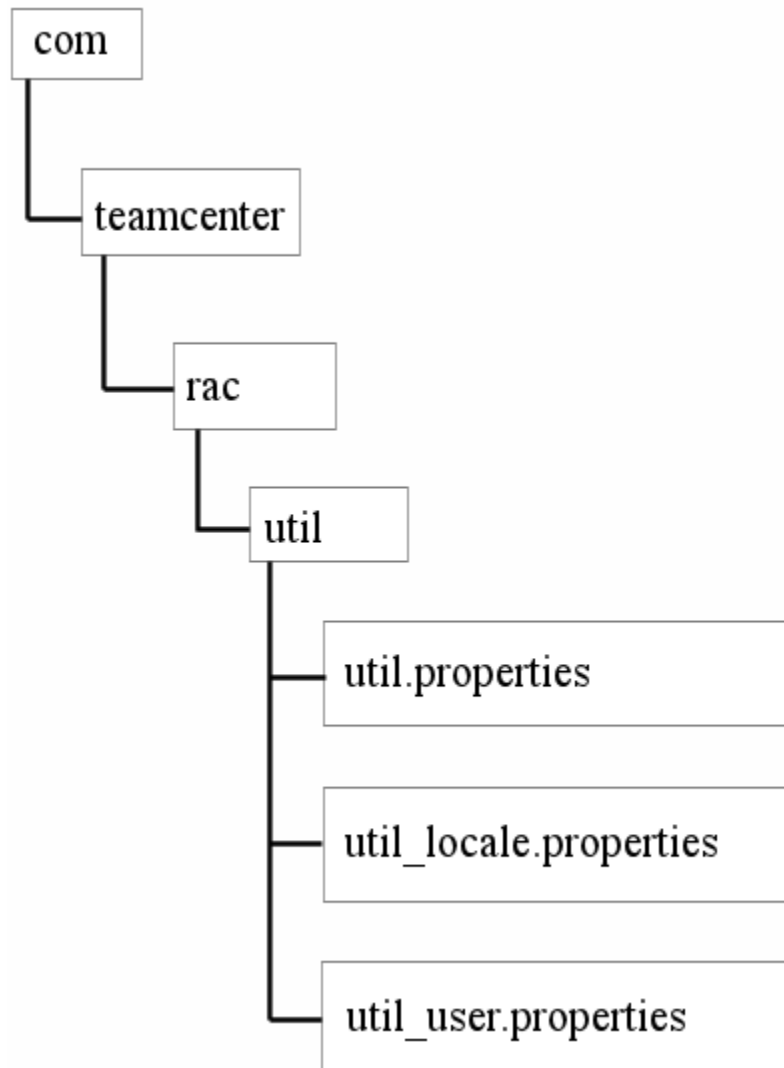
Registry

The **Registry** class contains registry information and provides a means of obtaining information stored in a resource bundle, extending the functionality of the resource bundle by way of encapsulation. This class provides the ability to native data types and instance classes via key registry entries. The registry files must match the package name.

The following code shows an example of a registry file.

```
# comments
import=com.teamcenter.rac.util
myLabel=My Label:
myIcon=images\myIcon.gif
ok=OK
ok.MNEMONIC=O
cancel=Cancel
cancel.MNEMONIC=C
```

The import statement in the preceding figure imports another registry file. It can also import multiple files using a comma delimiter. The following figure shows the hierarchy of the registry files in the **com.teamcenter.rac.util** package.



Hierarchy of the `com.teamcenter.rac.util` package

The keys and values inside the **util_user.properties** file override those defined in the **util_locale.properties** and **util.properties** files. The locale version property files are provided for localization and the user property files are provided so that users can define their own values. If no user version is defined, the default values in the **util.properties** file are used.

References to **Registry** objects are obtained by the **getRegistry()** static methods provided in this class (as shown in the following code example):

```

Registry reg = Registry.getRegistry ( this );
String label = reg.getString ( "myLabel" );
ImageIcon myIcon = reg.getImageIcon ( "myIcon" );
    The following is the corresponding registry file read by the above code:
# comments
import=com.teamcenter.rac.util
myLabel=My Label:
myIcon=images\myIcon.gif
  
```

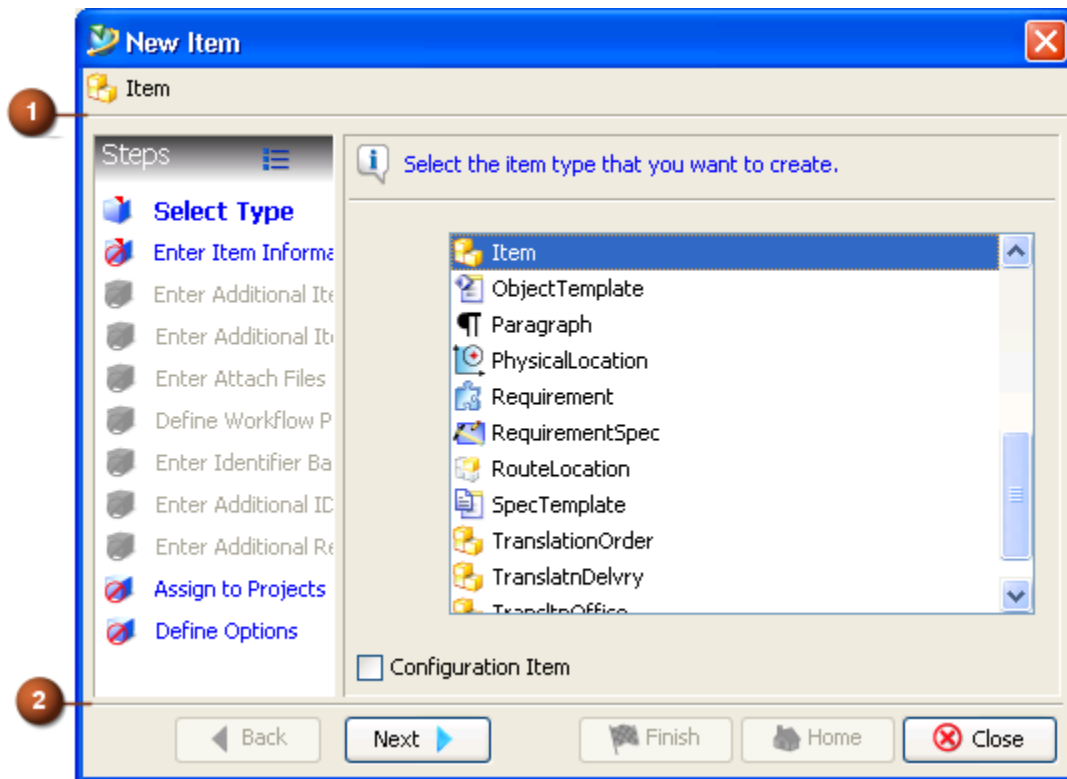
```

ok=OK
ok.MNEMONIC=O
cancel=Cancel
cancel.MNEMONIC=C

```

Separator

The **Separator** component visually separates user interface components (as shown in the following figure). Separators can be oriented either horizontally or vertically.



Separator in New Item dialog box

- 1 **Separator** used to separate dialog box header from the body
- 2 **Separator** used to separate dialog box footer from the body

The code in the following example uses the **Separator** component:

```

import java.awt.*;
import java.awt.event.*;
import java.util.*;
import javax.swing.*;
import com.teamcenter.rac.util.*;
public class SeparatorTest extends JPanel
{
    SeparatorTest ()
    {

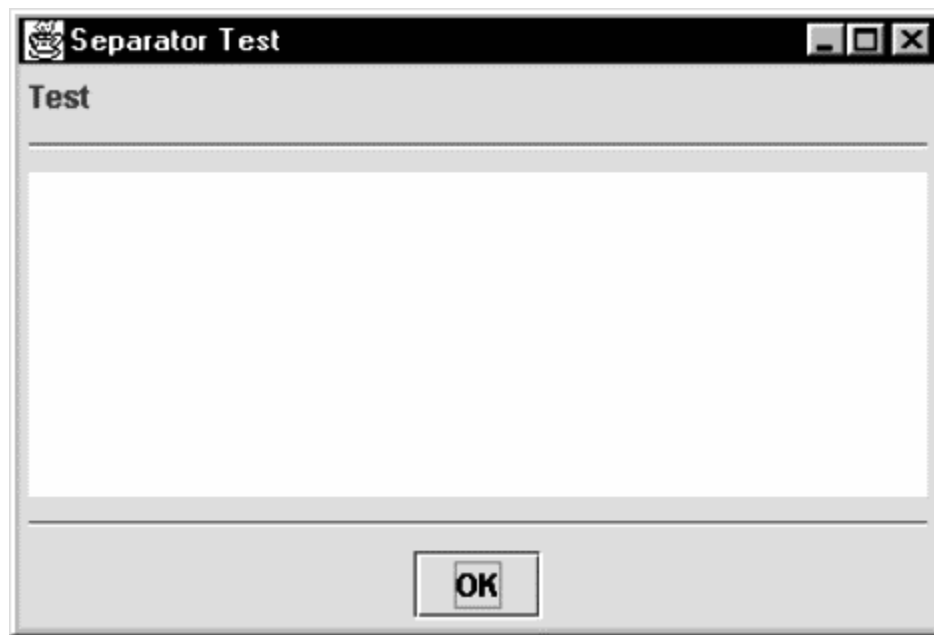
```

```

        this.setLayout ( new VerticalLayout(10,4,4,4,4) );
        JLabel label = new JLabel ( "Test" );
        JTextArea text = new JTextArea ( 10, 3 );
        JButton okButton = new JButton ( "OK" );
        this.add ("top.nobind.left.top", label);
        this.add ("top.bind", new Separator());
        this.add ("bottom.nobind.center.center", okButton);
        this.add ("bottom.bind", new Separator());
        this.add ("unbound.bind", text);
    }
    public static void main(String s[])
    {
        WindowListener l = new WindowAdapter()
        {
            public void windowClosing(WindowEvent e) {System.exit(0);}
        };
        Frame f = new Frame("Separator Test");
        f.addWindowListener(l);
        f.add("Center", new Separatortest());
        f.pack();
        f.setSize(new Dimension(350,350));
        f.show();
    }
}

```

The following figure shows the results of the code.



Separator produced from sample code

SplitPane

The **SplitPane** component is similar to the **JSplitPane** component. It extends from the **JPanel** component and creates a split pane with two panels: either left and right, or top and bottom.

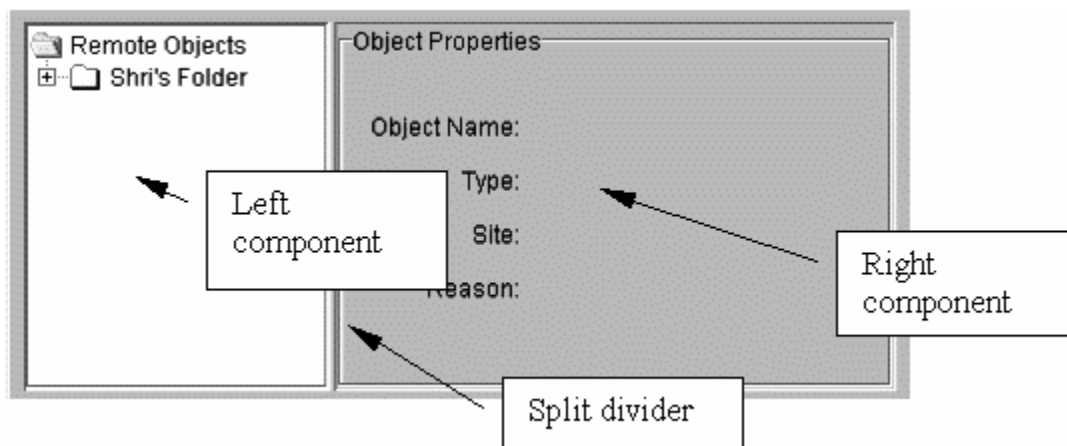
Note:

This component is a Swing class. Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens Digital Industries Software encourages you to customize Teamcenter using SWT/JFace components. Siemens Digital Industries Software will discontinue Swing/AWT support in a future version.

The code in the following example creates the **SplitPane** component. It adds two components, one for the left pane and one for right. The divider is set at 45 percent of the total size of the **SplitPane** component.

```
SplitPane splitPane = new SplitPane ( SplitPane.HORIZONTAL_SPLIT );
JPanel leftPanel = new JPanel ();
JPanel rightPanel = new JPanel ();
// add components to splitPane
splitPane.setLeftComponent ( leftPanel );
splitPane.setRightComponent ( rightPanel );
splitPane.setDividerLocation(0.45);
splitPane.setDividerSize(2);
```

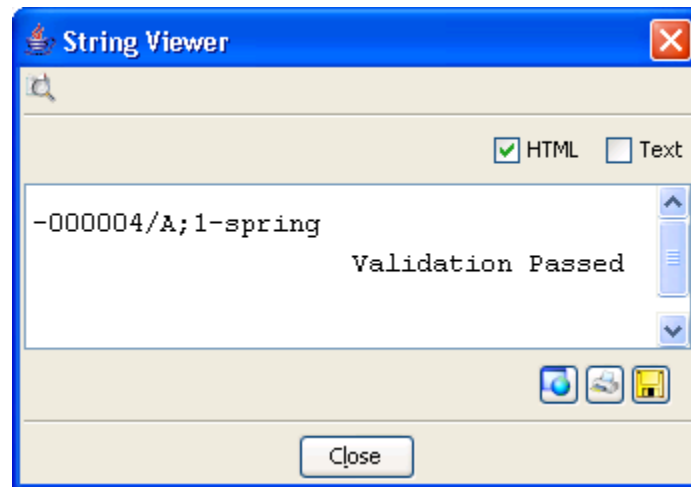
The following figure shows the pane produced by the code.



SplitPane component used in a dialog box

StringViewerDialog

The **StringViewerDialog** class loads a string array and displays text in a scrollable format. This class extends from the **AbstractDialog** class. The dialog box allows you to save and print the string array text and can also act as a simple text editor. In the following figure, the entire panel is a **StringViewerDialog** component.



StringViewerDialog component

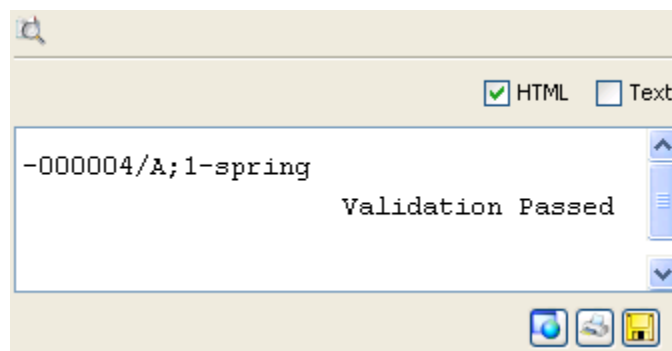
The code in the following example creates a **StringViewerDialog** component.

```
StringViewerDialog dlg = new StringViewerDialog (file);
dlg.setVisible(true);
```

StringViewerPanel

The **StringViewerPanel** component displays a text file or string array using a **JTextArea** component to store and render the text. A scroll pane ID handles text scrolling. The **StringViewerPanel** component serves as both a simple text editor and a file viewer and supports print and saving files.

The following figure shows the Validation report, constructed using the **StringViewerPanel** component.



Validation report

The code in the following example creates a **StringViewerPanel** component:

```
private StringViewerPanel stringViewerPanel = null;
stringViewerPanel = new StringViewerPanel(file);
parentPanel.add ( "unbound.bind.center.top", stringViewerPanel );
```


Troubleshooting rich client customization

Common problems in rich client customization

Eclipse startup error

If you get the error shown in the following figure during Eclipse's startup, either the Java Runtime Environment is not installed or the **PATH** statement does not contain the *JDK-installation-directory\bin* directory.



Customizations from a new plug-in do not appear

Teamcenter has a base set of Eclipse plug-ins for the rich client. They are located in the *TC_ROOT\portal\plugins* directory, and the file names start with **com.teamcenter**. When you add a new plug-in, you deploy it into this directory.

If you add or remove a plug-in when customizing the rich client and your changes do not appear, delete the **Teamcenter** subdirectory in the user's home directory on the client. This clears the cache.

On a Windows client, it is typically the *%HOMEDRIVE%%HOMEPATH%\Teamcenter* directory. On a Linux client, it is typically the *\$HOME/Teamcenter/* directory.

Unable to load application error

When you start the rich client, you may get the following error:

```
Unable to Load Application: application-name
Details java.lang.reflect.InvocationTargetException
```

This error can occur when you expose custom classes using Eclipse but do not include the class information in the **MANIFEST.MF** file. To correct this error, ensure that you have added your plug-in on the **Runtime** tab in the **Exported Packages** pane in Eclipse. This adds class information to the **MANIFEST.MF** file.

Previous to Teamcenter 2007.2, the classpath alone was sufficient to handle custom class information. In later Teamcenter versions, you must add your plug-in to the **Exported Packages** pane on the project **Runtime** tab to ensure that the **MANIFEST.MF** file is correctly updated.

Rich client debugging

Rich client debugging tools

You can debug your customizations to the rich client by using the following tools:

- **Print Object** view
Displays the selected object's internal attribute values. Use it to determine if there is an incorrect value for an attribute.
- **Communication Monitor** view
Shows you the calls between the rich client and server. Use it to determine if your data is being correctly exchanged between the server and client.
- **Performance Monitor** tool
Tracks the calls between the server and the database. Use it to determine how fast the server and the database are communicating.
- **DB Walker** view
Examines the database. This view is for Siemens Digital Industries Software internal use only.

There are also standard Eclipse views that can help you debug your customization.

Debug using the Print Object view

You can use the **Print Object** view in the rich client to find objects or retrieve information about objects in the Teamcenter database.

Retrieve an object by UID

Enter an object's tag (UID) here to retrieve that object from the database. Once retrieved, you can copy a URL link to that object, and then paste it within the rich client or into an email, for example.

Copy a link to an object

Use the **Copy** button  to copy a URL link to this object onto the rich client and Windows clipboards.

See previously referenced objects

Use the drop-down list at the top-right of the view to switch between previously viewed objects.

Filter which attributes you see

Use the two drop-down lists at the bottom of the view to control which attributes are displayed.

Save attributes to a file

Use the **SAVE** button to save the shown attributes to a file for future reference.

Debug using the Communication Monitor view

1. In the rich client, choose **Window**→**Show View**→**Communication Monitor**.
The values appear in the **Communication Monitor** pane at the bottom of the rich client.
2. To choose what you want to see in the monitor, click the **Menu** button  in the **Communication Monitor** pane and choose one or more of the following:
 - **Show Server Calls**
Displays an entry for each call to the server.
 - **Show Stack Trace**
Displays the call stack trace for each call to the server.
 - **Show Request**
Displays the XML request sent to the server.
 - **Show Response**
Displays the XML response returned by the server.
 - **Show Timing**
Displays the length of the server call in seconds.
3. If you want to clear the data, click the **Clear** button  in the pane.
4. If you want to save the data to a file, click the **Save As** button  in the pane.

Debug using the Performance Monitor tool

1. Set the **TC_PERFORMANCE_MONITOR** environment variable to **0**.
The tool appears the next time you open the rich client.
2. To see the data from the server and client, click the **Report** button. The data is also logged along with the text in the **Log comment** box. It also resets the counters.

- The SQL and server CPU statistics are retrieved from the server.
- **Wallclock time since reset** is the time since the last reset.
Times are shown in milliseconds. If the top of the Performance Monitor states that the Hi-Res timer is in use, times are accurate to 1 millisecond. Otherwise the standard operating system clock is in use; Microsoft Windows uses a 60 Hertz clock, so times on Windows are accurate to about 16 milliseconds.
- **Client CPU time** is available only with the Hi-Res timer.
- **Server calls made** is a count of all calls made to the server, not including the call to get the SQL statistics.

Note:

If you select the **Reset on first server call** check box, the Performance Monitor is reset after the next server call after you click the **Report** button.

3. To clear the data and reset all counters, click the **Reset** button.

Note:

If you select the **Reset on first server call** check box, the Performance Monitor is reset after the next server call after you click the **Reset** button.

4. To save the data to a file, click the **Save** button.

Debug using Eclipse views

Some Eclipse views may be helpful when debugging your customizations:

- **Console view**
Shows the standard output, standard error, and standard input for your program. Used to debug problems with customizations and to monitor rich client activity.
For more information, see the following URL in the Eclipse help:
http://help.eclipse.org/indigo/index.jsp?topic=/org.eclipse.jdt.doc.user/reference/views/console/ref-console_view.htm
- **Progress view**
Shows the progress of background jobs. You can connect this view to your customizations if you want to see the progress when your customizations run.
For more information, see the following URL in the Eclipse help:

http://help.eclipse.org/indigo/index.jsp?topic=/org.eclipse.platform.doc.isv/guide/runtime_jobs_progress.htm

Enabling client-side logging

Changing the logging level and location

Teamcenter uses the **log4j** mechanism for logging. You can change logging parameters in the **TcLogger.properties** file, located in the `TC_ROOT\portal\plugins\configuration_version` directory, to specify the level and scope of logging, as well as changing the log file location.

You can change the logging level for the entire rich client, a package, or a single class, as well as the log location:

- Rich client debug-level logging
To enable debug-level logging for the entire rich client, change the **log4j.rootLogger** line in the **TcLogger.properties** file to **DEBUG**. The line should look like this:

```
log4j.rootLogger=DEBUG, TcLoggerConsoleAppender, TcLoggerFileAppender,
TcLogContext
```

- Package debug-level logging
To enable debug-level logging for a package, add a line in the **TcLogger.properties** file for it. For example, if you want to enable debug-level logging for Structure Manager, the line looks like this:

```
log4j.logger.com.teamcenter.rac.pse=DEBUG
```

- Single class debug-level logging
To enable debug-level logging for a single class, add a line in the **TcLogger.properties** file for it. For example, if you want to enable logging for the **TCSession** class, the line looks like this:

```
log4j.logger.com.teamcenter.rac.kernel.TCSession=DEBUG
```

- Log location
By default, the rich client log is the *operating-system-user-name_TcRAC.log* file in your operating system's temporary directory. You can change the location by changing the **log4j.appender.TcLoggerFileAppender.file** entry in the **TcLogger.properties** file. This is the default location:

```
log4j.appender.TcLoggerFileAppender.file=${osgi.instance.area}/
${user.name}_TcRAC_${timestamp}.log
```

Adding appenders

You can easily add or remove an appender to a logger. The content of all appender output is identical unless you add a filter to the content. Each appender supports a pattern layout that determines the format of the output. By default, the rich client has two kinds of appenders:

- A console appender which outputs to the console. To see console output in the rich client outside of Eclipse, you must use the **-consolelog** flag on the command line when you run the rich client.

- A file appender which outputs to a file.

Pattern layouts

Use a pattern layout to include more information in the console or log file. Each appender has a pattern layout, which is a substitution string for the output.

- **c**
Display the logger (category) name.
- **C**
Display the fully qualified class name of the caller issuing the logging request.
- **d**
Display the date of the logging event.

`%d{HH:mm:ss,SSS}`
- **F**
Display the file name where the logging request was issued. This slows execution.
- **l**
Display the location information of the caller that generated the logging event. This slows execution.
- **L**
Display the line number from where the logging request was issued. This slows execution.
- **m**
Display the message.
- **M**
Display the method name where the logging request was issued. This slows execution.
- **n**
Insert a new line.
- **p**
Display the priority of the logging event (**DEBUG**, **WARN**, **INFO**).
- **r**

Display the number of milliseconds elapsed since the start of the application until the creation of the logging event.

- **t**

Display the name of the thread that generated the logging event.

- **x**

Display the nested diagnostic context (NDC) associated with the thread that generated the logging event.

For example:

- **%-5p: %m%n**

This layout produces the following log message:

```
ERROR: There is something wrong here!
```

- **%n%-5p: %d{HH:mm:ss,SSS} %x - %C{1}.%M:%L%n%m%n**

This layout produces the following log message:

```
INFO : 11:51:54,871 - Class.method:90
Session logging via TcLoggerFileAppender into file
C:\Windows\temp\smithj_TcRAC.log
```

Add logging to your code

1. Import the Apache logger:

```
import org.apache.log4j.Logger;
```

2. Add a static logger to the file at the top of the class:

```
private final static Logger logger = Logger.getLogger( MyClass.class );
```

3. Specify the logger output you want:

- **Error condition**

```
logger.error( String [,Throwable] );
```

- **Warning condition**

```
logger.warn( String [,Throwable] );
```

- **Information condition**

```
logger.info( String [,Throwable] );
```

- **Debug condition**

```
logger.debug( String [,Throwable] );
```

4. (Optional) Add debug control flags to the **TcLogContext** appender defined in the **TcLogger.properties** file:
 - a. Add the boolean flag to the **TcLogger.properties** file.
 - b. Add the getter and setting for the flag to the **com.teamcenter.rac.util.log.TcLogContext** line.
 - c. Use the getter to control your debug code:

```
if( TcLogger.getLogContext().getMyFlag() ) doSomething();
```

Listener leaks

Events in Java are fired by means of listeners. An object registers interest with a target object, so that when an event occurs the listening object is notified. For this relationship to be maintained, the target object must maintain a reference to the listening object. The Java memory management facilities look only to delete objects from virtual memory when the objects are no longer referenced by any other Java object. The problem at hand is the removal of listeners.

Rich client does not currently have a system in place to facilitate the removal of listeners. This creates two issues:

- It causes a memory leak.
The memory leak issue exists because references are maintained to Java objects that are no longer needed. This creates the situation in which the garbage collector runs but is unable to remove the old objects because they are still tied as listeners. Under this condition, the virtual memory of the Java VM eventually runs out.
- It begins to impact performance of the UI, because old components are being needlessly updated.
The performance issue is more prevalent than the memory leak. System performance begins to deteriorate quickly under certain UI conditions. The use of the viewer illustrates this, because as new viewers are displayed, they add their components to the session, attached as listeners. The UI appears sluggish and eventually becomes unusable.

The **InterfaceSignalOnClose** and **SignalOnClose** classes are used to remedy listener leaks:

- **InterfaceSignalOnClose**

The **InterfaceSignalOnClose** class requires the implementation of the **closeSignaled()** method. This interface is designed to signify the desire to be notified when closure is to commence. The **closeSignaled()** method is designed to remove any listeners that were created during the life of the object.

This interface signifies that the implementing class registers interest to be known when closure occurs. The implementing class is required to implement the **closeSignaled()** method. The **closeSignaled()** method is invoked when closure is commencing (as shown in the following code):

```
public interface InterfaceSignalOnClose
{
    public void closeSignaled();
}
```

Example: An implementation of the **closeSignaled()** method can look like the following:

```
public void closeSignaled()
{
    TCSession session = (TCSession) application.getSession();
    if (session != null)
    {
        session.removeAIFComponentEventListener( this );
    }
}
```

- **SignalOnClose**

The **SignalOnClose** class is designed to signal the processing of the components to detach themselves from listeners and prepare to be closed. This class contains a single method, **close()**, which is designed to be passed a reference to a **Container** object. The **Container** object is the start of a recursive walk down the component tree to look for instances of **InterfaceSignalOnClose** classes. If instances are found, the classes are notified that closure is commencing. At this point, it is the responsibility of the implementing class to take appropriate action.

A. Rich client customization reference

Adding a third-party JAR file to your plug-in	A-1
Command line options for rich client startup	A-1
Coding standards	A-5
File organization	A-5
Naming conventions	A-5
Property conventions	A-6
Source code conventions	A-6
Dialog box standards	A-7
Property beans	A-8
Common Teamcenter command IDs	A-26
Plug-in locations of perspectives	A-27
Application Integration Framework (AIF)	A-28
AIF customization and development	A-28
Integrating the Application Integration Framework (AIF) desktop with the Eclipse workbench	A-29
AIF context sensitivity	A-30

A. Rich client customization reference

Adding a third-party JAR file to your plug-in

Sometimes, you may need to add a third-party JAR file to your plug-in project. To accomplish this, add the JAR file to a project library.

1. To create a folder to hold the JAR file, right-click your Eclipse project, choose **New**→**Folder**, and create a **lib** folder. This stands for *libraries* and can contain all the third-party JAR files required by the project.
2. To import the JAR file, right-click the **lib** folder, choose **Import**→**General**→**File System**, select the directory that contains the JAR file on your system, and select the JAR file to import.

The JAR file is imported into the **lib** folder.

3. To add the **lib** folder to the classpath, on the **Runtime** tab, click the **Add** button and select the **lib** folder.
4. To ensure the new JAR file is built with the project, click the **Build** tab, and in the **Binary Build** pane, select the check box next to the **lib** folder.
5. Click the **Save** button.
6. To update the class path, right-click the project and choose **PDE Tools**→**Update classpath**.

The JAR file is added to the **Referenced Libraries** container under the project.

7. To add the JAR to the project Java build path properties, perform the following:
 - a. Right-click the project, choose **Properties**, and select **Java Build Path** in the left pane.
 - b. In the **Java Build Path** dialog box, click the **Add Library** button, select **User Library**, click **Next**, select the **lib** folder, and click **Finish**.
 - c. In the **Java Build Path** dialog box, select the **lib** library, click the **Add JARs** button, select the JAR file under the **lib** folder, and click **OK**.

Command line options for rich client startup

You can specify the following command line options during rich client startup.

Note:

- You can specify these same command line options in Eclipse when you test your customization. Choose **Run**→**Debug Configurations** or **Run**→**Run Configurations**, select the application in the left pane that you want to test, click the **Arguments** tab, and enter the command line options in the **Program arguments** box.
- You can also use some of these command line arguments (such as **-attach**) when constructing a URL to launch the rich client in a four-tier environment. To obtain a URL in the four-tier rich client, right-click an object such as an item and choose **Copy**, and then paste the resulting URL into the address bar of a Web browser. This allows you to launch the rich client and automatically open the copied object. For example:

```
http://svi6w101:7001/tc/launchapp?-attach=true&-s=226TCSession&
-o=QNG11_93oEfenBAAAAAAAAAAAAA
```

In this example, **launchapp?** launches the rich client, and **-attach** launches within an already-running rich client session if one is available. (While not rich client command line options, the **-s** argument designates the session, and the **-o** argument designates the object to open.)

If you have multiple rich client installations on your system, the constructed URL only opens the most recently installed rich client because the registry is updated at each installation.

- **-arch** *architecture*

Defines the processor architecture on which the Eclipse platform is running. The Eclipse platform ordinarily computes the optimal setting using the prevailing value of Java **os.arch** property. If specified here, this is the value that the Eclipse platform uses. The value specified here is available to plug-ins as **BootLoader.getOSArch()**. Example values: **x86**, **sparc**, **PA-RISC**, **ppc**.

- **-attach**

Attaches the new client to an existing session.

For example, a user launches CATIA from the rich client and works on a structure. The same structure is also loaded in Structure Manager. Selecting a part in CATIA, the user wants to synchronize the same selections in Structure Manager. This is where the **-attach** argument is useful to indicate that the synchronization of the selections should happen in an existing rich client session.

- **-application** *applicationId*

Specifies the application to run. Applications are declared by plug-ins supplying extensions to the **org.eclipse.core.runtime.applications** extension point. This argument is typically not required. If specified, the value overrides the value supplied by the configuration. If not specified, the Eclipse Workbench is run. For example, to launch My Teamcenter, use the following command:

```
portal.bat
-application=com.teamcenter.rac.ui.perspectives.navigatorPerspective
```

- **-clean**

Cleans cached data used by the OSGi framework and Eclipse run time. This is useful if you have new plug-ins you have added to your environment. Try to run Eclipse once with this argument if you observe startup errors after installation, update, or using a shared configuration.

Note:

When you start the rich client, if your customization changes still do not appear in the user interface after running the **-clean** argument, delete the Teamcenter subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client.

On a Windows client, it is typically the **%HOMEDRIVE%%HOMEPATH%\Teamcenter** directory. On a Linux client, it is typically the **\$HOME/Teamcenter/** directory.

If you delete this directory, the last state of the rich client is lost, and the user interface appears as it does at initial startup.

- **-configuration** *configurationFileURL*
The location for the Eclipse platform configuration file, expressed as a URL. The configuration file determines the location of the Eclipse platform, the set of available plug-ins, and the primary feature. Note that relative URLs are not allowed. The configuration file is written to this location when the Eclipse platform is installed or updated.
- **-consolelog**
Mirrors the Eclipse platform's error log to the console used to run Eclipse. It is effective when combined with **-debug**.
- **-data** *workspacePath*
The path of the workspace on which to run the Eclipse platform. The workspace location is also the default location for projects. Relative paths are interpreted relative to the directory that Eclipse was started from.
- **-detach**
Detaches the client from an existing session.
Starts clients as separate sessions. This is the default behavior, even if the **-detach** option is not specified.
- **-debug** [*optionsFile*]
Puts the platform in debug mode and loads the debug options from the file at the given location, if specified. This file indicates which debug points are available for a plug-in and whether or not they are enabled. If a file location is not given, the platform looks in the directory that eclipse was started from for a file called **.options**. Both URLs and file system paths are allowed as file locations.
- **-dev** [*classpathEntries*]
Puts the platform in development mode. The optional classpath entries (a comma separated list) are added to the run-time classpath of each plug-in. For example, when the workspace contains plug-ins being developed, specifying **-dev bin** adds a classpath entry for each plug-in project's directory named **bin**, allowing freshly generated class files to be found there. Redundant or nonexistent classpath entries are eliminated.

- **-DskipRegReload** *args*

If you are starting the rich client inside the Eclipse IDE, this option prevents the registry database from loading if it already exists, which can save up to a minute or more during startup depending on your system. You can add this argument to your run/debug configuration. However, if you use this argument, any changes you make to the registry property files are not used. If you are making changes to the registry database, do not use this argument.

- **-initialize**

Initializes the configuration being run. All run-time related data structures and caches are refreshed. This is useful with shared installs; running Eclipse once with this option from an account with write privileges improves startup performance.

- **-nl** *locale*

Defines the name of the locale on which the Eclipse platform is running. The Eclipse platform ordinarily computes the optimal setting automatically. If specified here, this is the value that the Eclipse platform uses. The value specified here is available to plug-ins as **BootLoader.getNL()**. For example, you can use the following: **en_US** or **fr_FR_EURO**.

- **-nosplash**

Runs the platform without putting up the splash screen.

- **-os** *operatingSystem*

Defines the operating system on which the Eclipse platform is running. The Eclipse platform ordinarily computes the optimal setting using the prevailing value of Java **os.name** property. If specified here, this is the value that the Eclipse platform uses. The value specified here is available to plug-ins as **BootLoader.getOS()** and used to resolve occurrences of the **\$os\$** variable in paths mentioned in the plug-in manifest file. For example, you can use one of the following: **win64**, **macosx**, or **linux**.

- **-perspective** *perspectiveId*

The perspective to open in the active workbench window on startup. If this parameter is not specified, the perspective that was active on shutdown will be opened.

- **-plugincustomization** *propertiesFile*

The location of a properties file containing default settings for plug-in preferences. These default settings override default settings specified in the primary feature. Relative paths are interpreted relative to the directory that Eclipse was started from.

- **-product** *productId*

The ID of the product to run. The product gives the launched instance of Eclipse its personality, and determines the product customization information used. This replaces **-feature**, which is still supported for compatibility.

- **-refresh**

Option for performing a global refresh of the workspace on startup. This reconciles any changes that were made in the file system since the platform was last run.

- **-showlocation** [*workspaceName*]

Option for displaying the location of the workspace in the window title bar. The optional workspace name argument displays the provided name in the window title bar instead of the location of the workspace.

- **-vm** *vmPath*
The location of Java Runtime Environment (JRE) to use to run the Eclipse platform. If not specified, the JRE is at **jre**, sibling of the Eclipse executable. Relative paths are interpreted relative to the directory that Eclipse was started from.
- **-vmargs** *args*
When passed to Eclipse, this option customizes the operation of the Java Virtual Machine (VM) used to run Eclipse. If specified, this option must come at the end of the command line. The given arguments are dependant on the VM that is being run.

Coding standards

File organization

The following file and directory structure standards should be used when developing the rich client customization code:

- All package names must be lowercase. Do not use space characters in package names.
- The general package registry should have the same name as the last package name. For example, for the **com.mycompany.rac.explorer** package, the file for the **ResourceBundle** object that contains the registry information is **explorer.properties**.
- Image files associated with a particular package must be located within the images directory below the package.
- Image file names must consist of all lowercase letters.

Naming conventions

The following table describes the recommended naming convention for the various Java types.

Java type	Rule	Example	Comment
Interface	Interface [<i>name</i>], I [<i>name</i>]	InterfaceAIFOperationListener, ISelectionService	Teamcenter standard
Abstract class	Abstract [<i>name</i>]	AbstactAIFApplication	Teamcenter/Java standard

Java type	Rule	Example	Comment
Exception class	[name]Exception	SomethingHappenedException	Java standard
Variable naming	Lowercase first word, uppercase first letter of other words.	factoryName	Java standard
Accessor methods	Getting: Use getXXX , except for Booleans, where XXX is allowed. Setting: use setXXX .	getFactoryName(), setFactoryName(), isReady()	Java standard
Source files	Same as Class name (including case).	ISelectionService.java	Java requirement
Class names	Uppercase first character of each word.	ComponentManager	Java standard

Property conventions

Property files can be categorized in three ways:

- Core development
- Localization
- User properties

The reason for this distinction is that customers modify the user property files while maintaining the links to the core development property files. The following table describes the properties files using **explorer** as an example.

File	Description
explorer.properties	Base property file
explorer_locale.properties	Property file for the purpose of localization
explorer_user.properties	Customer created property file

Source code conventions

The source conventions follow the Sun Java source code standards that match the industry norms for Java development.

Dialog box standards

The following standards should be used when customizing the rich client dialog boxes:

- Dialog boxes must always be modal unless the situation requires that they be nonmodal, such as when the user selects additional information when the dialog box is visible.
- Mnemonics should be used for common dialog box buttons, such as **OK**, **Apply**, and **Cancel**.
- The initial location of the dialog box must be screen centered, and the sizing of the dialog box must be adjusted with a sizing factor.
- Dialog boxes should include a default focus when displayed.
- **Text field/area policies:**
 - Allow only the maximum number of characters that the property can accept.
 - Send an audible beep when the maximum is reached.
 - Set all text area components to the initial size of 3 rows by 30 columns.
 - Select the text when the focus is gained inside the text field.
 - Set word wrapping to **true** for text area components.
- **Color policies**

Whenever possible, use the default color provided by the base component. Allow the current look and feel to determine the color.

 - If it is not possible to use the default color, use the **SystemColor** class.
 - If neither the default color nor the **SystemColor** class suffice, define the color in the property files so users can change it.
- **Font policies**

Whenever possible, use the default font provided by the base component. If the default font is not sufficient, try one of the following options:

 1. Offset the font based on the size of the current font. Do not hard code the font name, because the font may not be available on all platforms.
 2. Mention the font in the property file.

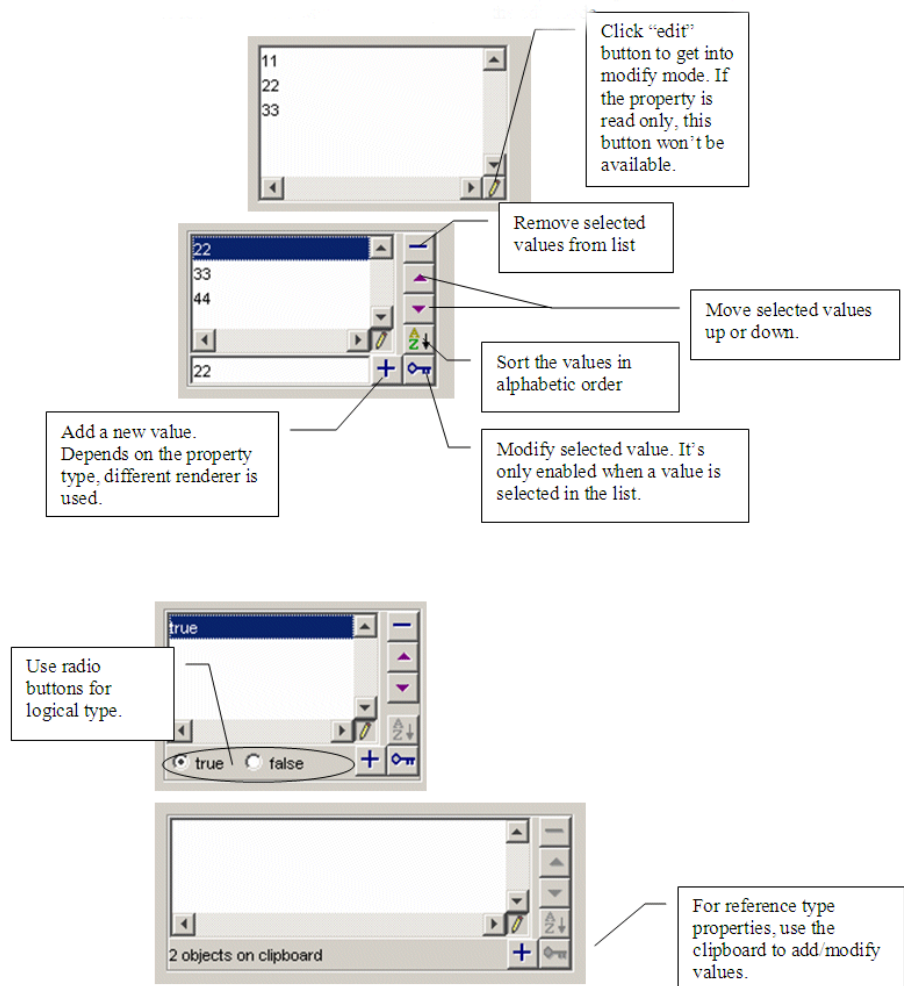
Property beans

The following table lists JavaBeans that you can use to customize the properties display.

Note:

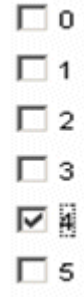
To display a red asterisk in the upper-right corner of a UI widget to indicate a mandatory property, use the Business Modeler IDE to set the **Required** property constant to **true**.

JavaBean	Description
PropertyArray	Renders all array type properties. It is composed with a list box and buttons to access the values in the list box (as shown in the following figure). Based on the type of the property, a different renderer is used for modifying or adding values. If the property is read only, users cannot enter edit mode.



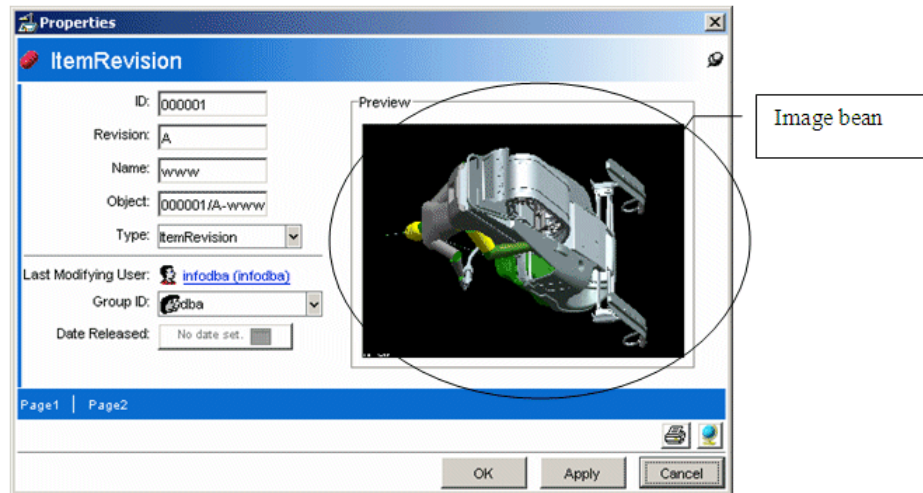
PropertyArray

JavaBean	Description
PropertyCheckbox	property The property name presented by the bean.
	modifiable Indicates whether the property is modifiable. If not, the edit button is not available and the user cannot enter edit mode.
	Renders any nonarray type property, except reference/relation type, using JCheckBox. A flag indicates whether the value is saved only when the check box is selected or whether it is always saved. There is a variable for the value to use on saving for a selected button or for the deselected button. If the flag is set to save regardless of whether the button is selected or deselected, both the selected value to save and the deselected value to save must be set. The following figure illustrates an implementation of the PropertyCheckbox bean. PropertyCheckbox Properties: property The property name presented by this bean. When a component is provided, the check box is selected if the property value is same as the selected value. modifiable Indicates if the property is modifiable. If not, the check box is disabled. selectedValue Specifies the value used for saving when the check box is selected. deselectedValue Specifies the value to use for saving when the check box is deselected. saveOnlyWhenSelected Indicates to save only when check box is selected or to always save.

JavaBean	Description
PropertyCheckboxOptionLov	Presents each value in the LOV as a check box (as shown in the following figure). This bean is designed for any type property that has a LOV attached. If the property is not an array, the check boxes are added to a button group; otherwise, they are not added to the button group and multiple selections are allowed.  PropertyCheckboxOptionLov Properties: property The property name presented by the bean. modifiable Indicates whether the property is modifiable. If not, the check boxes are disabled. lovName Specifies the name of the LOV that the bean will use. If undefined, the LOV information is retrieved from the property descriptor.
PropertyImage	This bean is limited to items, item revision, datasets, and BOM view revisions. It walks down these objects to find a dataset with a displayable image and displays that image. It uses the same logic as Engineering Process Management Visualization to find the image dataset and read the search order defined in the tcviewer.properties file. For example, the following figure shows an image attached to the selected item revision.

JavaBean

Description



PropertyImage

PropertyLabel

Renders any nonarray type property and displays the value of the property as the label text. Users cannot change the label text; therefore, save does not apply to this bean.

Properties:

property

The property name presented by the bean. When a component is provided, the property value is displayed as the label text.

PropertyLogicalPanel

This bean is used to present a logical type property. It uses two buttons, one for value **true** and another for **false**.

property

The property name presented by the bean.

modifiable

Indicates whether the property is modifiable. If not, the buttons are disabled.

Note:

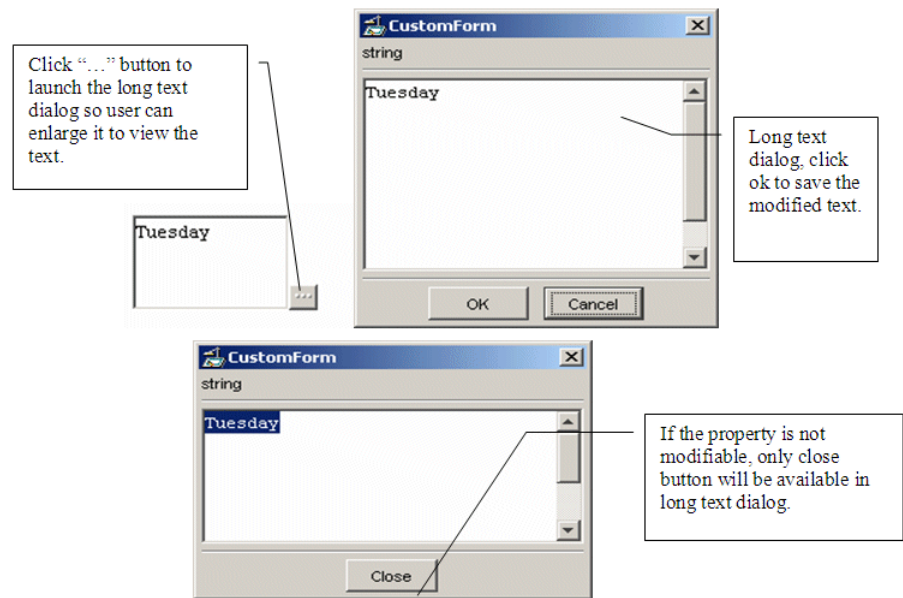
For Boolean properties, if neither true or false are selected, the value of the property is NULL.

PropertyLongText

This bean can be used with any string or note type properties, but is generally used to render text with lengths over 2500 characters. It contains a text area and a button. The long text dialog box is displayed by clicking the button, making it easier to browse the text (as shown in the following figure).

JavaBean

Description



PropertyLongText

Properties:

property

The property name presented by the bean.

modifiable

Indicates whether the property is modifiable. If not, the text area cannot be edited and only the close button is available in the long text dialog box.

PropertyLOVButton

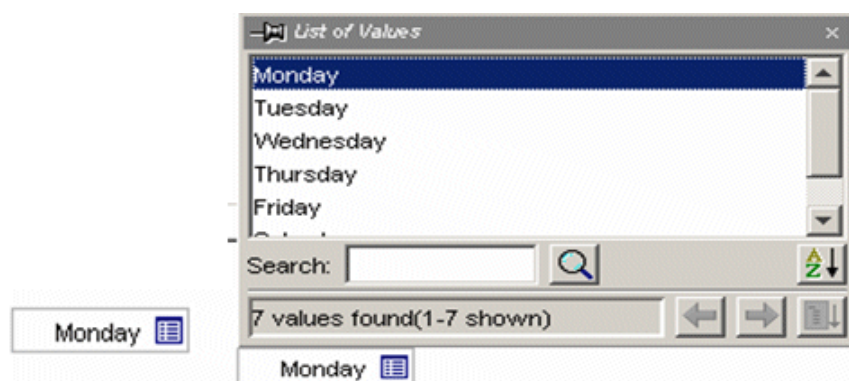
As of Teamcenter 10.1, this bean is deprecated and is replaced by the **PropertyLOVDisplayer** bean.

Used for any property that has an LOV attached to it except logical type.

It uses **LOVPopupButton** to present this property (as shown in the following figure).

JavaBean

Description



LOVPopupButton

Properties:

property

The property name presented by the bean. When a component is provided, the button text is set to the property value.

modifiable

Indicates whether the property is modifiable. If not, the button is disabled.

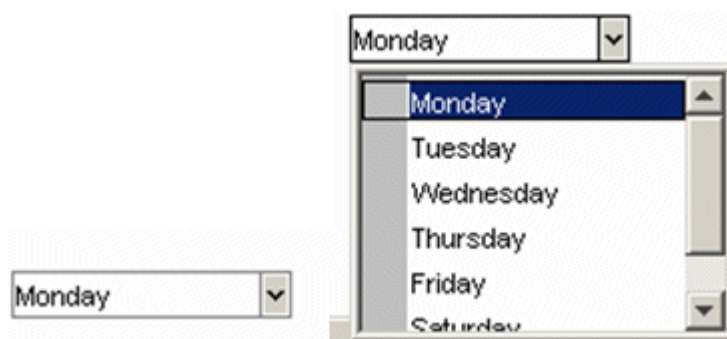
lovName

Specifies the name of the LOV that the bean uses. If undefined, the LOV information is retrieved from the property descriptor.

**PropertyLOV
Combobox**

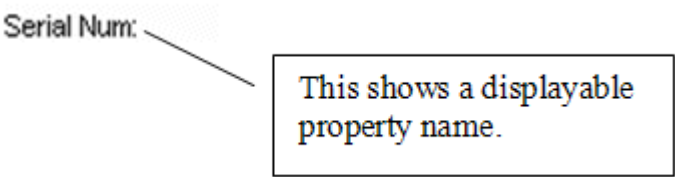
As of Teamcenter 10.1, this bean is deprecated and is replaced by the **PropertyLOVDisplayer** bean.

This bean is similar to the **PropertyLOVPopupButton** bean; however, it uses **LOVComboBox** rather than **LOVPopupButton** to present the property (as shown in the following figure).



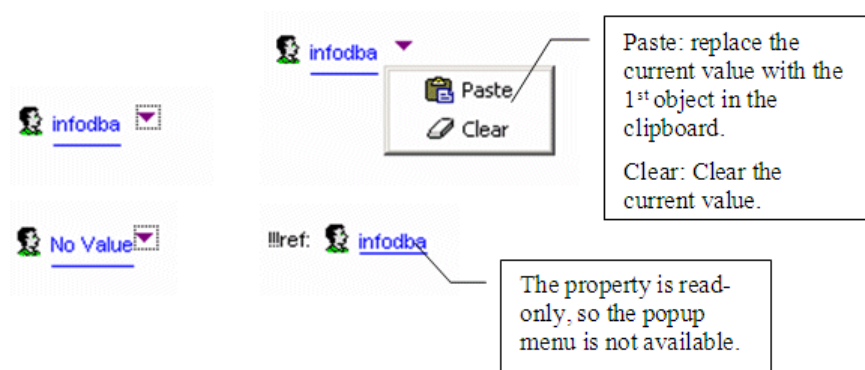
PropertyLOVPopupButton

Properties:

JavaBean	Description
	property The property name presented by the bean. modifiable Indicates whether the property is modifiable. If not, the combo box is disabled. lovName Specifies the name of the LOV that the bean will use. If undefined, the LOV information is retrieved from the property descriptor.
PropertyNameLabel	Renders the name of a property (as shown in the following figure). By specifying the name property, it shows either the displayable name or real name of the property according to the setting. This bean can be used along with other beans that display the property value.
	
	PropertyNameLabel Properties: property Specifies the property name presented by the bean. displayName Indicates whether the displayable name or real name is used. colon Indicates whether or not a colon is displayed after the name.
PropertyObjectLink	This bean renders reference type properties. A shortcut menu is provided for users to modify the value (as shown in the following figure). If the value is not modifiable, the shortcut menu is not available. When the link is clicked, the system displays a dialog box with properties of this reference component.

JavaBean

Description



PropertyObjectLink

Properties:

property

The property name presented by the bean.

modifiable

Indicates whether the property is modifiable. If not, the shortcut menu is not available.

PropertyPanel

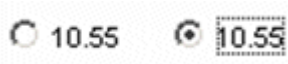
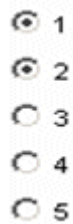
This bean is a generic container that allows the integrator to control how property values are stored and displayed. You must override the load and save methods to implement this standard behavior. You can combine one or more UI components within **JPanel** to provide custom behavior.

For example, if a property in a form called **status** is an integer and is either **1** or **2** meaning **Running** or **Stopped**, you may want two toggle buttons with radio behavior to represent this property. To accomplish this, use the **PropertyPanel** bean with two **JToggleButton**s contained within.

You must override the load and save methods of the **PropertyPanel** to determine the selection state from the two buttons. The **PropertyPanel** is designed for special behavior beyond the scope of the other property beans described in this manual.

PropertyRadioButton

The usage of this bean is same as that of the **PropertyCheckbox** bean; however, rather than using **JCheckBox**, **JRadioButton** is used. The following figure illustrates an implementation of the **PropertyRadioButton** bean.

JavaBean	Description
	 PropertyRadioButton Properties: property The property name presented by the bean. When a component is provided, the button is selected if the property value is the same as the selected value. modifiable Indicates whether the property is modifiable. If not, the button is disabled. selectedValue Specifies the value used for saving when the button is selected. deselectedVaue Specifies the value to use for saving when the button is deselected. saveOnlyWhenSelected Indicates to save only when the button is selected or to always save.
PropertyRadioButtonOptionLov	Same as the PropertyCheckboxOptionLov bean, except it uses buttons (as shown in the following figure).  PropertyRadioButtonOptionLov
PropertySlider	Renders any numeric type property. For double or float types, the value is cast to an integer. For string or note types, the value is converted to an integer if possible. Upon save, the value set on the slider is converted to the corresponding property type and saved. The following figure illustrates an implementation of the PropertySlider bean.

JavaBean

Description

**PropertySlider**

Properties:

property

The property name presented by the bean. When a component is provided, the slider value is set according to the property value.

modifiable

Indicates whether the property is modifiable. If not, the slider is disabled.

PropertyTextArea

Renders any nonarray type property except reference/relation type. This bean wraps the rendering of the given POM property into the **JTextArea**, and is generally used for longer text (as shown in the following figure). Upon saving, it attempts to convert the string to the corresponding type. If it cannot convert the string to the required type, an exception is thrown.

**PropertyTextArea**

Properties:

property

Specifies the property name presented by the bean. When a component is provided, the property value is displayed inside the text area.

modifiable

Indicates whether the property is modifiable. If not modifiable, the text area cannot be edited.

PropertyTextField

Renders any nonarray type property, except reference/relation type properties (as shown in the following figure). This bean wraps the rendering of the given POM property into **JTextField**. Upon saving,

JavaBean	Description
	the bean attempts to convert the string to the corresponding property type. If it cannot convert the string to the required type, an exception is thrown.  PropertyTextField Properties: property Specifies the property name presented by the bean. When a component is provided, the property value is displayed inside the box. modifiable Indicates whether the property is modifiable. If not modifiable, the text box cannot be edited.
PropertyToggleButton	The usage of this bean is the same as the PropertyCheckbox bean; however, this bean uses JToggleButton rather than JCheckBox. The following figure illustrates an implementation of the PropertyToggleButton bean.  PropertyToggleButton Properties: property The property name presented by the bean. When a component is provided, the button is selected if the property value is the same as the selected value. modifiable Indicates whether the property is modifiable. If not, the button is disabled. selectedValue Specifies the value used for saving when the button is selected. deselectedVaue

JavaBean

Description

Specifies the value to use for saving when the button is deselected.

saveOnlyWhenSelected

Indicates to save only when the button is selected or to always save.

PropertyToggleButtonOptionLov

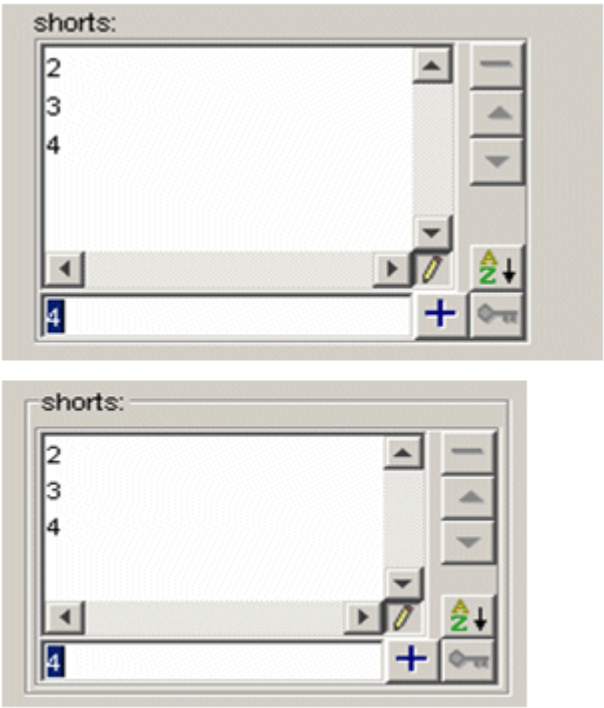
Same as **PropertyCheckboxOptionLov**, except it uses buttons (as shown in the following figure).



PropertyToggleButtonOptionLov

TitledPropertyArray

Displays the property name above the property array (as shown in the following figure) and is similar to the **PropertyArray** bean.



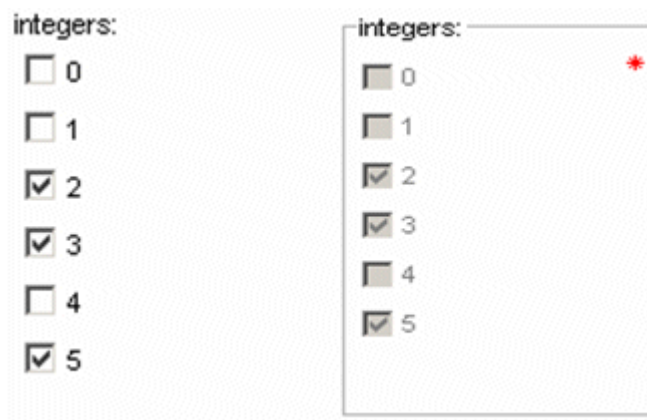
TitledPropertyArray

JavaBean	Description
	bordered Specifies whether a border is drawn around the panel. In addition, you can apply the properties of the PropertyArray bean.
TitledPropertyCheckbox	Displays the property name above the check box (as shown in the following figure). This bean is similar to the PropertyCheckbox bean and actually contains two beans: PropertyNameLabel and PropertyCheckbox.



TitledPropertyCheckbox

	bordered Indicates whether a border is drawn around the check box. In addition, you can apply the properties of the PropertyCheckbox bean.
TitledPropertyCheckboxOptionLov	Displays the property name above the check boxes (as shown in the following figure). This bean is similar to the PropertyCheckboxOptionLov bean.



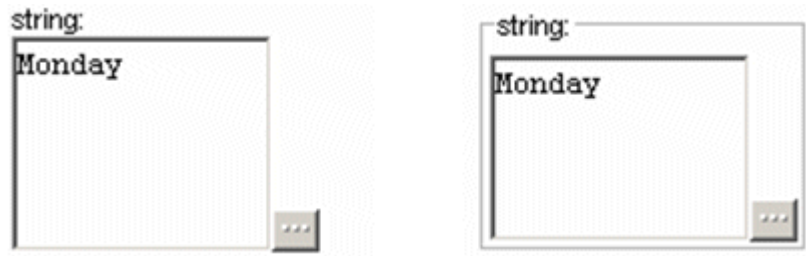
TitledPropertyCheckboxOptionLov

bordered	Indicates whether a border is drawn around the check boxes.
-----------------	---

JavaBean	Description
	In addition, you can apply the properties of the PropertyCheckboxOptionLov bean.
TitledPropertyLabel	Displays the property name above the label (as shown in the following figure). This bean is similar to the PropertyLabel bean, and it actually contains two beans: PropertyNameLabel and PropertyLabel.  The figure shows two side-by-side labels. The left label, representing PropertyLabel, displays the text 'string:' above 'Monday'. The right label, representing TitledPropertyLabel, displays the text 'string:' above a rectangular box containing the text 'Monday'. TitledPropertyLabel Properties: bordered Indicates whether a border is drawn around the label. In addition, you can apply the properties of the PropertyLabel bean.
TitledPropertyLogicalPanel	Displays the property name above the box (as shown in the following figure). This bean is similar to the PropertyLogicalPanel bean.  The figure shows two side-by-side logical panels. The left panel, representing PropertyLogicalPanel, displays the text 'logical:' above two radio buttons labeled 'true' and 'false', with the 'false' button selected. The right panel, representing TitledPropertyLogicalPanel, displays the text 'logical:' above a rectangular box containing the same two radio buttons labeled 'true' and 'false', with the 'false' button selected. TitledPropertyLogicalPanel bordered Indicates whether a border is drawn around the text area. In addition, you can apply the properties of the PropertyLongText bean. Note: For Boolean properties, if neither true or false are selected, the value of the property is NULL.

JavaBean**Description****TitledProperty
LongText**

Displays the property name above the long text panel (as shown in the following figure). This bean is similar to the **PropertyLongText** bean.

**TitledPropertyLongText****bordered**

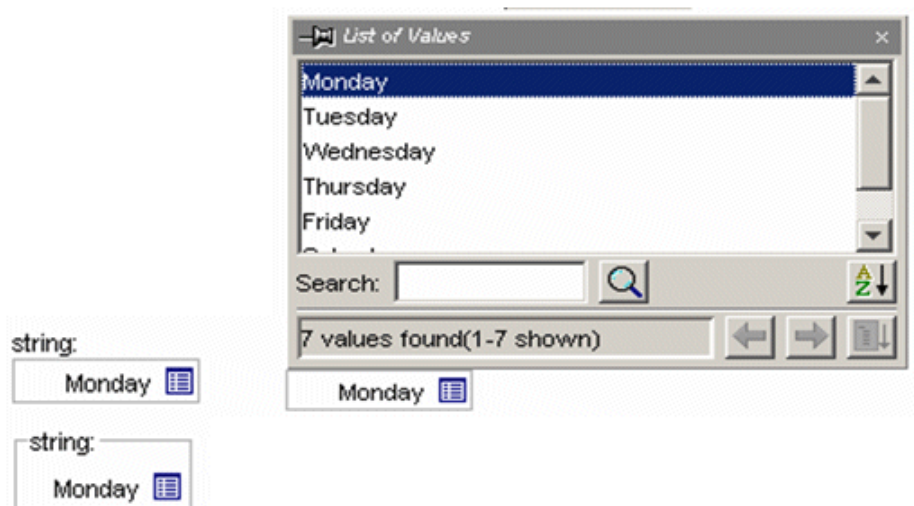
Indicates whether a border is drawn around the text area.

In addition, you can apply the properties of the **PropertyLongText** bean.

**TitledProperty
LOVButton****Caution:**

As of Teamcenter 10.1, this bean is deprecated and is replaced by the **TitledPropertyLOVDisplayer** bean.

Displays the property name above the LOV button (as shown in the following figure). This bean is similar to the **PropertyLOVButton** bean and actually contains two beans: **PropertyNameLabel** and **PropertyLOVButton**.

**TitledPropertyLOVButton****bordered**

JavaBean	Description
	Indicates whether a border is drawn around the LOV popup button. In addition, you can apply the properties of the PropertyLOVButton bean.
TitledPropertyLOVCombobox	Displays the property name above the LOV combo box (as shown in the following figure). This bean is similar to the PropertyLOVCombobox bean. Caution: As of Teamcenter 10.1, this bean is deprecated and is replaced by the TitledPropertyLOVDisplayer bean.  The figure shows two UI elements side-by-side. The left one is a simple text label 'string:' above a dropdown menu showing 'Monday'. The right one is a text label 'string:' above a dropdown menu showing 'Monday', but the entire element is enclosed in a rectangular border. TitledPropertyLOVCombobox
	bordered Indicates whether a border is drawn around the LOV combo box. In addition, you can apply the properties of the PropertyLOVCombobox bean.
TitledPropertyObjectLink	Displays the property name above the link (as shown in the following figure). This bean is similar to the PropertyObjectLink bean.  The figure shows two UI elements side-by-side. The left one is a text label 'ref:' above a link 'infodba' with a small icon to its left and a dropdown arrow to its right. The right one is the same element but enclosed in a rectangular border. TitledPropertyObjectLink

JavaBean

Description



TitledPropertyRadioButton

bordered

Indicates whether a border is drawn around the button.

In addition, you can apply the properties of the **PropertyRadioButton** bean.

**TitledPropertyRadio
ButtonOptionLov**

Displays the property name above the buttons (as shown in the following figure). This bean is similar to the **PropertyRadioButtonOptionLov** bean.



TitledPropertyRadioButtonOptionLov

TitledPropertySlider

Displays the property name above the slider (as shown in the following figure). This bean is similar to the **PropertySlider** bean, and it actually contains two beans: **PropertyNameLabel** and **PropertySlider**.

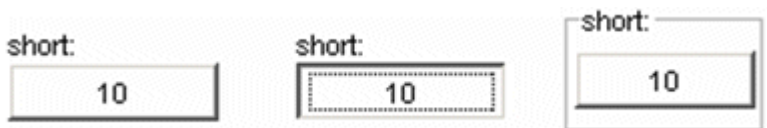


TitledPropertySlider

bordered

Indicates whether a border is drawn around the slider.

JavaBean	Description
TitledPropertyTextArea	In addition, you can apply the properties of the PropertySlider bean. Displays the property name above the text area (as shown in the following figure). This bean is similar to the PropertyTextArea bean and actually contains two beans: PropertyNameLabel and PropertyTextArea.  TitledPropertyTextArea Properties: bordered Indicates whether a border is drawn around the text area. In addition, you can apply the properties of the PropertyTextArea bean.
TitledPropertyTextField	Displays the property name above the text box (as shown in the following figure). This bean is similar to the PropertyTextField bean and actually contains two beans: PropertyNameLabel and PropertyTextField.  TitledPropertyTextField Properties: bordered Indicates whether a border is drawn around the text box. In addition, you can apply the properties of the PropertyTextField bean.
TitledPropertyToggleButton	Displays the property name above the button. This bean is similar to the PropertyToggleButton bean. The following figure illustrates an implementation of the TitledPropertyToggleButton bean.

JavaBean	Description
	 TitledPropertyToggleButton bordered Indicates whether a border is drawn around the button. In addition, you can apply the properties of the PropertyToggleButton bean.
TitledPropertyToggle ButtonOptionLov	Displays the property name above the buttons (as shown in the following figure). This bean is similar to the PropertyToggleButtonOptionLov bean.  TitledPropertyToggleButtonOptionLov

Common Teamcenter command IDs

Teamcenter command IDs have many uses. The most common being to add a command to a rich client menu or toolbar, or adding a command to a page or objectSet using an XML style sheet.

Note:

- The preferred method to locate command IDs is to use the rich client command-line utility `DumpCMSConfigInfo`. It will generate a **CSV** file containing an accurate listing of command IDs for your specific rich client configuration, as well as one for your view IDs, and one for your application context IDs as well.

```
teamcenter -application com.teamcenter.rac.util.DumpCMSConfigInfo
```

- When a command to create a new object is added within the **objectSet** tag, only commands for **New Item**, **New Dataset**, **New Part**, and **New Other** actions paste the new object with the relation specified in the **<objectSet source="relation.business-object">** tag. All other new object types use the default paste relation.

The following code example shows creating a new object with the **com.teamcenter.rac.newDesign** command ID. However, the default paste relation is used instead of the **IMAN_specification** relation in the **objectSet source** tag:

```
<objectSet source = "IMAN_specification.Design Revision "
defaultdisplay = "tableDisplay" sortBy = "object_string" sortdirection =
"ascending">
.
.
.
<command actionKey = "newDesignAction" commandId = "com.teamcenter.rac.NewDesign"
renderingHint = "commandbutton"/>
```

The following tables show a few common examples of a menu command, and their associated command IDs.

My Teamcenter File→New menu commands

Menu command	Command ID
File→New→Item	com.teamcenter.rac.newItem
File→New→Folder	com.teamcenter.rac.newFolder

My Teamcenter Edit menu commands

Menu command	Command ID
Edit→Cut	org.eclipse.ui.edit.cut
Edit→Copy	org.eclipse.ui.edit.copy
Edit→Paste	org.eclipse.ui.edit.paste

Plug-in locations of perspectives

The following table lists a few common plugins that define perspectives in the rich client to get you started. If you want to customize rich client perspectives, look for these plugins in the **TC_ROOT\portal\plugins** directory. The perspectives are declared in the plugins with the **org.eclipse.ui.perspectiveExtensions** extension point in the **plugin.xml** file.

To hide perspectives, you can use the **HiddenPerspectives** preference.

Partial list of plugin locations of perspectives

Perspective	Plug-in
Multi-Structure Manager	<code>com.teamcenter.rac.cme.collaborationcontext</code>
My Teamcenter	<code>com.teamcenter.rac.ui</code>
Organization	<code>com.teamcenter.rac.organization</code>
Query Builder	<code>com.teamcenter.rac.querybuilder</code>
Workflow Designer	<code>com.teamcenter.rac.workflow.processdesigner</code>
Workflow Viewer	<code>com.teamcenter.rac.workflow.processviewer</code>

Application Integration Framework (AIF)

AIF customization and development

In versions prior to Teamcenter 2007, the rich client ran on its own Application Integration Framework (AIF). The AIF and rich client were both written using Java. The rich client used Swing for its user interface and all customization mechanisms were Teamcenter-developed.

In versions prior to Teamcenter 2007, the Application Integration Framework (AIF) was an integration framework that enabled developers and customers to build custom interfaces to applications for Teamcenter, NX, and personal use. This framework provided the foundation through which applications could be launched and executed in a standard manner. It also provided the basic design for applications, the base classes and methods, and a methodology for creating and handling events generated by the user interface. In addition, the AIF provided tools to handle the registration of components, represented by JavaBeans, as well as a mechanism for locating and passing messages to those components.

This framework allowed products to be built using a standard interface and methodology. It was modular and dynamic with regard to how applications were registered and launched, allowing applications to be written in a plug-and-play manner and to be released independently of the framework. Applications could be independent of both Teamcenter and NX, but the framework provided simple mechanisms to use Teamcenter as the primary data management system. The **AIFDesktop** component provided the main window users interacted with during the course of their session.

In Teamcenter 2007, the rich client was hosted within the Eclipse rich client platform (RCP) framework. The RCP is a general purpose application framework that provides strong support for modular and extensible component-based development through the use of plug-ins. The rich client took limited advantage of the Eclipse framework. There was a single Teamcenter perspective that all Teamcenter applications used. The user interface was mostly Swing running with the aid of the SWT_AWT bridge.

Starting with Teamcenter 8, the rich client took advantage of many Eclipse features and introduced many SWT-based controls. Some of the rich client changes include:

- Each application became its own perspective.
- Menus were declarative.
- Eclipse OSGi services were used.
- Many Teamcenter extension points, services, and SWT controls were added.

Note:

Teamcenter still supports the AIF in the Eclipse RCP. However, you should make every effort to migrate your customization to use the Eclipse RCP menu, toolbar, and status bar functionality.

Integrating the Application Integration Framework (AIF) desktop with the Eclipse workbench

If you have older AIF desktop customizations, you can integrate them with the newer Eclipse RCP. However, although Teamcenter still supports the AIF in the Eclipse RCP, you should make every effort to migrate your customization to use the Eclipse RCP menu, toolbar, and status bar functionality.

In the rich client, the main integration point is an *application*. The active application defines what is showing and its layout in the main pane area. The current application controls the contents of the main menu bar and toolbar. The navigation pane is always on and helps you to select the current application. In the rich client, there is a main banner that identifies the current application and how to switch between active applications. The main mechanism for defining the set of applications has been the **portal.properties** file. This file is still supported and is augmented by the **aif_application** extension point. The extension point supports three different types of scenarios:

- Adding new traditional or legacy-based applications without modifying the **portal.properties** file.
- Adding new pure SWT-based applications.
For more information about how pure SWT user interface components work together in a workbench, see the following Web site:

<http://www.eclipse.org/articles/Article-UI-Workbench/workbench.html>

- Adding hybrid applications or defining existing legacy applications to be hybrid to allow Swing-based user interface components from within an Eclipse perspective that can also manage pure SWT-based user interface components simultaneously. Though these components can co-exist, there is no additional infrastructure for these different user interface components to collaborate.

The rich client desktop is built on top of the Eclipse RCP workbench, including the menubar, toolbar, and status bar. The RCP workbench is augmented with additional shell trim that defines the main application

switcher/banner bar and the navigation pane. The remaining area is the current active Eclipse perspective. By default, there is a simple Teamcenter perspective that simply holds a tabbed stack of views. Each rich client application is forced to be associated with an Eclipse perspective. Whenever a rich client application is activated, the associated Eclipse perspective is made active. By default, an application is associated with the Teamcenter perspective so legacy applications are not forced to define a perspective to be associated with. If any application also contains a non-null main Swing JPanel, that panel is wrapped in an AWT_SWT bridge view and placed in the stack of views. Since each rich client application is associated with an Eclipse perspective, you might want to read more about what an Eclipse perspective is.

For more information, see the following Web site:

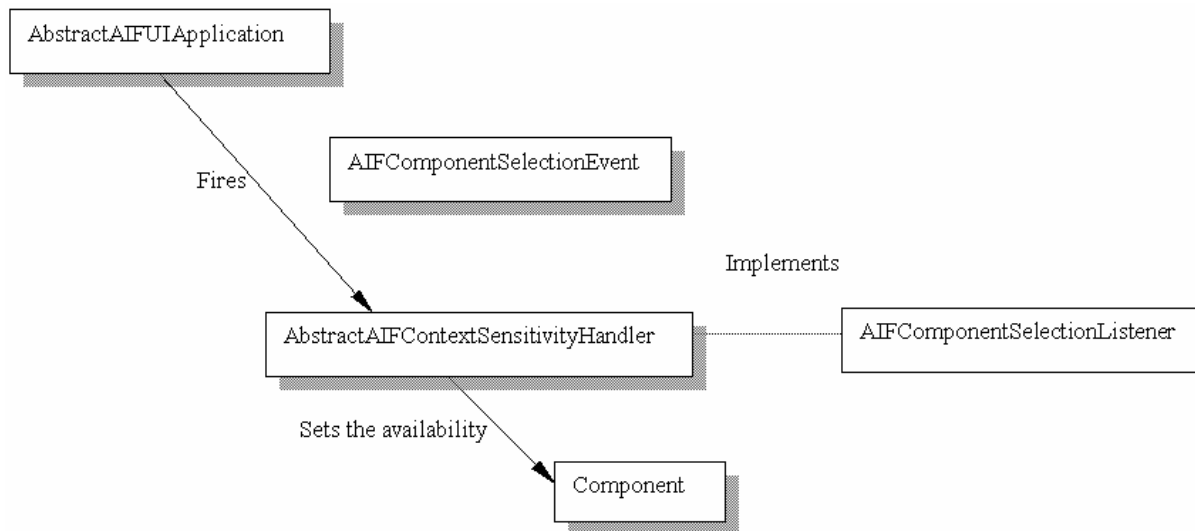
<http://www.eclipse.org/articles/using-perspectives/PerspectiveArticle.html>

AIF context sensitivity

Introduction to AIF context sensitivity

The Application Integration Framework (AIF) supports the concept of context sensitivity for all UI components. Context sensitivity is controlling the availability of UI components when certain conditions or states exist. In some cases, context sensitivity can be confusing to the user because he or she may not know what to do to activate an option. One of the techniques currently used to solve this is to change the tooltip text.

The model for the context sensitivity system is tied to the application where the action appears. There is a selection listener associated with the application such that when something is selected within the application, the application notifies all listeners that a selection is changed. The idea behind this model is that the application fires the event to all listeners and each UI component has an associated *handler* that knows the UI component it is working for. The handler contains the logic that determines the validity of the UI component with which it is associated. In basic terms, the handler typically sets a component to disabled (**setEnabled(false)**). However, the handler is flexible and is designed to allow any action to be taken upon the associated UI component. For example, instead of disabling a component you may want to write a handler that sets a UI component visibility based upon a certain state. The handler interrogates the application for the state, and based on the result invokes the UI components **setVisible()** method. The handler simply implements one method. That method is responsible for making the verdict and applying the appropriate action to the UI component.



Context Sensitivity object model

AIF context sensitivity registration

The registration process is where the application, handler, and UI component are bound together based on the context sensitivity object model. Each application maintains a list of handlers (listeners) that are notified whenever a selection is made within the application.

To register a handler (listener) with an application, invoke the following:

```
// c is an instance of a Component
MyHandler h = new MyHandler ( c );
app.addSelectionListener ( h );
```

This is typically done within the menu bar and toolbar construction. This can also be done within the application panel construction for components located there.

There is a special implementation for Teamcenter applications that utilizes the action system for the menu bar and toolbar. Register your handler within the **actions.properties** file, and it is automatically used with the associated command. Therefore, to register the simple **RequiredSelectionHandler** handler, which is prepackaged for use, add it to your action definition within the **actions.properties** file, as follows:

```
<yourCommand>.SELECTION_HANDLER=
com.teamcenter.rac.aif.common.contextsensitivity.RequiredSelectionHandler
```

No other registration is required for the command. When the selections change within the application, your command is available only when something is selected. If nothing is selected, it is not available.

If you want to keep components within the toolbar and menu bar synchronized like a state selection, you can use this mechanism by manually triggering the selection event and having your handlers refer to a class variable within the application for the state. The system is written generically so that it can handle a variety of cases.

Write a handler for AIF context sensitivity

Base AIF provides the framework for context sensitivity as well as one implemented handler, **RequiredSelectionHandler**. The **RequiredSelectionHandler** handler looks at the given application, and if one or more components are selected, the associated components are enabled. Otherwise, the component is disabled. This is enabled for many actions that currently appear within the menu bar and toolbar, more notably the cut, copy, and paste actions.

Use the following steps to write your own handler:

1. Write a class that subclasses the **AbstractAIFContextSensitivityHandler** class.
2. Provide the implementation method within the class public void **componentSelected** (**SelectionEvent e**). The implementation of the **componentSelected()** class interrogates the application to get the desired data and takes the appropriate action, such as checking the number of components selected within the application and setting the state of the component. The following code shows the source for the **RequiredSelectionHandler** handler:

```
public class RequiredSelectionHandler extends
AbstractAIFContextSensitivityHandler
implements AIFComponentSelectionListener
{
    public RequiredSelectionHandler ( Component cmp )
    {
        super ( cmp );
    }
    public void componentSelected ( AIFComponentSelectionEvent e )
    {
        AbstractAIFUIApplication a = e.getApplication();
        Component c = getComponent();
        if ( a.getTargetContexts() != null )
        {
            c.setEnabled ( true );
        }
        else
        {
            c.setEnabled ( false );
        }
    }
}
```

3. Register your handler with the action by either adding to the registry or manually registering it within the UI parent in which it is contained (menu bar, toolbar, panel).

To manually initiate the firing of a selection event, invoke the **fireSelectionEvent()** method within the application (**a.fireSelectionEvent()**).

B. Customizing form and properties display

Why would I use forms?	B-1
Developing forms using JavaBeans	B-2
Process for developing forms using JavaBeans	B-2
Create a sample custom form using JavaBeans	B-3
Using the Teamcenter property beans	B-9
Developing custom property beans	B-10
Developing forms by extending the abstract class	B-11
Process for developing forms by extending the abstract class	B-11
Create a sample custom form by extending the abstract class	B-12
General comments	B-19
Form performance issues	B-20
Form events	B-21
Form user interface display components	B-21
Displaying a form	B-22

B. Customizing form and properties display

Why would I use forms?

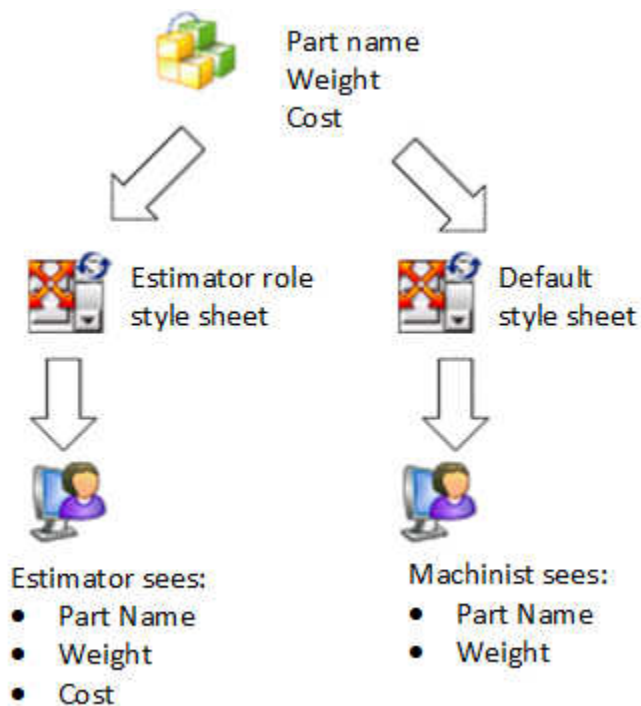
A form is a specialized business object whose sole purpose is to hold properties. Siemens Digital Industries Software recommends that you store your properties on your main business objects, item revision types for example, instead of using forms. If you want to track usage information in assemblies, put those properties on the relation object.

In the following examples, you have three properties you wish to track:

- Part name
- Weight
- Cost - this should only be visible to **Estimators**

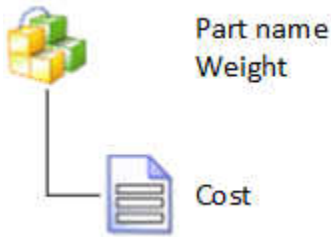
Example - style sheet (preferred)

Instead of using forms, the preferred method is to use style sheets to filter properties for users by group or role. In this example, the **Cost** property is only visible to users in the **Estimator** role.



Example - separate form

Another way to restrict the read or write access to a property separately is to place it on an attached form, and then control the access to the form separately.



This method also has the option of being able to add or modify form properties after the main business object has been released, if desired.

Custom forms

If you must use forms, there are several ways to modify their layout.

- Style sheets - XML definition, stored in the database, no deploy. Siemens Digital Industries Software recommends using this method.
- JavaBean - requires Java coding, must be deployed.
- AbstractRendering - requires Java coding, must be deployed. In places where Swing is still used (double-click a form, for example) you must implement the `IOpenService` to trigger your code.

Notes

- Character (**char**) and character array (**char []**) data types are not supported in forms. Use a string (**string**) or a string array (**string []**) data type with a length of **1**.
- The default string value for **False** is an empty string. Therefore, end users must click the **Show empty properties** link on the rendering page to see this logic attribute if its value is set to **False**.

Developing forms using JavaBeans

Process for developing forms using JavaBeans

1. Launch your IDE and create a new **JPanel**.
2. Add components to the new **JPanel**. To select a library to add components, select the one that includes the **com.teamcenter.rac** plug-in files. If the library containing the plug-in files is not listed, add it.
3. Locate and select a property bean class.

Note:

The property JavaBeans are located in the **com\teamcenter\rac\stylesheets** directory.

4. Repeat the previous steps to add additional JavaBeans.
The property JavaBeans are now ready to use.

Create a sample custom form using JavaBeans

You can create a custom Java panel (**JPanel**) assigned to a custom business object and display the properties using JavaBeans.

In this example, when a user selects a custom **ItemRevision** business object and chooses the **Viewer** view or **View→Properties**, the custom panel is displayed. The custom properties on the business object are set on the Java panel using property beans (JavaBeans).

Note:

Panel rendering is registered to the business object by placing the following command into a custom **stylesheet_user.properties** file:

```
custom-business-object-name.JAVARENDERING.  
package-name.custom-java-panel-name
```

1. Create a custom item business object.
 - a. In the Business Modeler IDE, create a custom item business object as a child of the **Item** business object, for example, **A5_MyItem**.

Note:

The **A5_** portion of the name is an example of a naming prefix. When you create a Business Modeler IDE project, you are required to define a unique prefix that will be affixed to the name of all custom data model items you create to show that they belong to your custom data model. You can use your own prefix, but remember that the following coding examples use this example prefix.

- b. In the **Business Objects** view, open the item revision business object (for example, **A5_MyItemRevision**), click the **Properties** tab, and create the custom persistent properties you want to display in the panel, for example:
 - **a5_MyDate**
Select the **Date** attribute type.

Tip:

Type a display name for each of these new properties, for example, **Test Date**.

- **a5_MyDouble**
Select the **Double** attribute type.
- **a5_MyFlag**
Select the **Boolean** attribute type.
- **a5_MyLongString**
Select the **LongString** attribute type.
- **a5_MyLOV**
Select the **String** attribute type. Attach an LOV to this property, for example, **BillCodes**.
- **a5_MyRef**
Select the **TypedReference** attribute type. Choose a reference business object, for example, **Item**.

- c. Set the **Enabled** property constant to **True** for each of the new properties. This means the property is enabled for display in the user interface.
- d. Deploy the custom template from the Business Modeler IDE to your Teamcenter server. If you use the deployment wizard, select the **Generate Server Cache?** check box to generate shared server cache that contains the new data model.
- e. After deployment, test your new business object in the Teamcenter rich client by creating an instance of it.
For example, in the My Teamcenter application, choose **File→New→Item**.
Your new business object appears in the **New Item** dialog box. Choose your new business object and launch the **New Item** wizard.

2. Create the Eclipse project.

- a. In Eclipse, choose **File→New→Project**.
- b. In the **New Project** dialog box, select **Plug-in Project**. Click **Next**.
- c. In the **Project name** box, type **com.mycom.custompanel**. Click **Next**.
- d. Under **Options**, ensure the **Generate an activator** and **This plug-in will make contributions to the UI** check boxes are selected. Click **Next**.
- e. Clear the **Create a plug-in using one of these templates** check box. Click **Finish**.

3. Update the project tabs.
 - a. Update the **Dependencies** tab.
 - A. In Eclipse, click your project tab and click its **Dependencies** tab.
 - B. Under **Required Plug-ins**, click the **Add** button.
 - C. Select the following plug-ins from the list by holding down the Ctrl key while you click them:


```
com.teamcenter.rac.aifrcp
com.teamcenter.rac.common
com.teamcenter.rac.external
com.teamcenter.rac.kernel
com.teamcenter.rac.neva
com.teamcenter.rac.tcapps
com.teamcenter.rac.util
```
 - D. Click **OK**.
 - b. Update the **Runtime** tab.
 - A. Click your project's **Runtime** tab.
 - B. Under **Exported Packages**, click the **Add** button.
 - C. Select your project and click **OK**.
 - c. Update the **Extensions** tab.
 - A. Click your project's **Extensions** tab.
 - B. Click the **Add** button.
 - C. Select the **com.teamcenter.rac.util.tc_properties** extension point.
 - D. Click **Finish**.
 - d. Save the project by choosing **File→Save All**.
4. Create the project files.
 - a. Create the custom viewer panel.
 - A. In **Package Explorer**, right-click your project and choose **New→Package**.

- B. In the **Name** box, type **com.teamcenter.rac.stylesheet**. This is the path name where rich client style sheet files are located.
- C. Click **Finish**.
- D. Right-click the **com.teamcenter.rac.stylesheet** package and choose **New→File**.
- E. Open the project's **Runtime** tab, click the **Add** button, and add the **com.teamcenter.rac.stylesheet** package.
Note that the listed exported packages are **com.mycom.custompanel** and **com.teamcenter.rac.stylesheet**. This ensures that these packages are listed as exported packages in the **MANIFEST.MF** file.
- F. In the **File Name** box, type **CustomSamplePanel.java** and click **Finish**.
- G. Open the **CustomSamplePanel.java** file and enter the following code:

```
package com.teamcenter.rac.stylesheet;
import org.eclipse.swt.SWT;
import org.eclipse.swt.widgets.Display;
import org.eclipse.swt.widgets.Shell;
import com.teamcenter.rac.util.PropertyLayout;
import java.awt.BorderLayout;
import javax.swing.JLabel;
import javax.swing.JPanel;
/**
 * This example shows how to use the property java beans to display properties/form.
 * To use this example, please create an object with following attributes (or change the
 * property names used in this example to match the data model you have):
 * a5_MyDouble: double type
 * a5_MyDate:    date type
 * a5_MyFlag:    logical type
 * a5_MyLongString: long string type
 * a5_MyRef:     typed reference type
 * a5_MyLOV: a string type attached with LOV
 *
 * To register the usage of this panel, user needs to add a line to the
stylesheet_user.properties file,
 * with the format: <type_name>.JAVARENDERING=<package name>.CustomSamplePanel
 *
 * For example, I have this panel registered to display my custom item revision properties:
 *   A5_MyItemRevision.JAVARENDERING=com.teamcenter.rac.stylesheet.CustomSamplePanel
 *
 * When user selects A5_MyItemRevision and launches the properties dialog, this
CustomSamplePanel
 * should be displayed.
 */
public class CustomSamplePanel
    extends JPanel
{
    public CustomSamplePanel()
    {
        try
```

```

        {
            init();
        }
        catch( Exception e )
        {
            e.printStackTrace();
        }
    }
    private void init()
        throws Exception
    {
        this.setLayout( new BorderLayout() );
        this.setOpaque( false );
        JPanel jPanel1 = new JPanel();
        jPanel1.setLayout( new PropertyLayout() );
        jPanel1.setOpaque( false );
        JLabel jLabel1 = new JLabel("Test double");
        PropertyTextField doubleTextField = new PropertyTextField();
        doubleTextField.setProperty( "a5_MyDouble" );
        JLabel jLabel2 = new JLabel("Test date" );
        PropertyDateButton dateButton = new PropertyDateButton();
        //dateButton.setDate( (String) null );
        //dateButton.setDisplayFormat( "d-MMM-yyyy HH:mm:ss" );
        dateButton.setProperty( "a5_MyDate" );
        dateButton.setMandatory( true );
        JLabel jLabel3 = new JLabel("Test boolean");
        PropertyLogicalPanel logicalPanel = new PropertyLogicalPanel();
        logicalPanel.setProperty( "a5_MyFlag" );
        JLabel jLabel4 = new JLabel("Test longstring");
        PropertyLongTextPanel longTextPanel = new PropertyLongTextPanel();
        longTextPanel.setProperty( "a5_MyLongString" );
        JLabel jLabel5 = new JLabel("Test ref");
        PropertyObjectLink refLink = new PropertyObjectLink();
        refLink.setProperty( "a5_MyRef" );
        JLabel jLabel6 = new JLabel("Test Lov");
        PropertyLOVUIComponent lovUIComp = new PropertyLOVUIComponent();
        lovUIComp.setProperty( "a5_MyLOV" );

        this.add( jPanel1, BorderLayout.WEST );
        jPanel1.add( "1.1", jLabel1 );
        jPanel1.add( "1.2", doubleTextField );
        jPanel1.add( "2.1", jLabel2 );
        jPanel1.add( "2.2", dateButton );
        jPanel1.add( "3.1", jLabel3 );
        jPanel1.add( "3.2", logicalPanel );
        jPanel1.add( "4.1", jLabel4 );
        jPanel1.add( "4.2", longTextPanel );
        jPanel1.add( "5.1", jLabel5 );
        jPanel1.add( "5.2", refLink );
        jPanel1.add( "6.1", jLabel6 );
        jPanel1.add( "6.2", lovUIComp );
    }
}

```

Note:

If you want to set your panel's background color, you cannot use the **setOpaque(false)** tag. For more detail on the usage of this Java API, go to the following URL:

<http://download.oracle.com/javase/6/docs/api/javax/swing/JComponent.html#setOpaque%28boolean%29>

- b. Create a style sheet property file to render the new panel.
 - A. Right-click the **com.teamcenter.rac.stylesheet** package and choose **New→File**.
 - B. In the **File Name** box, type **stylesheet_user.properties** and click **Finish**. This file adds content to the **stylesheet.properties** file. The **_user** name in the file indicates that this is a custom file provided by a user.
 - C. Open the **stylesheet_user.properties** file and enter the following line:

```
A5_MyItemRevision.JAVARENDERING=
com.teamcenter.rac.stylesheet.CustomSamplePanel
```

- D. In the rich client, create a preference for this rendering. Choose **Edit→Options→Filters** and click the **Create a new preference definition** button in the upper left corner of the dialog box. In the **Name** box, type **A5_MyItemRevision.JAVARENDERING** and in the **Values** box, type **com.teamcenter.rac.stylesheet.CustomSamplePanel**.

- c. Save the new files by choosing **File→Save All**.

5. Package the project.

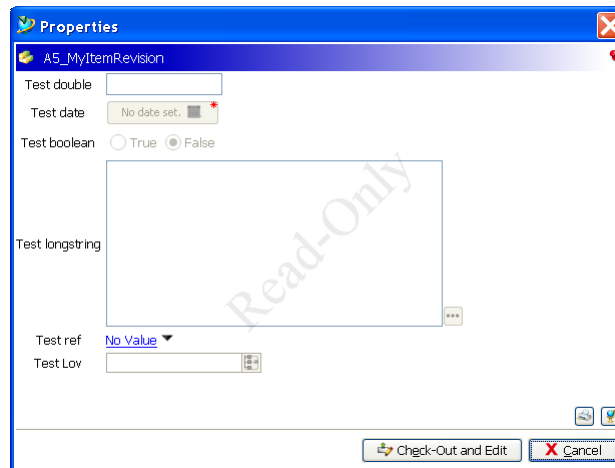
- a. Right-click the project, choose **Export→Plug-in Development→Deployable plug-ins and fragments**, select the **TC_ROOT\portal** directory as the destination, and click **Finish**. The **com.mycom.project-name** JAR file is automatically generated into the **TC_ROOT\portal\plugins** directory.
- b. Run the **TC_ROOT\portal\registry\genregxml** file to register the plug-in with the rich client. When the script is finished, a new **TC_ROOT\portal\registry\RegistryLoader.xml.gz** file is created.

Note:

If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the

properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory.

- c. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.
On a Windows client, it is typically the **%HOMEDRIVE%%HOMEPATH%\Teamcenter** directory.
On a Linux client, it is typically the **\$HOME/Teamcenter/** directory.
6. Verify the customization.
 - a. In the rich client, choose **File→New→Item** and select the new custom item type (for example, **A5_MyItem**), and create an instance of it.
 - b. Select the item revision and click the **Viewer** tab or choose **View→Properties** to see your new panel.



Custom form using JavaBeans

Using the Teamcenter property beans

You can create a **JPanel** component in which to contain the property beans, place them in the panel, and connect them directly to the form properties by entering in the property for the box. Once this is done, compile the form and register it with the rich client.

Registration is accomplished by adding an entry in the form of *Form-Type-Name.FORMJAVARENDERING=Your-Form-Class* to the **com.teamcenter.rac.stylesheet.stylesheet_user.properties** file. Once registered, the rich client uses the custom form when the form type is opened.

For example, if the form type is **MyFormType** and the form display class is named **com.mycompany.forms.MyForm**, the entry is added to the properties file, as follows:

```
My\ Form\ Type.FORMJAVARENDERING=com.mycompany.forms.MyForm
```

Often, a form must obtain the reference to the **TCComponentForm** component with which it is rendering. Each property bean has knowledge of the **TCComponentForm** class; however, for the **JPanel** component to recognize the form, you must include a constructor that includes the **TCComponentForm** component. When you use your IDE to create the **JPanel** component, you may be provided a default constructor, for example:

```
public MyPanel()
{
}
```

To reference the **TCComponentForm** component, include a constructor requesting a reference, as shown in the following example:

```
// IDE given
public MyPanel()
{
}
// User written
public MyPanel ( TCComponentForm f )
{
}
```

When the form loads, the system looks first for the constructor with a reference to the **TCComponentForm** component. If one is found, it is used. If not, the default constructor is used.

Developing custom property beans

Each property bean fundamentally knows how to load and save itself. When the form is loaded and ready to be displayed, each property bean is notified to load itself and its data. The property bean uses the defined property and obtains the value from the Teamcenter database. When the user clicks the **Save** button, each property bean is notified to save. When the form is loaded, the top-level container (the **JPanel** component) that is registered for the form type is recursively cycled to look for the **InterfacePropertyComponent** interface. When found, the beans are instructed to either load or save. The property beans can be nested as deeply as desired within containers and still be selected by the system.

To implement property beans, decide which user interface class to subclass. Then, identify a Teamcenter property bean and implement the **InterfacePropertyComponent** interface, which contains two methods:

```
public void load ( TCComponent ) throws Exception

public void save ( TCComponent f ) throws Exception
```

After you create the component, all other JavaBean rules apply. For example, you can attach icons for reference within an Integrated Development Environment (IDE), such as Eclipse, and attach property rendering rules.

To perform checks prior to loading the property beans, override the **checkObject** method as follows:

```
public void checkObject() throws Exception
{
    //required checks
}
```

Note:

All IDEs that support JavaBeans work with the property beans.

To increase the efficiency of property beans, Siemens Digital Industries Software recommends that you also implement the **InterfaceBufferedPropertyComponent** class. This requires a method that has the following signature:

```
public TCFormProperty saveProperty ( TCComponent f )
throws Exception
```

This method should only use the **setValue<type>Data** calls to the form property and return it. Therefore, all properties in the property bean system are collected and saved in one call. This increases the efficiency of the property bean.

Siemens Digital Industries Software provides both **save()** and **saveProperty()** methods to allow for flexibility in form storage. All property beans delivered with Teamcenter use the **saveProperty()** method. If you choose to override any of the base property beans, Siemens Digital Industries Software recommends that you override the **saveProperty()** method.

Developing forms by extending the abstract class

Process for developing forms by extending the abstract class

Perform the following steps to create a user interface form:

1. Create a form type on the server side.
The form type can be created in Teamcenter using the Business Modeler IDE application. All methods require you to enter the form type name: POM storage.
2. Extend the schema.
3. Create the form panel.
Create a Java object that extends from the **AbstractRendering** class, which requires the implementation of two methods: **loadRendering()** and **saveRendering()**. The **loadRendering()** method is invoked when the form is constructed and reads the values from the **TCComponent** class to populate the form interface. The **saveRendering()** method writes the values from the user interface components to the **TCComponent** class. Override the **checkObject()** method to enable performance of checks prior to loading the form.

4. Register the form panel with the rich client.

Once a form class is written on the client, it must be registered in the **stylesheet_user.properties** registry file to be recognized within Teamcenter. The key within the **stylesheet_user.properties** file points to the implemented **AbstractRendering** class definition and is comprised of the form type name appended with **.FORMJAVARENDERING**. The associated client-side definition file for the Item Revision Master form type is shown in the following code example.

```
ItemRevision\ Master.FORMJAVARENDERING=com.teamcenter.rac.form.ItemRevisionMaster
```

The backslash character and a space (\) in the string create a space. If the backslash character is not used, the space is misinterpreted and the form is displayed using the automatic form display. Java interprets the key as **item** and does not parse past the space, considering it the delimiter for the key/value combination.

Create a sample custom form by extending the abstract class

You can extend the abstract class (**AbstractRendering**) to display properties for a custom business object.

In this example, you create a custom Java form assigned to the **ItemRevisionMaster** form. When a user chooses **File**→**New**→**Item** to launch the **New Item** wizard for a custom **Item** business object, a new form is displayed on the **Define additional item revision information** page of the wizard.

This example extends the **AbstractRendering** component. The sample code uses the **getRenderingModified** and **isRenderingModified** methods. These methods ensure the values are copied from the **New Item** wizard to the form.

Note:

The **isRenderingModified()** method is required to use this customized form in the **Viewer** view.

Note:

Rendering of the form is tied directly to the business object by placing the following command into a custom **stylesheet_user.properties** file:

```
custom-business-object-name.FORMJAVARENDERING.  
package-name.custom-java-form-name
```

1. Create a custom item business object.
 - a. In the Business Modeler IDE, create a custom item business object as a child of the **Item** business object, for example, **A5_MyItem**.

Note:

The **A5_** portion of the name is an example of a naming prefix. When you create a Business Modeler IDE project, you are required to define a unique prefix that will be affixed to the name of all custom data model items you create to show that they belong to your custom data model. You can use your own prefix, but remember that the following coding examples use this example prefix.

- b. Deploy the new custom item business object from the Business Modeler IDE to your Teamcenter server.
- c. After deployment, test your new business object in the Teamcenter rich client by creating an instance of it.
For example, in the My Teamcenter application, choose **File→New→Item**. Your new business object appears in the **New Item** dialog box. Choose your new business object and launch the **New Item** wizard. Observe the boxes on the **Define additional item revision information** page. These are provided by the item revision master form. In the following steps, you are going to create your own custom form to replace it and to provide different boxes on this page.

Default form in the item creation wizard

2. Create the Eclipse project.
 - a. In Eclipse, choose **File→New→Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Click **Next**.
 - c. In the **Project name** box, type **com.mycom.masterform**. Click **Next**.

- d. Under **Options**, ensure the **Generate an activator** and **This plug-in will make contributions to the UI** check boxes are selected. Click **Next**.
 - e. Clear the **Create a plug-in using one of these templates** check box. Click **Finish**.
3. Update the project tabs.
- a. Update the **Dependencies** tab.
 - A. In Eclipse, click your project tab and click its **Dependencies** tab.
 - B. Under **Required Plug-ins**, click the **Add** button.
 - C. Select the following plug-ins from the list by holding down the Ctrl key while you click them:

com.teamcenter.rac.aifrcp
com.teamcenter.rac.common
com.teamcenter.rac.external
com.teamcenter.rac.kernel
com.teamcenter.rac.neva
com.teamcenter.rac.tcapps
com.teamcenter.rac.util
 - D. Click **OK**.
 - b. Update the **Runtime** tab.
 - A. Click your project's **Runtime** tab.
 - B. Under **Exported Packages**, click the **Add** button.
 - C. Select the **com.mycom.custompanel** and **com.teamcenter.rac.stylesheet** packages and click **OK**.
 - c. Update the **Extensions** tab.
 - A. Click your project's **Extensions** tab.
 - B. Click the **Add** button.
 - C. Select the **com.teamcenter.rac.util.tc_properties** extension point.
 - D. Click **Finish**.
 - d. Click the **plugin.xml** tab and replace the contents of the file with the following:

```
<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.2"?>
<plugin>
  <extension point="com.teamcenter.rac.util.tc_properties">
    <plugin_properties pluginName="com.mycom.masterform"/>
  </extension>
</plugin>
```

e. Save the project by choosing **File→Save All**.

4. Create the project files.

a. Create the custom item revision master form.

A. Right-click the **com.mycom.masterform** package and choose **New→File**.

B. In the **File Name** box, type **A5_MyItemMaster.java** and click **Finish**.

C. Open the **A5_MyItemMaster.java** file and enter the following code:

```
package com.mycom.masterform;

import com.teamcenter.rac.stylesheet.AbstractRendering;
import com.teamcenter.rac.kernel.TCComponent;
import com.teamcenter.rac.kernel.TCException;
import com.teamcenter.rac.kernel.TCProperty;
import com.teamcenter.rac.util.MessageBox;
import com.teamcenter.rac.util.PropertyLayout;
import com.teamcenter.rac.util.VerticalLayout;

import java.util.HashMap;
import java.util.Map;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JTextField;

public class A5_MyItemMaster extends AbstractRendering {

  /**
   *
   */
  private static final long serialVersionUID = 1L;
  private JTextField data1_jtextfield;
  private JTextField data2_jtextfield;
  private TCProperty data1_tcproperty;
  private TCProperty data2_tcproperty;
  private TCComponent comp; // TcEng legacy used TCComponentForm, use TCComponent now

  public A5_MyItemMaster( TCComponent c)throws Exception
  {
    super ( c );
    comp = c;
    loadRendering();
  }
}
```

```

public void loadRendering() throws TCException
{
    initializeUI();
    data1_tcproperty = comp.getTCProperty("user_data_1");
    data2_tcproperty = comp.getTCProperty("user_data_2");
    data1_jtextfield.setText(data1_tcproperty.getStringValue());
    data2_jtextfield.setText(data2_tcproperty.getStringValue());
}

public void saveRendering()
{
    try
    {
        data1_tcproperty.setStringValueData(data1_jtextfield.getText() );
        data2_tcproperty.setStringValueData(data2_jtextfield.getText() );
        TCProperty[] ps = new TCProperty[2];
        ps[0] = data1_tcproperty;
        ps[1] = data2_tcproperty;

        // TcEng legacy used setTCProperties or setFormTCProperties,
        // use component.setTCProperties(ps);

        component.setTCProperties( ps );
    }
    catch ( Exception ex )
    {
        MessageBox.post(ex.getMessage(), null, MessageBox.ERROR);
    }
}

public boolean isRenderingModified()
{
    if( data1_tcproperty != null && !
data1_jtextfield.getText().equals( data1_tcproperty.getStringValue() ) )
    {
        return true;
    }
    if( data2_tcproperty != null && !
data2_jtextfield.getText().equals( data2_tcproperty.getStringValue() ) )
    {
        return true;
    }
    else
    {
        return false;
    }
}

@Override
public Map getRenderingModified ()
{
    Map modifiedRendering = new HashMap<String, Object> ();
    if( data1_tcproperty != null && !
data1_jtextfield.getText().equals( data1_tcproperty.getStringValue() ) )
    {
        data1_tcproperty.setStringValueData(data1_jtextfield.getText() );
        modifiedRendering.put( "user_data_1", data1_tcproperty );
    }
}

```

```

    }
    if( data2_tcproperty != null && !
data2_jtextfield.getText().equals( data2_tcproperty.getStringValue() ) )
    {
        data2_tcproperty.setStringValueData( data2_jtextfield.getText() );
        modifiedRendering.put( "user_data_2", data2_tcproperty );
    }
    return modifiedRendering;
}

private void initializeUI()
{
    setLayout ( new VerticalLayout() );
    JPanel mainPanel = new JPanel( new PropertyLayout());
    mainPanel.setOpaque(false);
    mainPanel.setEnabled(false);
    // Create all the text fields
    data1_jtextfield = new JTextField(15);
    data2_jtextfield = new JTextField(15);
    //Add components to Panel
    mainPanel.add("1.1.right.center",new JLabel("User Data One"));
    mainPanel.add("1.2.left.center", data1_jtextfield);
    mainPanel.add("2.1.right.center",new JLabel("User Data Two"));
    mainPanel.add("2.2.left.center", data2_jtextfield);
    add("unbound.bind", mainPanel);
}
/**
 *The following code is optional depending on whether you want to pre-load the
 *property data from the original (selected) item revision during a Revise.
 * Without it the values are reset/left blank.
 */
@Override
public void setValues(String[] props, Object[] values)
{
    if ( props != null && props.length > 1 )
    {
        for( int i=0; i<props.length; i++ )
        {
            if( props[i].equals( "user_data_1" ) )
            {
                data1_jtextfield.setText(values[i].toString());
            }
            else if( props[i].equals( "user_data_2" ) )
            {
                data2_jtextfield.setText(values[i].toString());
            }
        }
    }
}
}

```

Note:

Note how this file uses the **TCComponent** method to set properties on components. This method uses the underlying Teamcenter Services API. Prior to Teamcenter 9, the **SOAPPropertyHelper** method, which was a temporary wrapper

on top of legacy code, could be used to set properties on components even though it was unpublished.

- b. Create a style sheet property file to render the new form.
 - A. In **Package Explorer**, right-click your project and choose **New→Package**.
 - B. In the **Name** box, type **com.teamcenter.rac.stylesheet**. This is the path name where rich client style sheet files are located.
 - C. Click **Finish**.
 - D. Right-click the **com.teamcenter.rac.stylesheet** package and choose **New→File**.
 - E. In the **File Name** box, type **stylesheet_user.properties** and click **Finish**. This file adds content to the **stylesheet.properties** file. The **_user** name in the file indicates that this is a custom file provided by a user.
 - F. Open the **stylesheet_user.properties** file and enter the following line:

```
A5_MyItemRevisionMaster.FORMJAVARENDERING=
com.mycom.masterform.A5_MyItemMaster
```

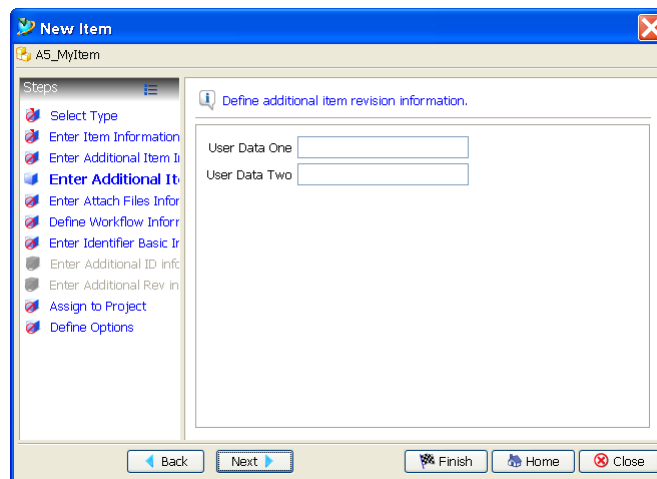
- c. Save the new files by choosing **File→Save All**.

5. Package the project.
 - a. Right-click the project, choose **Export→Plug-in Development→Deployable plug-ins and fragments**, select the **TC_ROOT\portal** directory as the destination, and click **Finish**. The **com.mycom.project-name** JAR file is automatically generated into the **TC_ROOT\portal\plugins** directory.
 - b. Run the **TC_ROOT\portal\registry\genregxml** file to register the plug-in with the rich client. When the script is finished, a new **TC_ROOT\portal\registry\RegistryLoader.xml.gz** file is created.

Note:

If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory.

- c. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.
On a Windows client, it is typically the **%HOMEDRIVE%%HOMEPATH%\Teamcenter** directory.
On a Linux client, it is typically the **\$HOME/Teamcenter/** directory.
6. Verify the customization.
 - a. In the rich client, choose **File→New→Item** and select the new custom item type (for example, **A5_MyItem**), and click **Next**.
 - b. Assign the new item an ID and name and click **Next**.
 - c. Click **Next** on the **Define additional item information** page.
 - d. On the **Define additional item revision information** page, you should see your new item revision master form.



Custom form in the item creation wizard

General comments

The form user interface is not limited to creation using an integrated development environment (IDE) or to the use of any Java component. Third-party Java components can be used within the form.

The Eclipse IDE can be used to generate the contents of the form. Once created, you need only add the code required to read the property values from Teamcenter and set the property values within the associated component on the panel. Other components, such as **LOVButton**, make it easier to render the form properties.

If you upgrade from a previous version of Teamcenter that used the note attribute in the form storage class, instances created from that class will continue to use the note attribute. If you create a new storage class, it uses the string attribute, and instances created from that class use the string attribute.

Starting with Engineering Process Management 2005, you should use the new style sheet package (**com.teamcenter.rac.stylesheet**) instead of the old form package (**com.teamcenter.rac.form**). The old form package is deprecated. If you want to still use the old custom forms in the new package, move the entries you added to the **form_user.properties** file to the **stylesheet_user.properties** file.

Form performance issues

System performance when forms are saved depends largely on the way the API is used. All API components for saving and loading forms are found in the **TCComponentForm** component. The object retrieves form values and can represent all types of data found in Teamcenter. It is patterned after the **TCProperty** object. The **TCComponentForm** class is responsible for handling the retrieval and storage of properties to a form. The **TCFormProperty** object handles the data representation of the value (or property).

It is more efficient to save all form properties in one call, because this minimizes network traffic and limits the save action to one commit to the database. If each property is saved individually, each requires a network call and a commit to the database, which degrades system performance.

For example, a form containing 20 properties takes approximately two seconds to save when all properties are saved in one call and approximately 25 seconds to save when each property is saved individually. There are legitimate cases for saving properties both individually and collectively. Therefore, take care when deciding which API to use to achieve a desired result. The following examples illustrate the different usages of the form API and assume a component **f** of type **TCComponentForm**:

- **Obtaining one form property**

```
TCFormProperty p = f.getFormTCProperty("my_prop_name");
```

- **Obtaining all form properties**

```
TCFormProperty[] props = f.getAllFormProperties();
```

- **Saving one form property**

```
TCFormProperty p = f.getFormTCProperty("my_prop_name");
// Get the property to set
p.setStringValue ( "abc" ); // Set it.
At this point it is saved to the db.
```

- **Saving several form properties**

```
TCFormProperty[] ps = f.getAllFormProperties();
// Get the property to set
ps[0].setStringValueData ( "abc" );
// Sets the value but is not saved.
ps[1].setStringValueData ( "def" );
// Sets the value but is not saved.
f.setTCProperties(ps); // Now is saved to the db.
```

In summary, performance gains depend on how form data is saved. Whenever possible, obtain an array of properties and set their values by using methods such as **setStringValueData()** as opposed to the **setStringValue()** method. The **setStringValue()** method sets the value and performs the save immediately. The **setStringValueData()** method sets the value but relies on a subsequent call to perform the commit. Finally, for an array of properties, make a call such as **setTCProperties()** to increase efficiency.

Form events

When a form is displayed, the size is governed by the standard of preferred size for a dialog box. However, it may be necessary to control the dialog box size prior to displaying the form. In this event, implement the **InterfaceRendererEvent** interface within the form display class. This interface forces the implementation of two methods: **dialogDisplayed (OpenFormDialog dlg, boolean state)**. The method is called before the dialog box is displayed. It is the place where the **setBounds()** method for the dialog box can be called.

When a form is displayed, the **okToModify()** method is invoked. If the form is modifiable, it is constructed and displayed as designed. However, if the form is not modifiable, logic is executed to determine what should or should not be editable. When a read-only form is displayed, the components shown in the following table are modified.

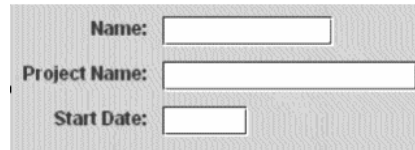
Component	Read-only effect
JTabbedPane	None
JSplitPane	None
JPanel	None
JLabel	None
JScrollPane	None
JProgressBar	None
JTextComponent	Edit ability set to False
All others	Disabled

When a form is not modifiable, all **Container** objects, such as **JTabbedPane**, are ignored and the remaining components are disabled. This is because **Container** objects allow users to traverse and work through them to view the data. You may want to control the read-only ability of a component within a form, in which case you must override the read-only logic by implementing the **InterfaceRendererEvent** interface and providing body to the **setReadOnly()** method.

Form user interface display components

The rich client has changed the strategy of forms in Teamcenter. Forms within the rich client are user interface representations only. Teamcenter stores the information for the form; therefore, when the

form is loaded and displayed in the rich client, all data is obtained from the server and placed into user interface components. The base component for a rich client form is a **JPanel** component that provides flexibility for the placement of forms within different parent containers. The following figure shows a form panel.

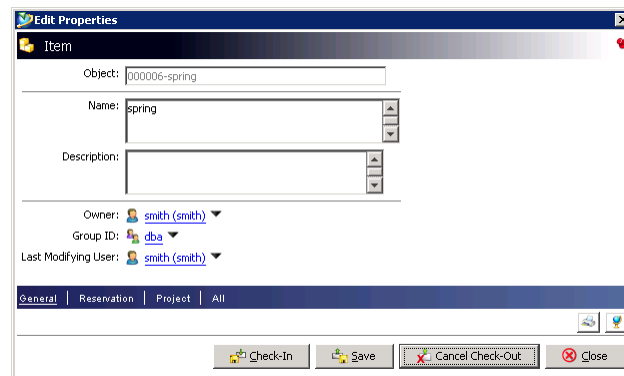


A screenshot of a form panel with a light gray background. It contains three labels with corresponding text input fields: "Name:" followed by a single-line text box, "Project Name:" followed by a wider single-line text box, and "Start Date:" followed by a date picker control.

Form user interface display panel

Displaying a form

When a form is displayed, the user interface definition for the form is constructed, populated, and placed within a container. In the rich client, a form can be displayed under two paradigms: dialog box and viewer. The dialog box displays when the form is opened using the **Open** menu or the form is double-clicked.



A screenshot of a dialog box titled "Edit Properties". The dialog has a tabbed interface with tabs for "General", "Reservation", "Project", and "All". The "General" tab is active. It contains several fields: "Object:" with the value "000006-spring", "Name:" with the value "spring", and "Description:" which is empty. Below these are dropdown menus for "Owner:" (smith (smith)), "Group ID:" (dba), and "Last Modifying User:" (smith (smith)). At the bottom, there are four buttons: "Check-In", "Save", "Cancel Check-Out", and "Close".

Form displayed in a dialog box

The viewer paradigm is used when the form is selected and the **Viewer** tab is active (the following figure). The contents of nonwritable form are displayed as read-only. If the user has write access, the form can be edited.

The screenshot shows a software window titled '000006-spring'. It has a menu bar with 'Summary', 'Details', 'Viewer', 'Impact Analysis', and 'JT Preview'. The 'Viewer' tab is active. The form contains the following fields:

- Object:** 000006-spring
- Name:** spring
- Description:** Spring for assembly
- Owner:** smith (smith)
- Group ID:** dba
- Last Modifying User:** smith (smith)

At the bottom, there is a tab bar with 'General', 'Reservation', 'Project', and 'All'. Below the tabs are four buttons: 'Check-In', 'Save', 'Cancel Check-Out', and 'Clear'.

Form displayed in the rich client viewer

A reusable Java component is used for form panel construction. Given the component reference to the form, call the **RenderingLoader.load(TCComponent)** method, which constructs and returns the form panel. The form definition within the rich client is flexible and supports a wide variety of applications. When a form panel is loaded, the **RenderingLoader** looks to see if there is a registered form panel for the component. If so, the registered form panel is returned. If not, an automatic form is constructed and returned.

Index

A

- Abstract class naming conventions A-5
- AbstractDialog class 4-125
- AbstractPopupButton class 4-96
- AbstractProgressDialog class
 - Commands 4-82
 - Features 4-80
- AbstractProgressDialog class 4-80
- AbstractUINode class 4-92
- AccessDialog component 4-89
- Accessor method naming conventions A-6
- actions.properties file A-31
- AIFDesktop component A-28
- all XML element 2-26
- Appearance keys 3-7
- Application Integration Framework (AIF) A-28
- application startup option A-2
- arch startup option A-2

B

- Base properties files 3-7
- break XML element 2-28

C

- centerToScreen method 4-95
- Classes
 - AbstractAIFContextSensitivity Handler A-32
 - AbstractDialog 4-95
 - AbstractPopupButton 4-96
 - AbstractProgressDialog 4-80
 - AbstractProgressDialog 4-82
 - AbstractUINode 4-92
 - DeleteOperation 4-83
 - GenericTableModel 4-97
 - InterfaceSignalOnClose 4-135
 - JTextArea 4-98
 - JTextField 4-98
 - MessageBox 4-118
 - Naming conventions A-6
 - SignalOnClose 4-135
 - StringViewerDialog 4-125
 - SystemColor A-7
 - TCComponentForm B-20
 - TCConstants 4-93
 - TCFormProperty B-20
 - TCTypeRenderer 4-93
- classificationProperties XML element 2-29
- classificationTrace XML element 2-31
- clean startup option A-3

- close() method 4-135
- closeSignaled() method 4-135
- Color format 3-8
- Colors
 - Conventions A-7
- column XML element 2-33
- com.teamcenter.rac.stylesheet.stylesheet_user.properties B-9
- Command line startup options A-1
- Command Suppression
 - Constraints 4-30
 - Customization 4-27
- Components
 - AccessDialog 4-89
 - ExpansionRule 4-84
 - GroupPanel 4-84
 - MLabel 4-120
 - MRUButton 4-87
 - OpenByNameButton 4-87
 - OrgSelectionDialog 4-88
 - ReferencerUINode 4-91
 - ReferencersPanel 4-89
 - RolePanel 4-92
 - Separator 4-123
 - SplitPane 4-124
 - StringViewerPanel 4-126
 - TCComponentUINode 4-92
 - TableModel 4-97
 - UserPanel 4-94
 - iTextArea 4-98
 - iTextField 4-98

- configuration startup option A-3
- consolelog startup option A-3
- Context menu suppression 4-72
- Context sensitivity A-30
 - Object model A-31
 - Registration A-31

Conventions

- Color A-7
- Dialog box A-7
- File organization A-5
- Fonts A-7
- Naming A-5
- Property A-6
- Source code A-6
- Text field A-7

- Conventions, syntax definitions 1-3
- customPanel XML element 2-37

D

- data startup option A-3

Data tabs display, customizing 3-8
debug startup option A-3
DeleteOperation class 4-83
detach startup option A-3
dev startup option A-3
Dialog box conventions A-7
dialogDisplayed() method B-21
Displaying
Files in the viewer 3-4
DskipRegReload startup option A-4

E

Eclipse

Workbench A-29

Examples

AbstractDialog class 4-95
AbstractPopupButton code 4-96
AbstractProgressDialog component 4-82
GenericTableModel component 4-97
HorizontalButtonLayout layout manager 4-99
HorizontalLayout layout manager 4-105
MLabel component 4-120
MRUButton component 4-87
MessageBox component 4-118
OpenByNameButton component 4-88
OrgSelectionDialog component 4-89
PropertyLayout layout manager 4-110
ReferencersPanel component 4-91
Registry file 4-121
Registry file references 4-122
Separator component 4-123
SplitPane component 4-125
StringViewerDialog component 4-126
StringViewerPanel component 4-126
TCComponentUINode component 4-93
TCTypeRenderer class 4-94
UserPanel component 4-95
VerticalButtonLayout layout manager 4-103
VerticalLayoutManager 4-114

Examples

GroupPanel component 4-85
RolePanel component 4-92

Exception class naming conventions A-6

ExpansionRule component 4-84

F

Files

Organization conventions A-5
Properties 3-7
actions.properties A-31
com.teamcenter.rac.stylesheet.stylesheet_
user.properties B-9

fireSelectionEvent() method A-32

Font conventions A-7

Font format 3-8

Forms

Comments B-19
Customizing B-1
Definition B-1
Developing B-2
Display components B-21
Displaying B-22
Events B-21
Performance issues B-20

G

generate_client_meta_cache utility 2-16

GenericTableModel class 4-97

getChildren() method 4-84

getOperations() method 4-83

GroupPanel component 4-84

H

Handlers

Registering A-31
RequiredSelectionHandler A-31
Writing A-32

header XML element 2-41

HorizontalLayout layout manager 4-105

I

Image file naming conventions A-5

image XML element 2-43

initialize startup option A-4

initializeDialog() method 4-82

Initiate selection events A-32

Instantiation keys 3-7

Interface components

InterfaceBufferedPropertyComponent B-10
InterfacePropertyComponent B-10
InterfaceRendererEvent B-21

Interface naming conventions A-5

InterfaceBufferedPropertyComponent component B-10

InterfacePropertyComponent component B-10

InterfaceSignalOnClose class 4-135

iTextArea component 4-98

iTextField component 4-98

J

JPanel component B-9

JTextArea class 4-98

JTextField class 4-98

L

label XML element 2-45

Layout managers

- Horizontal ButtonLayout 4-99
- HorizontalLayout 4-105
- PropertyLayout 4-110
- Vertical ButtonLayout 4-103
- VerticalLayout 4-114

listDisplay XML element 2-48

Listener leaks 4-134

Locale properties files 3-7

Localization 4-22

Localization keys 3-7

M

Manufacturing Process Planner tabs 3-10

Menu commands

- Toggle 4-66

MessageBox class 4-118

Methods

- RenderingLoader.load() B-23
- centerToScreen 4-95
- close() 4-135
- closeSignaled() 4-135
- componentSelection() A-32
- dialogDisplayed() B-21
- fireSelectionEvent() A-32
- getChildren() 4-84
- getOperations() 4-83
- initializeDialog() 4-82
- okToModify() B-21
- save() B-11
- saveProperty() B-11
- setBounds() B-21
- setEnabled() A-30
- setFormProperties() B-20
- setReadOnly() B-21
- setStringValue() B-20
- setStringValueData() B-20
- setVisible() A-30
- startOperation() 4-83
- startProcess() 4-83

MLabel component 4-120

mpp.properties file 3-10

MRUButton component 4-87

N

nl startup option A-4

nosplash startup option A-4

Note attribute B-19

O

objectSet tag 2-50

okToModify() method B-21

OpenByNameButton component 4-87

OrgSelectionDialog component 4-88

os startup option A-4

P

Packages

- Naming conventions A-5

page XML element 2-53

parameter XML element 2-59

perspective startup option A-4

Perspectives

- Plug-ins A-27

Plug-ins, perspectives A-27

plugincustomization startup option A-4

Process business object 3-10

product startup option A-4

Properties

- Display B-1
- File conventions A-6
- Files A-6

Properties files 3-6

Property beans

- Developing B-10
- Location B-2
- PropertyArray A-8
- PropertyCheckbox A-9
- PropertyCheckboxOptionLov A-10
- PropertyImage A-10
- PropertyLOVButton A-12
- PropertyLOVCombobox A-13
- PropertyLabel A-11
- PropertyLogicalPanel A-11
- PropertyLongText A-11
- PropertyNameLabel A-14
- PropertyObjectLink A-14
- PropertyPanel A-15
- PropertyRadioButton A-15
- PropertyRadioButtonOptionLov A-16
- PropertySlider A-16
- PropertyTextArea A-17
- PropertyTextField A-17
- PropertyToggleButton A-18
- PropertyToggleButtonOptionLov A-19
- TitledPropertyArray A-19
- TitledPropertyCheckbox A-20
- TitledPropertyCheckboxOptionLov A-20
- TitledPropertyLOVButton A-22
- TitledPropertyLOVCombobox A-23
- TitledPropertyLabel A-21
- TitledPropertyLogicalPanel A-21

- TitledPropertyLongText A-22
- TitledPropertyObjectLink A-23
- TitledPropertyPanel A-23
- TitledPropertyRadioButton A-23
- TitledPropertyRadioButtonOption Lov A-24
- TitledPropertySlider A-24
- TitledPropertyTextArea A-25
- TitledPropertyTextField A-25
- TitledPropertyToggleButton A-25
- TitledPropertyToggleButtonOption Lov A-26
- Using B-9
- PropertyArray bean A-8**
- propertyChangeListener 4-89**
- PropertyCheckbox bean A-9**
- PropertyCheckboxOptionLov A-10**
- PropertyImage bean A-10**
- PropertyLabel bean A-11**
- PropertyLayout layout manager 4-110**
- PropertyLogicalPanel bean A-11**
- PropertyLongText bean A-11**
- PropertyLOVButton bean A-12**
- PropertyLOVCombobox bean A-13**
- PropertyNameLabel bean A-14**
- PropertyObjectLink bean A-14**
- PropertyPanel bean A-15**
- PropertyRadioButton bean A-15**
- PropertyRadioButtonOptionLov bean A-16**
- PropertySlider bean A-16**
- PropertyTextArea bean A-17**
- PropertyTextField bean A-17**
- PropertyToggleButton bean A-18**
- PropertyToggleButtonOptionLov bean A-19**

R

- rac_command_suppression variable 4-27**
- Referencers layout managers**
 - In general 4-90
 - ReferencersReverseHorizontal NodeLayout 4-90
 - ReferencersTreeLookNodeLayout 4-90
 - ReferencersVerticalNodeLayout 4-91
- ReferencersPanel component 4-89**
- ReferencersReverseHorizontalNodeLayout layout manager 4-90**
- ReferencersTreeLookNodeLayout layout manager 4-90**
- ReferencersVerticalNodeLayout layout manager 4-91**
- ReferencerUINode component 4-91**
- refresh startup option A-5**
- Registry 3-6**
- Registry keys**
 - Appearance 3-7
 - Instantiation 3-7
 - Localization 3-7
 - Structural 3-7

- Syntax 3-8
- rendering XML element 2-65**
- RequiredSelectionHandler handler A-31**
- ResourceBundle object 3-6**
- Rich client**
 - Debugging tools 4-128
- RolePanel component 4-92**

S

- save() method B-11**
- saveProperty() method B-11**
- section XML element 2-66**
- Separator component 4-123**
- separator XML element 2-70**
- setBounds() method B-21**
- setEnabled() method A-30**
- setReadOnly() method B-21**
- setVisible() method A-30**
- showlocation startup option A-5**
- SignalOnClose class 4-135**
- Source**
 - File naming conventions A-6
 - Code conventions A-6
- SplitPane component 4-124**
- startOperation() method 4-83**
- startProcess() methods 4-83**
- String array format 3-8**
- StringViewerDialog class 4-125**
- StringViewerPanel component 4-126**
- Structural keys 3-7**
- Style sheets**
 - Create 2-13
 - Search 2-10
- Supported types 3-8**
- Suppressing commands 4-27**
- Syntax definition conventions 1-3**

T

- Tab customization 3-10**
- Table viewer 4-31**
- tableDisplay XML element 2-71**
- TableModel component 4-97**
- TCComponentUINode component 4-92**
- TCConstants class 4-93**
- TCTypeRenderer class 4-93**
- Text**
 - Field conventions A-7
- thumbnailDisplay XML element 2-73**
- TitledPropertyArray bean A-19**
- TitledPropertyCheckbox bean A-20**
- TitledPropertyCheckboxOptionLov bean A-20**
- TitledPropertyLabel bean A-21**
- TitledPropertyLogicalPanel bean A-21**
- TitledPropertyLongText bean A-22**

TitledPropertyLOVButton bean A-22
TitledPropertyLOVCombobox bean A-23
TitledPropertyObjectLink bean A-23
TitledPropertyPanel bean A-23
TitledPropertyRadioButton bean A-23
TitledPropertyRadioButtonOptionLov bean A-24
TitledPropertySlider bean A-24
TitledPropertyTextArea bean A-25
TitledPropertyTextField bean A-25
TitledPropertyToggleButton bean A-25
TitledPropertyToggleButtonOptionLov bean A-26
Toggle menu 4-66
Tree viewer 4-40
treeDisplay XML element 2-75

U

User
 Properties files 3-7
UserPanel component 4-94

V

Variable naming conventions A-6
VerticalLayout layout manager 4-114
Viewer, displaying files 3-4
visibleWhen expression 4-28
vm startup option A-5
vmargs startup option A-0

Siemens Digital Industries Software

Headquarters

Granite Park One
5800 Granite Parkway
Suite 600
Plano, TX 75024
USA
+1 972 987 3000

Americas

Granite Park One
5800 Granite Parkway
Suite 600
Plano, TX 75024
USA
+1 314 264 8499

Europe

Stephenson House
Sir William Siemens Square
Frimley, Camberley
Surrey, GU16 8QD
+44 (0) 1276 413200

Asia-Pacific

Suites 4301-4302, 43/F
AIA Kowloon Tower, Landmark East
100 How Ming Street
Kwun Tong, Kowloon
Hong Kong
+852 2230 3308

About Siemens Digital Industries Software

Siemens Digital Industries Software is a leading global provider of product life cycle management (PLM) software and services with 7 million licensed seats and 71,000 customers worldwide. Headquartered in Plano, Texas, Siemens Digital Industries Software works collaboratively with companies to deliver open solutions that help them turn more ideas into successful products. For more information on Siemens Digital Industries Software products and services, visit www.siemens.com/plm.

This software and related documentation are proprietary and confidential to Siemens.

© 2020 Siemens. A list of relevant **Siemens trademarks** is available. Other trademarks belong to their respective owners.